The pdp64 Architecture Manual

A 64-bit extension of the PDP-11

Digital Equity Collective's Pragmatic Digital Processor, 64 bit

David Walther

Copyright © 2026 David Walther. Released under the BSD 2-Clause licence (the file `COPYING` of the repository).
The source of this book is https://github.com/moebiusV/pdp64 (`docs/streamlined/`); the instruction dictionary is generated from the machine-readable specification. Typeset with Typst in EB Garamond and DejaVu Sans Mono.

CONTENTS

About this manual ii

Part I — The architecture 1

Chapter 1 — Introduction 2

Chapter 2 — The bus 7

Chapter 3 — Graphics: the terminal and its devices 11

Chapter 4 — Boot, the BIOS, the disk label and the GPT 14

Chapter 5 — The device driver model 18

Chapter 6 — The kernel: modes, exceptions and memory management 23

Chapter 7 — Multiple processors 41

Chapter 8 — Vector work, AI and array languages: the device route 51

Chapter 9 — The instruction set 55

Chapter 10 — Conclusion: from FPGA to silicon 112

Part II — The instruction reference tables 115

The instruction reference tables 116

The second page: one-parcel abbreviations 117

Assembler aliases and old spellings 119

The architectural constants 120

The mnemonics, alphabetically 123

The opcodes, in order 128

Part III — Appendices 130

Appendix A — Possible improvements not decided on yet 131

Appendix B — Decisions pending 134

Appendix C — Glossary 140

Appendix D — The ABI, version 1 143

Appendix E — The assembly language and the object format 144

About this manual

Each section is a few numbered aphorisms followed by a commentary. The aphorisms say what is true of the machine, in plain short statements, and read on their own as the main text. The commentary under them gives, as they apply: the *Precedent* (what earlier machines did), the *Trade-offs* (what the choice costs), the *Improvements* (what was changed from the precedent), the *Benefits*, the *Modern comparables* (what current architectures do), and, for an instruction that is not obvious, its *Use*. Every decision also carries an *Impact* line saying what it means for an efficient **kernel**, for efficient **compilation** and for an efficient **hardware layout**: the three things a processor designer asks of a choice. An instruction as plain as `MOV` or `ADD` needs no statement of use. The form is the one the author chose (errata 56); the reference entries of the full manual (errata 57) will follow the layout of the classic CPU manuals, in which every instruction has its encoding diagram, its operation and its effects.

Each aphorism carries a status. *(D)* means decided, with the erratum or the revision that fixed it where useful. *(P)* means a proposal that the author has not yet settled. *(S)* means a stub: a place held for material the author is to supply. An aphorism with no tag describes the architecture as it is built and tested.

Numbers. Where a figure is measured, the text says what was measured; where it is an estimate, it says so. A claim about another machine is the author's account, checked as far as the fact-check record shows (`../fact-check-2026-10-22.md` and the verification appendix of `../decisions-pending.md`).

The PDF. The same text as a book, `book/pdp64-manual.pdf`, built by `tools/mkbook.py` from the sources of this directory (Typst; see `book/README.md`).

Where the rest is. The full instruction set text is `../pdp64-isa.md`; every decision with its reasons is `../errata.md` and `../CHANGES-SINCE-R20.md`; the decisions still open, with five options each, are `../decisions-pending.md`.

Part I — The architecture

Chapter 1 — Introduction

The pdp64 is a 64-bit successor to the PDP-11, designed to be as pleasant to program as the PDP-11 and the VAX-11 were, and as small to build as the PDP-11 was. It is a little-endian machine with sixteen 64-bit registers, instructions of two, four or six bytes, and the PDP-11's addressing modes and branches, brought up to date with what fifty years of compilers and operating systems have shown to matter. This chapter says why it exists, what made its ancestors good, and the principles the rest of the book follows.

1.1 Why the machine exists

1.1.1 The PDP-11 and the VAX-11 were pleasant, even fun, to program in assembly language.

1.1.2 Afterward computer design went in many directions, and that pleasure was largely lost.

1.1.3 The pdp64 carries it forward, using the lessons of the last fifty years: which instructions are needed to get things done and which are not, what a processor is actually used for, and which instructions it needs the most.

1.1.4 The PDP-11 is a source, not an obligation. What earned its place stayed, what did not was dropped, and what fifty years showed to be missing was added.

Commentary. **Precedent.** The claim in 1.1.1 is old and widely made of the PDP-11: a small, regular instruction set that a programmer could learn in a week and keep in mind, with a set of addressing modes that fit the C language so closely that the language and the machine seem to have grown together. **Improvements.** The measurements overturned several received ideas, which is the reason the project looked rather than copied. A second unsigned multiply turned out to be the same instruction as the first, while the instruction that returns the *high half* of a product, which multi-precision arithmetic needs, was missing. A rotate by a register count has almost no users, because cryptographic code rotates by constants. A divide is rare, because compilers turn division by a constant into a multiply and a shift. Dedicated absolute-value, minimum and maximum instructions are worth under a tenth of a percent of compiled code. And CRC instructions become unnecessary once a carry-less multiply exists. **Benefits.** A machine built on these findings is small enough for a single engineer to hold in mind, and, as later chapters show, small enough to fit in a modest FPGA. **Modern comparables.** The same re-examination produced the RISC designs of the 1980s and, more recently, the compact RISC-V and LoongArch instruction sets; the pdp64 differs by keeping the PDP-11's memory operands, addressing modes and variable-length encoding where those were found to save code and to cost little.

1.2 What made the PDP-10, the PDP-11 and the VAX-11 good to program

1.2.1 The PDP-10 gave a programmer one word per instruction that said everything: the operation, the register, the index register, the indirect bit and the address.

1.2.2 The PDP-10's registers were also memory: the sixteen accumulators answered to the addresses 0 to 15.

1.2.3 The PDP-10 had half-word and byte operations, stack and skip instructions, and a procedure call that was one instruction.

1.2.4 The PDP-11 made every instruction and every addressing mode available with every register, so that the language had few exceptions to remember.

1.2.5 The PDP-11's program counter and stack pointer were ordinary registers, and its autoincrement and autodecrement modes made `*p++` and `*--p` single instructions.

1.2.6 The PDP-11 had no input or output instructions: a device was a few memory locations, and a driver was an ordinary program.

1.2.7 The VAX-11 widened the PDP-11 to 32 bits and added three-operand arithmetic, a complete virtual-memory system, and instructions for the jobs programs actually do: procedure calls, queues, character strings, array indexing and polynomial evaluation.

Commentary. **Precedent.** These are the qualities the author has in mind, stated here as the book's starting point rather than as a history. The PDP-10's simplicity of format and its habit of making the machine's own state addressable (1.2.1, 1.2.2) are the origin of the idea that the machine should hold together in the head. The PDP-11's orthogonality (1.2.4) is the property that mattered most to assembly programmers and, later, to compiler writers: an instruction that takes a register takes an address, an immediate or a stack slot with the same syntax. Its memory-mapped devices (1.2.6) are the reason the pdp64's devices are still eight addresses each (chapter 5). **Trade-offs.** Each of these qualities had a price that the later machines paid. The VAX-11's richness (1.2.7) bought programmability and cost an unbounded amount of hidden state in its longest instructions, so that a page fault in the middle of a character move could not be resumed without a special bit in the status word. The Motorola 68000, which took the PDP-11's orthogonality and widened it, could not restart an instruction after a bus error and so could not support demand-paged virtual memory until its successor patched the problem. **Improvements.** The pdp64 keeps what made programming pleasant and removes what made implementation hard: no hidden instruction state (every instruction can be restarted), at most one memory operand, a fixed small maximum instruction length, and no mode bits that change what unrelated instructions mean unless their benefit outweighs the cost. **Benefits.** The measurements in the instruction-set chapter show where this paid off: the processor fits in about 16,000 FPGA lookup tables and the compiler's code is the size of `gcc -Os`. **Modern comparables.** Of today's architectures, x86-64 kept the memory operands and variable-length encoding of this lineage; AArch64 and RISC-V chose fixed-length register-to-register encodings; the pdp64 sits between them, closer to x86 in the form of its instructions and closer to RISC-V in the discipline of its encoding.

1.3 More minimal than its ancestors, where hindsight allows

1.3.1 In several ways the pdp64 is more minimal than the PDP-11 and the VAX-11: it gives up theoretical completeness where fifty years of use showed the completeness was never exercised, and it spends what that saves on power that was.

1.3.2 Not every instruction works with every addressing mode on every operand: at most one operand of an instruction is a memory reference. *(D)*

1.3.3 The deferred addressing modes are gone; an indirection is a `JMPI`, a `CALLI` or an ordinary load. *(D)*

1.3.4 The ALU has one width, there is no arithmetic in the narrow sizes, and there is no conversion opcode group: conversions are moves. *(D)*

1.3.5 There is one rotate direction, and no slot exists only to complete a table.

1.3.6 The VAX's packed decimal, string editing and `CALLS`/`CALLG` procedure-call instructions are absent. *(D, instruction-set text, "explicitly rejected")*

1.3.7 There are no transcendental instructions, no separate floating-point register file and no rounding-mode bits. *(D)*

1.3.8 What the saving buys is power that programs use: a bit unit (population count, count of leading and trailing zeros, byte reversal, bitfields, a carry-less multiply), block operations that restart from their registers, a fused multiply-add with the IEEE exception flags, the full set of atomics, UTF-8 instructions, and a user-memory guard. *(D)*

Commentary. **Precedent.** The PDP-11 was orthogonal, any operation with any mode on any register, and the VAX carried that to completeness: every operation in every data type, with up to six operands, each operand of any mode. Both were designed when memory was dear and compilers were weak, and the completeness did two things at once: it made assembly programming regular and pleasant, and it made a compiler's job simple. **Trade-offs.** The completeness was never free. It bought a decoder that had to parse specifier after specifier before the next instruction's address was known, and an instruction that could fault in the middle with unbounded state to save. What hindsight lets the pdp64 do is separate the two benefits: keep the regularity that programmers and compilers use (every register is general, every mode means the same thing everywhere it is allowed) and drop the completeness that nobody exercised (a memory-to-memory arithmetic instruction, a deferred mode on every operand, a sized version of every operation). The price is a few more instructions in rare cases, such as a load and a store where the VAX had one memory-to-memory move. **Improvements.** Each piece of dropped completeness was weighed against what it freed: the one memory operand lets the fault matrix collapse to three rows with the third empty and pays for sixteen registers and a 12-bit displacement in the same parcel; dropping the sized ALU removes byte-lane multiplexing and partial-register writes altogether; dropping the transcendentals leaves the `FMA` datapath, which is what accurate library maths is built from. **Benefits.** The decoder is small, the exception machinery is small, and the die area that completeness would have used goes to a bit unit and a block-move sequencer that together cost less than 5% of the core. **Modern comparables.** The RISC machines of the 1980s went further and made every instruction register to register

except loads and stores; x86 kept one memory operand and survived on it; the pdp64 takes the middle position, which keeps the compact code of a memory operand and the restartability of a load/store design.

1.4 The principles

1.4.1 The design follows these principles, in this order: **code size**; **easy recoverability**, meaning fewer possible errors and an easier recovery path, so less recovery code; **speed and efficiency through fewer gates and tighter hot loops**, by careful choice of instructions. *(D, README; recorded by the author 2026-10-22)*

1.4.2 All the rules are soft rules: a rule gives way when breaking it is worth more than it costs, with the cost written down. *(D, errata 42)*

1.4.3 Between two designs that both work, prefer by this order: the one with no strange side effects (an instruction does what its name says and nothing else); the one that does not push an existing instruction into a longer encoding; and the simpler one (do not multiply entities without benefit).

Commentary. **Precedent.** The earlier statement of the constraints, still in the instruction-set text, ranked low transistor count first, bounded fault surface second, pleasantness in assembler third, a good compiler target fourth and code density fifth. The author's present ordering puts code size first and folds the gate count into the third principle with speed. The two orderings agree on almost everything, because a small, regular machine is also compact in code and cheap in gates; they differ in which wins when they conflict, and this book follows the present one. **Trade-offs.** "Easy recoverability" is the principle that most often changes a decision. It means counting the cost of an option not only in gates and bytes but in the code a kernel must write to handle its failures: a feature that adds a new fault, a new error return or a new half-finished state costs recovery code in every system that uses it, and a feature that makes an operation repeatable from the start costs none. **Improvements.** The soft-rules principle replaced an earlier text that listed two rules as having no exceptions: "no mode bits" and "natural alignment of quad accesses". They remain strong defaults, because breaking either has a large cost, but each is now weighed like any other choice. Three mode bits are accepted in the design on those terms: the translation-enable bit that is set once at boot, the user-memory guard that lives in the status word, and the sticky floating-point flags, which never change what an instruction does. **Benefits.** A reader can tell why a choice was made by asking which principle it serves. **Modern comparables.** Most architectures state no priorities; the RISC-V project's published principle of keeping the base small and the extensions optional is the nearest in spirit.

1.5 What has been built and measured

1.5.1 The machine is specified by machine-readable grammars; the assembler, linker, simulator, a compiler back end and the QEMU target are generated from or checked against them, so that the numbers cannot drift apart.

1.5.2 On 827 real source files that every compiler accepts, the first compiler's output, in code and data, is within 0.2% of `gcc -Os` and 6.5% smaller than `clang -Os` on amd64, using 1.19 and 1.10 times as many (shorter) instructions.

1.5.3 516 C test programs pass on the simulator and on QEMU.

1.5.4 The finish line before silicon is an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh`. *(D, errata 52)*

Commentary. **Precedent.** The habit of generating every tool from one description, rather than writing each and hoping they agree, is the discipline that other instruction-set projects adopted late; here it was the starting rule, and it is why a change to an opcode number is made once. **Trade-offs.** The measurements are of compiled C at the compiler's size-optimising setting; they are static counts of what is written, not run-time measurements, and the compiler is one of two planned (the LLVM back end, which OpenBSD needs, is at the stage where its assembler and object writer agree with the project's own on every form tested). **Benefits.** The comparison gives the architecture a number against which later changes are measured: the last change to the flags cost 0.39% in code size and was accepted for the benefit it gave to faster future implementations. **Modern comparables.** `gcc` and `clang` at `-Os` are the standard measuring sticks for code size on x86-64 and AArch64; the pdp64 figure is compared with them on the same source files.

1.6 Does the instruction set matter?

1.6.1 The instruction set is not interchangeable with any other: it fixes what every implementation must preserve, and so what a processor costs to build, to verify, to program and to secure.

1.6.2 The common view, that a good implementation hides the instruction set, is true of one thing: the peak speed of a very large core, where a great deal of hardware can absorb almost any encoding.

1.6.3 It is not true of the cost of a small core, the size of the code, the amount of software a system needs, or the number of ways a system can fail; those follow the instruction set, and they can be measured.

1.6.4 Each of the rows below is a measurement or an estimate from this project, and each is explained in the chapter named beside it.

What differs	The figure	Where
The logic of a complete implementation, same company and era	the PDP-11's T-11, a full integer core, was about 12,000 transistors; the MicroVAX 78032, a subset of the VAX with no floating point, about 125,000	10.1, 10.2
What it takes to find where the next instruction starts	pdp64: two bits of the first parcel; PDP-11: both operand specifiers; x86: prefixes, opcode, ModR/M and SIB in sequence; VAX: every specifier in order	9.4
Whether flags block a faster implementation	68.2% of compiled instructions wrote a flag and 1.5% read one, so the flag rules were changed at a measured cost of 0.39% in code size	9.3, 10.3
Size of compiled code	for the same 827 source files and the same compiler family, the pdp64 output is 0.86 times the size of its amd64 output, and 1.002 times `gcc -Os`	1.5
System-call path, instructions in and out (paper estimates)	x86-64 Linux about 45 and 30; AArch64 about 60 and 45; RISC-V about 66 and 45; VAX about 11 and 8; m68k 6 and 8; pdp64 4 to 5 and 4	6.5
Lines of kernel code to start a second core	x86: 1,528 plus 1,258 of real-mode code; arm64: 124 or 127; RISC-V: 108 or 58; pdp64: one register write	4.5, 7.8
Machine-dependent code in an OpenBSD port	riscv64 35,600 lines; arm64 61,613; amd64 79,167	10.4
Errata, workaround and quirk lines per 1,000 code lines in Linux's architecture directories	x86 0.8; arm64 0.7; arm 0.5; powerpc 0.4; mips 0.3; m68k 0.1; riscv 0.1; s390 0.0	pain-points report
Hardware-erratum options in Linux's configuration	arm64 64; arm 32; riscv 14	pain-points report

Commentary. **Precedent.** The view that instruction sets are equivalent has real support. Published comparisons of x86 and ARM implementations (the 2013 study usually cited is "Power Struggles: Revisiting the RISC vs. CISC Debate on Contemporary ARM and x86 Architectures", an unverified citation here) found that at the same performance level the instruction set explains little of the difference in power or speed, and the very large cores of both families have converged. The pdp64 does not dispute that. **Trade-offs.** The claim is about steady-state performance of comparable large cores, and the table is about other costs. The table's figures are measurements of real machines and real software, not controlled experiments: some of the differences between architectures in the Linux figures are partly a matter of age and deployment (x86 is the oldest and most widely deployed), and the system-call and start-up figures are estimates and line counts of what is written, not run times. They show that the instruction set is correlated with these costs, and the chapters show the mechanism in each case. **Improvements.** Each row is a place where this design made a different choice from the architecture it is compared with, and the choices were made against these costs, not against peak speed. **Benefits.** A reader who thinks the instruction set does not matter can check any row against its chapter: the answer is in what the instruction set obliges an implementation to do (precise exceptions, ordering, flag behaviour, instruction length, restart) and in the code a system must then write. **Modern comparables.** RISC-V's own case for a clean base instruction set and x86's survival by microarchitecture are the two sides of the argument.

1.7 How the book is organised

1.7.1 Chapter 2 is the bus. Chapter 3 is graphics: the terminal, a modernised C64 with its blitter, and how it drives HDMI and the other ports. Chapter 4 is boot. Chapter 5 is the device driver model. Chapter 6 is the kernel: modes, exceptions and memory management. Chapter 7 is multiple processors. Chapter 8 is vector work, AI and array languages, done by devices. Chapter 9 is the instruction set. Chapter 10 is the path to silicon.

1.7.2 Each section states its aphorisms first and its commentary after; the status tag *(D)* or *(P)* marks what is decided and what is proposed.

Commentary. **Precedent.** The form follows the old commentary traditions, in which a short text can be read straight through and the discussion of each line sits beneath it; and the layout of the classic CPU manuals for the reference material,

in which every instruction has a fixed entry (errata 56 and 57). **Benefits.** A designer who wants only the architecture reads the aphorisms; one who wants the reasons reads on.

Chapter 2 — The bus

The processor reaches the world through one bus, Q64, on which every device is a handful of memory locations, the bus itself is a device that says what is plugged into it, and every transfer of data is a descriptor in memory that a device carries out and marks done. This chapter states what the processor owes the bus, how the bus enumerates and maps devices and tells the processor when one comes or goes. How a driver talks to a device, the eight-register layout and the default devices are in chapter 5. Where the design is decided it is tagged *(D)*; where it is the author's proposal not yet settled it is *(P)*. Interrupts and modes are chapter 6, and the other cores are chapter 7. The full discussion is in `../pdp64-q64-bus.md`; the options still open are in `../decisions-pending.md` (R1 to R8, Q10).

2.1 Q64 and the address space

2.1.1 Q64 is the system bus: the Q-bus of the PDP-11 widened to 64-bit data and a 40-bit physical address. *(D)*

2.1.2 Devices live in the ordinary address space; the processor has no input or output instructions. *(D)*

2.1.3 Each device occupies its own 4 KB page of physical address space, so that one device can be mapped to one protection domain. *(P)*

2.1.4 Every page has a memory type: normal cached, normal uncached, device strongly ordered, or device weakly ordered. *(D)*

2.1.5 The detailed transactions of Q64 (widths, bursts, arbitration, timing) are not yet defined; the proposal is to take AXI as the transport and to treat Q64 as the programming model on top of it. *(P, decision R4)*

2.1.6 The top bit of the 40-bit physical address could select the space: set, the access goes to a device; clear, it goes to memory. Memory would then be 512 GB and device space 512 GB, with one bit and no range comparison deciding where an access goes. This is the VAX's arrangement (below). *(D, the author, 2026-10-22, errata 60; decision P19)*

Commentary. **Precedent.** The PDP-11's Unibus put every device in the address space at a fixed location, which is why a PDP-11 driver is an ordinary program; the Q-bus did the same with multiplexed lines. **Trade-offs.** The memory type is per page, not per access, so the code that writes a device register need not know that it is a device; the price is one page per device, which wastes address space but not memory. Using AXI as the transport saves a bridge between two protocols (estimated at about 600 lookup tables on the FPGA) and loses the Unibus's locked read-modify-write as a native bus transaction. **Improvements.** A 40-bit physical address puts devices and memory in one terabyte, with room for every device to sit in its own page. **Benefits.** An uncached, strongly ordered page needs no special instruction to reach a register. **Modern comparables.** RISC-V and ARM systems use memory-mapped devices on AXI or TileLink; x86 keeps both port I/O and memory-mapped devices. **Impact.** *Kernel:* Devices are pages in the kernel's address map, found by the bus controller. *Compiler:* Nothing special. *Hardware:* Address decode is by page; the transport is AXI if the proposal is accepted.

Commentary on 2.1.6. **Precedent.** The VAX-11/780 divided its physical address space in two: memory in the low half and the I/O space above `0x20000000` (the 4.3BSD kernel's `NEX780`), which holds one 8 KB window (`NEXSIZE`) per adaptor slot, with the Unibus adapters' device registers reached in a space of their own beyond it; the bit that is set for the whole of I/O space is the one the hardware decodes first (read in the 4.3BSD `nexus.h`, which also records the 8600's, 8200's and MicroVAX's different bases, so the VAX itself kept changing the map from model to model, which the pdp64's single fixed bit would avoid); the PDP-11 put its devices in a fixed page at the top of the address space (a V7 driver's pointer to its registers is a constant in that page). **Trade-offs.** One bit makes the first routing decision in the bus fabric trivial and makes two classes of bug detectable: a page-table entry that maps a normal cached page onto device space, or a device page onto memory, can be a fault because the entry's memory type and the address bit disagree; and the bus controller can refuse a device's DMA into device space, so that no device can write another's registers, which is a free partial protection until devices have a translation unit. The cost is a change to the address map (the emulator's timer and console move into the upper half) and, if the bit were a forty-first, one more bit in every translation-buffer entry and every bus address. The reading of "bit 40" that costs

nothing is the top bit of the existing 40-bit address (bit 39 counting from zero), because the page-table entry already holds physical page numbers up to that bit and DRAM then fills exactly 39 bits, the width of a virtual address. **Benefits.** Fewer gates in decode, and fewer ways to get a device mapping wrong. **Modern comparables.** Most systems split the map by address ranges chosen by the platform; the VAX and the PDP-11 used a fixed region; x86 has a separate port space. **Impact.** *Kernel:* a device mapping that is cached by mistake is a fault, not a silent bug, and a driver cannot be given a DMA address that reaches registers. *Compiler:* none. *Hardware:* a one-bit decode at the head of the fabric and a consistency check in the page-table walk.

2.2 What the processor owes the bus

2.2.1 The processor owes the bus exactly three things: a memory type per page, a `FENCE` instruction whose meaning against Q64 is defined, and atomics that can reach device memory. *(D)*

2.2.2 A driver can be sure a descriptor is visible before it writes the doorbell: it builds the descriptor, executes `FENCE`, and writes the doorbell register. *(D)*

Commentary. **Precedent.** The PDP-11 needed none of this because there was one processor and no cache. **Trade-offs.** Three obligations is the whole contract, which keeps the device side free to change without respecifying the processor. **Benefits.** A device and a driver can be written against these three promises without knowing the processor's pipeline. **Modern comparables.** ARM's device-memory types and `DMB`/`DSB`, and RISC-V's `FENCE` with I/O bits, play the same roles. **Impact.** *Kernel:* One `FENCE` before the doorbell is the whole ordering rule. *Compiler:* `FENCE` is an intrinsic. *Hardware:* The memory type is two bits per page-table entry and the bus honours it.

2.3 When an access fails

2.3.1 A read or instruction fetch of non-existent memory is a precise abort with cause `BUSFAULT`, at the faulting instruction. *(D)*

2.3.2 A posted write that times out, and an uncorrectable memory error, are machine checks: imprecise and not restartable. A `FENCE` before a probe makes even a posted write precise at a known point. *(D)*

2.3.3 `PROBER` and `PROBEW` test whether an address can be read or written without faulting. *(D)*

2.3.4 Resetting the bus is a write to a register of the bus controller, not an instruction. *(D)*

Commentary. **Precedent.** A PDP-11 read timeout was an unrestartable bus error, fatal to the process; the `nofault` idiom of V6 and V7 existed to probe for devices without dying. **Improvements.** The read timeout became a precise, restartable abort, and `PROBE` replaces the idiom. **Benefits.** A driver that probes for a device handles a missing one in a few lines of ordinary fault-handling code, which is the recoverability principle at work. **Modern comparables.** x86 machine checks and ARM's `SError` correspond to the imprecise class; PCI returns all ones for a read of a removed device, which every driver must then learn to distrust, and the pdp64 avoids that convention. **Impact.** *Kernel:* A driver probes a missing device in a few lines of ordinary fault handling. *Compiler:* None. *Hardware:* A timeout counter in the bridge and a way to make a read abort precise.

2.4 Coherence and the hand-off

2.4.1 Between processors the data caches are coherent without any software action, by snooping with three states (modified, shared, invalid), for two to four cores. *(D)*

2.4.2 Between a processor and a DMA device they are not: buffers are mapped uncached and the hand-off is explicit. *(D)*

2.4.3 The hand-off is five steps: the processor builds the descriptors and data in uncached memory, executes `FENCE`, writes the device's doorbell register; the device walks the descriptors, moves the data by DMA, writes the result and a status word, and raises an interrupt; the processor reads the result. *(D)*

Commentary. **Precedent.** The 11/70's cache watched Unibus writes so that DMA appeared coherent; the VAX's Unibus adapter had buffered data paths that a driver had to purge. **Trade-offs.** Snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off, so two mechanisms are used, each where it is cheap; the cost is that the processor reads received data from uncached memory, which is slow, and a cache-maintenance instruction would remove it for a non-coherent device that may come later. **Benefits.** No driver can forget a cache clean or invalidate, because there is none to forget. **Modern comparables.** x86 DMA is coherent; ARM and MIPS systems with non-coherent DMA need a DMA-mapping layer in the kernel to place cache maintenance correctly. **Impact.** *Kernel:* Uncached DMA buffers

mean no driver cache code. *Compiler:* None. *Hardware:* A snoop port on each data cache for core coherence only; no snoop port for DMA.

2.5 Atomics on the bus

2.5.1 The atomics are defined on a monitor: a load-linked takes a reservation, a write by any initiator clears it, and a store-conditional succeeds only if it is intact; the Unibus's `DATIP`/`DATO` locked pair is not an architectural primitive. *(D, errata 45)*

2.5.2 A device that writes memory shared with a core takes part by clearing reservations. *(D)*

Commentary. **Precedent.** The PDP-11's locked read-modify-write and the VAX's interlocked instructions are bus locks. **Trade-offs.** A lock is simple and a bottleneck as cores multiply; a monitor maps onto AXI's exclusive access and the memory systems of FPGAs and modern chips, at the cost of forward-progress rules. **Modern comparables.** RISC-V `LR`/`SC`, AArch64's exclusive loads and stores, and the large-system atomics that perform the operation at the memory. **Impact.** *Kernel:* Lock words and lock-free structures use the atomics of chapter 7. *Compiler:* C11 atomics lower to them. *Hardware:* A reservation monitor and an exclusive access, not a bus lock.

2.6 The bus as a device

2.6.1 The bus controller is itself a device with eight memory-mapped registers, through which the processor detects what is on the bus and asks it to map a device to an address range. *(D, errata 49)*

2.6.2 The bus alerts the processor when a device enters or leaves. *(D, errata 49)*

Offset	Register	Meaning
`0x00`	`ID`	bus controller, version, number of slots
`0x08`	`CONTROL`	enable events, rescan, reset the bus
`0x10`	`STATUS`	event pending, queue overflowed, busy
`0x18`	`SLOT`	select a slot
`0x20`	`INFO`	for the selected slot: present, type, version, window size, interrupt level
`0x28`	`MAPADDR`	the address at which the device's window appears
`0x30`	`MAPCTL`	enable the mapping; request an orderly eject
`0x38`	`EVENT`	next event (arrival, departure, fault) and its slot; zero if none

2.6.3 Each slot starts at a fixed 4 KB page; `MAPADDR` is for devices that need a particular address or size. *(P, decision N1)*

2.6.4 An arrival or departure puts an event in a small queue and raises an interrupt; if the queue overflows a flag says events were lost and the driver rescans the bus, which is also what it does at boot. *(P, decision N2)*

2.6.5 When a device leaves, its slot is marked absent at once: a read of its window is a precise bus fault, a posted write is dropped and counted, and its in-flight bus transactions are aborted. *(P, decision N3)*

2.6.6 The queued work of a departed device stays in memory untouched, so the driver can read, save, resend or fail it. *(P, the author's suggestion)*

Commentary. **Precedent.** The VAX scanned its interconnect: `probenexi()` in 4.3BSD reads each nexus slot's status register, whose type field says what is plugged in; the Unibus had no enumeration at all, with devices at switch-set addresses and `badaddr()` probes, and nothing told the kernel when a device left. PCI sizes each device's windows by writing all ones to its base registers and assigns addresses; Amiga AutoConfig lets each board announce itself at a fixed address and be told where to live; NuBus gave each slot a fixed share of the address space. **Trade-offs.** A fixed page per slot needs no assignment step and limits the slot count; assignment on demand supports any layout at a few gates of logic per slot. **Improvements.** On a surprise removal PCI returns all ones, so every driver must treat all ones as a possible error; the pdp64 uses the precise bus fault the instruction set already defines, so the driver has a single recovery path. The queue's overflow rule makes boot and recovery the same code. **Benefits.** The kernel needs one fixed address, the bus controller's, to find every device, and the hand-off at boot shrinks to memory, cores and that address. **Modern comparables.** PCI Express enumerates through configuration space with hot-plug signalled by a slot-status interrupt; USB hubs report port changes through a status-change endpoint; NVMe and `virtio` keep their queues in host memory and re-queue them after a reset. **Impact.** *Kernel:* Enumeration is one loop over the slots; hot-plug is an interrupt and a queue the kernel drains. *Compiler:* None. *Hardware:* A slot register, an information register, an event queue of about eight entries and one presence bit per slot.

2.7 Still open

The bus chapter's open items, each with five options in the decisions document: the transport (R4), the interrupt controller and routing (R1), starting the other cores and inter-processor interrupts (R2, B2), coherence beyond four cores (R3), the register layout and descriptor details (R5; chapter 5), DMA addressing and protection (R6), errors raised by devices (R7), virtualization (R8), and whether to use `virtio` devices in the emulator to reach an OpenBSD login sooner (Q10 and the roadmap).

Chapter 3 — Graphics: the terminal and its devices

The pdp64's display is a modernised and extended Commodore 64 built as memory-mapped devices: a video device, a sound device and a blitter, each with the eight-register face of chapter 5, and a driver for each that is a page of register writes. The pdp64 is an extension of the PDP-11 and its terminal is an extension of the C64: the same idea (a machine whose video and sound are chips with registers, not a graphics card with a programming model) at a modern width. The processor never sees the display protocol: HDMI, like SATA and USB, is a transmitter chip on the board with its own standard, and the pdp64's side of it is a small set of registers. This chapter sits after the bus because the first thing a user sees is the terminal, and before boot because the terminal is the first device the boot software drives. Tags: *(D)* decided; *(P)* proposed; *(S)* stub, to be filled when the author supplies the source.

3.1 The terminal: VIC-64X, SID-64X and BLT64

3.1.1 The video device `VIC-64X` is the Commodore 64's video chip (the VIC-II) extended: a framebuffer in the address space with 48-bit pixels, scanned out through a line buffer. *(D, the author's statement; the formats are to be written down)*

3.1.2 The sound device `SID-64X` is the C64's sound chip (the SID) extended: voices with filters, with 48-bit audio samples. *(D, the author's statement; the formats are to be written down)*

3.1.3 The blitter `BLT64` copies and combines rectangles of pixels with raster operations: registers for the source and destination base addresses, the strides, the width and height, a raster operation (the sixteen binary operations of source against destination and the common ternary ones), a mask or transparent colour, and a control register with direction, go, busy and interrupt on done. *(D, FPGA plan section 12)*

3.1.4 Each of the three is a device in a slot: it is enumerated by the bus, mapped into its own 4 KB page, and found by its identification register like any other (chapters 2 and 5). *(D)*

Commentary. **Precedent.** The Commodore 64's VIC-II and SID were chips whose registers a program poked directly, with a screen in memory that the video chip read by direct memory access; Rob Pike's Blit had a framebuffer in the address space plus a bit-block-transfer engine, as did the Amiga and every 1980s windowing system. **Trade-offs.** The blitter covers scrolling a terminal, moving a window, drawing a glyph, filling a rectangle and exclusive-oring a cursor, which is the working set of a text-and-windows machine; it does not do three-dimensional graphics. `MCPY` is no substitute: it is one-dimensional and occupies the processor, where a blit has a stride and a raster operation and runs concurrently. 48-bit pixels (presumably 16 bits per channel) exceed what any display connector carries today, so the display device converts to the connector's depth on the way out; 48-bit audio samples likewise exceed what a converter takes, so the sound device keeps the precision for mixing and filtering and reduces it at the output. **Improvements.** The C64's video and sound registers were scattered and irregular; the pdp64 versions use the uniform register face, so one driver skeleton serves them. **Benefits.** The blitter is about 1,500 lookup tables and a line buffer, nearly all address arithmetic; the whole terminal is a handful of small devices. **Modern comparables.** Every embedded display controller has a 2-D blitter and a scan-out engine; sound chips are now audio codecs behind an I2S interface. **Impact.** *Kernel:* a framebuffer, a sound and a blit driver are each a mapped page and a few register writes; no cache maintenance is needed because the framebuffer is uncached. *Compiler:* none. *Hardware:* a scan-out engine, a line buffer, a small address-arithmetic unit for the blitter and a mixer for the sound device, each independent of the processor and of the others.

3.2 The stub: the C64 documentation

3.2.1 The register-level definitions of `VIC-64X`, `SID-64X` and `BLT64` (register names, offsets, bit fields, the 48-bit pixel and sample formats, the sprite, raster-interrupt and voice models inherited from the C64) are not in the documents found so far and are to be supplied by the author. *(S)*

3.2.2 Until they are, the chapter states only what the author has stated and what the FPGA plan fixes: the devices exist, they follow the eight-register convention in the terminal's slots, and the interrupt, FENCE and uncached-framebuffer rules of section 3.4 apply. *(S)*

Commentary. This section is a placeholder and is the place the C64-derived specification belongs; the author's earlier work on the C64 design is the source, and the phase-7 plan treats the video chip as the visible fun part that must wait until the platform exists. **Impact.** *Kernel:* none until defined. *Compiler:* none. *Hardware:* the block RAM budget (section 3.4) must be settled before the video device is built.

3.3 How the terminal drives HDMI and the other ports

3.3.1 The display connector is standard kit: HDMI (and DVI and DisplayPort) is a transmitter chip that takes pixels and sync signals in parallel and does the link protocol, the encoding and the electrical signalling itself; on the first board it is the DE10-Nano's ADV7513, set up once over an I2C-style control bus. *(D, the author's statement; the board is FPGA plan section 3)*

3.3.2 The video device's output side is therefore a pixel stream with horizontal and vertical sync, a clock and a data-enable, which is what every such transmitter accepts; the device's registers set the mode (width, height, blanking, pixel clock, the pixel format), the base address of the framebuffer and the display on and off. *(P)*

3.3.3 Other ports are the same shape: a different transmitter (VGA digital-to-analogue converter, DisplayPort, an LVDS panel) takes the same pixel stream, so the pdp64's side does not change, and each port is one more slot or one more output of the video device. *(P)*

3.3.4 The sound device's output is likewise a standard digital audio stream (I2S or the HDMI audio channel) into a converter or the transmitter, and the processor never sees the protocol. *(P)*

3.3.5 The ports that carry data, SATA for storage and USB for peripherals, follow the same pattern: the protocol is in the chip or controller block, and the processor's side is a slot with the common registers and a ring (chapter 5). *(D)*

Commentary. **Precedent.** The MiSTer framework on the DE10-Nano already does exactly this (the retro cores produce a pixel stream and the framework's scaler and the HDMI transmitter do the rest), which is why the plan buys HDMI there. **Trade-offs.** The control bus of the transmitter needs a small amount of one-time setup that a boot routine does by writing a few registers; the pixel clock must be generated to the mode, which on an FPGA is a phase-locked loop and in silicon is the same block. **Improvements.** Because the interface is a stream and a register set, replacing the transmitter or the connector changes nothing above the video device, as a SATA controller behind its ring can be replaced without changing the disk driver. **Benefits.** No part of the operating system carries an HDMI, SATA or USB protocol stack: each is one driver of the common shape. **Modern comparables.** Single-board ARM computers with a display controller and an HDMI transmitter; the video output of every SoC. **Impact.** *Kernel:* a display driver sets a mode and a base address and nothing else; hot-plug of a monitor is a bus event if the transmitter reports one. *Compiler:* none. *Hardware:* a pixel generator, a clock and an off-chip transmitter; no protocol logic in the pdp64's own gates.

3.4 The framebuffer, the memory type and the memory budget

3.4.1 The framebuffer is mapped with the uncached memory type, or the processor's writes sit in the data cache while the video device scans out stale memory; a FENCE separates writing a device's registers from setting its go bit. *(D, FPGA plan section 12.2)*

3.4.2 On the first board the block RAM (about 5.5 megabits on the DE10-Nano) cannot hold a modern frame (a 1920 by 1080 frame at 32 bits is 8.3 MB), so the framebuffer lives in main memory and the block RAM holds the scan-out line buffer of a few kilobytes. *(D, FPGA plan section 12.3)*

3.4.3 The block RAM budget (caches, translation buffer, the blitter's scratch and the sound voices' state) is settled before the video device is built. *(D, FPGA plan risk 4)*

3.4.4 If the address-space split of section 2.1.6 is adopted, the framebuffer is ordinary memory in the lower half and the devices' registers are in the upper half, so a page-table entry that maps the framebuffer as a device (or a register page as cached memory) is detectably wrong. *(P)*

Commentary. **Trade-offs.** An uncached framebuffer makes a single pixel write slow and a long run of writes no slower per byte, which suits the blitter (it writes the bus directly) and punishes pixel-by-pixel drawing from the processor, which is what a blitter is for. **Modern comparables.** Write-combining memory types on x86 and ARM serve the same purpose.

Impact. *Kernel:* the mapping rule is one memory-type value in the page-table entry. *Compiler:* none. *Hardware:* the scan-out reads main memory by bus mastering, which competes with the processor for bandwidth.

3.5 A standard graphics card

3.5.1 A PCI Express graphics card is not excluded by the architecture: a bridge could present PCI Express functions as slots on the bus, enumerated like any other device. *(P, the author asks; this replaces the earlier plan's flat refusal)*

3.5.2 The earlier plan's reasons against remain facts to weigh: PCI Express configuration space, base-address mapping and message-signalled interrupts, a driver stack for a programming model that for some vendors must be reverse-engineered, the card's firmware and the initialisation a PC's firmware performs, and no hard PCI Express block on the first board (the Cyclone V and the Zynq 7020 lack one; a Zynq UltraScale+ has it). *(D, as facts)*

3.5.3 On the other side PCI Express is a standard enumerated bus, which suits the bus-as-a-device design (each function a slot, as each USB device is), and OpenBSD carries graphics drivers for several vendors' cards ported from Linux *(not confirmed in the sources read). (P)*

Commentary. **Trade-offs.** The terminal of this chapter works on the first board and costs a few thousand lookup tables; a standard card needs a root complex, which is a large block when it is not already on the chip, and a graphics driver ported to the pdp64, which is a larger task than the architecture. **Modern comparables.** PowerPC and RISC-V workstations plug in PCI Express graphics cards with open drivers. **Impact.** *Kernel:* a PCI Express card would need a large graphics driver ported. *Compiler:* none. *Hardware:* a PCI Express root complex and bridge to the bus.

Chapter 4 — Boot, the BIOS, the disk label and the GPT

The machine starts the way the PDP-11 did: a small ROM in the controller chip loads one block from the boot device and jumps to it, and every later stage is an ordinary program, open source, compiled by the project's own compiler. This chapter says what starts the machine, what takes the place of a BIOS, how the disk is laid out, how other cores and devices are started, and why UEFI is not used. The full review of how other systems boot is `../boot.md`. Tags: *(D)* decided; *(P)* proposed and not yet settled.

4.1 What takes the place of a BIOS

4.1.1 The pdp64's "BIOS" is the controller chip's boot ROM and its monitor. It brings up the machine, loads the first block, and offers a console monitor when nothing boots. It has no resident services and nothing is called after boot. *(P)*

4.1.2 The boot software is open source and part of the specification: the ROM and monitor, the first-block program, the second-stage loader and the boot tools. *(D, errata 47)*

4.1.3 Everything in the project, the boot software included, is under the BSD 2-clause licence. *(D, errata 48)*

Commentary. **Precedent.** A BIOS did four jobs: hardware bring-up and self-test, the first code (read the boot sector and jump), a runtime library of device services called by software interrupt (the "I/O" in its name), and configuration. UEFI replaced the second and third with a loader file and a set of boot and runtime services, and the other architectures have the same roles under other names: Open Firmware on SPARC and PowerPC, U-Boot with trusted firmware on ARM, OpenSBI on RISC-V, the SRM console on Alpha, the console processor on the VAX, and the boot ROMs and `ODT` monitor on the PDP-11. **Trade-offs.** The pdp64 needs only the first, second and fourth jobs in a small form. The third disappears because the devices have one uniform register layout (chapter 5) that the loader and the kernel share, so the loader needs no firmware to read a disk. The cost is that the second-stage loader carries its own drivers, which the author has settled is small because a driver drives a small register set (the common eight and a few of its own). **Improvements.** There is no firmware API to specify, no compatibility layer, no resident management code and no closed blob at the bottom of the trust chain. **Benefits.** The failure cases reduce to one: the first block did not load, and the monitor is the recovery. **Modern comparables.** x86 firmware keeps resident system-management code that the operating system cannot see; arm64 and RISC-V servers need trusted firmware and, usually, UEFI beside it. **Impact.** *Kernel:* There are no firmware services to call or to bug-fix. *Compiler:* The ROM, loader and tools are built by the project's own compiler. *Hardware:* A boot ROM of a few hundred bytes to a few kilobytes, with a driver per boot device.

4.2 The reset state and the first block

4.2.1 At reset the controller brings up memory, holds every core in reset, and looks for a boot device in a fixed order: disk or flash, then SD card, then the serial line. *(P)*

4.2.2 It reads block 0 of the first device that answers into low memory, checks the `0x55AA` signature in the last two bytes, and starts core 0 at that address. It does not parse a partition table. *(P)*

4.2.3 Core 0 starts in kernel mode at priority 7 with translation off (every address physical), `VBR` zero and the user-memory guard clear; `r0` holds the boot device number, `r1` the address of the boot-information block, `r14` the top of the first 64 KB of memory, and the other registers are zero. *(P)*

4.2.4 If no boot device answers, or a key is held at power-up, the monitor runs on the serial console. *(P)*

Commentary. **Precedent.** The PDP-11 read block 0 of the boot device into low memory and jumped to it, from a boot ROM or a few words keyed in on the front panel; the VAX console processor did the same with the boot block and passed the boot device in registers; the mainframe's IPL record and the Alpha console's boot block are the same idea. In V7 the second stage was about 250 lines of PDP-11 assembler that read a pathname and loaded the kernel. **Trade-offs.** A boot ROM with a driver per boot device is more code in the controller than a "jump to a fixed flash address" scheme, and much

less than firmware; the first block must fit in one sector. **Improvements.** The reset state is specified in full, a thing the earlier machines left to the console. **Benefits.** The first stage is the same on every board. **Modern comparables.** A PC's BIOS reads one 512-byte sector; ARM boards load a first-stage loader from the boot medium; UEFI runs a file from a FAT partition. **Impact.** *Kernel:* The reset state is fully specified, so the first kernel code makes no guesses. *Compiler:* The first block is 446 bytes of assembler. *Hardware:* The ROM reads one block and starts one core.

4.3 The disk: a GPT around the BSD label

4.3.1 The disk carries a GUID partition table with a protective MBR; the operating system's own partition holds the BSD disk label, which names the root, swap and other partitions. *(D, errata 50)*

4.3.2 Block 0 is the protective MBR, so the first stage lives in its 446 boot-code bytes, carrying a short block list (start block, count, load address) to the second stage; neither the ROM nor the first stage parses the GPT. *(P, decision B8)*

4.3.3 Inside the BSD partition, the label is at sector 1, offset 0; the boot area is 8192 bytes at the start of the partition; the file system begins after it, its superblock at offset 8192. *(D, errata 51)*

Sector (512 bytes) of the BSD partition	Contents
0	the partition's own boot block
1	the disk label, at offset 0
2 to 15	the rest of the 8 KB boot area
16 on	the file system, superblock at offset 8192

4.3.4 Sector counts are 64-bit throughout; the label has 16 partitions and its sizes carry high fields beyond 32 bits. *(D)*

Commentary. **Precedent.** V7 had no partition table on the disk: each driver compiled in an array of eight sizes and starting cylinders (`hp_sizes[8]` in `hp.c`), and changing the layout meant recompiling the kernel. The disk label arrived with 4.3BSD-Tahoe in 1988, in sector 0 or 1 "possibly offset from the beginning to leave room for a bootstrap", with the magic number `0x82564557`; OpenBSD's label is its descendant, with the same magic number, 16 partitions and 64-bit sizes. In the OpenBSD tree seven architecture headers put the label at sector 1 offset 0 (amd64, i386, arm, arm64, riscv64, hppa, powerpc64); Alpha and SPARC64 put it at sector 0. The PC used the MBR, a four-partition table in sector 0 with 446 bytes of boot code, and UEFI requires the GPT, whose header is in the second block, with a backup at the end of the disk and a protective MBR in block 0. **Trade-offs.** The label alone would have been the smallest design and would have been unreadable to other systems' tools; the GPT wrapper costs a GPT reader in the second stage and the kernel (OpenBSD's kernel already has one) and puts the protective MBR in block 0, and gives a disk that other operating systems and partitioning tools recognise. Following the majority on the offsets means the port adds one header of a dozen lines. **Improvements.** The first stage does not depend on a GPT that may be damaged, so a corrupted table cannot stop the machine reaching the second stage, which can repair it. **Benefits.** The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. **Modern comparables.** BSDs on UEFI machines use a GPT with the label inside an OpenBSD-type partition and a FAT partition for the loader; Linux has no scheme of its own. **Impact.** *Kernel:* The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. *Compiler:* None. *Hardware:* None.

4.4 The second stage and the hand-off

4.4.1 The second stage is a program in C that understands the root file system, reads the GPT and the label, loads `/bsd` (an ELF file) and jumps to it with `r0` and `r1` as before and a command line in the boot-information block. *(P)*

4.4.2 The boot-information block names the memory ranges, the core count, the address of the bus controller and the slots of the boot device and the console; the kernel enumerates every other device by asking the bus. *(P)*

4.4.3 The block's layout is defined once, in a grammar from which the headers are generated, and carries a checksum and a version. *(P)*

Commentary. **Precedent.** The VAX passed a restart parameter block, the Alpha a hardware restart parameter block, both fixed-layout records of the machine; the PDP-11 configured the kernel for the machine at build time and probed device addresses; x86 uses ACPI tables, and arm64 and RISC-V use a device tree. **Trade-offs.** A fixed-layout block with a checksum has two failure cases (bad checksum, unknown version), against the parser a device tree or ACPI requires; a device tree has the advantage that OpenBSD's arm64 and riscv64 ports already parse one and attach drivers from it, which could save kernel code if the port reuses many of their drivers. **Improvements.** Because the bus describes its own devices, the block shrinks to memory, cores and the bus controller's address. **Benefits.** The hand-off is a struct; the generated header cannot

disagree with the loader. **Modern comparables.** The device tree and ACPI are the two standard answers. **Impact.** *Kernel:* The hand-off is a structure the kernel reads. *Compiler:* The header is generated from one grammar, so it cannot drift from the loader. *Hardware:* None.

4.5 Starting the other cores and the devices

4.5.1 The controller holds every core except core 0 in reset; a write to a core's start register in the controller gives it an entry address and an argument and releases it into kernel mode at priority 7, with its core number in `r0` and the argument in `r1`. *(P, decision B2)*

4.5.2 A stuck core is reset by a write to a core-reset register. *(P)*

4.5.3 A device is initialised by writing its `INIT` bit and waiting for `READY`; a device that wants firmware is given it through a descriptor operation once the root file system is available, and a device the boot depends on needs no firmware. *(P, decision B4)*

Commentary. **Precedent.** On x86 the boot core sends INIT and START-UP interrupts through the APIC and each other core wakes in 16-bit real mode, then has to be walked through protected mode into 64-bit mode by a trampoline: the Linux `smpboot.c` is 1,528 lines and the real-mode code another 1,258. arm64 uses a firmware call (PSCI) in 124 lines or a spin table in 127; RISC-V asks the firmware to start a hart in 108 lines or spins in 58. The VAX's console processor started its other processors. **Trade-offs.** The pdp64 has no mode ladder, so a core can start in its normal mode at any address and the whole protocol is one register write, at the cost of a small register block in the controller. **Improvements.** There is no trampoline and no firmware call. **Benefits.** Fewer lines than any system measured and one recovery path, a core reset. **Modern comparables.** PSCI, the SBI hart-state calls, and x86's INIT and START-UP sequence. **Impact.** *Kernel:* Starting a core is a register write and a device start is `INIT`. *Compiler:* None. *Hardware:* A start register and a reset hold per core; an `INIT` bit per device.

4.6 Why UEFI is not used

4.6.1 The pdp64 does not use UEFI: its boot is the block-0 route of this chapter, its hand-off is the boot-information block, and its devices describe themselves. *(P, the author's inclination; a decision to confirm)*

4.6.2 The hand-off and the second stage are kept simple enough that a UEFI shim could be written by someone who wanted one. *(P)*

Commentary. **Precedent.** UEFI is a firmware specification of boot services, runtime services, a memory map, the GPT, the PE/COFF executable format and a driver model, implemented usually by the open-source EDK II or by a vendor. Its aim is that one installer boots on any machine that offers it. OpenBSD's arm64 and riscv64 ports use UEFI and a device tree, and its own loaders for them are 7,195 and 4,513 lines; for amd64 the loaders total 15,440 lines, including a BIOS boot block of 862 lines and a second stage of 189 beside the UEFI loader of 4,593. **Trade-offs.** UEFI would let existing generic loaders run unchanged, offers a standard memory map and a secure-boot chain, and costs a great deal of firmware (an EDK II port to a new architecture, in a new executable format), adds failure modes (variable storage, drivers, boot-order state), enlarges the attack surface, and still needs a device tree or ACPI to describe the devices. **Improvements.** The pdp64 has one architecture and one platform description, which is the case UEFI was not built for. **Benefits.** Smaller (principle 1), fewer failures (principle 2), fewer gates and less flash (principle 3). **Modern comparables.** x86 servers, arm64 servers and RISC-V servers all use UEFI; OpenBSD itself still ships a BIOS-style loader beside the UEFI one on amd64. **Impact.** *Kernel:* No firmware interface, no UEFI services, no memory-map translation. *Compiler:* No PE/COFF format in the toolchain. *Hardware:* A smaller flash and no firmware stack.

4.7 Recovery

4.7.1 The controller's ROM includes a monitor reached on the serial console when nothing boots: examine and deposit memory and registers, load a program over the serial line, start at an address or from a named device, reset a core, reset the machine. *(P)*

4.7.2 A hardware watchdog restarts the machine if the kernel hangs. *(P, decision P7)*

Commentary. **Precedent.** The PDP-11's console `ODT`, a monitor in ROM for examining and depositing memory and starting at an address, and the VAX console processor were the way in when nothing booted. **Trade-offs.** A monitor is a few hundred bytes of ROM and needs a working serial port and nothing else. **Benefits.** The same monitor brings up a new board before

any boot device works, and it is the simplest recovery path of all: it needs the least code to write. **Modern comparables.** U-Boot's console and OpenSBI's debug console fill this role; PC firmware has a setup screen. **Impact.** *Kernel:* A monitor is the way in when nothing boots. *Compiler:* None. *Hardware:* A serial port and a few hundred bytes of ROM.

4.8 Still open

The boot chapter's open items, in `../boot.md`: the address the first block is loaded to, the partition type of the boot partition (and whether a dedicated boot partition is needed at all, since the first stage's block list can point at the second-stage file in the root file system), the device-tree option, the details of the GPT (to be checked against the UEFI specification), and the PDP-11 power-up behaviour and the VAX and Alpha boot protocols (to be checked against their manuals).

Chapter 5 — The device driver model

A driver on the pdp64 is a short ordinary program: it maps a device's page, writes its registers (at least eight), hands the device descriptors in memory, and finds them marked done, and because the device is always the same shape the same driver skeleton serves a disk, a network card and a serial line. This chapter is the driver writer's view of the memory-mapped registers and of DMA: what a driver sees, the uniform eight-register convention and its descriptor, the life of a driver from probe to detach, how data moves and what the driver need not do, the default devices, how errors and departures are handled, interrupts, and how this maps onto OpenBSD's driver interfaces. The bus the devices sit on is chapter 2. Tags: *(D)* decided; *(P)* proposed and not yet settled; the drivers are expected to be trivially small because a device is eight registers, which the author has settled.

5.1 What a driver sees

5.1.1 A device is at least eight 64-bit registers at the start of its own 4 KB page (a larger device takes more pages, section 5.2.1), and the page is mapped with the device memory type, strongly ordered. *(P for the page per device; D for the memory types)*

5.1.2 A driver reaches a register with an ordinary load or store; there are no input or output instructions and no port space. *(D)*

5.1.3 The registers are 64 bits wide, naturally aligned and accessed whole; the machine is little-endian throughout, so a driver has no byte-order or byte-lane cases. *(D)*

5.1.4 Accesses to a device page happen in program order; one `FENCE` separates the last write to memory the device will read from the write of the doorbell. *(D)*

Commentary. **Precedent.** A V7 driver is a structure laid over a fixed address: `rk.c` declares `struct device { int rkds; int rker; int rkcs; int rkwc; caddr_t rkba; int rkda; }` and a pointer to it at `0177400`, and reads and writes the fields. **Trade-offs.** One page per device wastes address space and no memory, and lets the kernel give one device to one protection domain. **Improvements.** The register's address is a virtual address the kernel maps, so the driver is position-independent and the same code serves any slot. **Benefits.** There is no accessor layer to learn and nothing to get wrong with endianness or alignment. **Modern comparables.** Linux and BSD wrap register access in an accessor (`readl`, `bus_space_read_8`) to hide port and memory differences and byte order; on the pdp64 those wrappers reduce to a load or a store. **Impact.** *Kernel:* No accessor layer: a register access is a load or a store. *Compiler:* Register access is a 64-bit `volatile` load or store through a pointer to a structure. *Hardware:* A device is decoded by page address; its eight registers are an eight-way decode within the page.

5.2 The device convention: eight registers (at least) and DMA

5.2.1 Each device is at least eight memory-mapped registers and moves its data by DMA. *(D, errata 29)* Eight is the minimum every device has, not a limit: a device that needs more (a controller with many queues, a framebuffer, a display's mode and palette registers) maps a larger window in its slot, the first eight registers keep the common layout, and the rest are the device's own. *(D, the author, 2026-10-22, errata 61)*

5.2.1a The storage interfaces are all this one pattern: SCSI, ATA, SATA and NVMe each present a command queue in host memory, a doorbell and a completion in memory, which is the ring and the descriptor of 5.2.3 with a different descriptor body; an NVMe controller's submission and completion queues are the closest existing form of it. A disk driver for any of them is the same skeleton with a different command encoding. *(D, the author, 2026-10-22, errata 61)*

5.2.2 The first three registers are the same in every device: `ID`, `CONTROL` and `STATUS`. *(P)*

Offset	Register	Meaning
`0x00`	`ID`	type, variant, version, feature bits (read-only)
`0x08`	`CONTROL`	enable, `INIT` (initialise yourself), interrupt enable
`0x10`	`STATUS`	`READY`, `ERROR`, `BUSY`, error code
`0x18`	`RING_BASE`	physical address of the descriptor ring

Offset	Register	Meaning
0x20	RING_SIZE	the ring's size in entries (a power of two)
0x28	DOORBELL	write the producer index to start work
0x30	IRQ	cause bits (complete, error, attention); write 1 to clear
0x38	DEVICE	device-specific (for the polled devices, the data register)

5.2.3 A transfer is a 32-byte descriptor in a ring in host memory: four 64-bit words giving an operation and flags, a status field the device writes, a length, a buffer address, an argument (a block number, for a disk) and a result the device writes. *(P)*

```
word 0:  OP(8) FLAGS(8) STATUS(16) LENGTH(32)     FLAGS: OWN, STARTED, INTERRUPT, MORE
word 1:  ADDRESS (physical)
word 2:  ARGUMENT
word 3:  RESULT (written by the device)
```

5.2.4 The host sets OWN when it hands a descriptor to the device; the device clears it when it is finished, after writing the data, the status and the result. Progress is therefore in memory, not in the device. *(P)*

5.2.5 INIT makes a device initialise itself without touching the ring; a device may take a firmware image through an ordinary descriptor operation; boot-critical devices work without any image. *(P)*

5.2.6 Operations common to all devices: no-op, identify (the device writes an identification block into a buffer) and load-firmware; each device adds its own from 0x10. *(P)*

5.2.7 A device that carries traffic in two directions (a network card, a serial line) is two slots, one ring each; a console, a timer and an entropy source are polled devices with their data in the last register. *(P)*

Commentary. **Precedent.** The V7 RK11 disk is six registers (drive status, error, control and status, word count, bus address, disk address) with DMA by a word count and a bus address; the 4.3BSD DEUNA Ethernet is four registers plus transmit and receive rings in memory, each ring entry holding a segment length, an address, flags and errors, with an OWN bit that is zero when the descriptor belongs to the driver. The proposal is the second design, kept uniform. **Trade-offs.** A single ring per device makes duplex devices two slots, which is simple and slightly unusual, in exchange for in-order completion and no out-of-order bookkeeping; the ownership bit costs one status write per descriptor. **Improvements.** Because progress lives in the descriptor and not in a device register, a driver can recover from a departed or reset device with one routine: scan the ring for descriptors that are not done, then read, save, resend or fail them. **Benefits.** A driver is a skeleton shared by all devices, and the hardware is one reusable ring engine that fetches a descriptor, hands the operation to the device's own logic and writes the status back. **Modern comparables.** virtio rings, NVMe's submission and completion queues, the descriptor-done bit of Intel Ethernet controllers and the command lists of SATA controllers are all rings with a done mark; the 16550 serial chip is the classic eight-register device. **Impact.** *Kernel:* One driver skeleton serves every device, and the layout is the same for a disk, a network card and a serial line. *Compiler:* The descriptor is a 32-byte aligned C structure, and register offsets are generated constants. *Hardware:* One reusable ring engine fetches a descriptor, hands the operation to the device's logic and writes the status back.

5.3 The life of a driver

5.3.1 *Probe.* The kernel reads the bus controller's slots (section 2.6), reads each present device's ID register, and matches the type and version against the drivers it has. *(P)*

5.3.2 *Attach.* The driver maps the slot's page, writes INIT, waits for READY, and reads the device's identification block by submitting an identify descriptor. *(P)*

5.3.3 *Run.* The driver puts the ring in uncached memory, writes RING_BASE and RING_SIZE, fills descriptors with OWN set, executes FENCE, and writes DOORBELL; a descriptor is complete when the device has cleared OWN, which the driver sees by polling or by the interrupt. *(P)*

5.3.4 *Fail.* A failed descriptor completes with an error code in its status; a device that stops answering is reset with INIT and its ring reprogrammed from the first descriptor that is not done. *(P)*

5.3.5 *Detach.* A departure event, or a bus fault from an absent slot, tells the driver to stop; it reads, saves, resends or fails the descriptors still queued. *(P)*

Commentary. **Precedent.** A V7 disk driver is a strategy routine that queues a buffer, a start routine that writes the registers, and an interrupt routine that completes the buffer; 4.3BSD's Unibus drivers add a probe routine that tests whether a device

answers at an address and an attach routine, which BSD's autoconfiguration calls in order. **Trade-offs.** The five steps are the same for every device, so the work that differs is the personality: what the operations mean. **Improvements.** Probe reads the bus and not a configuration file, and detach is a defined path because the bus reports departures. **Benefits.** A new device is a new set of operations in a driver that already has the skeleton. **Modern comparables.** OpenBSD's autoconfiguration calls a driver's `match`, `attach` and `detach` functions; Linux's driver model has `probe` and `remove`. **Impact.** *Kernel:* The five steps are the same for every driver, so only the operations differ. *Compiler:* Nothing special. *Hardware:* The `INIT` and `READY` handshake is a small state machine in each device.

5.4 DMA and the hand-off

5.4.1 Buffers and descriptors are in memory mapped uncached, and the driver does no cache clean or invalidate. *(D)*

5.4.2 A device addresses physical memory directly, with 40-bit addresses, and the driver gives it physical addresses; until an address-translation unit for devices exists, every device must be able to address all of memory. *(P, decision R6)*

5.4.3 A buffer that crosses a page is described by several descriptors chained with the `MORE` flag; the driver does not make buffers contiguous. *(P)*

5.4.4 The device writes the data, then the status and result, then clears `OWN`; the driver reads `OWN` and then the data. *(P)*

Commentary. **Precedent.** The Unibus carried 18 address bits, so the VAX and the larger PDP-11s had map registers (`ubalonce` in 4.3BSD) translating device addresses to memory, and the VAX's adapter had data paths a driver had to purge before reading; the 11/70's cache watched DMA writes so that the data appeared coherent. **Trade-offs.** Uncached buffers make the processor's reads of received data slow, in return for a driver that cannot forget a cache operation; a non-coherent device that came later would call for cache-maintenance instructions. No address-translation unit means no protection from a faulty or malicious device. **Benefits.** The sequence build, `FENCE`, doorbell, complete is the whole of the hand-off. **Modern comparables.** ARM and MIPS systems with non-coherent DMA need a DMA-mapping layer that places cache maintenance; x86 DMA is coherent; `swiotlb` in Linux copies for devices that cannot address all memory. **Impact.** *Kernel:* No cache clean or invalidate in any driver, so none can be forgotten. *Compiler:* None. *Hardware:* DMA goes around the processor's data cache, so the cache needs no snoop port for devices.

5.5 The default devices

5.5.1 The **disk** has the operations read, write and flush at 512-byte sectors, with the block number in the argument word; its identification block gives the capacity in sectors and the sector size; the boot ROM reads block 0 with one descriptor. *(P)*

5.5.2 The **network card** is a transmit slot and a receive slot: the transmit ring carries frames to send, the receive ring carries buffers the host posts and the device fills, completing each with the frame length; link changes raise an attention interrupt. *(P)*

5.5.3 USB is presented by a controller chip that does the protocol and the enumeration and shows each attached device as a slot on the bus with the personality of its class: a mass-storage device looks like a disk, a keyboard or mouse like a receive-only device, a serial adapter like a console. *(P, decision for the author)*

5.5.4 The **console** is a polled serial device with a transmit-ready and a receive-ready bit and one data register; the **timer** is a 64-bit free-running counter, a compare register and a control register; the **entropy source** returns 64 random bits from its data register and fills buffers by descriptor. *(console and timer D, as built in the emulator; entropy P)*

5.5.5 A device raises one interrupt line at the level its slot asks for, with the vector numbered from the slot. *(P)*

Commentary. **Precedent.** The PDP-11 had a disk with eight registers, a serial line with four, and network controllers with rings; this is the same set with a common layout. **Trade-offs.** USB in eight registers is the hard case: a conventional host controller exposes command, event and transfer rings and many registers. Presenting each device as a bus slot moves the protocol into the controller chip, which costs gates or a small embedded processor in the controller and takes the USB stack out of the operating system; the alternative is a conventional ring-based host controller, which an existing operating-system USB stack can drive at the price of that large stack. The decision is the author's. **Improvements.** Treating every USB port as a hot-pluggable slot reuses the bus's own enumeration and hot-plug machinery instead of a second one. **Benefits.** A disk driver written for the bus disk works for a USB disk. **Modern comparables.** `virtio` block and network devices have the same shape; xHCI is the conventional USB controller; a USB-to-Ethernet adapter appears to the host as a network class device. **Impact.** *Kernel:* A disk driver written for the bus disk also drives a USB disk; the network card is two slots. *Compiler:* None.

Hardware: Each personality is a small block of logic behind the common ring engine; the USB protocol, if done in the controller, is where the gates go.

5.6 Errors, departure and recovery

5.6.1 A descriptor's status carries a code: success; bad operation; bad address (a DMA access faulted); bad length; media error; not ready; aborted (the device was reset); bad image. *(P)*

5.6.2 A device that is reset with `INIT` leaves its ring in memory untouched and clears its ring registers; the driver reprograms them and starts at the first descriptor that is not done. *(P)*

5.6.3 A device that leaves the bus leaves its queued work in memory; the driver finds it by scanning the ring from its last completed descriptor. *(P, the author's suggestion)*

5.6.4 Operations are repeatable where possible (reads, writes of the same data, sends, buffer posts, identify), so that resending an unfinished descriptor is safe; an operation that must not run twice carries a `STARTED` flag so the driver can tell "never started" from "started, outcome unknown". *(P)*

Commentary. **Precedent.** 4.3BSD's `ubareset()` resets the Unibus adapter and then calls each driver's reset routine, so each driver re-issues what was pending: resending as a design, driver by driver. **Trade-offs.** The cost is a status write per descriptor and the ordering rule between data and the done mark. **Improvements.** One recovery routine serves a device that fails and is reset, a device replaced by another, and, with a saved ring, a hand-over between cores. **Benefits.** This is the recoverability principle in the driver: fewer failure outcomes (the precise bus fault, not all ones on a removed device) and one path to recover from any of them. **Modern comparables.** NVMe and virtio drivers requeue outstanding requests after a reset; PCI returns all ones for a read of a removed device. **Impact.** *Kernel:* One recovery routine handles a failed, reset, replaced or departed device. *Compiler:* None. *Hardware:* The device writes data before status before clearing the ownership bit, and the bus keeps one presence bit per slot.

5.7 Interrupts from devices

5.7.1 A device raises one interrupt line, at the priority level its slot asks for, and its cause is in the `IRQ` register, where each bit is cleared by writing a 1. *(P)*

5.7.2 The vector of a device interrupt is its slot number added to 64. *(P)*

5.7.3 A polled device (the console, the timer, the entropy source) may be used with its interrupts masked. *(D)*

Commentary. **Precedent.** A PDP-11 device asserted one of four bus-request lines and supplied its own vector, set by switches on the card. **Trade-offs.** A vector derived from the slot needs no per-device switch and no assignment, and limits the device interrupts to 192 slots. **Benefits.** A driver's interrupt routine reads one register to learn why. **Modern comparables.** The causes and the write-one-to-clear convention are those of most memory-mapped controllers; message-signalled interrupts, which need no line, are added to the proposal later (decision R1). **Impact.** *Kernel:* An interrupt routine reads one register to learn why and clears it by writing the same bits. *Compiler:* None. *Hardware:* One line per device into an eight-level priority encoder, with the vector derived from the slot number.

5.8 Drivers in user space, and in OpenBSD

5.8.1 Because a device is one page, a driver can run in user space with only that page and its buffers mapped. *(P)*

5.8.2 Until devices have an address-translation unit, a user-space driver can instruct its device to write anywhere in memory, so it is as trusted as the device. *(P, decision R6)*

5.8.3 In OpenBSD the register accessors become loads and stores, the DMA mapping interface returns physical addresses with no cache-synchronisation work to do, and the autoconfiguration framework attaches a driver to each slot the bus reports. *(P, to be checked against the OpenBSD sources)*

Commentary. **Precedent.** Microkernels and exokernels run drivers in user space where the hardware lets one device be mapped to one process; the PDP-11's Unibus gave every device a fixed address and no protection. **Trade-offs.** Without a translation unit the mapping protects against stray register writes but not against DMA. **Benefits.** A driver can be tested, restarted and replaced without taking the kernel with it. **Modern comparables.** Linux's `VFIO` and `UIO` and DPDK run drivers in user space, relying on an I/O translation unit for protection. **Impact.** *Kernel:* The accessors reduce to loads and stores, the DMA interface to physical addresses, and attach to the bus's slots. *Compiler:* None. *Hardware:* One device per page lets the translation unit protect a device, which a device-sharing page layout would not.

5.9 Still open

The register layout and descriptor details (decision R5), whether USB is a per-slot controller or a conventional ring controller, whether the emulator uses `virtio` devices first to reach an OpenBSD login sooner, DMA addressing and protection (R6), errors raised by devices (R7), and the details of the interrupt vector and cause (R1). The proposal is in the working documents (`../decisions-pending.md`, `../pdp64-q64-bus.md`) and appendix B.

Chapter 6 — The kernel: modes, exceptions and memory management

An operating system on the pdp64 sees three things: a status word with a mode and a priority, an exception mechanism that writes only registers, and a three-level page table with software-managed accessed and dirty bits. This chapter is what a kernel writer needs: the modes and the privileged instructions, the exception and interrupt entry, the vector table, the system registers, translation and the page-table entry, the user-memory guard, system calls, and how the machine starts. It is the design called System v1, accepted on 2026-10-22 and implemented in the simulator and the QEMU target; the options and the comparisons with other architectures are in `../pdp64-system-architecture.md` (section 15) and `../pdp64-system-design.md`. Tags: *(D)* decided; *(P)* proposed.

6.1 Modes and the status word

6.1.1 There are three modes, with the PDP-11's encoding: kernel 0, supervisor 1, user 3. The supervisor mode exists in the status word and in the banked registers and nothing uses it yet. *(D, errata 36)*

6.1.2 The status word holds the flags in bits 0 to 3, the trace bit in bit 4, the interrupt priority in bits 5 to 7, the current mode in bits 8 and 9, the previous mode in bits 10 and 11, and the user-access guard in bit 12; the other bits are zero. *(D)*

6.1.3 `MTPS` in user mode writes only the flags; in kernel mode it writes every writable bit. `SPL #n` is kernel-only. *(D)*

6.1.4 The stack pointer `R14` and the direct-page register `DP` each have one copy per mode; the mode selects which one an instruction names. *(D)*

6.1.5 These instructions abort with the privileged-instruction vector outside kernel mode: `RTI`, `RTT`, `HALT`, `SPL`, `MTASID`, `MFASID`, `TLBI`, `TLBIA`, `TLBSYNC`, `MTDMW`, `MFSR`, `MTSR`, `PROBER`, `PROBEW`, `WFE`, `ISYNC`, `STUA`, `CLUA`, `PUSHX`, `POPX`. *(D)*

6.1.6 These are not privileged: `MTDP` and `MFDP` (in user mode, the thread pointer), `ICINV` and `ICINVA` (a JIT needs them), `FENCE`, `SEV`, `HINT`, `TRAP`, `BPT`, `NOP`, and `MTPS` and `MFPS` with the restriction above. *(D)*

Commentary. **Precedent.** The PDP-11 had kernel, supervisor and user modes, with a status word whose priority and mode fields the pdp64 keeps, and a stack pointer per mode so that a trap arriving in user mode finds a kernel stack without searching for one. **Trade-offs.** Three modes cost a third copy of two registers and no logic the kernel uses; supervisor mode has no user yet, and is kept because adding a mode later would change the status word and every saved copy of it. **Benefits.** Entry from user mode needs no stack switch code; a threaded program's pointer and a kernel's per-CPU pointer are the same register in different modes. **Modern comparables.** x86 has four rings of which two are used; AArch64 has exception levels 0 to 3; RISC-V has user, supervisor and machine modes. **Use.** `MFSR` and `MTSR` are the whole of system control; `PROBER` and `PROBEW` replaced the `nofault` idiom of V6 and V7. **Impact.** *Kernel:* Banked stack pointers mean a trap from user mode needs no stack switch, and the direct-page copies give a per-CPU pointer in the kernel and a thread pointer in user mode. *Compiler:* User-mode thread-local storage is one `MOVQ d(DP), Rd`. *Hardware:* Three copies of two registers, selected by the mode, and a mode comparison on privileged decode.

6.2 Exceptions and interrupts

6.2.1 Entry to an exception or an interrupt writes only registers: `EPC` receives the address of the faulting instruction (for an abort or a progress exception) or of the next instruction (for a trap or an interrupt), `EPSW` the old status word, `EVA` the faulting address (zero except for aborts), `ECAUSE` the vector number. *(D, errata 36)*

6.2.2 The new status word has the current mode kernel, the previous mode the old current mode, priority 7, and the trace bit and the flags cleared. *(D, errata 40)*

6.2.3 The new program counter is the 64-bit word at the physical address `VBR + 8 × vector`; a bus error on that fetch is a machine check that halts. *(D)*

6.2.4 The hardware pushes nothing. A handler that can be interrupted saves `EPC` and `EPSW` with `PUSHX`, lowers the priority with `SPL`, and before returning restores them with `POPX` and executes `RTI`. *(D, errata 37)*

6.2.5 `RTI` loads the program counter from `EPC` and the status word from `EPSW`; `RTT` does the same and sets the trace bit after the return. *(D)*

6.2.6 The vector table has 512 entries and `ECAUSE` equals the vector number. *(D)*

Vectors	Meaning
0	reset (not used by the QEMU machine)
1	machine check
2, 3, 4, 5	illegal instruction, privileged instruction, misaligned access, non-canonical address
6, 7, 8	instruction, load and store page fault
9, 10, 11	protection violation on instruction, load and store
12, 13	divide by zero, bus fault
14, 15	landing-pad violation, breakpoint
16 to 31	reserved
32 to 63	interrupts (in the virt machine, 32 timer, 33 console input, 34 console output)
64 to 255	reserved (the proposal assigns device interrupts here, vector 64 plus the slot)
256 + n	`TRAP #n` and `TRAPL #n`, n from 0 to 255

6.2.7 There are eight interrupt priority levels; the processor takes an interrupt between instructions when its level is greater than the priority in the status word; the timer is at level 6 and the console at level 4. *(D)*

Commentary. **Precedent.** The PDP-11 pushed the program counter and the status word on a stack and loaded new ones from a two-word vector, and its trap vectors conventionally held priority 7, so a PDP-11 handler began with interrupts masked and chose when to lower them; its interrupts took the priority of their device from the vector. The VAX pushed the program counter and the processor status longword and raised the priority to the interrupt's level for an interrupt, leaving it unchanged for an exception. **Trade-offs.** Entering in registers is short and cannot itself fault (no memory is written), and costs the handler an explicit save; it needs the priority raised on entry, because a higher-level interrupt could otherwise arrive before the handler had stored the pair and overwrite it, which is why the entry sets priority 7 for every exception and interrupt, a fixed choice where the PDP-11 let each vector choose. **Improvements.** `PUSHX` and `POPX` make the save and restore one instruction each; the aborts are precise, so a page-fault handler repairs the mapping and executes `RTI` to run the instruction again. **Benefits.** The kernel survey estimates a system-call entry at four or five instructions on paper, against 45 to 70 on RISC kernels; no entry code is needed to find a stack. **Modern comparables.** AArch64 and RISC-V latch the return address and status in registers; x86 pushes five words on the stack; every one of them masks interrupts on entry. **Impact.** *Kernel:* Entry is four or five instructions on paper; priority 7 on entry removes the code that would otherwise guard the saved state. *Compiler:* Handlers written in C need a short assembler stub to save and restore; no other code generation is affected. *Hardware:* Entry writes registers only, so it cannot fault, and the vector fetch is one 64-bit read.

6.3 System registers

6.3.1 The system registers are reached by `MFSR` and `MTSR` with an 8-bit register number. *(D, errata 32)*

Number	Register	Meaning
0	`PTBR`	physical address of the level-2 page table
1	`EPC`	exception program counter
2	`EVA`	exception virtual address
3	`ECAUSE`	exception cause (the vector number)
4, 5	`DMW0`, `DMW1`	direct-mapping windows (stored, not yet defined)
6, 7	`USP`, `SSP`	the user and supervisor stack pointers
8	`EPSW`	exception status word
9	`VBR`	vector base register
10	`MMUCTL`	memory-management control (bit 0 enables translation)
11	`ASID`	address-space identifier
12	`CPUID`	the core number (read-only)
13 and up		reserved (feature and identification registers are proposed here)

6.3.2 A write to `PTBR`, `ASID` or `MMUCTL`, or an `ICINV`, is guaranteed to take effect only after the next `ISYNC`. *(D, errata 39)*

Commentary. **Precedent.** The VAX had `MTPR` and `MFPR` with an internal-register number, the same design. The PDP-11 kept its control registers at fixed high addresses and reached them with ordinary moves. **Trade-offs.** A general pair of instructions with 243 free numbers costs two opcodes and no decoding as the registers are added; separate instructions per register would have spent opcode space with every register. **Benefits.** A new register is a number, not an instruction. **Modern comparables.** AArch64's `MRS` and `MSR`, RISC-V's `CSRR` and `CSRW`, PowerPC's `MFSPR` and `MTSPR`; the kernel survey found privileged-register access to be 48% of all the compiler-invisible instructions in four kernels. **Impact.** *Kernel:* A single pair of instructions reaches every system register, so adding a register adds a number and no instruction. *Compiler:* The compiler reaches them through an intrinsic with a constant register number. *Hardware:* A control-register file addressed by an 8-bit number, with a read-only decode for the identification registers.

6.4 Memory management

6.4.1 Virtual addresses are 39 bits, translated by three levels of nine bits over 4 KB pages: address bits 38 to 30 index level 2, 29 to 21 level 1 and 20 to 12 level 0; `PTBR` holds the physical address of the level-2 table. *(D)*

6.4.2 A page-table entry is 64 bits: `V` (valid) in bit 0, `R`, `W`, `X`, `U` and `G` (global) in bits 1 to 5, `A` (accessed) and `D` (dirty) in bits 6 and 7, the memory type `MT` in bits 8 and 9, and the physical page number in bits 12 to 39, in place; all other bits are zero, and a set reserved bit is a page fault. *(D)*

6.4.3 A non-leaf entry has `V` set and `R`, `W` and `X` clear and points to the next table; a leaf at level 2 maps 1 GB, at level 1 maps 2 MB and at level 0 maps 4 KB. *(D)*

6.4.4 A user-mode access needs `U`; kernel mode may read and write user pages but may not execute them; `W` without `R` is reserved. *(D)*

6.4.5 The accessed and dirty bits are managed by software: an access to a page with `A` clear, or a write to a page with `D` clear, raises the page fault of its kind and the kernel sets the bit; the hardware walker never writes a page table. *(D, errata 36)*

6.4.6 Translation is off at reset (`MMUCTL` is zero and every address is physical) and is enabled by bit 0 of `MMUCTL`; the `ASID` selects non-global entries. *(D, errata 44)*

6.4.7 `TLBI rN` invalidates the translation of the page containing the address in `rN` for the current address space and the global entries; `TLBIA` invalidates all; both broadcast to every core and `TLBSYNC` waits for the broadcast to finish. *(D)*

6.4.8 The memory type of a page is one of normal cached, normal uncached, device strongly ordered and device weakly ordered. *(D)*

Commentary. **Precedent.** The PDP-11 had eight segments per mode, each with a base and a length, and no page tables; the VAX had a linear page table per region with a hardware modify bit and no reference bit, so 4.3BSD emulated reference bits in software. **Trade-offs.** Software-managed bits cost one extra fault on the first touch of each page and one on the first write, and they make the walker small and make it safe against the chip bugs of hardware management that other architectures have had; a hardware-managed option, advertised in a feature register, is the planned next step. A fourth page-table level would give 48 bits at the price of a longer walk on every miss. **Improvements.** The entry has the physical address in place, so no shifting; a leaf at any level makes large pages free to use for the kernel's own mapping. **Benefits.** A walker that only reads is small; a kernel's page-fault handler is the same code for first touch, first write and demand paging, differing only in the bits it sets. **Modern comparables.** The geometry is RISC-V's Sv39; x86-64 and AArch64 use four levels and hardware-managed accessed and dirty bits; MIPS refills its translation buffer in software. **Impact.** *Kernel:* The handler sets accessed and dirty bits; leaves at any level let the kernel map itself with 1 GB pages; `TLBI` broadcasts so there is no shootdown code. *Compiler:* Position-independent code uses PC-relative addressing; the canonical 39-bit form means pointers need no masking. *Hardware:* The walker only reads; the physical page number is in place, so no shifting; three levels of nine bits keep the walk to three reads.

6.5 The user-memory guard and system calls

6.5.1 With the guard bit `UA` clear, a kernel-mode data access to a user page faults; `STUA` sets it and `CLUA` clears it; exception entry clears it and `EPSW` keeps the old value. *(D, errata 43)*

6.5.2 `TRAP #n` (n up to 15, two bytes) and `TRAPL #n` (n up to 255) enter the kernel through vector 256 + n; the system-call number and arguments travel in registers by the ABI. *(D)*

6.5.3 A copy routine opens the guard, copies, and closes it; every error path out of the copy must close it, and entry clearing it bounds the damage if one does not. *(D)*

Commentary. **Precedent.** The PDP-11 reached user memory with the explicit instructions MFPI and MTPI, since its kernel and user address spaces were separate, so accidental access was impossible; the VAX read user addresses directly after a PROBER check. **Trade-offs.** The guard is a mode bit, accepted because it lives in the status word, which is saved anyway; it makes a stray kernel pointer a clean fault at the cost of an open and a close around every user copy. **Benefits.** A kernel bug that dereferences a user pointer by mistake no longer succeeds silently. **Modern comparables.** x86 SMAP with STAC and CLAC, AArch64 PAN and RISC-V SUM. **Use.** The user copy routines (`copyin`, `copyout`, `copyinstr`). **Impact.** *Kernel:* Every user copy opens and closes the guard; every error path must close it, and entry clearing it bounds the damage of one that does not. *Compiler:* The copy routines are written in assembler or inline assembler; system-call stubs are one TRAP. *Hardware:* One status-word bit and one check on a kernel-mode data access to a user page.

6.6 Reset and the first kernel

6.6.1 At reset core 0 starts in kernel mode at priority 7 with translation off, the guard clear and VBR zero; the kernel sets VBR, builds page tables, writes PTBR, sets MMUCTL bit 0 and executes ISYNC. *(D for the QEMU machine; P for the architectural reset, chapter 4)*

6.6.2 The QEMU machine starts at the entry point of the ELF given with `-kernel`, with the kernel stack at the top of memory; the option `kernel-mode=on` turns off the system-call stand-in of user-mode programs so that TRAP enters the kernel. *(D)*

6.6.3 The first operating system is OpenBSD, with Unix/32V as the fallback if OpenBSD proves too hard. *(D, errata 41)*

Commentary. **Precedent.** The PDP-11 and VAX turned the memory manager on with a bit in a status register and a jump to a mapped address, each needing a transition in the boot code. **Trade-offs.** Translation-off at reset is the same arrangement, and the single bit is a machine-wide mode bit accepted for that reason (errata 44). **Benefits.** The boot code runs at physical addresses until the kernel is ready, with no identity-map trampoline of the kind x86 needs. **Modern comparables.** RISC-V's `satp` with a bare mode, AArch64's `SCTLR.M`. **Impact.** *Kernel:* Boot code runs at physical addresses until the kernel is ready, with no identity-map trampoline. *Compiler:* The first kernel code is position-independent and uses no virtual address until translation is on. *Hardware:* Translation-enable is one bit.

6.7 System instructions, exceptions and interrupts

6.7.1 There are three modes (kernel, supervisor, user); supervisor is defined and unused. System instructions executed outside kernel mode abort with the privileged-instruction vector. *(D)*

6.7.2 TRAP `#n` (0 to 15, two bytes) and TRAPL `#n` (vectors 0 to 255) enter the kernel; RTI returns, and RTT returns and sets the trace bit after. *(D)*

6.7.3 An exception or interrupt writes only registers: EPC receives the address of the faulting instruction (or the next one, for a trap or interrupt), EPSW the old status word, EVA the faulting address and ECAUSE the vector number; the new program counter is the 64-bit word at `VBR + 8 × vector`. *(D, errata 36)*

6.7.4 Entry sets the priority to 7 for every exception and every interrupt; the handler lowers it. *(D, errata 40)*

6.7.5 There are eight interrupt priority levels; SPL `#n` sets the priority without touching the flags. *(D)*

6.7.6 MFSR and MTSR read and write the system registers by number; TLBI, TLBIA and TLBSYNC invalidate translations (broadcast to other cores); MTASID and MFASID set the address-space tag; ICINV and ICINVA invalidate the instruction cache; PROBER and PROBEW test an address; MFCPUID identifies the processor. *(D)*

6.7.7 UA in the status word guards user memory: with it clear, a kernel-mode data access to a user page faults; STUA sets it and CLUA clears it; entry clears it. *(D, errata 43)*

Commentary. The modes and exceptions are sections 6.1 to 6.3 and the multiprocessor instructions are chapter 7. **Precedent.** The PDP-11 pushed the program counter and status word on a stack and loaded new ones from a two-word vector; its trap vectors conventionally held priority 7. **Trade-offs.** Entering in registers is short and cannot fault, and costs the handler an explicit save; raising the priority on entry closes the window in which an interrupt could overwrite the saved state. **Improvements.** PUSHX and POPX save the pair in one instruction, and the user-memory guard turns a stray kernel pointer into a clean fault. **Benefits.** The kernel survey estimates system-call entry at four or five instructions against 45 to 70 on RISC kernels (a paper estimate, not a measurement). **Modern comparables.** AArch64 and RISC-V save the return address and status in registers on exception entry; x86 pushes five words; x86's SMAP with STAC/CLAC, AArch64's PAN and RISC-V's SUM are the user-memory guards. **Use.** PROBER and PROBEW replace the `nofault` idiom of V6 and V7; ISYNC follows a write to the page-table base or the vector base. **Impact.** *Kernel:* This is the kernel's whole instruction vocabulary: MFSR, MTSR, TLBI, RTI, SPL, ISYNC,

the probes. *Compiler:* The compiler never emits them except through intrinsics. *Hardware:* The system group takes its sub-opcode from the register field, and the privilege check is one comparison on the current mode.

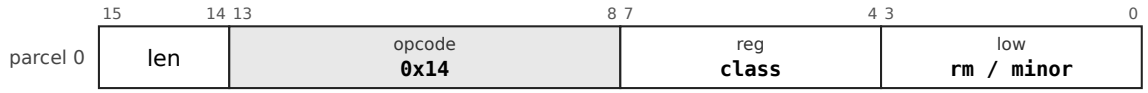
NOP: No operation and landing pad

Mnemonics: `NOP`, `LPAD`

Syntax

```
NOP
LPAD
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`NOP`	0	`0x0`
`LPAD`	0	`0x9`

Operation

```
nothing
```

Flags None written.

Exceptions None.

`LPAD` marks a legal target of an indirect jump or call; no instruction enforces it in ABI version 1.

Why it is here. `NOP` is common and has no story; `LPAD` is the landing pad. Landing pads, `LPAD`, mark the legal targets of indirect jumps and calls so that jump-oriented attacks can be stopped at the transfer, as Intel's indirect-branch tracking and ARM's branch target identification do and OpenBSD enforces. ABI version 1 leaves them off (an open item of the instruction set, section 12.3 of the text), and the assembler provides `.lpad`; the kernel pain-points and mitigations reports (this chat, 2026-10-22) record why deciding later gets more expensive (appendix A, section A.1).

Cf. Appendix A, section A.1 (landing pads).

Status specified; **not yet implemented** in the reference executor.

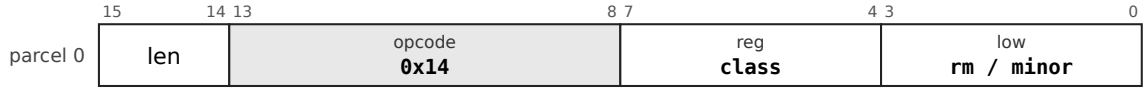
RTI: Return from interrupt or exception

Mnemonics: `RTI`, `RTT`

Syntax

```
RTI
RTT
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`RTI`	0	`0x1`
`RTT`	0	`0x2`

Operation

```
PC <- EPC ; PSW <- EPSW          ; a PSW with the reserved mode 2 in either mode field aborts
RTT: and then set the trace bit T after the return
```

Flags Replaced by `EPSW`, including the flags.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

`RTT` sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

Why it is here. `RTT` stays despite looking redundant against `RTI` (ISA revision 17): it sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

Cf. 6.2 (exceptions and interrupts), 6.7.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

RET: Return from subroutine

Mnemonics: `RET`

Syntax

```
RET
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`RET`	0	`0x3`

Operation

```
PC <- M[SP] ; SP <- SP + 8
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Not privileged, though it is in the system escape: it is the two-byte partner of `CALL`.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

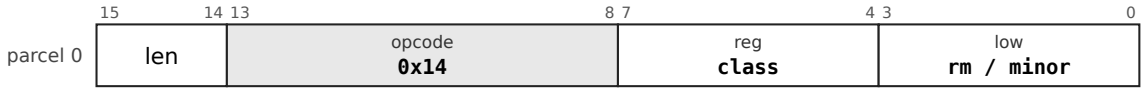
HALT: Halt the processor

Mnemonics: `HALT`

Syntax

```
HALT
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`HALT`	0	`0x4`

Operation

```
stop the processor
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Cf. 6.6.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

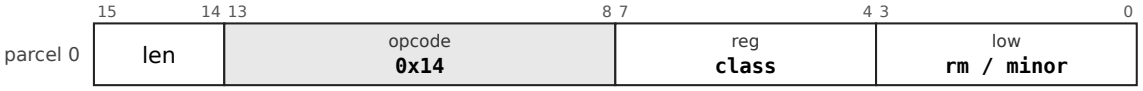
BPT: Breakpoint

Mnemonics: `BPT`

Syntax

```
BPT
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`BPT`	0	`0x5`

Operation

```
take the breakpoint vector with EPC = the address of the next instruction
```

Flags None written.

Exceptions Enters the kernel by the `BPT` vector.

A trap like any other, but kept as its own two-byte instruction because a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has.

Why it is here. `BPT` is just a trap vector, but a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has; that is why it stayed when `IOT` and `EMT` were dropped in ISA revision 17.

Cf. 6.2.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

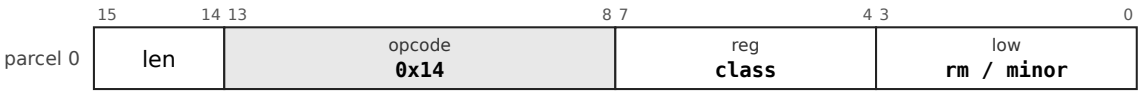
FENCE: Memory fence

Mnemonics: `FENCE`

Syntax

```
FENCE
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`FENCE`	0	`0x6`

Operation

```
order the memory accesses before it against those after it, for every initiator that observes them
```

Flags None written.

Exceptions None.

Owed to devices (one `FENCE` separates writing a device's registers from setting its go bit) and to other cores.

Why it is here. ISA revision 15 declared the memory model sequentially consistent and so needs `FENCE` only for what is outside the model: devices and the framebuffer (one `FENCE` between writing a device's registers and setting its go bit, section 3.4 of the book) and instruction streams. The record says the `_DB` barrier forms of the atomics were added so that a `FENCE` is not needed in a lock acquire.

Cf. 7.2 (the memory model), 2.2 (what the processor owes the bus).

Status specified; **not yet implemented** in the reference executor.

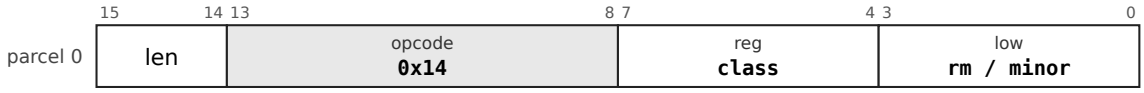
WFE: Wait for an event

Mnemonics: `WFE` (aliases: `WAIT`)

Syntax

```
WFE
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`WFE`	0	`0x7`

Operation

```
sleep until an interrupt is pending or another core executes SEV
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

`WAIT` is an assembler alias. Whether `WFE` wakes on a pending but masked interrupt (the idle-loop question) is open.

Why it is here. `WAIT` was a PDP-11 instruction (sleep until an interrupt) and was dropped in ISA revision 17 as subsumed by `WFE`, which sleeps until an interrupt or another core's `SEV`; on a uniprocessor it behaves exactly as `WAIT` did. `WFE` and `SEV` came with the SMP additions of revision 15 (the design chat's question about symmetric multiprocessing). The wake rule (whether a pending but masked interrupt wakes it) was an accidental decision that the author asked to see written up like the others; it is open item P1.

Cf. 7.6 (waiting and signalling).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

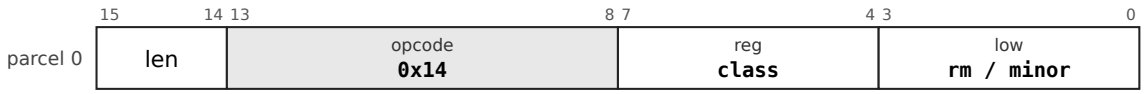
SEV: Send an event

Mnemonics: `SEV`

Syntax

```
SEV
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`SEV`	0	`0x8`

Operation

```
wake every core waiting in WFE
```

Flags None written.

Exceptions None.

Why it is here. The signalling half of `WFE`, added in ISA revision 15 so that a core waiting for a lock can sleep instead of spinning and be woken by the core that releases it.

Cf. 7.6.

Status specified; **not yet implemented** in the reference executor.

TLBA: Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache

Mnemonics: `TLBIA`, `TLBSYNC`, `ICINVA`

Syntax

```
TLBIA
TLBSYNC
ICINVA
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`TLBIA`	0	`0xA`
`TLBSYNC`	0	`0xB`
`ICINVA`	0	`0xC`

Operation

```
TLBIA   : invalidate every translation-buffer entry on this core and, by broadcast, on the others
TLBSYNC : wait until the invalidations this core started have completed on every core
ICINVA  : invalidate the whole instruction cache
```

Flags None written.

Exceptions Privileged (`TLBIA`, `TLBSYNC`); the reference executor treats `ICINVA` as a no-op.

Why it is here. ISA revision 15 respecified `TLBI` and `TLBIA` as broadcasting and added `TLBSYNC` to wait for the other cores, and `ICINV` and `ICINVA`: "the absence of an I-cache invalidate through revision 14 was a hole", because a Lisp runtime that compiles a function cannot work without it.

Cf. 7.5 (translation and instruction-cache coherence).

Status implemented in the reference executor for `TLBIA`, `TLBSYNC`; not yet for the others.

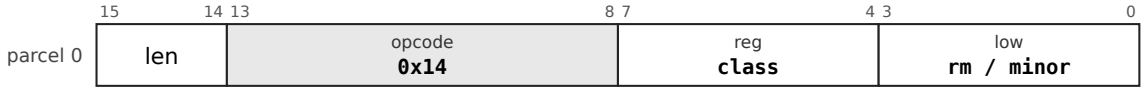
PUSHX: Push and pop the exception state

Mnemonics: `PUSHX`, `POPX`

Syntax

```
PUSHX
POPX
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`PUSHX`	0	`0xF` (`SYSX`), sub-opcode `0` in parcel 1
`POPX`	0	`0xF` (`SYSX`), sub-opcode `1` in parcel 1

Operation

```
PUSHX: SP <- SP - 16 ; M[SP] <- EPSW ; M[SP + 8] <- EPC
POPX : EPSW <- M[SP] ; EPC <- M[SP + 8] ; SP <- SP + 16
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. `SP` must be 8-byte aligned (a misaligned-access abort); both words are checked before either is touched.

Four bytes (the `SYSX` escape, sub-opcodes 0 and 1) and two bytes on page 2. For a handler that wants to be interrupted: it can save the exception state and lower the priority.

Why it is here. The author decided on 2026-10-22: “add pushx popx”, then “i like the pushx popx savings; they might not show up high in the instruction count, but they’d save bytes and cycles in a very tight, very often used kernel loop, no?” A handler that can be interrupted must save `EPC` and `EPSW` before it lowers the priority; with only `MFSR` that costs two moves and two stores (sixteen bytes) on every such kernel entry, and the same to restore. It came out of the System v1 design (errata 36 and 37). Page 2 has the two-byte forms.

Cf. 6.2, 6.7.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

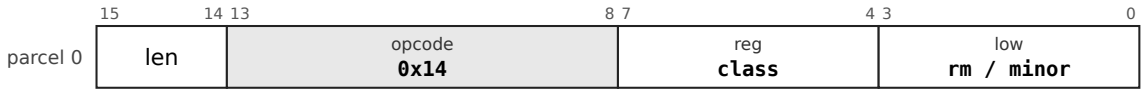
ISYNC: Instruction synchronisation

Mnemonics: `ISYNC`

Syntax

```
ISYNC
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`ISYNC`	0	`0xF` (`SYSX`), sub-opcode `2` in parcel 1

Operation

```
make earlier writes to PTBR, ASID, MMUCTL and VBR, and earlier ICINVs, take effect for the instructions that
follow
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A non-zero pad byte is an illegal instruction.

The architecture promises the effect only after the next `ISYNC`; an in-order implementation may make it immediate, and software must not rely on that.

Why it is here. Adopted on 2026-10-22 (errata 39) from the out-of-order readiness review, which the author asked for (“i’m ok with the fpga/asic being in order, but want to be ready for the out of order large scale design”): it makes earlier writes

to PTBR, ASID, MMUCTL and VBR take effect for what follows, so an in-order machine may make them immediate and a speculative one has a place to serialise.

Cf. 7.5, 6.3.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

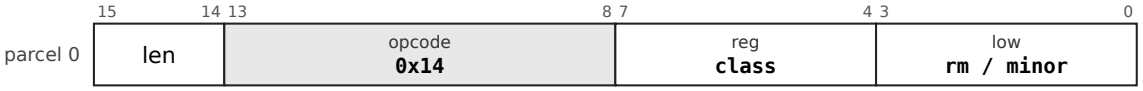
HINT: Hint

Mnemonics: HINT

Syntax

```
HINT #n        ; n = 0 to 255
```

Encoding



The reg field is the **class**. Class 0 takes a minor opcode from low; the other classes take a register or a 4-bit immediate from low.

Mnemonic	Class (reg)	Minor (low)
HINT	0	0xF (SYSX), sub-opcode 3 in parcel 1

Operation

```
no architectural effect
```

Flags None written.

Exceptions None.

Defined codes: 0 a plain no-op hint, 1 pause (spin-wait), 2 speculation barrier. Every other code is a reserved hint that executes as a no-op; software shall not use it. Not privileged.

Why it is here. Adopted with ISYNC on 2026-10-22 (errata 39): a no-op with a byte of code, defined for 0 (plain), 1 (pause, for spin-wait loops) and 2 (speculation barrier), every other code a reserved hint that executes as a no-op so that a later machine can give it a meaning without breaking an old binary.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

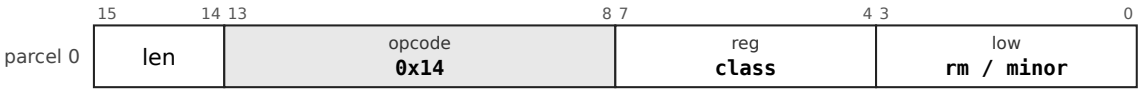
MFSR: Move from and to a system register

Mnemonics: MFSR, MTSR

Syntax

```
MFSR Rd, name
MTSR name, Rs
```

Encoding



The reg field is the **class**. Class 0 takes a minor opcode from low; the other classes take a register or a 4-bit immediate from low.

Mnemonic	Class (reg)	Minor (low)
MFSR	0	0xD, system register number in parcel 1
MTSR	0	0xE, system register number in parcel 1

Operation

```
MFSR: Rd <- system register `name`
MTSR: system register `name` <- Rs         ; read-only registers are an illegal instruction
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. An unassigned system register is an illegal instruction.

The register is chosen by name (a number in parcel 1); the instruction is four bytes. A write to `PTBR`, `ASID` or `MMUCTL` flushes the translation buffer.

Name	Number	Access
`PTBR`	0	read and write
`EPC`	1	read and write
`EVA`	2	read only
`ECAUSE`	3	read only
`DMW0`	4	read and write
`DMW1`	5	read and write
`USP`	6	read and write
`SSP`	7	read and write
`EPSW`	8	read and write
`VBR`	9	read and write
`MMUCTL`	10	read and write
`ASID`	11	read and write
`CPUID`	12	read only

Why it is here. The author asked on 2026-10-22: “actually didn’t you notice some missing instructions that a paging os needs? you should add them in now to, to the docs, the grammar, and the backend”. The instruction-set text named `PTBR`, `EPC`, `EVA` and `ECAUSE` and had no instruction to write the first or read the others, so a kernel could neither install a page table nor learn why it was entered. One general pair reaches all of them by name (errata 32). The user and supervisor stack pointers are in the set because any kernel must read and write the user stack pointer to build a signal frame, copy a process or switch context.

Cf. 6.3 (system registers).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

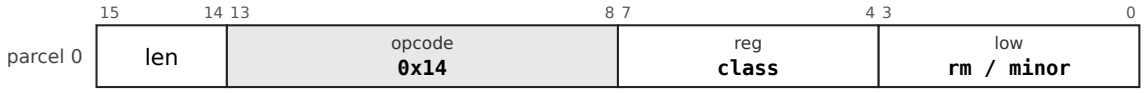
MTPS: Move to and from the processor status word

Mnemonics: `MTPS`, `MFPS`

Syntax

```
MTPS Rs
MFPS Rd
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`MTPS`	`0x1`	operand in `low`
`MFPS`	`0x2`	operand in `low`

Operation

```
MTPS: PSW <- Rs          ; in user mode only the flags N Z V C are written
MFPS: Rd <- PSW
```

Flags `MTPS` writes the flags from `Rs`; `MFPS` writes none.

Exceptions A `MTPS` value with the reserved mode 2 in a mode field aborts.

Cf. 6.1 (modes and the status word).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

SPL: Set the interrupt priority level

Mnemonics: `SPL`

Syntax

```
SPL #n        ; n = 0 to 7
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`SPL`	`0x3`	operand in `low`

Operation

```
PSW.priority <- n
```

Flags None written. The condition flags are untouched.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A priority above 7 is an illegal instruction.

Kept despite `MTPS` writing the whole word because it sets the priority alone: a kernel raising priority around a critical section must not clobber the flags.

Why it is here. `SPL` stays despite `MTPS` writing the whole status word (ISA revision 17): it sets the priority alone, leaving the condition codes untouched, so a kernel raising priority around a critical section does not clobber flags. It is the PDP-11 instruction of the same name.

Cf. 6.2.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

TRAP: System call and trap

Mnemonics: `TRAP`, `TRAPL`

Syntax

```
TRAP #n       ; n = 0 to 15, two bytes
TRAPL #n      ; n = 0 to 255, four bytes
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`TRAP`	`0x4`	operand in `low`
`TRAPL`	`0xF`	operand in `low`

Operation

```
enter the kernel by vector (TRAP_BASE + n), with EPC = the address of the next instruction
```

Flags None written.

Exceptions Enters the kernel; not privileged.

The replacement for the PDP-11 `IOT` and `EMT`. `TRAPL` reaches the vectors 0 to 255 with `n` in the high byte of parcel 1.

Why it is here. `TRAPL` was added (errata 5) so that the vector range of 256 and the two-byte common case could both be kept: `TRAP #n` for `n` below 16 is two bytes and the long form reaches 256. `IOT` and `EMT`, which were `TRAP` at two particular vectors on the PDP-11, were dropped in ISA revision 17.

Cf. 6.2, 6.5 (system calls).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

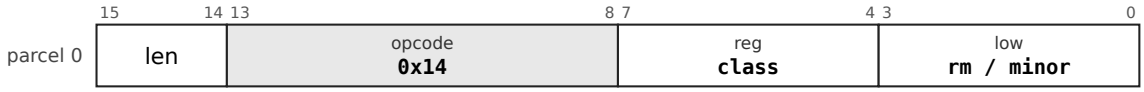
TLBI: Invalidate a translation

Mnemonics: `TLBI`

Syntax

```
TLBI Rs
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`TLBI`	`0x5`	operand in `low`

Operation

```
invalidate the translation-buffer entry for the page containing the address in Rs, on this core and by
broadcast on the others
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Why it is here. Respecified as broadcasting in ISA revision 15 (SMP section): a translation changed by one core must be invalidated on all of them, and `TLBSYNC` waits for it.

Cf. 7.5, 6.4.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MTASID: Move to and from the address-space identifier

Mnemonics: `MTASID`, `MFASID`

Syntax

```
MTASID Rs
MFASID Rd
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`MTASID`	`0x6`	operand in `low`
`MFASID`	`0x7`	operand in `low`

Operation

```
MTASID: ASID <- Rs[15:0]          ; the translation buffer is flushed in the reference executor
MFASID: Rd <- ASID
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

The `ASID` tags translation-buffer entries so a context switch is one store.

Why it is here. The `ASID` tags translation-buffer entries so a context switch is one store and no flush, which preserves the constant-time context switch the earlier map-set design was for (ISA section 8.1). The record gives no further history.

Cf. 6.4.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

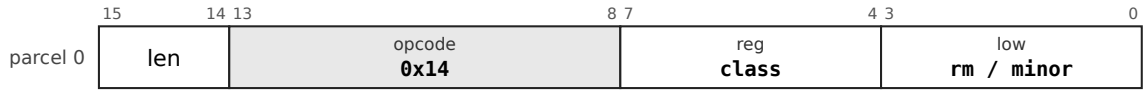
MTDP: Move to and from the direct pointer

Mnemonics: `MTDP`, `MFDP`

Syntax

```
MTDP Rs
MFDP Rd
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`MTDP`	`0x8`	operand in `low`
`MFDP`	`0x9`	operand in `low`

Operation

```
MTDP: DP <- Rs
MFDP: Rd <- DP
```

Flags None written.

Exceptions None.

`DP` is the thread pointer in user mode and the per-CPU pointer in kernel mode; there is one copy per mode.

Why it is here. `DP` entered at revision 1 as the descendant of the 6502's zero page, kept for code density (a 12-bit displacement in four bytes); it turned out to be the thread pointer in user mode and, in kernel mode, the per-CPU pointer that an SMP kernel wants (ISA sections 2.2 and 19.3). `MTDP` and `MFDP` read and write the current mode's copy.

Cf. 7.4 (per-CPU state).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

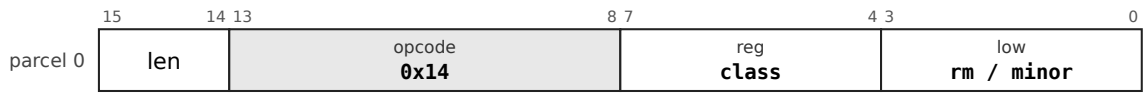
MTDMW: Move to a direct-mapping window

Mnemonics: `MTDMW`

Syntax

```
MTDMW Rs
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`MTDMW`	`0xA`	operand in `low`

Operation

```
DMW0 <- Rs          ; storage only: the operand names no window and the windows are not yet defined
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

The direct-mapping windows are deferred (errata 44).

Why it is here. Taken from LoongArch's direct-mapping windows (ISA section 15.1), which stop the kernel taking translation-buffer misses on its own text. The author decided on 2026-10-22 that the windows are deferred, `MMUCTL` stays, and the instruction is storage only for now (errata 44).

Cf. 6.4.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MFCPUID: Read the processor identifier

Mnemonics: `MFCPUID`

Syntax

```
MFCPUID Rd
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`MFCPUID`	`0xB`	operand in `low`

Operation

```
Rd <- CPUID
```

Flags None written.

Exceptions None.

The index of this core, for per-CPU data.

Why it is here. Added in ISA revision 15 with the SMP section, because per-CPU data needs each core to know which one it is.

Cf. 7.4, 7.8.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

ICINV: Invalidate the instruction cache for an address

Mnemonics: `ICINV`

Syntax

```
ICINV Rs
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`ICINV`	`0xC`	operand in `low`

Operation

```
invalidate the instruction-cache line containing the address in Rs
```

Flags None written.

Exceptions The reference executor treats it as a no-op; privilege as for the other cache-maintenance instructions.

Why it is here. Added with `ICINVA` in ISA revision 15: instruction-cache invalidation, by address and whole. The reason is the one given for `TLBA`: a runtime that compiles a function cannot work without it.

Cf. 7.5.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

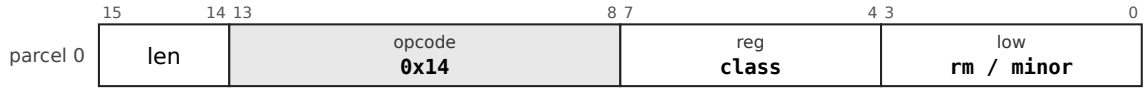
PROBE: Probe an address for read or write access

Mnemonics: `PROBER`, `PROBEW`

Syntax

```
PROBER Rd, Rs
PROBEW Rd, Rs
```

Encoding



The `reg` field is the **class**. Class 0 takes a minor opcode from `low`; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Class (`reg`)	Minor (`low`)
`PROBER`	`0xD`	operand in `low`
`PROBEW`	`0xE`	operand in `low`

Operation

```
Rd <- 1 if the address in Rs is readable (PROBER) or writable (PROBEW) in the previous mode, else 0
no fault is taken
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Taken from the VAX `PROBER` and `PROBEW`: the kernel validates a user address before using it.

Why it is here. Restored from the VAX `PROBER` and `PROBEW` in the idioms the revision 13 inventory brought back (ISA section 14.1): the kernel validates a user address before using it, and it replaces the `nofault` global of the V7 kernel for the address-validation case.

Cf. 6.5 (the user-memory guard), 6.4.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

6.8 Faults, restart and memory management

6.8.1 Faults fall in four categories: traps (the instruction completes), aborts (it is annulled and restarted), progress (a block operation stopped part-way, restarted from its registers) and machine checks (not restartable). *(D)*

6.8.2 The register update of an auto-increment or auto-decrement mode is held until the instruction retires and discarded by an abort, so a handler never undoes anything. *(D)*

6.8.3 A read of non-existent memory is a precise abort with cause `BUSFAULT`; a posted write that times out and an uncorrectable memory error are machine checks. *(D)*

6.8.4 Virtual addresses are 39 bits, translated by three levels of nine bits over 4 KB pages to a 40-bit physical address. *(D)*

6.8.5 A page-table entry carries the physical page number in place and read, write, execute and user permission bits; accessed and dirty bits are managed by software; one memory type per page selects cached, uncached, strongly ordered or weakly ordered. *(D, errata 36)*

6.8.6 Two direct-mapping windows, `DMW0` and `DMW1`, are stored but not yet defined; `MMUCTL` bit 0 enables translation and is zero at reset, so every address is physical until the kernel turns it on. *(D, errata 36 and 44)*

Commentary. **Precedent.** The PDP-11 recorded its register side effects in `MMR1` and left the undo to software; the VAX needed resumable microstate and a status-word bit; the 68000 had no restart path. **Trade-offs.** Thirty-nine bits is 512 GB, what a three-level table indexes with nine bits per level; a fourth level would give 48 bits at the price of a longer walk on every TLB miss. Software-managed accessed and dirty bits cost one extra fault on first touch and one on first write per page, in exchange for a simpler walker (appendix B). **Improvements.** The one-memory-operand rule makes the undo buffer a single entry and row 3 of the fault matrix provably empty, so there is no journal and no undo protocol. **Benefits.** An FPGA walker is small, and a core with no speculation has no page-table side channels. **Modern comparables.** RISC-V's Sv39 has the same geometry; x86-64 and AArch64 have 48-bit spaces with four levels and hardware-managed accessed and dirty bits. **Use.** The memory type on a page lets a driver map a device register uncached and strongly ordered without a special instruction. **Impact.** *Kernel:* Precise faults and software-managed accessed and dirty bits make one small page-fault handler for first touch, first write and demand paging. *Compiler:* No scheduling restriction arises from faults, because the register update of an addressing mode is held until retirement. *Hardware:* A single-entry undo buffer, a read-only walker, and (on the FPGA) a four-way set-associative translation buffer rather than a content-addressable one.

6.9 Still open

The format of the direct-mapping windows `DMW0` and `DMW1` (stored but without effect; the translation-enable bit stays, errata 44), the use of supervisor mode, hardware breakpoints and watchdogs, performance counters, machine-check reporting beyond halting, feature and identification registers from register 13 up, and a TLB invalidate by address-space tag (appendix B).

Chapter 7 — Multiple processors

The pdp64 is built so that adding a second core needs seventeen operations, one per-core register, and nothing a kernel writer has to work around: memory is sequentially consistent, the atomics are one instruction each, the per-CPU pointer is a register that already exists, and the other cores' translation buffers are invalidated by a single broadcast. This chapter collects everything the architecture adds for symmetric multiprocessing, what a kernel does with it, what was deliberately left out, and what must still be built on the device side. The instruction-set text is `../pdp64-isa.md`, section 17. Tags: *(D)* decided; *(P)* proposed.

7.1 The seventeen operations

7.1.1 The instructions that exist for multiple processors fall under one umbrella of seventeen operations: ten atomics and seven others. *(D)*

Group	Operations
Atomics (10)	compare-and-swap (`CAS`), swap (`SWAP`), fetch-and-add (`XADD`), fetch-and-and (`XAND`), fetch-and-or (`XOR_`), fetch-and-exclusive-or (`XXOR`), fetch-and-maximum (`XMAX`, with `XMAXU`), fetch-and-minimum (`XMIN`, with `XMINU`), load-linked (`LL`), store-conditional (`SC`)
Others (7)	`FENCE`, wait-for-event (`WFE`), send-event (`SEV`), `TLBI` (with `TLBIA` as its "all" form), `TLBSYNC`, instruction-cache invalidate (`ICINV`, with `ICINVA`), read the core number (`MFCPUID`)

7.1.2 Every atomic has a variant, `_DB`, that folds a barrier into the instruction, so that a lock acquire or release is one instruction. *(D)*

7.1.3 The atomics live in the atomic escape group and the other operations in the system group; the umbrella is conceptual, not an encoding. *(D)*

7.1.4 Supporting these are things the machine already had: the banked direct-page register, the address-space tag, the global-page bit and the banked stack pointer (section 7.4). *(D)*

Commentary. **Precedent.** The PDP-11/70's read-modify-write instructions were indivisible on the Unibus (`DATIP` and `DATO`); the VAX added interlocked instructions (`BBSSI`, `ADAWI` and the queue instructions) and multiprocessor models, and the 4.3BSD kernel on them still used `spl` for exclusion. **Trade-offs.** The set is the union of the three designs found on other machines (compare-and-swap, load-linked and store-conditional, fetch-and-operate), which is more atomic operations than any architecture surveyed, in exchange for nothing a kernel writer has to emulate. **Improvements.** The `_DB` forms were taken from LoongArch: a lock acquire should be one instruction, with no separate fence. **Benefits.** The seven non-atomic operations are the minimum that a kernel cannot synthesise: which core am I, wait and signal, invalidate everywhere, and make the instruction fetch see what the data side wrote. **Modern comparables.** RISC-V's `LR`/`SC` and `AMO` set, AArch64's exclusive loads and stores and its large-system atomics, x86's `LOCK`-prefixed read-modify-write. **Impact.** *Kernel:* Each of the seventeen operations is one instruction in the kernel's lock, idle, shootdown and per-CPU code. *Compiler:* C11 atomics and the kernel's intrinsics map to them directly. *Hardware:* The atomics live in one escape group and the other seven in the system group, so the decoder has two places to look.

7.2 The memory model

7.2.1 Memory is sequentially consistent: the result is as if all cores' accesses happened in one order that respects each program's order. *(D)*

7.2.2 A store buffer is allowed, provided loads snoop it and stores drain in program order. *(D)*

7.2.3 Quad accesses are naturally aligned, so every access is atomic within its own size; no misaligned access can be torn across cores. *(D)*

7.2.4 The machine is in order and does not speculate, so no speculative side channel exists to be mitigated. *(D, errata 38)*

7.2.5 The ABI says nothing about memory ordering, because with sequential consistency there is nothing to say. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX ran one processor at a time in the model that mattered, so the question did not arise; ARM's memory model took about a decade to formalise, and the C11 and C++11 models exist largely to paper over the differences between weak architectures. **Trade-offs.** Sequential consistency is the model with no cost to software and the highest cost to a large speculative core; software written against a weak model runs on a strong machine and not the reverse, so the choice is close to irreversible, which is why it was made once. If a large out-of-order core is ever built, total store order is the least painful retreat. **Benefits.** None of the barrier code that fills other kernels: the kernel survey found barrier vocabulary in 5,596 lines of Linux code and 3,016 lines of `memory-barriers.txt` documentation, and none of it arises here. **Modern comparables.** x86 and SPARC are total store order; ARM, POWER and RISC-V are weak. **Impact.** *Kernel:* None of the 5,596 lines of ordering code found in Linux, nor the barrier documentation, is needed. *Compiler:* The optimiser may move non-volatile accesses freely, and no fence is inserted for ordering. *Hardware:* In-order with a snooped store buffer; sequential consistency is nearly free on this core and the expensive part only appears on a speculative one.

7.3 The atomics in detail

7.3.1 `LL` takes a reservation on a granule of memory; a write to the granule by any initiator, core or device, clears it; `SC` succeeds exactly when the reservation is intact and otherwise writes nothing. *(D, errata 45)*

7.3.2 A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule. *(D)*

7.3.3 The read-modify-write atomics are single-copy atomic with respect to every initiator; the memory system may build them from an exclusive access with hardware retry. *(D)*

7.3.4 A device that writes memory shared with a core takes part by clearing reservations. *(D)*

Commentary. **Precedent.** A bus lock, the Unibus's `DATIP`/`DATO` pair, is simple and a bottleneck as cores multiply; a monitor maps onto the exclusive access of AXI and the memory systems of FPGAs and modern chips. **Trade-offs.** A monitor needs forward-progress rules and, under contention, can livelock if not specified; RISC-V's guarantee of eventual success for constrained sequences is the model. The pain-points report records that the kernels' simple lock does not scale (OpenBSD's ticket kernel lock measured 155% worse than an Anderson lock in one developer's test) and that Linux moved from test-and-set through ticket locks to queued locks; the instruction set provides the primitives and the lock design is the kernel's. **Benefits.** A kernel's lock needs one instruction; lock-free queues need compare-and-swap. **Modern comparables.** RISC-V `LR`/`SC` with its constrained-loop guarantee, AArch64 load-exclusive and `CAS`, x86 `LOCK CMPXCHG`. **Use.** `CAS` for lock-free structures and lock words; `XADD` for counters and ticket numbers; `LL`/`SC` for everything else. **Impact.** *Kernel:* A lock is one `_DB` instruction. *Compiler:* A `LL`/`SC` loop must be emitted in the constrained form, with no other memory access between the two. *Hardware:* A reservation granule tag in the cache controller and an exclusive-access path to memory.

7.4 Per-CPU state and cheap context

7.4.1 `MFCPUID` returns the number of the core that executes it. It cannot be synthesised and per-CPU data needs it. *(D)*

7.4.2 `DP`, the direct-page register, has a copy per mode; in kernel mode it is the per-CPU pointer and a field of the per-CPU structure is one four-byte instruction, `MOVQ d(DP), Rd`. *(D)*

7.4.3 The address-space tag makes a context switch constant-time, and global page-table entries and the direct-mapping windows keep the kernel's own mappings out of the invalidation path. *(D)*

7.4.4 The stack pointer is banked by mode, so an interrupt arriving in user mode on any core finds that core's kernel stack without a search. *(D)*

Commentary. **Precedent.** Every SMP kernel builds a per-CPU base by hand: x86 uses the `GS` base and a `swapgs` instruction, ARM uses `TPIDR_EL1`, RISC-V the supervisor scratch register, MIPS reserves two registers; the kernel survey found seven architectures paying for the problem of having no free register at the moment of entry. **Improvements.** `DP` was added for code density and turns out to be what both an SMP kernel and a threaded C runtime need most, in the mode where each needs it, and the two copies do not conflict. **Benefits.** No `swapgs`, no reserved scratch register, no code spent finding the CPU. **Modern comparables.** `GS` and `TPIDR` as above. **Impact.** *Kernel:* Per-CPU data is one four-byte `MOVQ d(DP), Rd`, with no `swapgs` and no reserved register. *Compiler:* The compiler addresses per-CPU variables relative to `DP`. *Hardware:* Three copies of `DP`, one per mode.

7.5 Translation and instruction-cache coherence

7.5.1 `TLBI` and `TLBIA` broadcast to all cores; `TLBSYNC` waits until the broadcast has completed everywhere. *(D)*

7.5.2 Instruction caches are not coherent with data caches; `ICINV` invalidates a line and `ICINVA` all of it, and neither is privileged, because a JIT needs them. *(D)*

7.5.3 A write to the page-table base, the address-space tag, the translation-enable bit or the vector base takes effect only after the next `ISYNC`. *(D, errata 39)*

Commentary. **Precedent.** x86 flushes remote translation buffers with an inter-processor interrupt and a software handshake, which is slow and intricate; AMD added a broadcast invalidate only late. **Trade-offs.** A broadcast is simpler in hardware and far faster for two to four cores; it needs the completion barrier, because a broadcast one cannot wait for is no use. Making instruction caches coherent would cost a snoop port for a case that almost never happens. **Improvements.** The kernel's own mappings are global or window-mapped and never part of a shootdown. **Benefits.** The pain-points report finds that remote translation-buffer shootdown, a source of deadlocks in the BSDs and of much code in Linux, does not arise. **Modern comparables.** ARM's broadcast `TLBI` and `DSB`; RISC-V's `SFENCE.VMA` still needs software shootdown. **Impact.** *Kernel:* A shootdown is `TLBI` then `TLBSYNC`; a module or JIT write is `ICINV` then `ISYNC`. *Compiler:* A JIT or a trampoline generator must call `ICINV` after writing code. *Hardware:* A broadcast on the interconnect and a counter for completion; the instruction cache has an invalidate port but no snoop port.

7.6 Waiting and signalling

7.6.1 `WFE` sleeps until another core executes `SEV` or an interrupt arrives, and `HINT #1` is the spin-wait pause. *(D, errata 39)*

7.6.2 `WFE` ends its sleep when an interrupt is pending even if the priority masks it, so that the idle loop is `SPL #7`, check for work, `WFE`, `SPL #0`, with no gap. *(P, the wake rule under decision)*

7.6.3 On one core `WFE` is the idle instruction, and `WAIT` is its assembler alias. *(D)*

Commentary. **Precedent.** x86's `PAUSE` tells a spinning core to back off; ARM's `WFE` lets it stop until an event. V7's idle routine lowered the priority and executed `wait` in a loop that rechecked the run queue after every interrupt, so a missed wake-up cost one clock tick. **Trade-offs.** On an FPGA a spinning core consumes memory bandwidth the other cores want, which is the reason to prefer a real wait. **Benefits.** An idle core costs nothing and wakes on a signal. **Modern comparables.** ARM `WFE` and `SEV`, RISC-V `WFI`. **Impact.** *Kernel:* An idle core waits on `WFE` and costs no bandwidth. *Compiler:* Spin-wait loops use `HINT #1`. *Hardware:* `WFE` stalls or gates the core's clock until an event or an interrupt.

7.7 Cache coherence

7.7.1 Between processors, coherence is transparent to software, by snooping with three states (modified, shared, invalid), for two to four cores. *(D)*

7.7.2 A DMA device is the opposite case: the buffers are uncached and the hand-off is explicit, because snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off. *(D)*

Commentary. **Trade-offs.** MESI would add an exclusive state to avoid a bus transaction when writing a clean line nobody else holds; at four cores the benefit is small and the state machine larger; a directory scales further and is not needed below about eight cores. Software-managed coherence is cheaper in hardware and is where concurrency bugs live, since every missing flush is a race that reproduces once a week. **Benefits.** The instruction set owes nothing to the coherence protocol, which is the correct division. **Modern comparables.** x86 snoops with MESIF or MOESI and directories across sockets; ARM's coherent interconnects use directories or snoop filters. **Impact.** *Kernel:* No coherence code, in either direction between cores. *Compiler:* None. *Hardware:* A snoop port on each data cache, about 500 lookup tables and a small state machine for three states, plus about 600 per core.

7.8 Starting and signalling the other cores

7.8.1 The controller holds every core but core 0 in reset; a write to a per-core start register gives a core an entry address and an argument and releases it into kernel mode at priority 7, with its number in `r0`. *(P, decision B2)*

7.8.2 An inter-processor interrupt is a write to a per-core register in the interrupt controller; no APIC-like machinery is needed. *(P)*

7.8.3 The routing of device interrupts to cores, and a shared second-level cache or a coherent interconnect, are device-side and not yet defined. *(P, decision R1)*

Commentary. **Precedent.** An x86 core wakes in 16-bit real mode and has to be walked through three modes by a trampoline (1,528 lines of start-up code and 1,258 of real-mode code in Linux); arm64 uses a firmware call in 124 lines or a spin table in

127; RISC-V 108 or 58; the VAX's console processor started the others. **Benefits.** A core can start in its normal mode at any address, so the whole protocol is one register write. **Modern comparables.** x86's INIT and START-UP interrupts, ARM's PSCI, RISC-V's hart-start call. **Impact.** *Kernel:* Starting a core is one register write, and a stuck core is reset by another. *Compiler:* None. *Hardware:* A start register and a reset-hold per core in the controller.

7.9 A kernel's view

7.9.1 The kernel starts a core by writing its start register, keeps its per-CPU structure at the address in `DP`, and takes a lock with one `_DB` atomic. *(P)*

7.9.2 It unmaps a page by changing the entry, issuing `TLBI`, and waiting with `TLBSYNC`. *(D)*

7.9.3 It makes new instructions visible by writing them, issuing `ICINV` and executing `ISYNC`. *(D)*

7.9.4 It idles a core with `WFE` and wakes it with `SEV` or an inter-processor interrupt. *(P)*

Commentary. This section is the checklist an operating-system port works through. Each item is one instruction or one register write, and each is a place where other architectures need a trampoline, a handshake or a fence.

7.10 What was avoided

7.10.1 The architecture has no weak ordering, no speculative execution, no hardware transactional memory, no software-managed coherence between processors, no coherent instruction caches, no inter-processor-interrupt shootdown as the main mechanism, and no memory-model-dependent ABI. *(D)*

Commentary. **Trade-offs.** Intel's transactional memory produced a decade of errata and side channels for a feature few programs used; compare-and-swap and `LL`/`SC` suffice. Each avoided item is a tax on every concurrent program, or on every implementation, in exchange for performance the small in-order core cannot use. **Impact.** *Kernel:* Each avoided feature is code a kernel does not have to write: no barriers, no software coherence, no shootdown interrupts. *Compiler:* Each is analysis the compiler does not have to do: no memory-model-dependent code generation. *Hardware:* Each is a block not built: no speculation, no transactional memory, no coherent instruction cache.

7.11 What the model does not give for free, and the road to out of order

7.11.1 A simple memory model does not make an atomic implementation simple by itself: cache coherence, exclusive ownership, invalidation, ordering and the interaction with interrupts still have to be designed and tested. *(the author's caveat)*

7.11.2 Out-of-order execution was never ruled out: the first implementations are in order, and the architecture is kept ready for a large out-of-order design. *(D, errata 38)*

7.11.3 Sequential consistency on a speculative core needs loads that are replayed when another core intervenes; the readiness review calls this awkward but defined, and the flag rules (errata 39), the reservation rules and the `ISYNC` and `HINT` instructions are what the review asked for. *(D)*

Commentary. **Precedent.** One instruction set often spans small in-order and large out-of-order cores: ARM's small and large cores, RISC-V's Rocket and BOOM, the x86 Pentium and P6, Alpha's 21064 and 21264. **Trade-offs.** What the pdp64 gains by refusing speculation is a smaller design with fewer error cases and none of the speculative side channels; what it gives up is the performance of an aggressive core, which an FPGA or a small chip could not build anyway. The readiness document (`../pdp64-ooo-readiness.md`) and the hardware-vulnerabilities report give the rules a future core would follow. **Benefits.** The decision is reversible in the direction that matters: total store order can be the retreat for a large core. **Impact.** *Kernel:* None today. *Compiler:* None today. *Hardware:* A speculative core would need loads that replay when another core intervenes, and the flag and reservation rules of errata 39 keep that path open.

7.12 The atomic and ordering instructions

7.12.1 The atomic escape provides `CAS`, `SWAP`, `XADD`, `XAND`, `XOR_`, `XXOR`, `XMAX`, `XMIN`, `XMAXU` and `XMINU`, each with a `_DB` variant that adds a barrier, and `LL` and `SC`. *(opcode `0x11`)*

7.12.2 `LL` takes a reservation on a granule of memory; any write to it by any initiator clears it; `SC` succeeds exactly when the reservation is intact. A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule. *(D, errata 45)*

7.12.3 The read-modify-write atomics are atomic with respect to every initiator, cores and devices alike. *(D)*

7.12.4 FENCE orders device accesses; WFE and SEV wait for and signal an event; ISYNC makes earlier system-register writes take effect; HINT #n has no architectural effect (1 pause, 2 speculation barrier, others reserved). *(D, errata 39)*

Commentary. **Precedent.** The PDP-11's read-modify-write instructions were indivisible on the Unibus (the DATIP/DATO pair); the VAX had interlocked queue and bit instructions. **Trade-offs.** A bus lock is simple and does not scale as cores multiply; a monitor (reservation) maps onto the memory systems that FPGAs and modern chips use, at the cost of forward-progress rules. **Improvements.** The pdp64 offers all three designs found on other machines (load-linked/store-conditional, compare-and-swap, fetch-and-operate), more atomic operations than any architecture surveyed. **Benefits.** A kernel lock needs one instruction. **Modern comparables.** RISC-V LR/SC and the AMO set; AArch64 load-exclusive and its large-system atomics; x86 LOCK-prefixed read-modify-write. **Use.** CAS for lock-free structures; XADD for counters; LL/SC for anything else; WFE for the idle loop. **Impact.** *Kernel:* A lock is one instruction, and the _DB forms need no separate fence. *Compiler:* C11 atomics map one-to-one, no ordering fences are emitted under sequential consistency, and a LL/SC loop must be generated in the constrained form (short, no other memory access inside). *Hardware:* The reservation is a granule tag in the cache controller, and the read-modify-write atomics use an exclusive access with hardware retry.

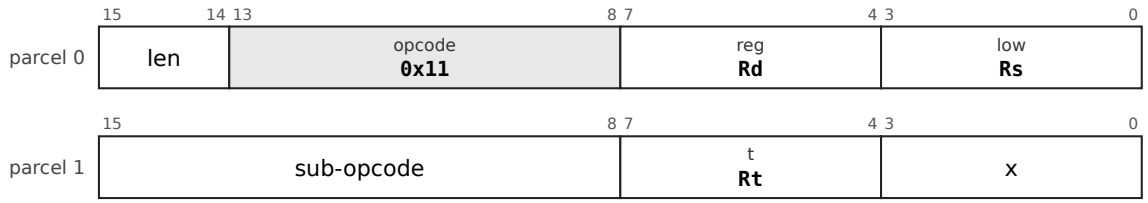
CAS: Compare and swap

Mnemonics: CAS, CASDB

Syntax

```
CAS expected, new, (address)        ; operand placement not yet fixed
CASDB ...                            ; with a full barrier
```

Encoding



From len = 01. In this group low is a register, never a mode. The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Mnemonic	Sub-opcode
CAS	0x00
CASDB	0x01

Operation

```
old <- M[address]
if old == expected then M[address] <- new
result <- old          ; the program compares result with expected to learn whether the swap happened
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Why it is here. ISA revision 13 expanded the atomic escape with barrier-fused variants, taken from LoongArch's pragmatism: a lock acquire is one instruction rather than an atomic followed by a FENCE, and that pair is in every critical section of the system. The design chat asked what is needed for symmetric multiprocessing and what can be learned from modern multi-core processors; revision 15 added section 17 (a sequentially consistent model, nearly free on an in-order machine) and the SYS additions. The author's question on 2026-10-21, "how many folded opcodes in system and smp? i thought atomics and smp would be in the same umbrella", made the seventeen-operation SMP umbrella. On 2026-10-22 the author chose

atomics defined on a reservation monitor and not a bus lock (decision P11, errata 45). The group is specified and not yet implemented in the reference executor.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

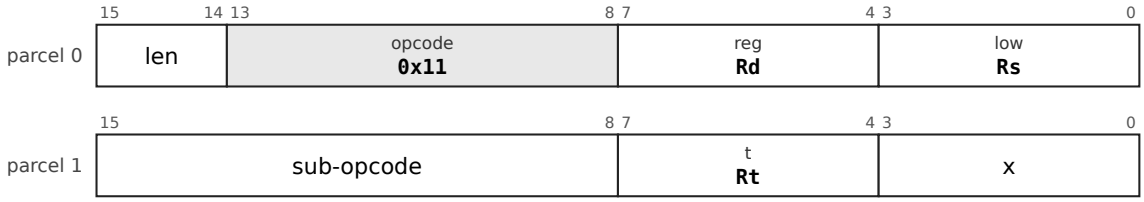
SWAP: Atomic exchange

Mnemonics: `SWAP`, `SWAPDB`

Syntax

```
SWAP value, (address)
SWAPDB ...
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Mnemonic	Sub-opcode
`SWAP`	`0x02`
`SWAPDB`	`0x03`

Operation

```
old <- M[address] ; M[address] <- value ; result <- old
```

Flags Not specified beyond “none for the read-modify-write forms”; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Why it is here. One of the read-modify-write atomics of the `ATOM` group (see `CAS`). The record gives no separate history for it.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

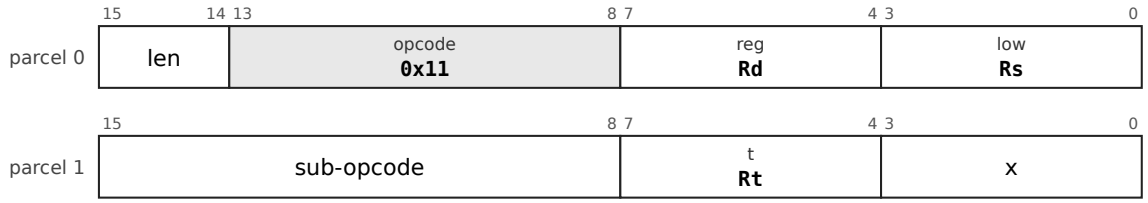
XADD: Atomic fetch and add

Mnemonics: `XADD`, `XADDDB`

Syntax

```
XADD value, (address)
XADDDB ...
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Mnemonic	Sub-opcode
XADD	0x04
XADDDB	0x05

Operation

```
old <- M[address] ; M[address] <- old + value ; result <- old
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Why it is here. One of the fetch-and-operate atomics of `ATOM` (see `CAS`), in the place of the PDP-11's read-modify-write instructions, which were indivisible on the Unibus through the `DATIP`/`DATO` pair (ISA section 9.13).

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

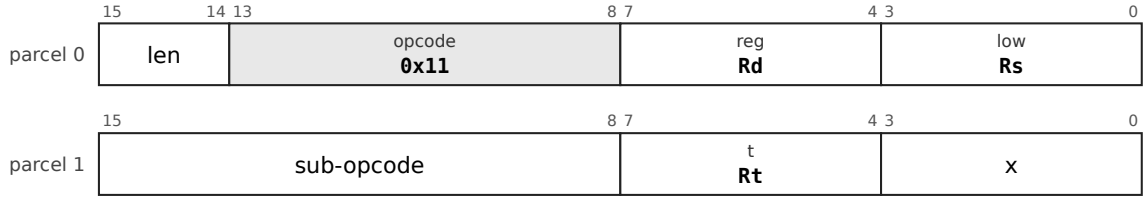
XAND: Atomic fetch and and, or, xor

Mnemonics: `XAND`, `XANDDB`, `XOR`, `XOR_DB`, `XXOR`, `XXORDB`

Syntax

```
XAND value, (address)
XOR_ value, (address)        ; the trailing underscore keeps the mnemonic distinct from XOR
XXOR value, (address)
each with a DB form
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Mnemonic	Sub-opcode
XAND	0x06
XANDDB	0x07
XOR_	0x08
XOR_DB	0x09
XXOR	0x0A

Mnemonic	Sub-opcode
XXORDB	0x0B

Operation

```
old <- M[address]
XAND: M[address] <- old AND value
XOR_: M[address] <- old OR value
XXOR: M[address] <- old XOR value
result <- old
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Why it is here. The fetch-and-and, or and xor atomics of ATOM (see CAS). The record gives no separate history for them.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

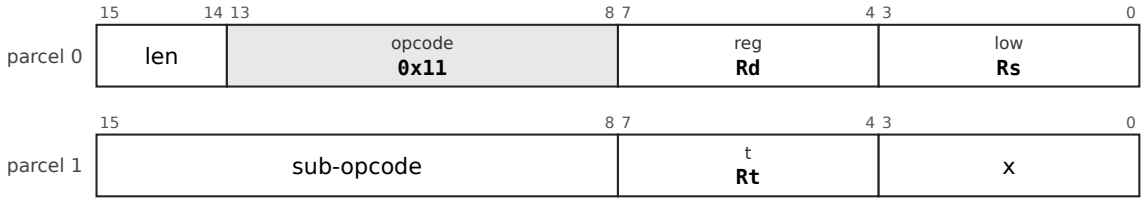
XMAX: Atomic fetch and minimum or maximum

Mnemonics: XMAX, XMAXDB, XMIN, XMINDB, XMAXU, XMAXUDB, XMINU, XMINUDB

Syntax

```
XMAX value, (address)
XMIN ..., XMAXU ..., XMINU ...
each with a DB form
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Mnemonic	Sub-opcode
XMAX	0x0C
XMAXDB	0x0D
XMIN	0x0E
XMINDB	0x0F
XMAXU	0x10
XMAXUDB	0x11
XMINU	0x12
XMINUDB	0x13

Operation

```
old <- M[address] ; M[address] <- max(old, value) (XMAX, XMAXU) or min (XMIN, XMINU) ; result <- old
signed for XMAX and XMIN, unsigned for the U forms
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Why it is here. The fetch-and-minimum and maximum atomics, borrowed from LoongArch's richer atomic set (ISA section 15.1): each is a couple of hundred lookup tables and saves a compare-and-swap retry loop.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

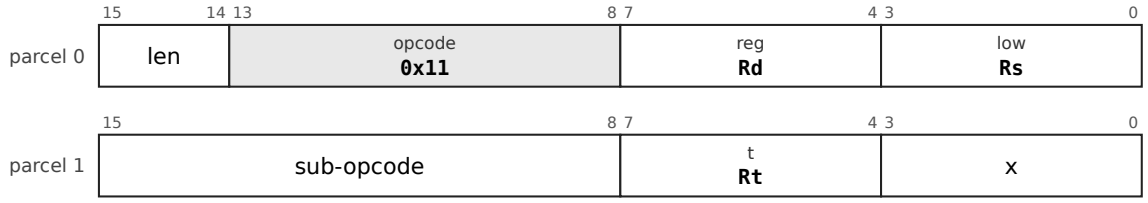
LLSC: Load-linked and store-conditional

Mnemonics: `LL`, `SC`

Syntax

```
LL (address), Rd
SC value, (address), result
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet.

Mnemonic	Sub-opcode
`LL`	`0x20`
`SC`	`0x21`

Operation

```
LL: Rd <- M[address] ; take a reservation on the granule containing address
SC: if the reservation is intact then M[address] <- value ; result <- 0 (success)
    else nothing is written ; result <- 1 (failure)
a write to the granule by any initiator, core or device, clears the reservation
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule (the RISC-V guarantee for constrained sequences). The text of the instruction set fixes the sub-opcodes of this group and its semantics but not yet the placement of the operands of each form; the forms below name the roles, and the assembler and the reference executor do not implement the group yet. The result convention shown for `SC` is the usual one and is not yet fixed in the text.

Why it is here. Load-linked and store-conditional, with the reservation made a monitor in errata 45 (the author's pick, decision P11, 2026-10-22), because a monitor maps onto the exclusive access of AXI and of FPGA and modern memory systems, where the Unibus's locked read-modify-write (`DATIP`/`DATO`) does not scale as cores multiply. A short sequence must eventually succeed if no other initiator keeps writing the granule, the RISC-V rule.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

7.13 Still open

The transport and the bus's own atomics mapping (decision R4), the interrupt controller and routing (R1), starting the other cores (R2, B2), how coherence scales beyond four cores (R3), the wake rule of `wfe` (P1), and lock scaling above a few cores, which is a kernel design question the instruction set does not settle.

Chapter 8 — Vector work, AI and array languages: the device route

Wide data-parallel work, whether neural-network inference or the primitives of an array language, is done by memory-mapped devices with their own memory and driven by descriptors, not by SIMD registers in the processor. There is no vector register file and no AVX2-style instruction set in the pdp64. This chapter states that decision, why it was taken, and how it serves the workloads the author named: AI and large-language-model inference, and the array languages APL and K. Graphics, the third, has its own chapter (chapter 3), since its devices are the first a user sees. It is the route ARM-based systems take for their neural-network and graphics engines, and the one the PDP-11 took for its attached array processors. The discussion of the alternatives is appendix B (S1 to S6); the accelerator designs are `../pdp64-lb64-accelerator.md` and section 12 of `../pdp64-fpga-plan.md`. Tags: *(D)* decided; *(P)* proposed.

8.1 The decision

8.1.1 The processor has no vector register file and no SIMD instruction set of the SSE, AVX2 or NEON kind. *(D, errata 59)*

8.1.2 Wide work is described to a device by descriptors in memory: an operation, the addresses of the arrays and their length. *(D)*

8.1.3 The processor owes such a device what it owes every device: a memory type per page, a `FENCE`, and atomics that reach device memory; the instruction set changes not at all. *(D, instruction-set text section 16)*

8.1.4 Packed-lane operations in the ordinary integer registers (a handful of instructions for bytes, halves and words in a 64-bit register) are not part of this decision; they remain a possibility to be settled by measurement. *(P, appendix A.10)*

Commentary. **Precedent.** The PDP-11's heavy numerical work went to attached array processors driven by library calls (the Floating Point Systems AP-120B attached to PDP-11 hosts, by the one brochure checked), and the VAX gave its 6000 series a vector extension of 16 registers of 64 elements, supported by VMS but little used by Unix. x86 went from MMX through SSE, AVX and AVX2 (256-bit) to AVX-512, a new width and a new set of state each time; ARM has NEON in the processor and scalable vectors beyond, and attaches its neural-network and graphics engines as memory-mapped devices; RISC-V has a scalable vector extension. **Trade-offs.** A device needs a setup (a descriptor) and a completion (a status), so small arrays are faster on the processor and large ones on the device; a SIMD register file has no setup and a cost in state, opcodes and width that grows with every generation. **Improvements.** The decision removes the questions that a vector register file raises: whether a context switch saves the vector state (x86's lazy saving of floating-point state was a security hole and Linux went back to eager restore), whether the kernel may use the registers, how wide they are, and what the calling convention does with them. **Benefits.** A processor that stays small and in order and that gains graphics and inference by adding devices. **Modern comparables.** ARM's Ethos neural engines and Mali graphics engines, Apple's matrix coprocessor reached through a library, Google's TPU on PCI Express. **Impact.** *Kernel:* no vector state to save at a context switch and no rule about the kernel using vector registers; a device is one more slot, mapped like any other. *Compiler:* no auto-vectoriser targets the device; the library and the language runtime call it at the level of an operation on an array. *Hardware:* no second register file, no wide load and store path, no opcodes spent; the processor's cost does not grow with the workload.

8.2 Why not SIMD in the processor

8.2.1 A SIMD register file is a second wide register file, which the instruction set deliberately avoided (floating point shares the general registers), and it brings wide loads and stores entangled with the alignment rules and a large demand on a first-page opcode map that is 58 of 64 full. *(D)*

8.2.2 A fixed-width design (128-bit NEON, 256-bit AVX2) bakes the width into every instruction, so a wider generation needs new instructions, and old binaries cannot use the new width. *(D)*

8.2.3 A scalable design (ARM SVE, RISC-V V) avoids that and is the most complex to specify and to implement; it is the only in-processor form that would suit this project's range from a small FPGA core to a possible large one. *(P)*

8.2.4 The compiled corpus shows little vectorisable code at the size-optimising setting; whether it shows more at higher optimisation is a measurement to make before any in-processor form is reconsidered. *(P, appendix B, S1)*

Commentary. **Precedent.** x86's history of SSE, AVX, AVX2 and AVX-512 is the argument against a fixed width: each added instructions and state, and AVX-512 was withdrawn from some client chips and later proposed to be unified. **Trade-offs.** A scalable vector extension would let one binary run on a small and a large implementation, at the price of the largest specification in the architecture and a compiler model that must handle an unknown vector length. **Improvements.** Moving the work to a device makes the length a descriptor field, which is the scalable idea without a register file. **Benefits.** The block RAM of a small FPGA, the scarce resource of the plan, is not spent on vector registers. **Modern comparables.** AVX2 on x86, NEON on ARM, SVE and the RISC-V vector extension. **Impact.** *Kernel:* nothing to save and no lazy-save hazard. *Compiler:* no vector types in the ABI and no vectorisation pass for the processor. *Hardware:* no area for a register file or wide data paths; the FPGA's memory goes to the caches and the device buffers.

8.3 AI and large-language-model inference

8.3.1 The inference accelerator, LB64, evaluates the layers of a binary-weight language model as sign-flip accumulates with no multipliers, taking a chain of descriptors and writing a status back on completion. *(D, LB64 document)*

8.3.2 A layer is two descriptors, the second accumulating into the first's result for the residual path, and a transformer block is a chain; the processor writes the chain once and is then idle. *(D)*

8.3.3 The work is partitioned between small engines, each doing one thing with its own local memory: LB64 for the projections and the feed-forward layers, a floating-point multiply-accumulate engine for the attention products, a vector engine for normalisation, softmax, activation and rotary position encoding, and the processor for the descriptor chains, sampling and the tokeniser. *(P, LB64 document, section 6.4)*

8.3.4 The accelerator has no instruction set: its computation has exactly one shape, so a programmable sequencer would spend its area on flexibility the work does not use. *(D)*

8.3.5 The processor keeps `POPCNT` and `XNORPOP` for the binary operations that small models and bitsets use; they are not what the accelerator uses. *(D)*

Commentary. **Precedent.** A descriptor chain is how every competent DMA engine since the PDP-11's RK11 has worked, and the partition into small single-purpose engines fed by chains is the idea of the 144-processor GA144 chip applied to a workload that suits it. **Trade-offs.** The accelerator is fast at what it does and does nothing else: attention is not helped, the element-wise work needs another engine, and a training accelerator is a different design again. The accelerator's descriptor is larger than the 32-byte descriptor of chapter 5, so the two must be reconciled: the proposal is that the standard descriptor's address points at a device-defined work block, which can chain, and the ring carries pointers to work blocks. **Improvements.** The earlier rationale for `XNORPOP`, which was wrong about which workload it serves, was corrected in the instruction-set text. **Benefits.** The lane count is set by the memory bandwidth and not by the amount of logic, so a small board can run a model the processor alone could not. **Modern comparables.** ARM's Ethos engines, Google's TPU, and the many inference accelerators on small boards. **Impact.** *Kernel:* a driver builds a chain and starts it; the kernel owns no model state and the only scheduling is of the device. *Compiler:* none; inference frameworks emit descriptors. *Hardware:* adders and sign-flip logic and no multipliers in LB64; the multiply-accumulate engine needs multiplier blocks; the memory interface sets the size.

8.4 Array languages: APL and K

8.4.1 Array languages express computation as operations on whole arrays: each primitive (add, multiply, compare, reduce, scan, index, where, sort) is one operation over contiguous data. *(the nature of the languages)*

8.4.2 A device that takes a descriptor naming an operation, the addresses of the arrays and a length is the natural implementation of those primitives: one descriptor per primitive, the length in the descriptor. *(P)*

8.4.3 The interpreter stays on the processor and sends a primitive to the device when the array is long enough to repay the setup; short arrays run on the processor. *(P)*

8.4.4 There is no vector length in the language and none in a register: the length is a field of the descriptor, so one implementation serves any array size. *(P)*

8.4.5 A vector engine for such primitives would provide element-wise arithmetic and comparison, selection, reductions, scans, gather and scatter, compress (`where`) and index generation, on integer, floating-point and bit-array types. *(P)*

Commentary. **Precedent.** Iverson's APL made whole-array operations the language, and its descendants J and Arthur Whitney's K (the language of the kdb+ database) are interpreters whose speed comes from running long loops over contiguous arrays in a few tight kernels; the Cray vector machines were built around the same shape of loop, and DEC offered APL for the PDP-11 (not confirmed in the sources read). **Trade-offs.** Offloading a primitive has a fixed cost, so the benefit is for arrays above a size threshold; reductions and scans carry a dependence along the array that the device must support directly; sorting and gather are bandwidth-limited. **Improvements.** The language supplies the parallelism: the programmer says "add these two arrays", so there is no vectorising compiler to find it, which is the hardest part of making an ordinary language fast on SIMD hardware. The interpretive overhead is paid once per primitive, not once per element, and the device streams the data at memory bandwidth. **Benefits.** Vector languages are easier to support (a small interpreter with a library of primitives that map onto descriptors) and more efficient (the per-element work is hardware, and the per-primitive work is one descriptor), and the same device that serves the array languages serves the elementwise work of inference (section 7.4) and the lane-wise work of graphics. **Modern comparables.** K and kdb+ interpreters use AVX2 intrinsics heavily on x86 *(not confirmed in the sources read)*; array-language compilers for GPUs (Futhark, Co-dfns) target accelerators through offload, the same shape as this. **Impact.** *Kernel:* a language runtime that drives a device from user space needs the device's page mapped into its address space, with the protection caveat of chapter 5 (without a translation unit a user-space driver can direct its device to write anywhere). *Compiler:* none; the interpreter or compiler for the array language lowers primitives to descriptors, and a conventional compiler never sees a vector. *Hardware:* a streaming engine with a small set of element-wise, reduce, scan and gather units behind the ring engine, sized to the memory bandwidth.

8.5 Number formats

8.5.1 The processor handles `binary64`, `binary32` and `binary16` as scalar floating point; other formats (bfloat16, 8-bit integers with 32-bit accumulation, binary weights, 8-bit floating point) are the device's. *(D for the processor; P for the devices)*

8.5.2 Each vector device names its operations and formats in its identification block and feature bits, so a library can choose by asking. *(P, with the feature registers of appendix B)*

Commentary. **Trade-offs.** A format fixed in the processor's instruction set is a format that cannot change when the machine-learning field changes it, as it has several times; a format in a device is replaced by replacing the device. **Modern comparables.** x86 has half precision, bfloat16 and `VNNI` dot products in the instruction set; ARM has `BF16`, `SDOT` and matrix extensions; accelerators handle the rest. **Impact.** *Kernel:* none. *Compiler:* only the scalar formats; the machine-learning formats are descriptor fields. *Hardware:* the devices' multiply-accumulate units, not the processor's, are built for the formats in use.

8.6 Driving a vector device

8.6.1 A program fills a descriptor naming the operation and the arrays, executes `FENCE`, and writes the doorbell; the device processes the descriptor, writes the results and a status, and clears the ownership bit; the program polls or takes the interrupt. *(P)*

8.6.2 A chain of descriptors runs without the processor, and a work block that is a chain is how a transformer layer or a sequence of array primitives is expressed. *(P)*

8.6.3 The buffers are uncached, as for every device, so the processor's reads of results are slow; a large result is left in the device's memory or read back by the next descriptor. *(P)*

Commentary. This is the same driver skeleton as for a disk (chapter 5), with an operation field that means "add these arrays". **Trade-offs.** The slow uncached readback is the cost of the explicit hand-off; an interpreter that needs many small results back on the processor should leave them in device memory. **Impact.** *Kernel:* one driver skeleton. *Compiler:* none. *Hardware:* one ring engine shared with every device.

8.7 Still open

The reconciliation of the accelerators' descriptors with the standard descriptor, the formats and operation set of the vector engine, the target workloads, and the evidence that would reopen the question of packed integer lanes: compile the corpus

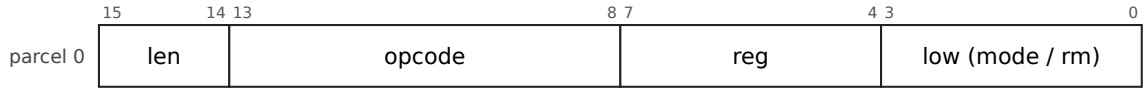
at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and estimate the block RAM cost (appendix B, S1 to S6).

Chapter 9 — The instruction set

The instruction set is built around one idea: the length of an instruction is known from its first two bits, so the cheapest instructions are two bytes and nothing is longer than six. This chapter first shows how the 64 opcodes are divided, then states the properties and principles that hold throughout, then the registers, the format and the addressing modes, then each group of instructions, then faults, restart and memory management, and ends with position-independent code, what was rejected, the idioms restored and lost, the comparison with the PDP-11 and VAX, what was taken from LoongArch, and what is still open. Every instruction has its entry (syntax, encoding drawing, operation, flags, exceptions) in the section of its group below; the atomic instructions are in section 7.12 and the system instructions in section 6.7; the reference tables hold the second page, the aliases, the constants and the lists of mnemonics and opcodes. The full text with every encoding is `../pdp64-isa.md`; every number here comes from the grammars that the tools are generated from.

9.1 How the opcode space is divided

9.1.1 Every instruction begins with a 16-bit parcel with four fields: `len` (2 bits), `opcode` (6 bits), `reg` (4 bits) and `mode` or `rm` (4 bits).



9.1.2 There are 64 opcodes on the first page. An operation gets a top-level opcode if its common case is register to register, because only then can it be two bytes; every other operation lives in an escape group and costs nothing for it.

9.1.3 Six escape groups, `0x0C` to `0x11`, read a sub-opcode from the second parcel: `BESC` (bits, fields, rotates, UTF-8), `MBLK` (block memory), `AESC` (divide, remainder, absolute, minimum, maximum, set), `MESC` (unaligned access, ALU to memory), `FESC` (fused multiply-add, binary32 and binary16, divide, square root, classify) and `ATOM` (atomics).

9.1.4 Three groups, `0x12` to `0x14`, take their sub-opcode from the register field: `XFER` (indirect jump and call, tag dispatch), `SOP` (single-operand instructions, jump, call, push, pop) and `SYS` (system control).

9.1.5 Every other run of opcodes is one kind, and its low bits select the operation directly: twelve moves (`0x00-0x0B`), thirteen ALU operations (`0x16-0x22`), four floating-point operations (`0x24-0x27`), three compare-and-branch forms (`0x28-0x2A`) and sixteen branches (`0x30-0x3F`).

9.1.6 Fifty-eight of the sixty-four opcodes are assigned, one ALU slot (`0x23`) is spare and five (`0x2B-0x2F`) are reserved.

9.1.7 When `len` is `11`, the instruction is a single parcel from a second page of 64 opcodes, 45 defined and 19 reserved, each an exact abbreviation of a first-page instruction.

Commentary. **Precedent.** The PDP-11 partitioned its opcodes by the addressing modes of the first word; the VAX by operand specifiers parsed in sequence; x86 by prefixes, opcode, ModR/M and SIB read one after another. Each step down that list costs more front-end logic before the address of the next instruction is known. **Trade-offs.** Putting an operation in an escape group makes its shortest form four bytes, since the sub-opcode lives in the second parcel, so the allocation is a frequency sort, and five groups were demoted for that reason (divide and remainder, rotate by a register count, floating-point divide and square root, and the read-modify-write memory forms) where the demotion cost nothing or almost nothing. **Improvements.** The six second-parcel escapes sit together so that "read a sub-opcode" is one range comparison on bits 13 to 8; the three register-field escapes sit together so that one range compare enables the mux that already feeds the register file's address inputs; the other runs are contiguous so their low bits are wired straight to the function selects. **Benefits.** The decoder is a few comparators and wires, and a new instruction usually costs a sub-opcode, not an opcode. **Modern comparables.** RISC-V groups its major opcodes by format and AArch64 by top-level encoding class; the pdp64 groups by the one property that matters most to its front end, whether the sub-opcode needs the second parcel. **Impact.** *Kernel:* Nothing in the kernel depends on the opcode map except the privileged group, which sits in one place. *Compiler:* The

instruction selector gets the two-byte form for every register-to-register operation and must charge four bytes for anything in an escape group, which its cost model has to know. *Hardware:* The decoder is a few range comparisons and wires: six escape groups are one compare on bits 13 to 8, three register-field escapes are one compare that enables a mux already feeding the register file, and every run's low bits go straight to its function select.

9.2 Properties and principles

9.2.1 The length of an instruction is a function of its `len` field alone: 0, 1 or 2 extension parcels, for 2, 4 or 6 bytes. *(D)*

9.2.2 At most one operand of an instruction may be a memory reference; the other, where present, is a register. The block operations are the one admitted exception. *(D)*

9.2.3 No instruction is longer than six bytes. *(D)*

9.2.4 A register is always written in full: narrow loads sign- or zero-extend, and there is no partial-register write. *(D)*

9.2.5 Compatibility is by conversion instructions, never by mode bits that change what other instructions mean. *(D, a strong default under errata 42)*

9.2.6 Every instruction can be restarted after a fault, and exceptions are precise; the machine has no hidden instruction state. *(D)*

9.2.7 Floating point is done in the general registers; there is no separate floating-point register file. *(D)*

9.2.8 Memory is sequentially consistent between processors. *(D)*

9.2.9 The ALU has one width, 64 bits; sizes appear only on loads, stores and conversions. *(D, errata 28)*

9.2.10 Addresses are canonical: 39 bits of virtual address, 40 bits of physical address. *(D)*

9.2.11 Instructions are aligned to two bytes; quad data accesses are naturally aligned, and the explicit unaligned forms exist for the cases that need them. *(D)*

9.2.12 An instruction does what its name says and nothing else; no flag is set that its entry does not list. *(D)*

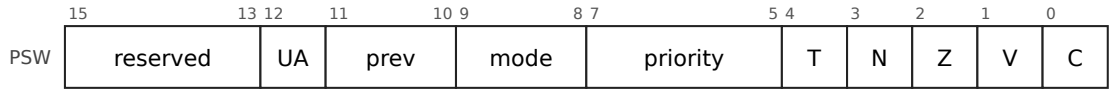
Commentary. **Precedent.** 9.2.2 is x86's ModR/M idea made architectural; it is also what lets the exception machinery be small (the PDP-11 recorded at most two register side effects per instruction because it had two specifiers; with one memory operand there is at most one). 9.2.6 exists because the Motorola 68000, which was very nearly this machine, could not restart an instruction after a bus error and so could not do demand paging until its successor retrofitted a fault model. 9.2.4 is the rule x86-64 adopted for 32-bit writes and failed to apply to the 8- and 16-bit forms. 9.2.7 is the arrangement RISC-V standardised as Zfinx. **Trade-offs.** 9.2.9 means narrow arithmetic is done wide and converted, which costs an extension instruction after some narrow operations; whether that cost is large enough to justify 32-bit forms of add and subtract is an open question that the compiled corpus can answer (appendix B). 9.2.8 is the cheapest model on a small in-order core and the most expensive on a large speculative one. 9.2.10 gives up the 48-bit space of x86-64 and AArch64 for a three-level page table. **Improvements.** The rules are ranked as defaults, not laws: each says what it protects and what breaking it costs. **Benefits.** Together they make the machine small, restartable and easy for a compiler; the measured code size is in chapter 1. **Modern comparables.** AArch64 and RISC-V have fixed-length register-to-register encodings and weak memory models; x86-64 has memory operands and total store order; the pdp64 takes memory operands, strong ordering and a bounded fault surface. **Impact.** *Kernel:* Restartable instructions and precise exceptions mean a page-fault handler repairs the mapping and returns, with no instruction-specific cases, and sequential consistency means no barrier code. *Compiler:* One memory operand keeps instruction selection and register allocation simple; a register written in full removes partial-register merges; the single ALU width costs an extension instruction after some narrow arithmetic. *Hardware:* One address generator, at most one register side effect, a single-entry undo buffer, no byte-lane multiplexing and a two-bit length are what keep the die small.

9.3 State: registers and flags

9.3.1 There are sixteen 64-bit general registers, `R0` to `R15`. `R14` is the stack pointer `SP`, with three copies (kernel, supervisor, user) chosen by the current mode; `R13` is the frame pointer by convention only; the others carry no meaning to the hardware. *(D)*

9.3.2 The program counter is not a general register; it is read with `LEA 0(PC), Rn`. *(D)*

9.3.3 The processor status word is 16 bits. *(D)*



(Bit positions as in System v1: `C` 0, `V` 1, `Z` 2, `N` 3, `T` 4, priority 5-7, current mode 8-9, previous mode 10-11, `UA` 12.)

9.3.4 The flags form two groups, `C` and `{N, Z, V}`, and no instruction writes part of `{N, Z, V}` or writes it conditionally. Moves, loads, stores and the compare-and-branch instructions write no flags; shifts write the same flags for every count; rotates and the funnel shift write none; the by-one left shift sets `V` to `N` xor `C`. *(D, errata 39 and the author's ruling)*

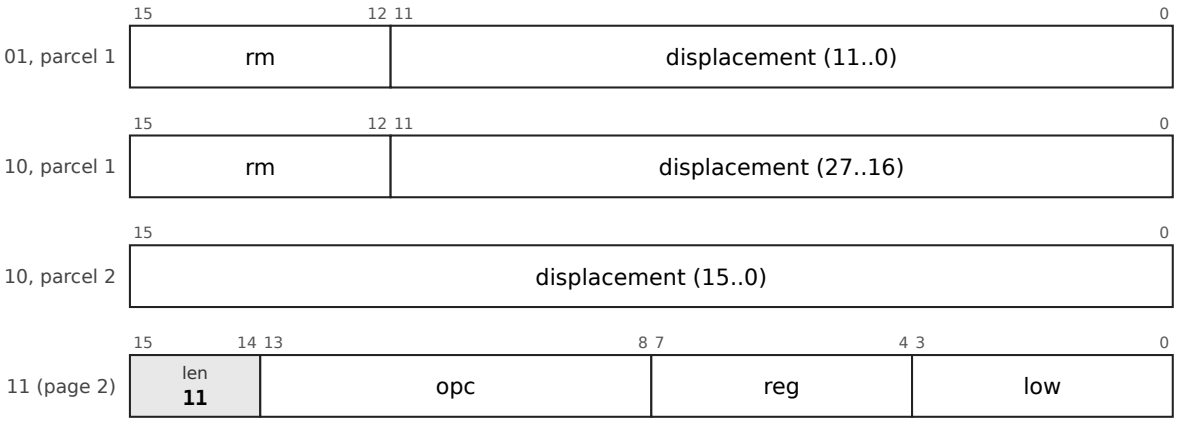
9.3.5 `DP`, the direct-page base, has three copies by mode. It makes `d(DP)` the shortest address form (a 12-bit displacement in four bytes); in kernel mode it is the per-CPU pointer, and in user mode the thread pointer. *(D)*

9.3.6 Other special registers: `FPSR` (five sticky floating-point flags), `ASID`, `PTBR`, `DMW0` and `DMW1`, `EPC`, `EPSW`, `EVA`, `ECAUSE`, `VBR`, `MMUCTL`; the system registers are read and written by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

Commentary. **Precedent.** The PDP-11's PC and SP were ordinary registers and its flags were set by nearly every instruction. The VAX put the PC in R15. **Trade-offs.** Keeping the PC out of the register file means a jump to a computed address uses `JMP`, not a write to a register, so that writing `SP` has no control-flow consequence and the hazard logic treats it like any other destination. Condition codes were kept (rather than a flagless design) because compare-and-branch and flag-setting arithmetic make for dense code; they were made cheap for a future out-of-order implementation by errata 39, because the compiled corpus showed that 68% of instructions wrote a flag and only 1.5% read one. **Improvements.** The status word carries a user-memory guard bit (9.14) and the per-mode `DP` has found two jobs nobody designed it for. **Benefits.** An exception handler finds a kernel stack and a per-CPU pointer without first searching for them; a threaded program's thread-local variable is one instruction. **Modern comparables.** x86-64 keeps `FS` and `GS` base registers for the same two jobs; AArch64 has `TPIDR_EL0` and `TPIDR_EL1`; RISC-V uses `tp` and the supervisor scratch register. **Impact.** *Kernel:* Banked stack and direct-page registers let entry find a kernel stack and a per-CPU pointer with no code, and the status word is saved by one `PUSHX`. *Compiler:* The flag rules decide when a compare may fuse with its branch (1,893 pairs stay separate because a second branch reads the flags) and what the compiler may move between a compare and a branch. *Hardware:* Two flag groups need one or two rename targets, not one per bit; the register file is 16 by 64 with two read ports and one write (about 250 lookup tables of distributed memory); the banked copies are a three-way mux.

9.4 The instruction format

9.4.1 The first parcel holds `len`, the opcode and two four-bit fields; `len` is the number of extension parcels (0, 1 or 2) when it is less than 3. *(D)*



9.4.2 Instructions are parcel-aligned and little-endian; an instruction may cross any boundary of 8 bytes, a cache line or a page. *(D)*

9.4.3 Most instructions are two or four bytes: register-to-register work and the whole conversion set is one parcel; any memory operand within ±2 KB is two; displacements to ±128 MB, 32-bit immediates and branches to ±4 GB are three. *(D)*

9.4.4 Where an addressing mode names no base register (immediate, absolute, PC-relative), the register nibble of the second parcel carries payload, so those modes reach 16 bits at four bytes and 32 bits at six. *(D)*

9.4.5 A constant wider than 32 bits is loaded from a literal pool with `MOVQ =K, Rd`, written with an explicit `=`; nothing is pooled silently. *(D, errata 33)*

Commentary. **Precedent.** The PDP-11 had the length in the first word, but as a function of both of its six-bit specifiers; UTF-8 made "the leading unit says how long" famous, but what matters here is that length is the *primary* partition, so the payload is budgeted by what each form needs and not carved from a fixed width. **Trade-offs.** Byte-granular encoding, as in x86, buys perhaps a tenth of the code size and costs a full alignment rotator and cross-line splice logic in the fetch path on a 64-bit datapath; the pdp64 refuses it. Forcing an instruction to stay inside an eight-byte window would simplify fetch and would cost 5% to 10% of code size in padding. **Improvements.** The fourth parcel of earlier revisions was removed (errata 31) and its code now selects a second page of one-parcel abbreviations; the measured average is near three bytes per instruction. **Benefits.** The front end is a two-bit counter, a two-deep parcel buffer and no feedback. **Modern comparables.** Thumb-2 and RISC-V's compressed instructions mix 16- and 32-bit forms too, with the length read from the low bits. **Impact.** *Kernel:* Page-2 forms make kernel text smaller with no source change. *Compiler:* The assembler picks the shortest form, branch relaxation is just a length choice, and the cost model knows only 2, 4 and 6 bytes. *Hardware:* The fetch unit is a two-bit down-counter and a two-deep parcel buffer with no feedback.

9.5 Addressing modes

9.5.1 Four bits select one of eight modes, present at four bytes and longer. *(D)*

Mode	Syntax	Effect
`0000`	`d(Rn)`	base plus displacement
`0001`	`d(Rn)(Rx*s)`	scaled index, `s` in {1, 2, 4, 8}
`0010`	`d(PC)`	PC-relative; the base is the first parcel of the instruction
`0011`	`d(DP)`	direct page
`0100`	`#imm`	immediate, sign-extended
`0101`	`@abs`	absolute, sign-extended
`0110`	`(Rn)+d`	post-increment by `d` (default: the operand size)
`0111`	`-(Rn)` or `(Rn)-d`	pre-decrement by the operand size, or by `d`

9.5.2 The deferred modes of the PDP-11 are removed; indirection is done with `JMPI`, `CALLI` and a load. *(D)*

9.5.3 At most one register side effect occurs per instruction, and it is committed at retirement; a fault discards it. *(D)*

Commentary. **Precedent.** The PDP-11 had eight modes for each of eight registers, with a deferred flavour of each, and it made `*p++` and `*--p` single instructions. **Trade-offs.** Dropping the deferred modes removes the case where a fault occurs after a register has been updated at an address that itself came from a load; keeping the strided autoincrement costs nothing, because the undo information was already a delta (the PDP-11's `MMR1` recorded an amount for the same reason). **Improvements.** The increment is a stride, so walking an array of 16-byte records is one instruction; the PC-relative modes count from the instruction itself, so a branch needs no adjustment and `LEA` reads the PC. **Benefits.** Address generation is one adder pass, never load-then-add, which is the difference between one cycle and a long path on an FPGA. **Modern comparables.** AArch64 has pre- and post-indexed forms with an immediate; x86 has scaled index with a base; RISC-V has base plus immediate only. **Impact.** *Kernel:* A strided autoincrement walks a table of records, and `d(DP)` reaches the per-CPU structure in one instruction. *Compiler:* The modes match C idioms (`*p++`, `a[i]`) one-to-one, and dropping the deferred modes removes the cases the selector would otherwise have to consider. *Hardware:* Address generation is one adder pass, never load-then-add, and the PC-relative base is the first parcel, so no adjustment is needed.

9.6 Moves, conversions and address formation

The instructions. Every instruction has an entry in the section of its group, laid out as in a processor reference manual: its mnemonics, its syntax, its encoding (a drawing of the parcels and the numbers), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the book explains the matter it touches (*Cf.*). The atomic instructions are in section 7.12 and the system instructions in section 6.7, where their subject is. The entries are generated from `spec/pdp64-mnemonics.hbnf`, the one place where the numbers of the architecture are written, so an encoding here cannot disagree with the assembler or the simulators; the words (operation, flags, exceptions) were checked against the reference executor, `common/`

pdp64exec.c, which sim64 and QEMU share, and an entry says when the executor does not implement an instruction yet. The one-parcel second page, the aliases, the constants and the lists of the mnemonics and the opcodes are in the reference tables.

The stories. An instruction that is not common (the PDP-11 core and the operations every architecture has carry none) is followed by *Why it is here*: the record of why it was chosen, from the author's questions in the original design chat (chat01.txt), the author's messages in the archive of the working session (dates are those of the archive, in UTC), the revision changelogs of the instruction-set text, the errata and the measurements. Where the record gives no reason, the paragraph says so.

Notation. Rd is the destination register (the reg field of parcel 0), Rs the first source (low), Rt the second (t of parcel 1), Ru the third (x); the assembler writes the destination last. rm is a general operand: a register at len = 00, or a memory operand, an immediate or an absolute address by one of the eight addressing modes (section 9.5). EA(rm) is its effective address. M[a] is memory, M8[a] one byte. sext and zext sign- and zero-extend. A len = 01 instruction is two parcels (four bytes) and a len = 10 instruction three (six bytes); len = 11 is page 2. N Z V C are the condition flags. In the drawings the bits of a parcel are numbered from 15 at the left, a shaded box is a fixed value, and a white box is a field that varies.

Cf. section 6.4 for the memory types that decide whether an access is cached, and section 6.8 for what a faulting load leaves behind.

9.6.1 MOVQ, MOVL, MOVW and MOVB load and store 64, 32, 16 and 8 bits; the loads sign-extend. MOVZL, MOVZW and MOVZB load with zero extension. *(opcodes 0x00-0x0A)*

9.6.2 With a register operand, the same opcodes are the conversion set: MOVB R1, R2 sign-extends the low byte, MOVZW R1, R2 zero-extends the low word.

9.6.3 LEA forms an effective address; LEA 0(PC), Rn reads the program counter. *(opcode 0x0B)*

9.6.4 Moves, loads and stores write no flags. *(D, errata 39)*

9.6.5 The explicit unaligned forms MOVUB, MOVUW, MOVUL and MOVUQ exist in both directions, in the memory escape. *(D)*

9.6.6 Page 2 abbreviates the common cases to one parcel: loads and stores through (rn), loads and stores at small offsets from SP, LEA from SP, and MOVQ #imm, rd with a four-bit immediate. *(D)*

Commentary. **Precedent.** The PDP-11 had MOV and MOVB with a byte sign-extended into a register; the VAX had a sized MOV for every width. **Trade-offs.** Keeping the sized loads costs four extra opcodes against a quad-only design, but without MOVL a struct field load becomes two MOVWs, a shift and an or, and ELF, timestamps and most on-disk and wire structures are full of 32- and 16-bit fields. **Improvements.** There is no separate CVT group: the conversions are the moves with a register source, so they cost nothing. **Benefits.** No partial-register state anywhere. **Modern comparables.** AArch64 and RISC-V have the same sized loads with sign and zero extension; x86-64 keeps 8- and 16-bit register writes that preserve the upper bits, which makes narrow writes depend on the old value. **Use.** LEA 0(PC) is how a position-independent program finds itself; the unaligned forms are for packed on-disk and network structures. **Impact.** *Kernel:* LEA 0(PC) lets position-independent kernel code find itself; the unaligned forms read packed on-disk and network structures. *Compiler:* Sized loads avoid the shift, mask and or of a struct-field read, and the conversions are moves, so no separate conversion selection exists. *Hardware:* No partial-register write anywhere; the conversions reuse the load-extend path.

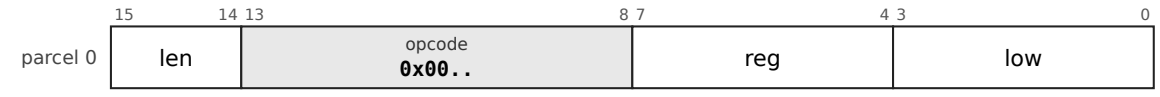
MOV: Move, load and store (64, 32, 16 and 8 bits)

Mnemonics: MOVQ, MOVL, MOVW, MOVB

Syntax

```
MOVQ rm, Rd        ; load
MOVQ Rs, rm        ; store
MOVL, MOVW, MOVB likewise
```

Encoding



reg is the register Rd (the source of a store); low is a second register (Rs) at len = 00, where the instruction is two bytes. At len = 01 or 10, low is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Mnemonic	Opcode (the load or direct form)
MOVQ	0x00
MOVL	0x02
MOVW	0x04
MOVB	0x06

The store form of each move has the opcode of its load form plus 1: MOVQ 0x01, MOVL 0x03, MOVW 0x05, MOVB 0x07.

Operation

```
size = 8 (MOVQ), 4 (MOVL), 2 (MOVW) or 1 (MOVB) bytes
load :  Rd <- sext(M[EA(rm)], size)          ; len = 00: Rd <- sext(Rs[8*size-1:0])
store:  M[EA(rm)] <- Rs[8*size-1:0]          ; len = 00: Rd <- sext(Rs[8*size-1:0])
```

Flags None written. (Errata 39: moves, loads and stores write no flags.)

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A register is always written in full: a narrow load sign-extends to 64 bits and there is no partial-register write anywhere. At len = 00 the same opcodes are the conversion set: MOVB R1, R2 sign-extends the low byte of R1 into R2. Page 2 abbreviates the loads and stores through (rn), at small offsets from SP and from a register, and MOVQ #imm, rd with a 4-bit immediate (see the page-2 table).

Cf. 6.4 for the memory type that decides whether the access is cached; 6.8 for what a faulting access leaves behind.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

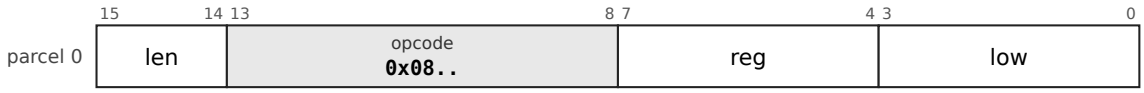
MOVZ: Move zero-extended (32, 16 and 8 bits)

Mnemonics: MOVZL, MOVZW, MOVZB

Syntax

```
MOVZL rm, Rd
MOVZW rm, Rd
MOVZB rm, Rd
```

Encoding



reg is the register Rd (the source of a store); low is a second register (Rs) at len = 00, where the instruction is two bytes. At len = 01 or 10, low is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Mnemonic	Opcode (the load or direct form)
MOVZL	0x08
MOVZW	0x09
MOVZB	0x0A

Operation

```
Rd <- zext(M[EA(rm)], size)      ; len = 00: Rd <- zext(Rs[8*size-1:0])
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

There is no zero-extending 64-bit move (it would be MOVQ). At len = 00 these are the zero-extending half of the conversion set.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

LEA: Load effective address

Mnemonics: LEA

Syntax

```
LEA d(Rn), Rd
LEA d(PC), Rd
LEA d(Rn)(Rx*s), Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- EA(rm)          ; no memory access
```

Flags None written.

Exceptions The operand must name memory; `len = 00` (a register) and the immediate mode are reserved. No memory access is made, so the instruction takes no memory fault. A reserved encoding aborts with the illegal-instruction vector.

`LEA 0(PC), Rd` is how the program counter is read: it puts the address of the `LEA` itself in `Rd`. At `len = 10` it forms any address within ±2 GB of itself PC-relatively in six bytes. The base of every PC-relative displacement is the address of the first parcel of the instruction.

Why it is here. The design chat opened with the questions "why is SP in the register file but not PC?", "what is the problem with letting programmers read and modify PC directly?" and "what if R7 was a read-only view of the PC?". The answer (ISA revision 13, section 9.3 of the book) was to keep the program counter out of the register file, because every register write would then be a potential branch, and to refuse the read-only view, which would fix for ever *which* PC is meant (ARM32 is stuck with `PC+8`). `LEA 0(PC), Rn` gives the capability without the commitment: the address of the instruction itself, in one instruction. The same instruction forms any address PC-relatively, which is what makes position-independent code free (section 9.16).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MOVU: Unaligned load and store

Mnemonics: `MOVUB`, `MOVUW`, `MOVUL`, `MOVUQ`

Syntax

```
MOVUQ rm, Rd        ; load, any alignment
MOVUQ Rs, rm        ; store, any alignment
MOVUW, MOVUL, MOVUB likewise
```

Encoding



Parcel 0's `low` is an addressing mode, as in the ordinary memory format; parcel 1 is `sub(4) rm(4) disp(8)` (16 more displacement bits per further parcel).

Mnemonic	Sub-opcode
`MOVUB`	`0x0`
`MOVUW`	`0x2`

Mnemonic	Sub-opcode
MOVUL	0x4
MOVUQ	0x6

Operation

```
size = 1, 2, 4 or 8 bytes
load :  Rd <- sext(M[EA(rm)], size)           ; the access may cross an alignment boundary
store:  M[EA(rm)] <- Rs[8*size-1:0]
```

Flags None written.

Exceptions Page and protection faults and bus errors as for any memory operand; **no alignment fault**.

The ordinary moves fault on a misaligned address (the alignment fault is the tag check of section 9.5); these are the explicit forms for data that is not aligned, in MESC.

Why it is here. Data alignment is architectural on this machine: an ordinary quad access to an address that is not a multiple of 8 aborts, and that abort doubles as the tag check for tagged pointers (ISA sections 6.2 and 7.4; the rejection of "a load form that ignores low pointer bits" in section 11 gives the reason). Packed on-disk and network structures still need unaligned access, so MESC carries explicit unaligned forms rather than weakening the rule for every load. The record gives no further history for these four.

Cf. 6.4, 6.8.

Status specified; **not yet implemented** in the reference executor.

9.7 Integer arithmetic

Cf. section 6.2 for the divide-by-zero vector (DIV, DIVU, MOD, MODU).

9.7.1 ADD, SUB, CMP, MUL and MULH are two bytes with register operands; MULHU (high half of an unsigned product), DIV, DIVU, MOD and MODU are in the arithmetic escape. *(D)*

9.7.2 CMP is written `CMP Rd, rm`, register first, and computes `Rd - rm`. *(D, errata 13)*

9.7.3 ADD sets C on a carry; SUB and CMP set C on a borrow, the PDP-11 sense, so BHIS and BCC mean "no borrow", unsigned greater or equal. *(D, errata 14)*

9.7.4 ADC and SBC add and subtract the carry; INC, DEC, NEG, CLR and TST are single-operand forms that work on a register or memory. *(opcode group SOP)*

9.7.5 ABS, MIN, MAX, MINU and MAXU, and `SET cc, Rd` (write 1 if the condition holds, else 0), are in the arithmetic escape. *(D, errata 23 and 25)*

Commentary. **Precedent.** ADD, SUB, CMP, BIT, BIS, BIC and the single-operand set are the PDP-11's, unchanged in name and meaning except for the order of CMP's operands. **Trade-offs.** The PDP-11 wrote `CMP a, b` to compute `a - b`; since the destination of an ALU operation here must be a register, a literal `src, dst` reading would force every comparison against an immediate or memory to be reversed, so the register comes first. Divide was demoted to four bytes because compilers turn division by a constant into MULH plus a shift, so a real divide is rare. **Improvements.** The high-half multiply that bignum code needs was added; a second unsigned multiply, which turned out to be the same instruction as the first, was not. **Benefits.** SET replaces a branch around a move for the very common "store the truth value of a comparison", and absolute value, minimum and maximum remove a compare-and-branch from small loops. **Modern comparables.** RISC-V has MULH, MULHU and MULHSU, and AArch64 SMULH and UMULH; x86 has a one-operand multiply that produces both halves; AArch64's conditional select and set instructions are the nearest to SET. **Use.** MULH and MULHU are for multi-precision arithmetic and for division by constants; ADC and SBC carry a multi-word add or subtract; MIN, MAX and ABS appear under a tenth of a percent of compiled code but each replaces a branch. **Impact.** *Kernel:* ADC and SBC carry multiword arithmetic for the kernel's 128-bit counters and for the bignum code in the base system. *Compiler:* MULH turns division by a constant into a multiply and a shift; SET materialises a comparison without a branch; CMP takes the register first, as the compiler orders its operands. *Hardware:* One adder on the carry chain; on an FPGA the multiplier is built from two to four multiplier blocks over four to eight cycles; the divide is a sequencer.

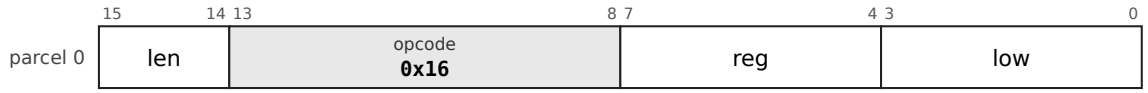
ADD: Add

Mnemonics: ADD

Syntax

```
ADD rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- Rd + rm
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a carry out of bit 63.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

All ALU operations are 64 bits, with no size field. Page 2 has `ADD #imm, rd` with a 4-bit immediate and `ADD #(8*n), SP`.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

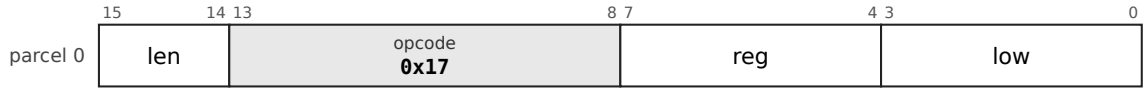
SUB: Subtract

Mnemonics: `SUB`

Syntax

```
SUB rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- Rd - rm
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Page 2 has `SUB #(8*n), SP`.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

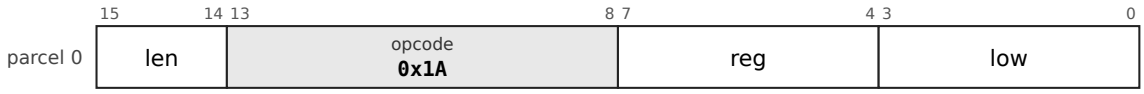
CMP: Compare

Mnemonics: `CMP`

Syntax

```
CMP Rd, rm        ; register first
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
tmp <- Rd - rm         ; the result is discarded
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The operands are in the order register, then the other operand, the reverse of the PDP-11 `CMP src,dst`: `CMP Rd, rm` computes `Rd - rm`, so a compare of a register against an immediate or memory operand does not reverse its branch condition. After `CMP`, `BHIS`/`BCC` mean unsigned `>=` (no borrow) and `BLO`/`BCS` mean a borrow occurred.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

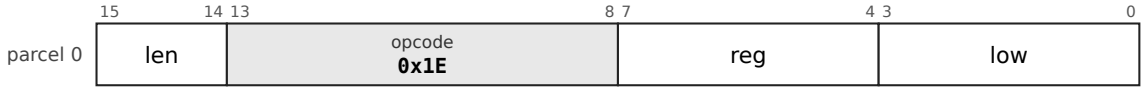
MUL: Multiply, low 64 bits

Mnemonics: `MUL` (aliases: `MULU`)

Syntax

```
MUL rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- (Rd * rm)[63:0]         ; sign-agnostic
```

Flags `N` and `Z` from the result, `V` set when the signed 128-bit product does not fit in 64 bits, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The low 64 bits of a two's-complement product are the same whether the operands are read as signed or unsigned, so there is one low-half multiply. The old mnemonic `MULU` is an alias of `MUL`. The high half is `MULH` (signed) and `MULHU` (unsigned).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

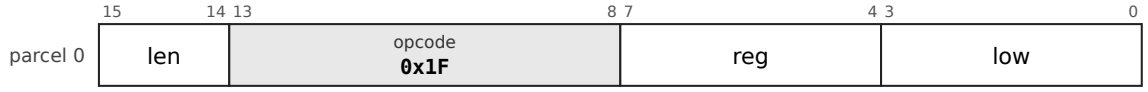
MULH: Multiply, high 64 bits, signed

Mnemonics: `MULH`

Syntax

```
MULH rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- (sext(Rd) * sext(rm))[127:64]
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With `MULH`, division by a constant is a multiply by a reciprocal and a shift, which is what compilers emit.

Why it is here. ISA revision 17 pruned instructions that were carried for continuity and found one duplicate hiding a genuine hole: `MUL` and `MULU` were the same instruction, because the low 64 bits of a two's-complement product are identical signed or unsigned, so two ALU slots held one operation, while nothing in the architecture produced the *high* half. Without it a 64 by 64 to 128-bit multiply, every bignum multiply, 128-bit fixed point and divide-by-a-constant by multiplying by the reciprocal were impossible. `MULH` took the duplicate's slot (ISA revision 17, section 4.2.2). It is also why a divide is rare in compiled code: compilers turn division by a constant into `MULH` and a shift.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

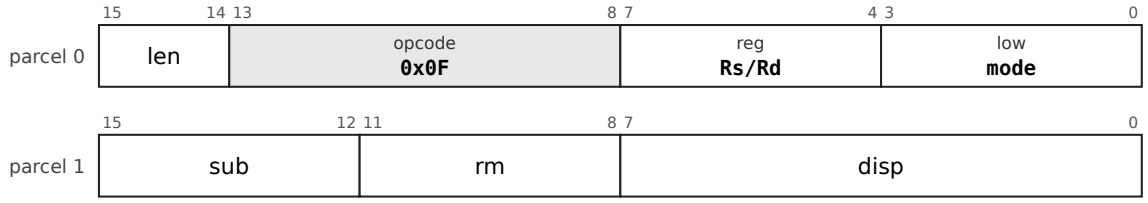
ADDM: ALU operation with a memory destination

Mnemonics: `ADDM`, `SUBM`, `BICM`, `BISM`

Syntax

```
ADDM Rs, rm
SUBM Rs, rm
BICM Rs, rm
BISM Rs, rm
```

Encoding



Parcel 0's `low` is an addressing mode, as in the ordinary memory format; parcel 1 is `sub(4) rm(4) disp(8)` (16 more displacement bits per further parcel).

Mnemonic	Sub-opcode
`ADDM`	`0x8`
`SUBM`	`0x9`
`BICM`	`0xA`
`BISM`	`0xB`

Operation

```
ADDM: M[EA] <- M[EA] + Rs
SUBM: M[EA] <- M[EA] - Rs
BICM: M[EA] <- M[EA] AND NOT Rs
BISM: M[EA] <- M[EA] OR Rs
```

Flags As for ADD, SUB, BIC and BIS.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Write permission is checked during the read access, so the write cannot fault after the read has succeeded.

Parcel 0's low nibble is a mode, as in the ordinary memory format, so these are MESC forms with a memory destination (`len >= 01`). INC, DEC, NEG, COM and CLR on a memory operand are in SOP.

Why it is here. Through ISA revision 19 these were top-level opcodes `0x20` to `0x23`. Revision 20 reallocated all sixty-four opcodes by one rule (a top-level opcode is for an operation whose common case is register to register) and moved them to MESC at no cost, because an instruction with a memory destination is at least four bytes anyway; that freed four top-level slots. They cover the case where the second operand is a register, for example `BISM Rmask, flags(DP)` (ISA section 4.13). The measurement of 2026-10-22 found 265 sites in the compiled corpus (1.6 KB), so they earn their place in the escape group but not in the opcode space (this chat, the instruction-set analysis).

Status specified; **not yet implemented** in the reference executor.

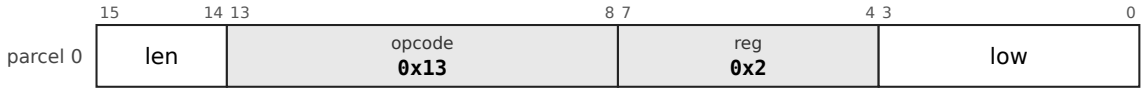
INC: Increment

Mnemonics: INC

Syntax

```
INC rm
```

Encoding



The reg field is the sub-opcode; rm and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- rm + 1
```

Flags N, Z and V as for ADD; C **unchanged**.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

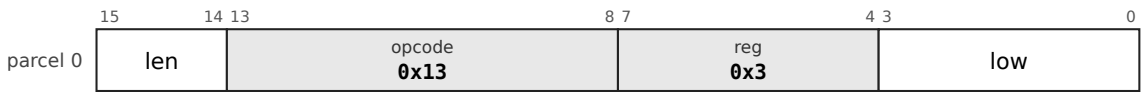
DEC: Decrement

Mnemonics: DEC

Syntax

```
DEC rm
```

Encoding



The reg field is the sub-opcode; rm and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- rm - 1
```

Flags N, Z and V as for SUB; C **unchanged**.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

NEG: Negate

Mnemonics: NEG

Syntax

```
NEG rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- 0 - rm
```

Flags N and Z from the result, V set on signed overflow, C set on a **borrow** (the PDP-11 sense): C = 1 when the unsigned operand being subtracted is the larger. (C = 1 unless the operand was 0.)

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

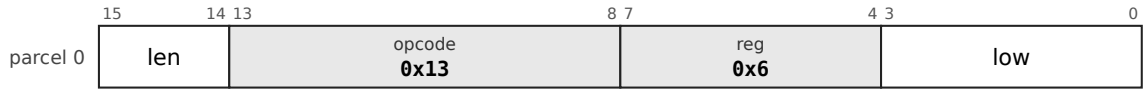
ADC: Add carry

Mnemonics: ADC

Syntax

```
ADC rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- rm + C
```

Flags N and Z from the result, V set on signed overflow, C set on a carry out of bit 63.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With ADD, multi-word addition.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

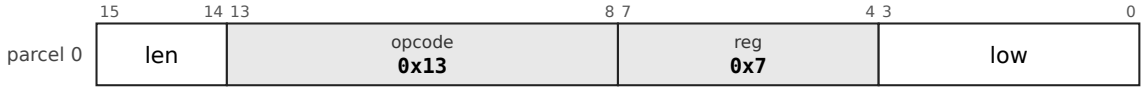
SBC: Subtract carry

Mnemonics: SBC

Syntax

```
SBC rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- rm - C
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

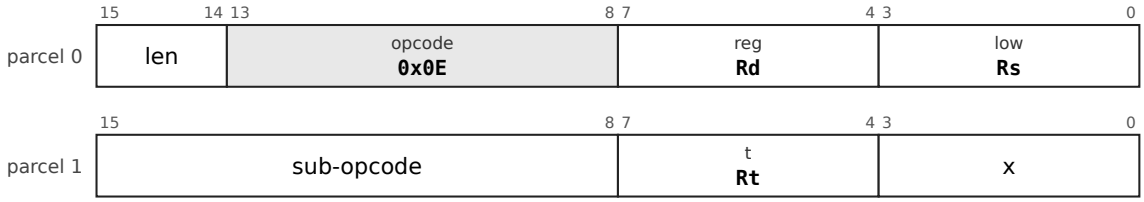
DIV: Divide and remainder, signed and unsigned

Mnemonics: `DIV`, `DIVU`, `MOD`, `MODU`

Syntax

```
DIV Ra, Rb, Rd
DIVU Ra, Rb, Rd
MOD Ra, Rb, Rd
MODU Ra, Rb, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Sub-opcode
`DIV`	`0x00`
`DIVU`	`0x01`
`MOD`	`0x02`
`MODU`	`0x03`

Operation

```
if Rb = 0 then divide-by-zero abort
DIV : Rd <- Ra / Rb  (signed, rounded toward zero; INT_MIN / -1 = INT_MIN)
DIVU: Rd <- Ra / Rb  (unsigned)
MOD : Rd <- Ra mod Rb (signed, the sign of Ra; any value mod -1 = 0)
MODU: Rd <- Ra mod Rb (unsigned)
```

Flags None written.

Exceptions Division by zero aborts with the divide-by-zero vector (cause `DIVZERO`). There is no overflow trap: `INT_MIN / -1` returns `INT_MIN`.

Division by a constant is a MULH and a shift, so a real divide is rare; this is why divide has no two-byte form.

Cf. 6.2 (the divide-by-zero vector).

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

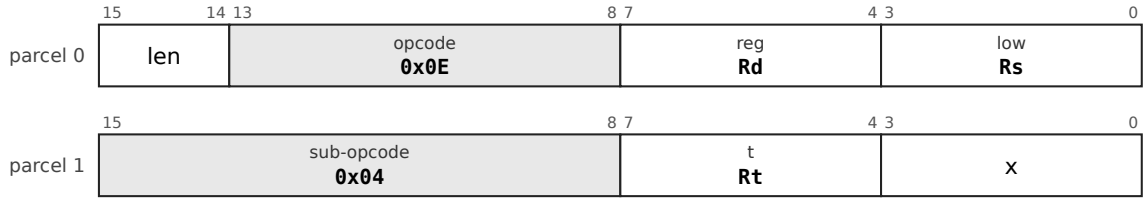
MULHU: Multiply, high 64 bits, unsigned

Mnemonics: MULHU

Syntax

```
MULHU Ra, Rb, Rd
```

Encoding



From len = 01. In this group low is a register, never a mode.

Operation

```
Rd <- (Ra * Rb)[127:64]    ; unsigned
```

Flags None written.

Exceptions None.

MULHSU (signed by unsigned) is reserved and not committed.

Why it is here. The unsigned partner of MULH, added in ISA revision 17 for the same reason (the high half of the product was missing). It went to BESC first and moved to AESC in revision 20, where arithmetic belongs. MULHSU, signed times unsigned for a bignum's top limb, was left reserved because it can be built from MULH with one correction.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

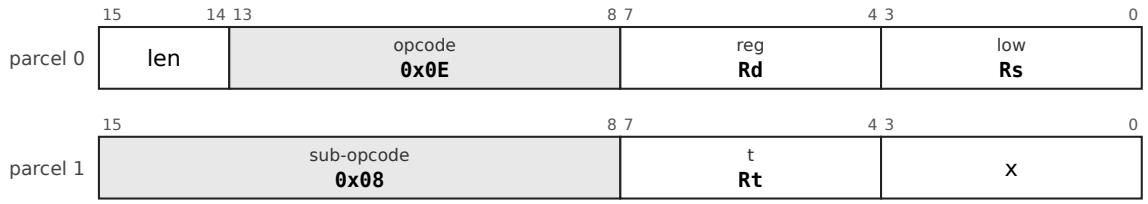
ABS: Absolute value

Mnemonics: ABS

Syntax

```
ABS Rs, Rd
```

Encoding



From len = 01. In this group low is a register, never a mode.

Operation

```
Rd <- |Rs|
```

Flags N and Z from the result, V set if Rs is the most negative value (the result is then still negative), C unchanged.

Exceptions None.

Why it is here. The author asked on 2026-10-21: "do we have abs opcode?" There was none for integers, only FABS. After the cost was worked out (about 150 to 250 lookup tables, no new operation by the folding rule, the compiler recognising `x < 0 ? -x : x` being the real cost) the author said "add them, and if we have to teach the front end about abs/min/max nodes that is fine". The measurement that followed found 18 uses of abs and 5 of its unsigned form in the compiled corpus, under 0.1% of code, so the case is clarity and the idiom more than size (errata 23; CHANGES section 14.4).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

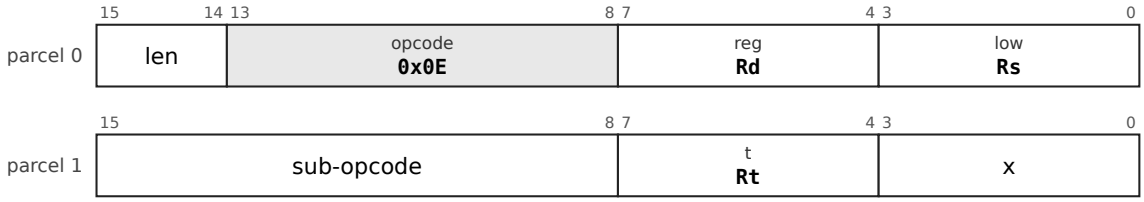
MINMAX: Minimum and maximum, signed and unsigned

Mnemonics: MIN, MINU, MAX, MAXU

Syntax

```
MIN Ra, Rb, Rd
MINU Ra, Rb, Rd
MAX Ra, Rb, Rd
MAXU Ra, Rb, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Sub-opcode
MIN	0x10
MINU	0x11
MAX	0x12
MAXU	0x13

Operation

```
MIN : Rd <- smaller of Ra, Rb (signed)        MINU: unsigned
MAX : Rd <- larger of Ra, Rb (signed)         MAXU: unsigned
```

Flags N and Z from the result, V = 0, C unchanged.

Exceptions None.

Why it is here. Asked for together with ABS on 2026-10-21 ("what would be the cost of adding abs/min/max for ints?"; "add them"), as the integer counterparts of FMIN and FMAX. By the folding rule (integer against float and signed against unsigned are qualifiers) they add no operation, and the hardware reuses the subtract-and-compare that CMP already does plus a 64-bit select. The corpus has 171 uses of MIN and 52 of MAX across 95 files, under 0.1% of code in all (errata 23; CHANGES section 14.4). The alternative the assistant raised, a general conditional select, was left on hold for want of a speed model.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

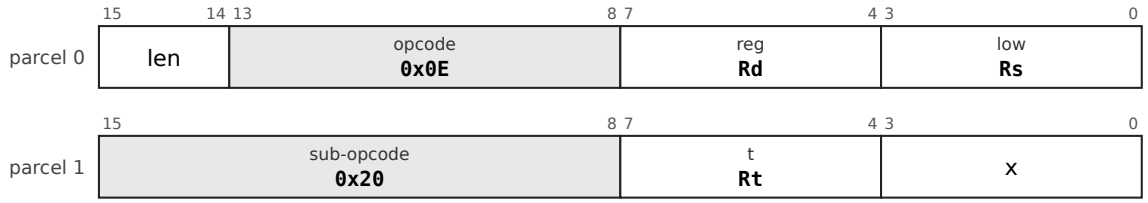
SET: Set a register from a condition

Mnemonics: SET

Syntax

```
SET cc, Rd        ; for example SET EQ, R1
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- 1 if condition cc holds on the current flags, else 0
```

Flags None written.

Exceptions None.

The condition is the `x` field of parcel 1, numbered as the low nibble of the branch opcodes (`al` 0, `ne` 1, `eq` 2, ... `los` 14).

Why it is here. The first instruction written with a condition word: the author's questions on 2026-10-21 were "why do them as single opcodes instead of as an opcode that takes a flag?" and "set eq instead of seteq", which fixed the form as one instruction, `SET cc, Rd`, with the condition in a field numbered like the low nibble of the branch opcodes (errata 25). It turns a comparison used as a value into 0 or 1 without a branch. gcc `-Os` for amd64 emits 673 `set<cc>` on the corpus, and `SET` would remove about six bytes per site, around 0.4% of code; the compiler cannot yet reach it because pass 1 turns such a comparison into branches before pass 2 sees it (CHANGES section 14.4).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

9.8 Logic, shifts and rotates

9.8.1 `AND`, `BIC` (bit clear), `BIS` (bit set, the or), `XOR` and `BIT` (test) are two-byte ALU operations; `COM` is one's complement. *(opcodes `0x18`, `0x19`, `0x1B`, `0x1C`, `0x1D`)*

9.8.2 `SHL`, `SHR` and `SAR` shift by a count taken from `rm` (a register, an immediate, or memory); `ASL` and `ASR` are the by-one forms in two bytes. *(D)*

9.8.3 A shift count of 64 or more shifts the value entirely away; `C` receives the last bit that left. *(D)*

9.8.4 `ROR` rotates right by a register count and `RORI` by an immediate; `ROL Rd, Rs, #n` assembles to `RORI` with `64 - n`; `RORI32` rotates the low 32 bits and sign-extends. Rotates write no flags. *(D)*

9.8.5 `DSHR` is a double-width funnel shift. It writes no flags. *(D)*

Commentary. **Precedent.** The PDP-11 had `ASL`, `ASR`, `ROL` and `ROR` by one through the carry; x86's `SHRD` and ARM's `EXTR` are the funnel shifts. **Trade-offs.** One rotate direction is enough, so the by-one `ROL` was removed from its two-byte slot; rotates are not through the carry because a carry chain is fragile when the carry flag is shared. **Improvements.** The shift count comes from `rm`, not from a fixed register like x86's `CL`, which costs a register shuffle in every hand-written loop. **Benefits.** Multi-precision shifting is `DSHR`, two instructions for any distance. **Modern comparables.** RISC-V and AArch64 have rotates by register and by immediate; x86 has `ROL`/`ROR` through the carry as well. **Use.** `RORI` and `RORI32` are for hashes and ciphers, which rotate by constants; `DSHR` shifts a bignum. **Impact.** *Kernel:* Bit tests and mask operations on the status and page-table words are single instructions. *Compiler:* The shift count may be any register, immediate or memory operand, so the compiler never has to move a count into a fixed register; hashes and ciphers rotate by constants. *Hardware:* The barrel shifter (about 450 lookup tables) serves shifts and rotates, and writing no flag for rotates removes a result path.

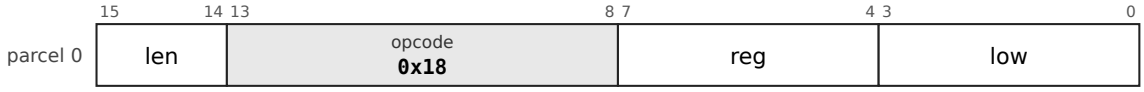
BIC: Bit clear

Mnemonics: `BIC`

Syntax

```
BIC rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- Rd AND NOT rm
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The assembler also accepts `BIC #imm, rm` as `AND #~imm, rm`.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

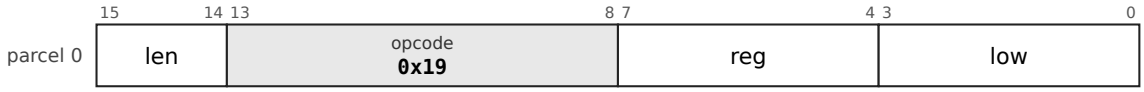
BIS: Bit set (inclusive or)

Mnemonics: `BIS`

Syntax

```
BIS rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- Rd OR rm
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BIT: Bit test

Mnemonics: `BIT`

Syntax

```
BIT rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
tmp <- Rd AND rm         ; the result is discarded
```

Flags N and Z from the result, V cleared, C unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

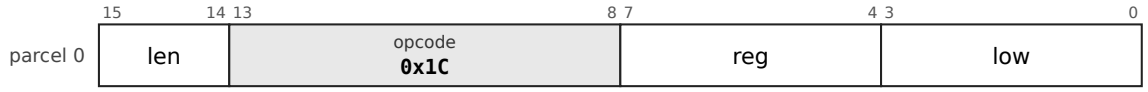
XOR: Exclusive or

Mnemonics: XOR

Syntax

```
XOR rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- Rd XOR rm
```

Flags N and Z from the result, V cleared, C unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

AND: And

Mnemonics: AND

Syntax

```
AND rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
Rd <- Rd AND rm
```

Flags N and Z from the result, V cleared, C unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

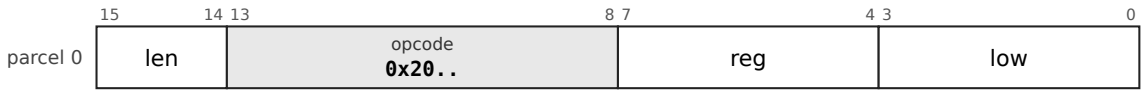
SHIFT: Shift left, shift right logical, shift right arithmetic

Mnemonics: `SHL`, `SHR`, `SAR`

Syntax

```
SHL rm, Rd
SHR rm, Rd
SAR rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Mnemonic	Opcode (the load or direct form)
`SHL`	`0x20`
`SHR`	`0x21`
`SAR`	`0x22`

Operation

```
n <- rm                                ; the count: a register, an immediate, or memory
SHL: Rd <- (n >= 64) ? 0 : Rd << n
SHR: Rd <- (n >= 64) ? 0 : Rd >>u n
SAR: Rd <- (n >= 64) ? (Rd < 0 ? -1 : 0) : Rd >>s n
```

Flags `N` and `Z` from the result, `V` cleared, `C` the last bit shifted out (bit 63 for `SHL`, bit 0 for `SHR` and `SAR`). **The same flags are written for every count**: a count of zero gives `N` and `Z` from the unchanged operand, `V` = 0 and `C` = 0 (errata 39).

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The count is read as an unsigned 64-bit value, so a count of 64 or more shifts the value entirely away. Page 2 has the three shifts by an immediate 0 to 15. `ASL` and `ASR` (in `SOP`) are the by-one forms in two bytes; rotates are in `BESC`.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

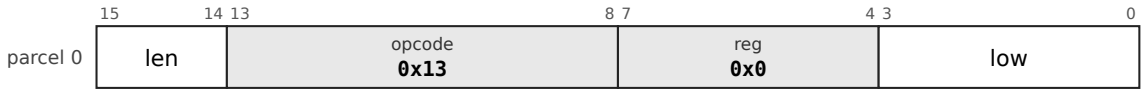
CLR: Clear

Mnemonics: `CLR`

Syntax

```
CLR rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- 0
```

Flags `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

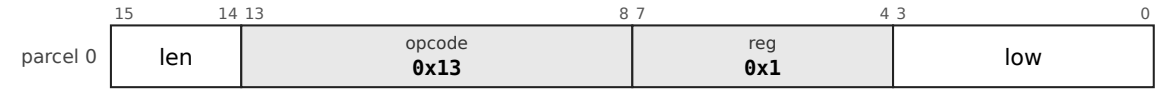
COM: Complement

Mnemonics: `COM`

Syntax

```
COM rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
rm <- NOT rm
```

Flags `N` and `Z` from the result, `V` = 0, `C` = 1 (the PDP-11 rule).

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A two-byte instruction for a register operand.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

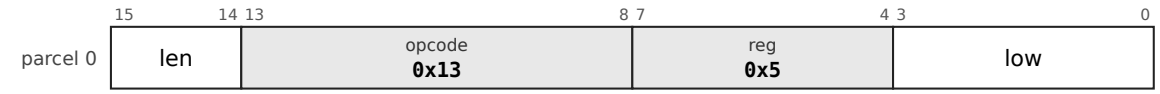
TST: Test

Mnemonics: `TST`

Syntax

```
TST rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
set flags from rm; nothing is written
```

Flags `N` and `Z` from the operand, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

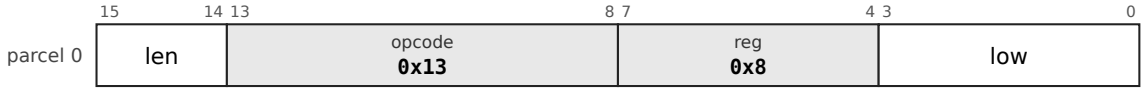
ASL: Arithmetic shift left by one

Mnemonics: `ASL`

Syntax

```
ASL rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
C <- rm[63] ; rm <- rm << 1
```

Flags `N` and `Z` from the result, `C` the bit shifted out of bit 63, `V` = `N` xor `C` (set if the sign changed: the PDP-11 rule).

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The by-one form of `SHL` in two bytes.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

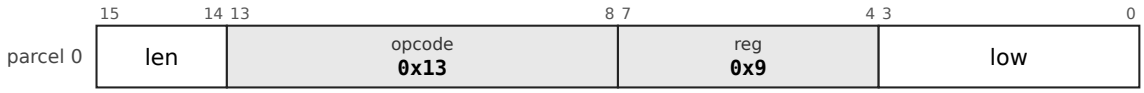
ASR: Arithmetic shift right by one

Mnemonics: `ASR`

Syntax

```
ASR rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
C <- rm[0] ; rm <- rm >>s 1
```

Flags `N` and `Z` from the result, `C` the bit shifted out of bit 0, `V` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The by-one form of `SAR` in two bytes.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

9.9 Bits, bytes and text

9.9.1 The bit-and-field escape provides `CLZ`, `CTZ`, `CLO`, `CTO` (count leading and trailing zeros and ones), `POPCNT`, `BSWAP`, `BSWAPW`, `BSWAPL` and `BITREV`. *(opcode `0x0C`)*

9.9.2 `EXTB` and `EXTZB` extract a bitfield at a position and width, sign- or zero-extended, and `INSB` inserts one; the position and width are in the third parcel, so these are six bytes. *(D)*

9.9.3 `XNORPOP` computes the population count of the exclusive-nor of two registers. *(D)*

9.9.4 `CLMULL` and `CLMULH` are the low and high halves of a carry-less multiply; no polynomial is built into the architecture. *(D)*

9.9.5 `UTF8LEN`, `UTF8DEC` and `UTF8ENC` find the length of, decode and encode a UTF-8 sequence, and reject overlong forms, surrogates, out-of-range values and bad continuation bytes. *(D)*

Commentary. **Precedent.** The PDP-11 had SWAB; the VAX had bit-field instructions and a table-driven CRC; x86 added POPCNT, LZCNT and PCLMULQDQ late, and committed CRC32 to one polynomial. **Trade-offs.** A bit unit is one shared structure: POPCNT, XNORPOP and a garbage collector's mark-bitmap scan all want the same 64-input adder tree, and CLZ and its relatives want a priority encoder over the same operand. **Improvements.** A carry-less multiply replaced a CRC instruction: no polynomial is baked in, and the same instruction serves CRC, the Galois-counter-mode authentication used in TLS, and hashing. Every UTF-8 security hole of the past is three comparators, which an instruction cannot omit and software often does. **Benefits.** The bit unit and the UTF-8 decoder together cost less than 5% of the die. **Modern comparables.** RISC-V's bit-manipulation and carry-less-multiply extensions, AArch64's CLZ, RBIT, REV and PMULL, and x86's PCLMULQDQ. **Use.** POPCNT for bitsets and Hamming distance; XNORPOP for binary-weight neural networks and bitset intersection; CLMUL for CRC, GCM and fingerprinting; UTF8* for text validation in parsers and kernels. **Impact.** *Kernel:* CTZ and CLZ find the first set bit in a run-queue or free-page bitmap; POPCNT counts bits in bitmaps; BSWAP converts network byte order. *Compiler:* The compiler maps `__builtin_clz`, `__builtin_popcount` and `__builtin_bswap` directly, and a carry-less multiply is the primitive for CRC and hashing code. *Hardware:* One bit unit holds a shared 64-input adder tree and a priority encoder, and the UTF-8 comparators are about 300 lookup tables.

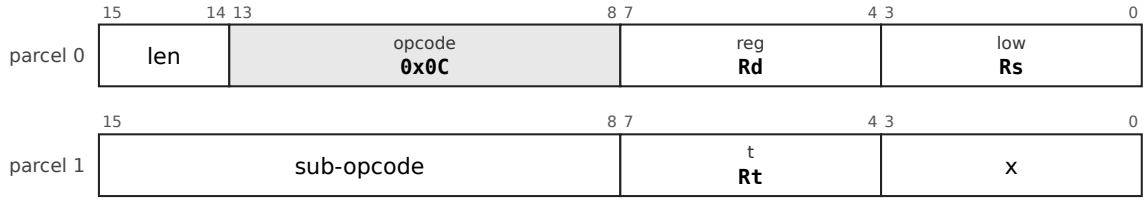
COUNT: Count leading and trailing zeros and ones

Mnemonics: CLZ, CTZ, CLO, CTO

Syntax

```
CLZ Rs, Rd
CTZ Rs, Rd
CLO Rs, Rd
CTO Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Register to register only; four bytes.

Mnemonic	Sub-opcode
CLZ	0x00
CTZ	0x01
CLO	0x02
CTO	0x03

Operation

```
CLZ: Rd <- number of leading zero bits of Rs       (64 if Rs = 0)
CTZ: Rd <- number of trailing zero bits of Rs      (64 if Rs = 0)
CLO: Rd <- number of leading one bits of Rs        (64 if Rs = all ones)
CTO: Rd <- number of trailing one bits of Rs       (64 if Rs = all ones)
```

Flags None written.

Exceptions None.

CTZ is the VAX FFS. Bit manipulation is core on a machine that does filesystem and bignum work, not an extension.

Why it is here. ISA revision 13 restored a bit-manipulation group, BESC, because revision 10's reorganisation had silently dropped BSWAP, EXTB, INSB and MOD ("a plain bug"): bit manipulation is not an extension on a machine that does filesystem work, bignum arithmetic and the like. They restore the VAX FFS and FFC idiom (find first set, find first clear; ISA section 14.1). LoongArch was the model for putting them in the base: it has CLO, CLZ, CTO, CTZ there, and that was "the decisive argument for giving it a top-level opcode" (ISA section 15.1). The compiler emits none of them yet, since C has no operator for them; the corpus shows zero uses (CHANGES section 14.4).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

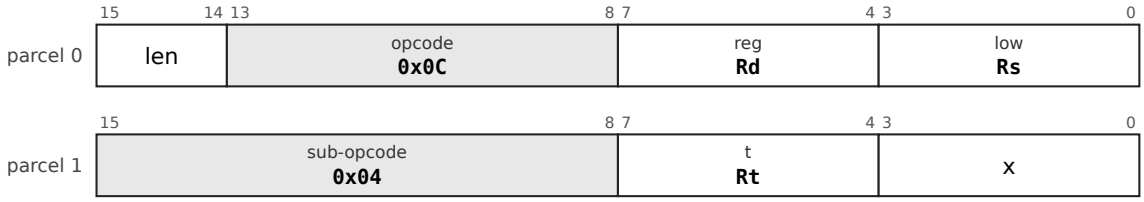
POPCNT: Population count

Mnemonics: `POPCNT`

Syntax

```
POPCNT Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- number of one bits in Rs
```

Flags None written.

Exceptions None.

Why it is here. One of the counts restored with `BESC` in ISA revision 13; it is also the building block of `XNORPOP`. A compiler needs `__builtin_popcount` to emit it, so the corpus shows no use (see `COUNT`).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

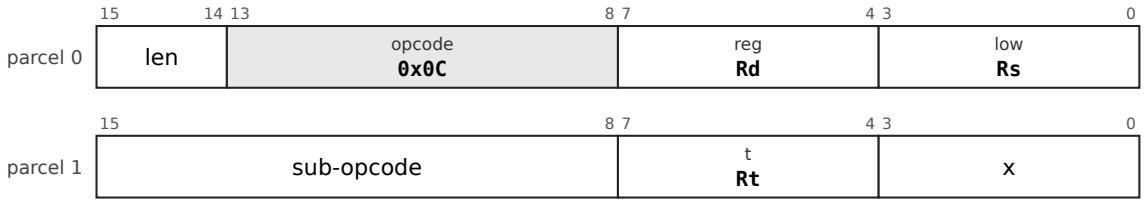
BSWAP: Byte swap

Mnemonics: `BSWAP`, `BSWAPW`, `BSWAPL`

Syntax

```
BSWAP Rs, Rd
BSWAPW Rs, Rd
BSWAPL Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Sub-opcode
`BSWAP`	`0x05`
`BSWAPW`	`0x06`
`BSWAPL`	`0x07`

Operation

```
BSWAP : Rd <- Rs with all 8 bytes reversed
BSWAPW: Rd <- Rs with the bytes of each 16-bit word reversed
BSWAPL: Rd <- Rs with the bytes of each 32-bit long reversed
```

Flags None written.

Exceptions None.

The generalisation of the PDP-11 SWAB to 64 bits.

Why it is here. The PDP-11 SWAB swapped the bytes of a word; revision 13 restored it generalised to 64 bits, with word and long variants, after revision 10 had dropped it by accident. SWAB is therefore subsumed, not lost (ISA section 14.3).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

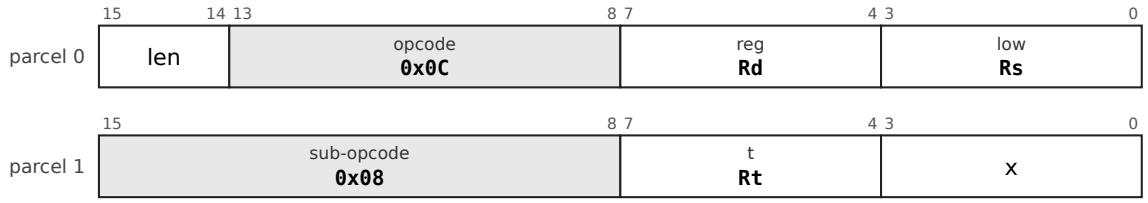
BITREV: Reverse the bits

Mnemonics: BITREV

Syntax

```
BITREV Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- Rs with bit i moved to bit 63-i
```

Flags None written.

Exceptions None.

Why it is here. Part of the BESC group of revision 13, borrowed with the other LoongArch bit operations (REVB, REVH, BITREV, ISA section 15.1). The record gives no workload from the corpus.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

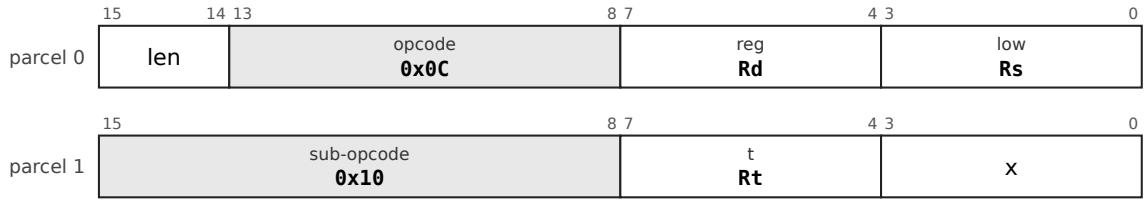
ROR: Rotate right by a register count

Mnemonics: ROR

Syntax

```
ROR Ra, Rb, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- Ra rotated right by (Rb mod 64)
```

Flags None written (rotates write no flags: errata 39).

Exceptions None.

There is no rotate left instruction: `ROL rs, #n, rd` is an assembler spelling of `RORI rs, #(64-n), rd`. No rotate is through carry; multi-word shifting uses DSHR.

Why it is here. The question that started revision 16 was why the ALU group had `ROR` but not `ROL`. The answer was that one rotate direction suffices and `ROR` is the one to keep: `ROL x, #n` is exactly `ROR x, #(64-n)`, and a funnel shifter yields `ROR` directly (ISA section 4.5.1). Revision 16 also stated for the first time that no rotate is through carry. In revision 20 `ROR` was demoted from the ALU group to `BESC`, "accepted, and revision 16's justification for its ALU slot was simply wrong": crypto rotates by compile-time constants, so a four-byte immediate form is needed either way. On 2026-10-22 the author took the recommendation that rotates write no flags (, errata 39).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

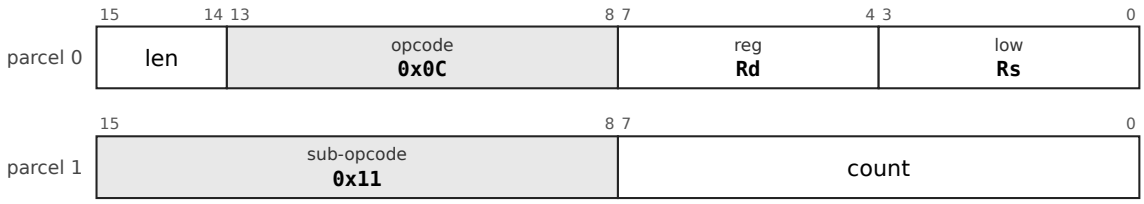
RORI: Rotate right by an immediate

Mnemonics: `RORI` (aliases: `ROL`)

Syntax

```
RORI Rs, #n, Rd
ROL Rs, #n, Rd        ; assembles to RORI Rs, #(64-n), Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Parcel 1 is `sub:8 count:8`.

Operation

```
Rd <- Rs rotated right by (n mod 64)
```

Flags None written.

Exceptions None.

Four bytes: the count is in the low byte of parcel 1.

Why it is here. Added with the register-count rotate in revision 16 and given its own mnemonic in errata 10, because an immediate count and a register count are different encodings. `ROL Rs, #n, Rd` is accepted by the assembler as `RORI Rs, #(64-n), Rd` (ISA revision 17, section 4.15): the complement costs nothing at assembly time, so there is no reason to make programmers do it, and a register-count `ROL` is rejected so that one mnemonic stays one instruction.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

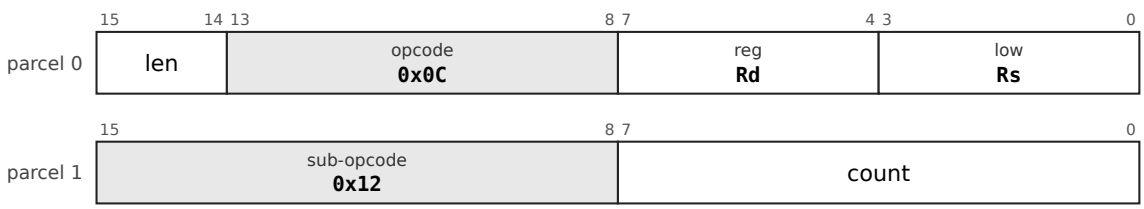
RORI32: Rotate right by an immediate, 32 bits

Mnemonics: `RORI32`

Syntax

```
RORI32 Rs, #n, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- sext32(ror32(Rs[31:0], n mod 32))
```

Flags None written.

Exceptions None.

The 32-bit rotate with the result sign-extended like every 32-bit result; the primitive of most hash functions and ciphers.

Why it is here. Added on 2026-10-22 with the second page (errata 31): a 32-bit rotate by a constant, the shape of crypto and hash inner loops (revision 20 had noted that crypto rotates by compile-time constants), estimated together with CBCC at 22 KB (1.3%) of the compiled image. Its first specification zero-extended the result; the author decided "make rori32 sign-extend like the rest", so that no instruction is needed after it (errata 34). It removed 44 MOVL instructions in the corpus, the demand analysis having already removed nearly all of them.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

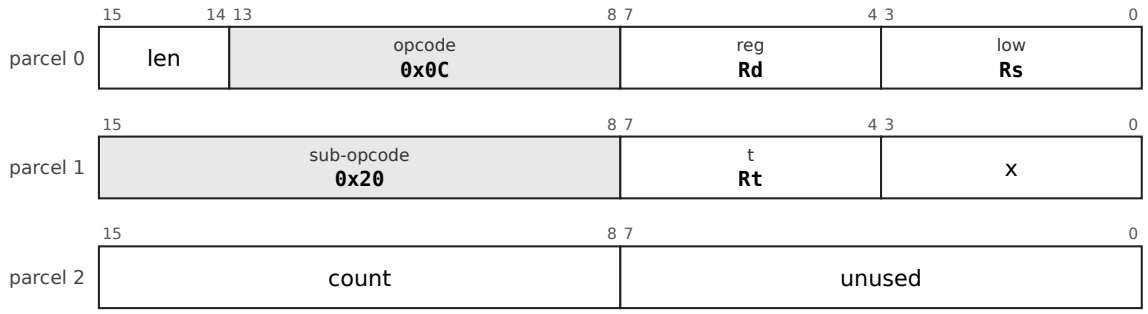
DSHR: Double-width funnel shift right

Mnemonics: DSHR

Syntax

```
DSHR Ra, Rb, #n, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
t <- Ra:Rb                    ; 128 bits, Ra the high half
Rd <- (t >> n)[63:0]          ; n = 0..63
```

Flags None written.

Exceptions None.

x86 SHRD, ARM EXTR. Six bytes (the count is in parcel 2). A 128-bit left shift by k is `DSHR hi, hi, lo, #(64-k)` then `SHL #k, lo`.

Why it is here. Added in ISA revision 16: a double-width funnel shift (x86 SHRD, ARM EXTR), the right primitive for multi-word shifting. Two instructions shift a 128-bit value by any distance, where rotate-through-carry took one bit per instruction, and that is why no rotate is through carry. It also subsumes ROR when both sources are the same register. It leaves every flag alone.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

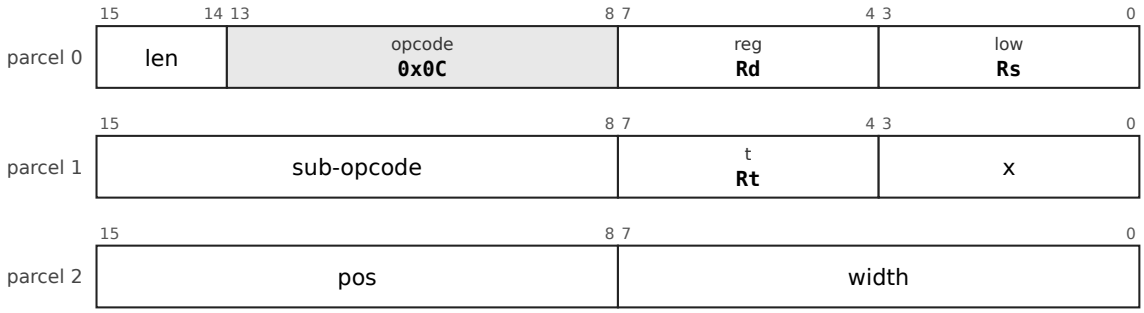
EXTB: Extract a bit field

Mnemonics: EXTB, EXTZB

Syntax

```
EXTB Rs, #pos, #width, Rd        ; sign-extended
EXTZB Rs, #pos, #width, Rd       ; zero-extended
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Sub-opcode
EXTB	0x21
EXTZB	0x22

Operation

```
Rd <- ext((Rs >> pos)[width-1:0])          ; sext for EXTB, zext for EXTZB; width 0 or > 64 means 64
```

Flags None written.

Exceptions None.

Six bytes (position and width are in parcel 2), so these exist at `len = 10` only.

Why it is here. Revision 10's reorganisation dropped bit-field extract and insert by accident; revision 13 restored them (the VAX EXTV, EXTZV and INSV idiom, ISA section 14.1). The compiler does not use them: the shift-pair extraction pattern occurs 151 times in 26 files of the corpus, about 600 bytes at best (0.03%), and there are four true bit-field extracts in all. They are for hand-written protocol and bit-stream code (CHANGES section 14.4).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

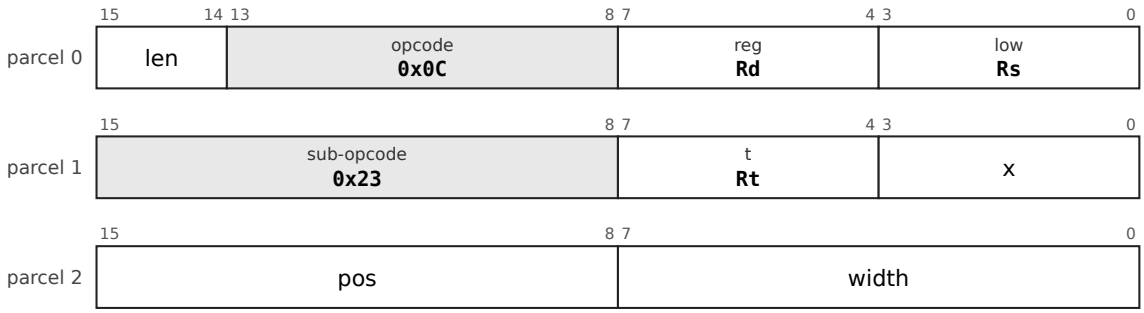
INSB: Insert a bit field

Mnemonics: `INSB`

Syntax

```
INSB Rs, #pos, #width, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
mask <- ((1 << width) - 1) << pos
Rd <- (Rd AND NOT mask) OR ((Rs << pos) AND mask)
```

Flags None written.

Exceptions None.

Six bytes. `Rd` is both source and destination.

Why it is here. Restored with `EXTB` in revision 13 (the VAX `INSV` idiom). The record names no use in the compiled corpus.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

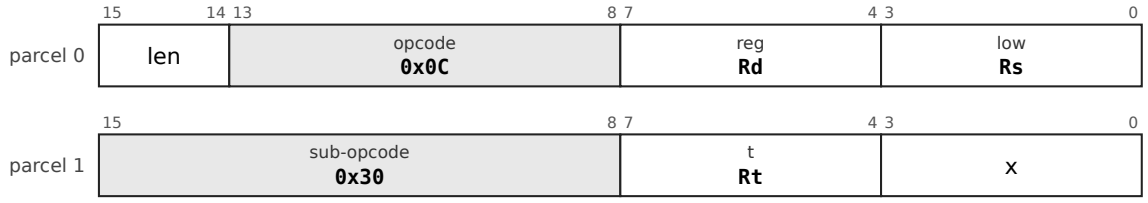
XNORPOP: Population count of the equality of two words

Mnemonics: `XNORPOP`

Syntax

```
XNORPOP Ra, Rb, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- popcount(NOT (Ra XOR Rb))
```

Flags None written.

Exceptions None.

For binary-activation networks, Hamming distance and bitset intersection cardinality. It does **not** serve sub-1-bit language models (book section 8.3).

Why it is here. Added in ISA revision 13 in answer to the author's design-chat question about the new language models that use single-bit encodings ("are there instruction sequences that would benefit from coalescing?"). Revision 14 corrected the reason: the instruction does **not** serve sub-1-bit language models, because the activations in the LittleBit work are FP16 and the inner loop is a conditional add, not a popcount. It stays for what it does serve: binary-activation networks, Hamming distance, bitset intersection counts and counting a collector's mark bitmap. The sub-1-bit kernel is bandwidth-bound and belongs to a bus-mastering accelerator, not an opcode (ISA section 4.6.2; chapter 8).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

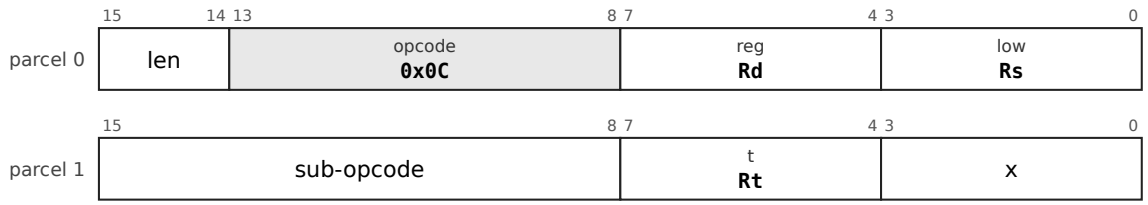
CLMUL: Carry-less multiply

Mnemonics: `CLMULL`, `CLMULH`

Syntax

```
CLMULL Ra, Rb, Rd
CLMULH Ra, Rb, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Sub-opcode
CLMULL	0x31
CLMULH	0x32

Operation

```
p <- carry-less (polynomial over GF(2)) product of Ra and Rb      ; 128 bits
CLMULL: Rd <- p[63:0]
CLMULH: Rd <- p[127:64]
```

Flags None written.

Exceptions None.

The primitive under every CRC (with a Barrett reduction), GF(2^k) arithmetic and GCM; no polynomial is committed to the architecture.

Why it is here. ISA revision 15 added carry-less multiply as the primitive beneath every CRC polynomial (a CRC is a carry-less multiply plus a Barrett reduction), GF(2^k) erasure codes and AES-GCM, chosen over a CRC instruction because x86 and ARM each committed a polynomial to the architecture. On 2026-10-21 the author asked "how about the 2 crc variants also?", the assistant added eight CRC32/CRC32C words, and the author withdrew them at: "ok, we have carryless mul no need for either crc" (errata 20).

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

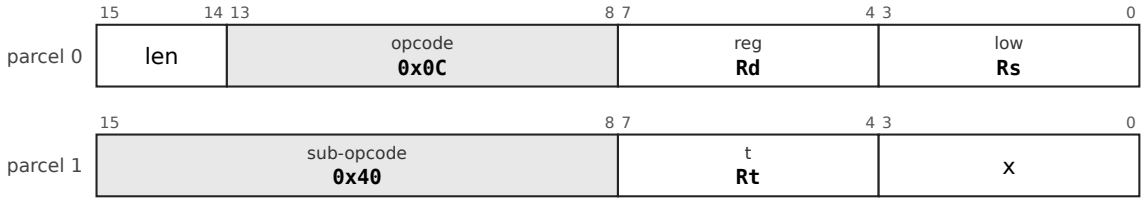
UTF8LEN: Length of a UTF-8 sequence from its leading byte

Mnemonics: UTF8LEN

Syntax

```
UTF8LEN Rs, Rd
```

Encoding



From len = 01. In this group low is a register, never a mode.

Operation

```
Rd <- length (1 to 4) implied by the leading byte Rs[7:0]; 0 if it cannot begin a valid sequence
```

Flags None written.

Exceptions None.

Why it is here. The design chat asked "do we have any special instructions for utf8?" and later whether the "string" functions being omitted included utf8enc and utf8dec. ISA revision 13 added three, because most of what UTF-8 needs is general bit manipulation but "the validity rules are where the bugs are". UTF8LEN is the exception that is nearly free: CLO on the leading byte, mapped to 1, 2, 3 or 4 (ISA section 4.6.1).

Status specified; **not yet implemented** in the reference executor.

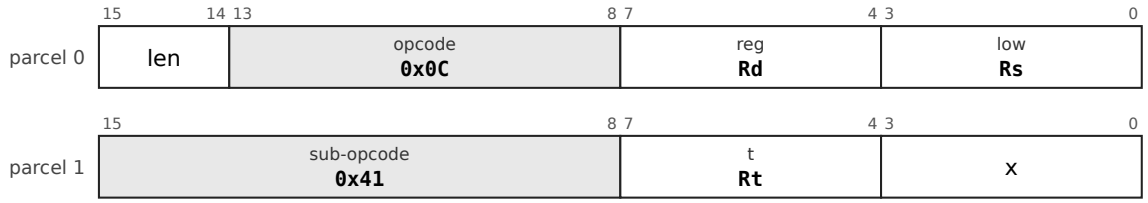
UTF8DEC: Decode a UTF-8 sequence

Mnemonics: UTF8DEC

Syntax

```
UTF8DEC Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rs holds up to 4 bytes, leading byte in bits 7..0
Rd <- the scalar value
V  <- 1 if the sequence is not a valid encoding
```

Flags `V` set for a bad continuation byte, an overlong form, a surrogate (U+D800 to U+DFFF), a value above U+10FFFF, or a leading byte of `0xC0`, `0xC1` or `0xF5` to `0xFF`; the length consumed is reported in a scratch field of the status word (details in the instruction-set text, section 4.6.1).

Exceptions None.

Why it is here. Added in ISA revision 13 (the design chat: "do we have any special instructions for utf8?"). Not for speed but for correctness: overlong forms, surrogates, values above U+10FFFF and the bytes that can never begin a sequence are each a historical security hole, each is three comparators in hardware, and software implementations get them wrong or omit them for speed (ISA section 4.6.1, about 300 lookup tables).

Status specified; **not yet implemented** in the reference executor.

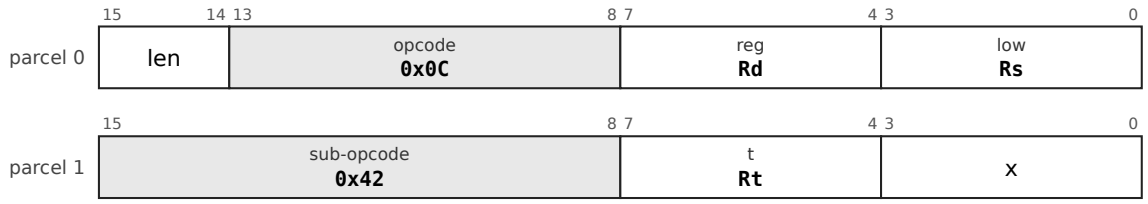
UTF8ENC: Encode a Unicode scalar value

Mnemonics: `UTF8ENC`

Syntax

```
UTF8ENC Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode.

Operation

```
Rd <- the UTF-8 bytes of the scalar value Rs, leading byte in bits 7..0
V  <- 1 if Rs is not a Unicode scalar value
```

Flags `V` set if `Rs` is not a Unicode scalar value (a surrogate or above U+10FFFF).

Exceptions None.

Why it is here. The encoding half of the UTF-8 pair added in ISA revision 13; it refuses surrogates and values above U+10FFFF so that an encoder cannot produce what a decoder must reject.

Status specified; **not yet implemented** in the reference executor.

9.10 Branches and control flow

Cf. section 6.7 for TRAP, RTI and RTT, and section 6.2 for how a trap enters the kernel; section 9.16 for CALLI and JMPI in position-independent code.

9.10.1 There is an unconditional branch `BR` and fourteen conditional branches, two bytes with an 8-bit displacement, four with 16 bits, six with 32 bits, counting two-byte parcels from the first parcel of the branch: ±256 bytes, ±64 KB, ±4 GB. *(opcodes `0x30-0x3E`)*

9.10.2 The conditions are `ne`, `eq`, `pl`, `mi`, `vc`, `vs`, `cc`/`hs`, `cs`/`lo`, `ge`, `lt`, `gt`, `le`, `hi` and `los`.

9.10.3 `SOB Rn, dst` subtracts one from `Rn` and branches if the result is nonzero, forward or backward. *(opcode `0x3F`)*

9.10.4 `CBCC` compares a register with a 5-bit immediate and branches (four bytes, ±1024 parcels); `CBW` takes a 16-bit immediate and a 16-bit displacement (six bytes); `CBRR` compares two registers and branches. None writes the flags. *(D, errata 34, 35 and 39)*

9.10.5 `JMP` and `CALL` take any address mode; `RET` returns; `JMPI` and `CALLI` jump and call through a pointer in memory; `JTAG` dispatches on the tag in the low three bits of a register; `BTAG` and `BNTAG` branch on a tag. *(D)*

9.10.6 A call pushes the return address on the stack; there is no link register. *(D, ABI)*

Commentary. **Precedent.** The complete PDP-11 branch set, which the long-branch problem of the PDP-11 (±256 bytes) made awkward, here reaches ±4 GB with no long-branch opcode and no trampoline, because the displacement widens with the length. `SOB` was the PDP-11's, which could only branch backward. **Trade-offs.** A compare-and-branch that writes no flags cannot feed a second branch that reads them, so a three-way compare is compare-then-branch; the compiler fuses only where the flags are dead afterwards, which cost about 0.4% in code size and was accepted. **Improvements.** The tag branches serve a Lisp or other tagged runtime in one instruction. **Benefits.** Branch relaxation in the assembler is just choosing a length. **Modern comparables.** RISC-V and MIPS are compare-and-branch machines with no flags; AArch64 has `CBZ` and `TBZ`; x86 uses flags like the pdp64's branches. **Use.** `JMPI` and `CALLI` are the shared-library call and the virtual call; `JTAG` is the dispatch of a dynamic-language interpreter. **Impact.** *Kernel:* `JMPI`, `CALLI` and `JTAG` are the shared-library call, the virtual call and the dispatch of an interpreter; `SOB` and `CBCC` are the loops of the kernel's inner routines. *Compiler:* Branch relaxation is choosing a length; fusing a compare and a branch is legal only where the flags are dead afterwards, so the compiler needs a flag-liveness analysis. *Hardware:* The next-PC mux takes the load-return path for the indirect forms; the compare-and-branch compare runs in the branch unit beside the ALU; displacements count parcels, so no shifter is needed.

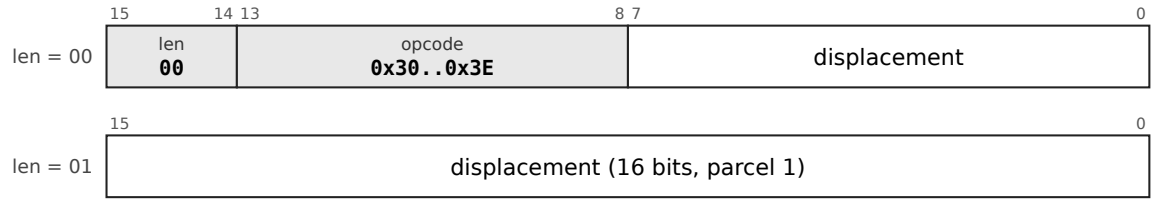
BCC: Branch on condition

Mnemonics: `BR`, `BNE`, `BEQ`, `BPL`, `BMI`, `BVC`, `BVS`, `BCC`, `BCS`, `BGE`, `BLT`, `BGT`, `BLE`, `BHI`, `BLOS` (aliases: `BHIS`, `BLO`)

Syntax

```
BR target
BNE target
BEQ target ... and the rest
```

Encoding



The displacement is signed and counts 2-byte parcels from the first parcel of the instruction: 8 bits at `len = 00` (about ±256 bytes), 16 bits at `len = 01` (±64 KB), 32 bits in parcels 1 and 2 at `len = 10` (±4 GB).

Mnemonic	Opcode
`BR`	`0x30` (condition 0)
`BNE`	`0x31` (condition 1)
`BEQ`	`0x32` (condition 2)
`BPL`	`0x33` (condition 3)
`BMI`	`0x34` (condition 4)
`BVC`	`0x35` (condition 5)
`BVS`	`0x36` (condition 6)

Mnemonic	Opcode
BCC	0x37 (condition 7)
BCS	0x38 (condition 8)
BGE	0x39 (condition 9)
BLT	0x3A (condition 10)
BGT	0x3B (condition 11)
BLE	0x3C (condition 12)
BHI	0x3D (condition 13)
BLOS	0x3E (condition 14)

Operation

```
if cond(c) then PC <- address of this instruction + 2 * disp
```

Flags None written.

Exceptions None; a branch to an unmapped or non-executable address faults on the fetch of the target.

Condition numbers: al 0 (BR), ne 1, eq 2, pl 3, mi 4, vc 5, vs 6, cc/hs 7, cs/lo 8, ge 9, lt 10, gt 11, le 12, hi 13, los 14. Conditions on the flags: ne !Z; eq Z; pl !N; mi N; vc !V; vs V; cc !C; cs C; ge N == V; lt N != V; gt !Z and N == V; le Z or N != V; hi !C and !Z; los C or Z. The displacement counts 2-byte parcels; the branch reaches ±4 GB with no long-branch instruction.

Cf. 6.7 for TRAP, RTI and RTT.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

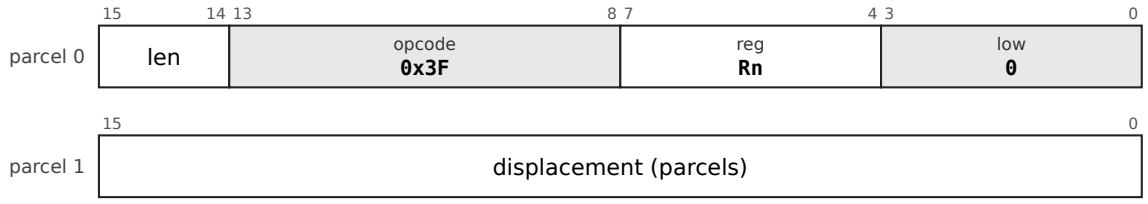
SOB: Subtract one and branch

Mnemonics: SOB

Syntax

```
SOB Rn, target
```

Encoding



From len = 01; the signed displacement is 16 bits (32 at len = 10).

Operation

```
Rn <- Rn - 1
if Rn != 0 then PC <- address of this instruction + 2 * disp
```

Flags None written.

Exceptions None.

Restored from the PDP-11, where it could branch only backward; here the signed displacement goes either way. Four bytes at least.

Why it is here. Restored from the PDP-11 in ISA revision 13 at the branch group's one free opcode. The PDP-11's could only branch backward (a 6-bit unsigned displacement); this one branches either way. It answers the design chat's question about the "cbnz idiom": it does not save bytes (DEC plus BNE is also four) but it saves an instruction issue, a cycle per iteration on a machine that does not pipeline. The measurement of 2026-10-22: "same size, one instruction fewer" (static dec; bne pairs exist).

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

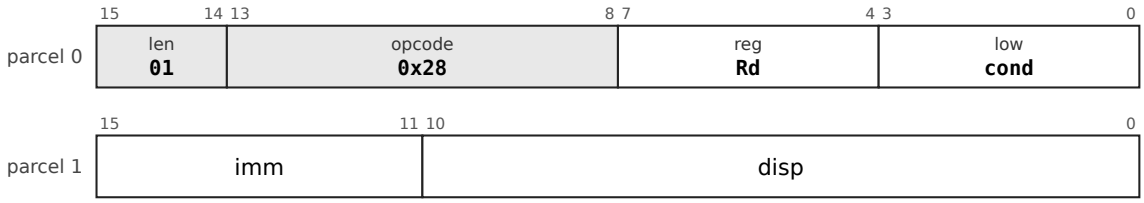
CBCC: Compare and branch on condition

Mnemonics: `CBNE`, `CBEQ`, `CBPL`, `CBMI`, `CBVC`, `CBVS`, `CBCC`, `CBCS`, `CBGE`, `CBLT`, `CBGT`, `CBLE`, `CBHI`, `CBLOS` (aliases: `CBHIS`, `CBLO`)

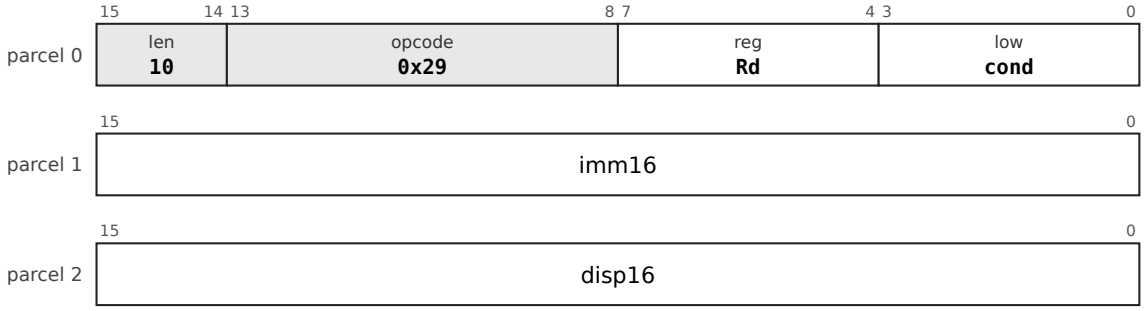
Syntax

```
CBNE Rd, #imm, target
CBEQ Ra, Rb, target ... and the rest
```

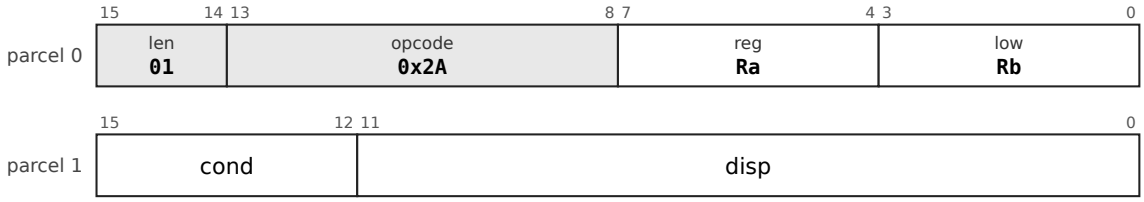
Encoding



`CBCC` (opcode `0x28`), four bytes: a signed 5-bit immediate (-16 to 15) and an 11-bit displacement in parcels. At `len = 10` (six bytes) parcel 1 is `imm(5) disp(26..16)` and parcel 2 `disp(15..0)`.



`CBW` (opcode `0x29`), six bytes: a 16-bit immediate and a 16-bit displacement in parcels.



`CBRR` (opcode `0x2A`), four bytes (six at `len = 10`, with a 28-bit displacement): compares two registers. The condition numbers are those of the branches: `al` 0, `ne` 1, `eq` 2, `pl` 3, `mi` 4, `vc` 5, `vs` 6, `cc`/`hs` 7, `cs`/`lo` 8, `ge` 9, `lt` 10, `gt` 11, `le` 12, `hi` 13, `los` 14. There is no `len = 00` form.

Operation

```
tmp <- Rd - (imm or Rb)         ; as CMP, for the decision only
if cond(c) on that comparison then PC <- address of this instruction + 2 * disp
```

Flags None written (errata 39): the instruction computes the comparison for its own decision and leaves `N Z V C` as they were.

Exceptions None.

Three encodings, chosen by the assembler: `CBCC` (`0x28`, a 5-bit signed immediate -16 to 15, a displacement of ±1024 parcels at four bytes or 27 bits at six), `CBW` (`0x29`, a 16-bit immediate and a 16-bit displacement, six bytes) and `CBRR` (`0x2A`, two registers, ±2,048 parcels at four bytes or 28 bits at six). There is no `len = 00` form. A target in another section is refused.

Why it is here. The author asked on 2026-10-22: "why do we need 8 byte instructions? only amd64 has more than 6 bytes. can we squeeze into 3 parcels?" The fourth parcel was removed and the freed length code became the second page (errata 31); at the same time a fused compare-and-branch, `CBCC`, was added in the reserved opcode `0x28`, estimated with `RORI32` at 22

KB (1.3%) of the image. Then, measured, step by step: the author asked for the wider form ("do the wider cbcc") and the **wide-immediate** **CBCC** removed 11,735 instructions (3.4%) and 1,828 bytes; "do the register-register compare-and-branch" added **CBRR**, which removed 4,631 instructions (1.4%) and 142 bytes (errata 34 and 35). The reason it is not a replacement for flags: LoongArch and RISC-V chose fused compare-and-branch instead of flags, and the ISA declines that (section 15.2) because it needs a comparator in the branch unit and a third read port; here the fused form is added beside the flags. Errata 39 then made CBCC write no flags.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BTAG: Branch on pointer tag

Mnemonics: `BTAG`, `BNTAG`

Syntax

```
BTAG Rn, #tag, target
BNTAG Rn, #tag, target
```

Encoding



`sense` is 0 for `BTAG` and 1 for `BNTAG`; `tag` is the 3-bit tag. The displacement is 16 bits (32 at `len = 10`).

Operation

```
BTAG : if (Rn & 7) == tag then PC <- address of this instruction + 2 * disp
BNTAG: if (Rn & 7) != tag then PC <- address of this instruction + 2 * disp
```

Flags None written.

Exceptions None.

For runtimes that keep a 3-bit type tag in the low bits of a pointer (`lowtag`). Four bytes at least; the displacement is 16 bits (32 at `len = 10`).

Why it is here. The design chat asked: "what instructions are there that make using the lowtag stuff easy for lisp type things?" ISA revision 13 answered with the tag branches `BTAG` and `BNTAG` and the table dispatch `JTAG`, "to make tagged pointers cheap to use rather than merely possible": three bits of tag and one of sense fit the `low` nibble exactly, so parcel 1 is a full displacement. Because quad-aligned objects have zero in the low three bits, the tag is free, and an unaligned access is a fault. The compiler for C never emits them (the corpus shows none); they are for Lisp and other tagged runtimes.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

JMP: Jump

Mnemonics: `JMP`

Syntax

```
JMP rm        ; a register, or an address by any memory mode
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
PC <- (register operand) ? Rm : EA(rm)
```

Flags None written.

Exceptions A jump to an unmapped or non-executable address faults on the fetch of the target.

A switch is a `CMP` against the limit, a `BHIS` to the default and a `JMP` through a scaled index.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

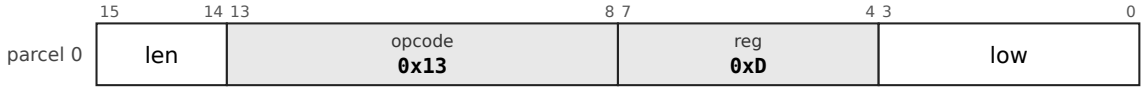
CALL: Call

Mnemonics: `CALL`

Syntax

```
CALL rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
t <- (register operand) ? Rm : EA(rm)
SP <- SP - 8 ; M[SP] <- address of the next instruction
PC <- t
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The return address is on the data stack where every unwinder and debugger expects it. `RET` pops it.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

JMPI: Jump and call through memory

Mnemonics: `JMPI`, `CALLI`

Syntax

```
JMPI rm        ; PC <- M[EA]
CALLI rm       ; push the return address, then PC <- M[EA]
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Mnemonic	`reg` (sub-opcode)
`JMPI`	`0x0`
`CALLI`	`0x1`

Operation

```
JMPI : PC <- M[EA(rm)]
CALLI: t <- M[EA(rm)] ; SP <- SP - 8 ; M[SP] <- address of the next instruction ; PC <- t
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. For CALLI the order is specified: load the target, then push, then transfer; a fault on the load leaves nothing changed.

The one deferred reference the architecture admits, for control transfer only: the PLT stub (`JMPI got_slot(PC)`) and virtual-method dispatch (`CALLI vtbl_off(Robj)`). It needs a memory operand, so it is four bytes at least.

Why it is here. ISA revision 18 reintroduced exactly one deferred reference, for control transfer only, in XFER: the PLT stub of every dynamically linked program (`JMPI got_slot(PC)`) and virtual-method dispatch (`CALLI vtbl_off(Robj)`), each two instructions and a scratch register without it. It is admitted because it breaks none of the reasons deferred modes were removed (no register is side-effected, the reference is in the branch path and not the operand path, a fault is clean by ordering; ISA section 4.9.1). The measurement of 2026-10-22: jump tables through JMPI are not smaller, because the table entries are 8 bytes against 8 for a case in a compare chain.

Status implemented in the reference executor for CALLI; not yet for the others.

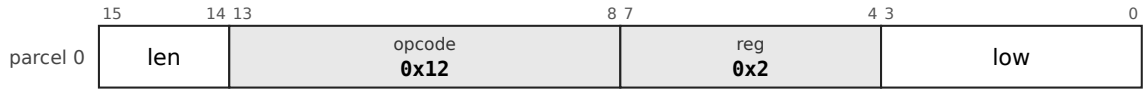
JTAG: Jump through a tag table

Mnemonics: JTAG

Syntax

```
JTAG d(Rb)(Rx*s)
```

Encoding



The reg field is the sub-opcode; rm and the mode give the single operand, at any length (section 9.5).

Operation

```
PC <- M[Rb + d + 8 * (Rx & 7)]
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Legal only with the scaled-index mode (0001). Dispatches on the low three bits of a tagged pointer through a table of eight entries; the 3-bit tag cannot index out of range.

Why it is here. The table-dispatch partner of BTAG, added in ISA revision 13 for the same question about tagged Lisp pointers. It is legal only in the scaled-index mode (errata 6), the one mode with a free register slot for the tagged value. It replaces the PDP-11 dispatch-table idiom @(Rn)+ better, because it needs no pointer register and cannot index out of range (ISA section 14.2).

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

9.11 The stack and block operations

Cf. section 6.8 for restarting a block operation after a fault (progress is held in registers) and section 6.4 for the page faults it can take; section 7.2 for how block operations order their accesses.

9.11.1 PUSH and POP move one register; PUSHM and POPM move a set named by a mask (a short mask form is in page 2). A mask containing R14 is reserved. *(D)*

9.11.2 PUSHX and POPX save and restore the exception state EPSW and EPC in one instruction each; both words are checked before either is accessed, so an abort leaves nothing changed. *(D, errata 37)*

9.11.3 The block operations MCPY, MMOV (overlap-safe), MSET, MCMP, MSCN and MSCNN take a destination pointer, a source pointer or value and a count in registers; MVAL validates a buffer as UTF-8. *(opcode 0x0D)*

9.11.4 A block operation keeps its progress in its registers: when it completes, the count is zero and the pointers are one past the end; when it stops early or is interrupted, they point where it stopped, and it restarts by running again. *(D)*

9.11.5 An implementation may move more than a byte per step, provided the registers are consistent with byte-at-a-time progress wherever an exception can be taken. *(D)*

Commentary. **Precedent.** The VAX's `MOVC5` is the canonical argument against such instructions: its operands could be memory, its architectural definition needed the original values to survive, and so it required hidden state and a first-part-done bit in the status word to resume. The PDP-10 had a block transfer, and x86's `REP MOVS` keeps its progress in registers as the pdp64's do. **Trade-offs.** This is the one admitted exception to the one-memory-operand rule, accepted because its mechanism, hidden state, is absent: the state is the registers, which are already architectural. A large copy can evict the whole data cache, and a non-temporal variant is an open question. **Improvements.** ARM reached the same design in 2022 with `FEAT_MOPS`, whose specification says the registers are updated so that progress is not lost. **Benefits.** The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors. **Modern comparables.** x86 `REP MOVSB` and `REP STOSB` with fast-string microcode; AArch64 `CPYP`/`CPYM`/`CPYE` and `SETP`/`SETM`/`SETE`; s390 `MVC`. **Use.** `memcpy`, `memmove`, `memset`, `memcmp` and `memchr` in the C library; `MVAL` in parsers and kernels. **Impact.** *Kernel:* `PUSHM`, `POPM`, `PUSHX` and `POPX` shorten entry, exit and the context switch, and the block operations are `copyin`, `copyout`, page clearing and `memcpy`. *Compiler:* A prologue and an epilogue are one instruction each, and `memcpy` and `memset` lower to the block operations. *Hardware:* The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors, with no hidden state.

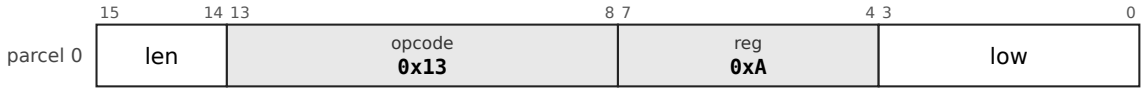
PUSHM: Push registers by mask

Mnemonics: `PUSHM`

Syntax

```
PUSHM #mask       ; a 16-bit immediate, bit n is Rn
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
for n = 0 to 15, if mask bit n:
    SP <- SP - 8 ; M[SP] <- Rn
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. A mask that includes `R14` (`SP`) is reserved.

Four bytes (mode `0100`, immediate). The registers are pushed in ascending order, so `POPM` with the same mask restores them. Page 2 has an 8-bit mask for `R8` to `R15`.

Why it is here. The VAX procedure entry mask was the useful half of `CALLS`; ISA revision 13 kept that half as `PUSHM` and `POPM` without the frame (which the book rejects: hardware call frames ran slower than the sequence on later VAXes). They began in `MBLK` and moved to `SOP` because a 16-bit mask does not fit beside `MBLK`'s sub-opcode; that used `SOP`'s last two free slots (CHANGES section 3.2). The second page has 8-bit forms for `R8` to `R15`, because all 2,392 push sites and 2,392 pop sites in the measured corpus used a mask within `R8` to `R15`.

Cf. 6.8 for the restart rule.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

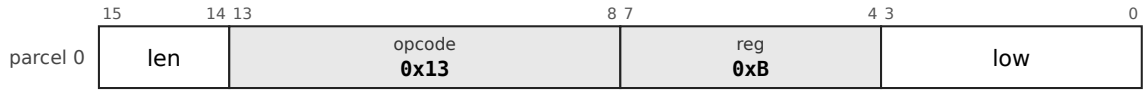
POPM: Pop registers by mask

Mnemonics: `POPM`

Syntax

```
POPM #mask
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
for n = 15 down to 0, if mask bit n:
    Rn <- M[SP] ; SP <- SP + 8
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The mirror of `PUSHM`; four bytes, with an 8-bit-mask form on page 2.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

PUSH: Push

Mnemonics: `PUSH`

Syntax

```
PUSH rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
v <- rm ; SP <- SP - 8 ; M[SP] <- v
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A one-parcel instruction for a register operand.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

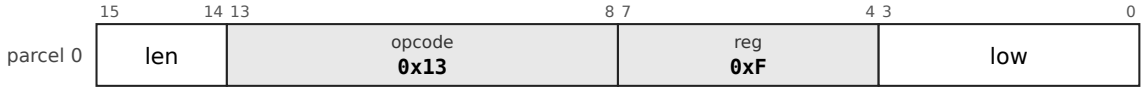
POP: Pop

Mnemonics: `POP`

Syntax

```
POP rm
```

Encoding



The `reg` field is the sub-opcode; `rm` and the mode give the single operand, at any length (section 9.5).

Operation

```
v <- M[SP] ; SP <- SP + 8 ; rm <- v
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

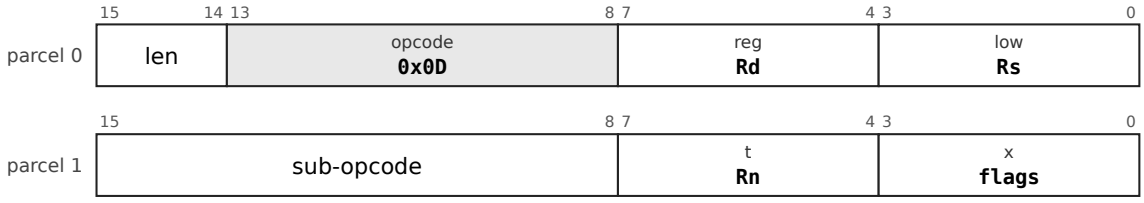
MCPY: Copy a block of bytes

Mnemonics: `MCPY`, `MMOV`

Syntax

```
MCPY Rd, Rs, Rn        ; destination pointer, source pointer, count
MMOV Rd, Rs, Rn        ; correct for overlap
```

Encoding



`reg` is the destination pointer, `low` the source pointer (or the value register `Rv`), parcel 1 `t` the count register `Rn`.

Mnemonic	Sub-opcode
`MCPY`	`0x00`
`MMOV`	`0x01`

Operation

```
MCPY:                           ; ascending; overlap is undefined
  while Rn != 0:  M8[Rd] <- M8[Rs] ; Rd <- Rd + 1 ; Rs <- Rs + 1 ; Rn <- Rn - 1
MMOV: as MCPY, but the direction is chosen from the pointers so that an overlapping copy is correct
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. The three registers are updated as work completes, so a fault leaves a consistent state and the instruction restarts from its registers.

The one granted exception to the one-memory-operand rule. All three registers are architectural state: on completion `Rn` is 0 and `Rd` and `Rs` point one past the end. An implementation may move more than a byte per step, so long as the register state at every point where an exception can be taken is consistent with a byte-at-a-time copy. The accesses made inside one block operation are unordered with respect to each other.

Why it is here. The design chat's questions: "maybe memory to memory mov is useful and should be allowed, especially with a loop" and "principles exceptions are allowed where there is a clear benefit" (which produced the exception rule of ISA section 1.1), then "is there any indirect addressing, so you could load r1 and r2 with addresses then `rep movsb r1,r2,10` to mov 10 bytes. and maybe an instruction to zero out a range of memory. is the cbnz idiom better? how about mmov

mcpy mset like ibm system z?". ISA revision 13 answered with MBLK: MCPY, MMOV, MSET, MCMP, MSCN, the one granted exception to the one-memory-operand rule, made safe by keeping the progress in architectural registers (the VAX MOVC5 kept it in hidden microstate, which is why it was rejected; ARM reached the same design in 2022 with FEAT_MOPS). MCPY replaces the PDP-11 MOV (R1)+,(R2)+ loop with one restartable instruction.

Cf. 6.8 (restart, progress faults); 7.2 (ordering of the accesses).

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

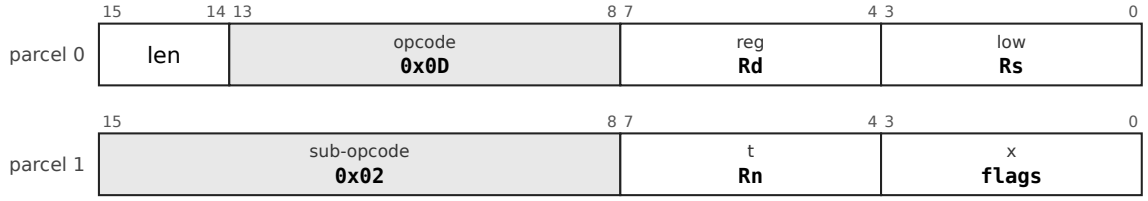
MSET: Fill a block of bytes

Mnemonics: MSET

Syntax

```
MSET Rd, Rv, Rn
```

Encoding



reg is the destination pointer, low the source pointer (or the value register Rv), parcel 1 t the count register Rn.

Operation

```
while Rn != 0:  M8[Rd] <- Rv[7:0] ; Rd <- Rd + 1 ; Rn <- Rn - 1
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

MSET with Rv = 0 is the range-zeroing instruction: bzero and the clear of calloc are one instruction.

Why it is here. Asked for in the design chat ("maybe an instruction to zero out a range of memory") and delivered in ISA revision 13 with MCPY; with a zero value it is the range-zeroing instruction, so bzero and the clearing of calloc need no separate opcode.

Cf. 6.8.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

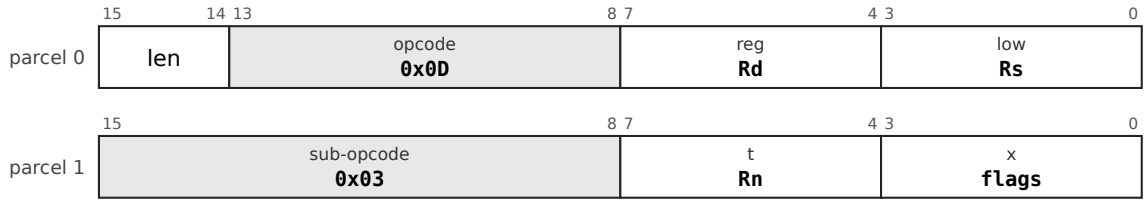
MCMP: Compare two blocks of bytes

Mnemonics: MCMP

Syntax

```
MCMP Rd, Rs, Rn
```

Encoding



reg is the destination pointer, low the source pointer (or the value register Rv), parcel 1 t the count register Rn.

Operation

```
while Rn != 0:
    x <- M8[Rd] ; y <- M8[Rs]
    if x != y then  flags <- flags of (x - y) ; stop      ; pointers left AT the differing byte
    Rd <- Rd + 1 ; Rs <- Rs + 1 ; Rn <- Rn - 1
if equal:  N = 0, Z = 1, V = 0, C = 0
```

Flags From the first differing byte as for `CMP` of the two bytes (`C` set if `x < y` unsigned, `N` and `V` from the signed difference); for equal blocks `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

When the instruction stops at a difference, `Rd` and `Rs` point at the differing bytes and `Rn` is the count remaining, so the position of the difference is in the registers.

Why it is here. The design chat asked about the "string" functions being omitted: "doesn't mcmp/mmov count?". It did: ISA revision 15 corrected the book-keeping, because the rejection of "string" instructions was wrong from revision 8 and visibly wrong by revision 13. Seven of the VAX's ten character-string instructions are present (`MOVC3`, `CMPC3`, `LOCC` and `SKPC` among them), and the rejection was always about `EDITPC` and packed decimal. `MCMP` is the `CMPC3` equivalent.

Cf. 6.8.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

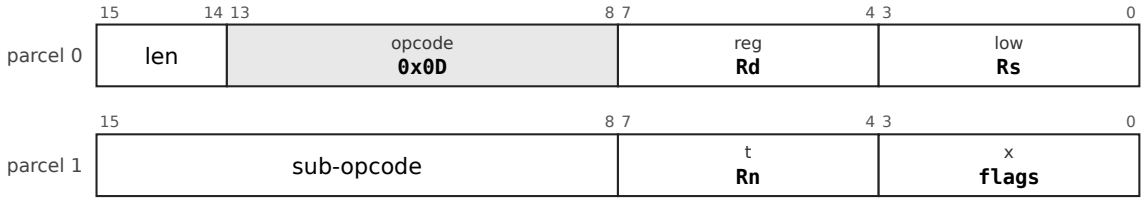
MSCN: Scan a block for a byte, or for a byte not equal to it

Mnemonics: `MSCN`, `MSCNN`

Syntax

```
MSCN Rd, Rv, Rn
MSCNN Rd, Rv, Rn
```

Encoding



`reg` is the destination pointer, `low` the source pointer (or the value register `Rv`), parcel 1 `t` the count register `Rn`.

Mnemonic	Sub-opcode
`MSCN`	`0x04`
`MSCNN`	`0x05`

Operation

```
while Rn != 0:
    hit <- (M8[Rd] == Rv[7:0])            ; MSCNN: hit <- (M8[Rd] != Rv[7:0])
    if hit then  Z = 1 ; stop             ; Rd points AT the byte found
    Rd <- Rd + 1 ; Rn <- Rn - 1
not found: Z = 0
```

Flags All four flags are written (errata 39): `Z` = 1 if the byte was found, else 0; `N`, `V` and `C` are cleared.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

`MSCN` is `memchr`; with an explicit count it gives `strnlen`, and composing it with `MCPY` gives a bounded `strcpy` in two instructions. There are no NUL-terminated string instructions.

Why it is here. MSCN is LOCC and memchr; **MSCNN was added in ISA revision 15 to fill the SKPC gap** (the first byte that is *not* equal). Both cost no opcode and almost no hardware. With an explicit count MSCN gives strnlen, and with MCPY a bounded strcpy in two instructions. NUL-terminated string instructions (the System/z MVST, SRST, CLST) are explicitly rejected, because they encode an unbounded read, the largest single source of memory-safety failures in C (ISA sections 11 and 14.2).

Cf. 6.8.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

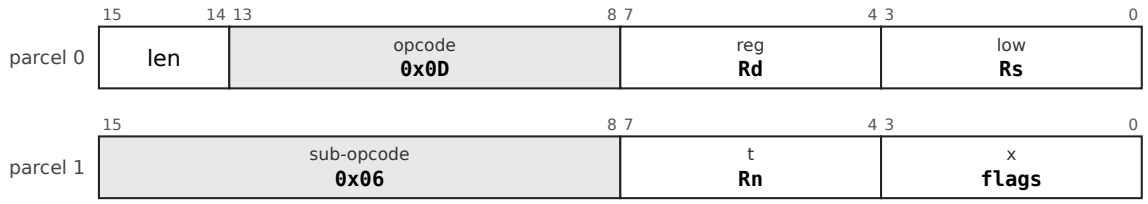
MVAL: Validate a block as UTF-8

Mnemonics: MVAL

Syntax

```
MVAL Rd, Rn
```

Encoding



reg is the destination pointer, low the source pointer (or the value register Rv), parcel 1 t the count register Rn.

Operation

```
scan Rn bytes at Rd; stop at the first byte that cannot be part of a valid UTF-8 sequence (the rules of
UTF8DEC)
```

Flags On completion Rn is 0 and the buffer is valid; otherwise Rd points at the offending sequence and Rn counts the bytes remaining (details in the instruction-set text, section 4.7.1).

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The UTF8DEC comparators wired to the block sequencer.

Why it is here. The design chat asked "what is mblk? what is a swar? validating an entire utf8 string sounds nice... do we do it often enough to justify the opcode? if utf8 is already 3 or 4 parcels, it's ok." ISA revision 15 committed MVAL: the UTF-8 validity comparators already built for UTF8DEC, wired to the MBLK streaming sequencer already built for MCPY, so it costs no opcode and almost no hardware. The alternative, a SWAR loop (SIMD within a register), runs at about two to three cycles a byte and is respectable, which is why the justification is cost and not throughput.

Cf. 6.8.

Status specified; **not yet implemented** in the reference executor.

9.12 Floating point

9.12.1 Floating point uses the general registers: a binary64 value fills one register, and the instruction, not the register, decides how its bits are read. *(D)*

9.12.2 FADD, FSUB, FMUL and FCMP are two-byte instructions; FDIV, FSQRT, the fused multiply-add family, FABS, FMIN, FMAX, FCLASS, the roundings and the binary32 forms are in the floating-point escape; binary16 has seventeen operations and conversions. *(D, errata 24 and 27)*

9.12.3 The five IEEE flags (invalid, divide by zero, overflow, underflow, inexact) accumulate in FPSR; no floating-point operation traps; FSFLAGS reads and clears them. *(D, errata 17)*

9.12.4 Every NaN that an operation creates is the canonical quiet NaN. *(D, errata 26)*

9.12.5 There are no rounding-mode or precision mode bits: the precision is in the opcode, and so is the rounding mode wherever it is not nearest-even. *(D)*

9.12.6 There are no transcendental instructions. *(D)*

Commentary. **Precedent.** The PDP-11's FP11 had `AC0` to `AC5` and mode bits for single and double precision; the VAX had F, D, G and H formats. **Trade-offs.** An integer operation applied to a register holding a `binary64` destroys it, with no trap and no tag; that is intended and must be said aloud in the ABI. A separate register file would cost a second set of read ports and a floating-point context; on a small implementation the register file is among the largest blocks. **Improvements.** The fused multiply-add is the largest single addition to the datapath (about 9,000 transistors) and the least negotiable; the IEEE exception flags are recorded without traps, which is what C99 `<fenv.h>` needs. **Benefits.** No transfer instructions between files and a single calling convention. **Modern comparables.** RISC-V Zfinx and Zdinx use the integer register file for the same reason; AArch64 and x86 have separate vector and floating-point files. **Use.** `FCLASS` begins every library maths function; `FMA` is the building block of accurate polynomial evaluation. **Impact.** *Kernel:* There is no separate floating-point state to save at a context switch beyond the registers already saved, and the sticky flags go with the status word. *Compiler:* One calling convention for integers and floats, fused multiply-add contraction, and no rounding-mode state for the compiler to track. *Hardware:* No second register file (one of the largest blocks of a small floating-point unit); the floating-point unit with fused multiply-add is about 34,000 transistors.

FADD: Floating add, subtract and multiply, binary64

Mnemonics: `FADD`, `FSUB`, `FMUL` (aliases: `FADDD`, `FSUBD`, `FMULD`)

Syntax

```
FADD rm, Rd
FSUB rm, Rd
FMUL rm, Rd
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Mnemonic	Opcode (the load or direct form)
`FADD`	`0x24`
`FSUB`	`0x25`
`FMUL`	`0x26`

Operation

```
Rd <- Rd + rm   (FADD)
Rd <- Rd - rm   (FSUB)
Rd <- Rd * rm   (FMUL)       ; IEEE 754 binary64, round to nearest even
```

Flags None written. The five IEEE exception flags are ORed into `FPSR`; no instruction traps.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Floating-point exceptions never trap.

Floating-point values live in the general registers; there is no separate register file and no mode bit. A NaN result is the canonical quiet NaN `0x7FF8000000000000`. The old spellings `FADDD`, `FSUBD`, `FMULD` are aliases.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

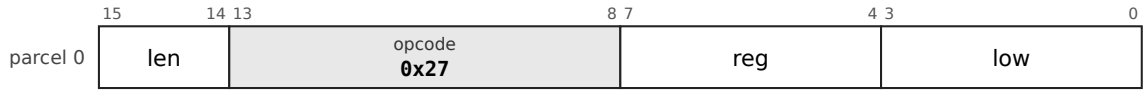
FCMP: Floating compare, binary64

Mnemonics: `FCMP` (aliases: `FCMPD`)

Syntax

```
FCMP Rd, rm        ; register first
```

Encoding



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00`, where the instruction is two bytes. At `len = 01` or `10`, `low` is the addressing mode and parcel 1 (and 2) carry the operand (section 9.5).

Operation

```
N <- Rd < rm ; Z <- Rd == rm ; V <- unordered ; C <- 0
```

Flags `N` set if `Rd < rm`, `Z` if equal, `V` if the operands are unordered (either is a NaN), `C` cleared. A signalling NaN raises `NV` in `FPSR`.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The signed branches do not yet treat the unordered case cleanly (`BLT` after an unordered compare takes `N xor V` = 1); this is an open item of the instruction set.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

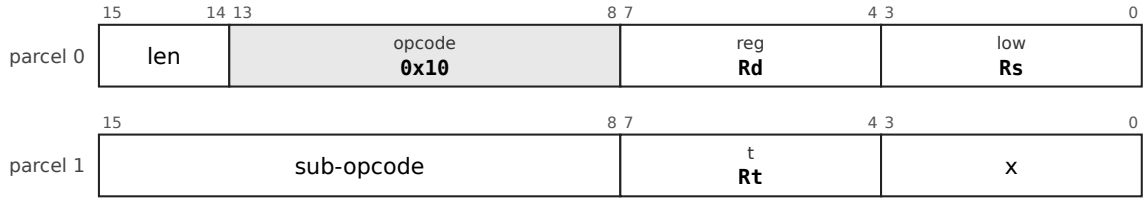
FDIV: Floating divide and square root, binary64

Mnemonics: `FDIV`, `FSQRT` (aliases: `FDIVD`, `FSQRTD`)

Syntax

```
FDIV Rs, Rd
FSQRT Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
`FDIV`	`0x00`
`FSQRT`	`0x01`

Operation

```
FDIV : Rd <- Rd / Rs
FSQRT: Rd <- sqrt(Rs)
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

`FDIV` and `FSQRT` are multi-cycle sequencer operations and rarely appear in a two-byte form, so they are in `FESC`. The old spellings `FDIVD`, `FSQRTD` are aliases.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

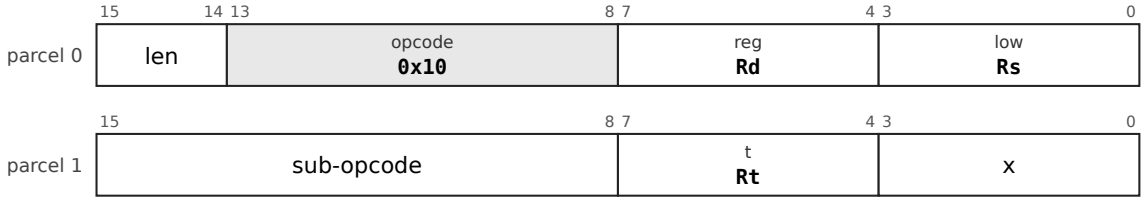
FSINGLE: Floating arithmetic, binary32

Mnemonics: FADD32, FSUB32, FMUL32, FDIV32, FSQRT32, FCMP32 (aliases: FADDS, FSUBS, FMULS, FDIVS, FSQRTS, FCMPS)

Syntax

```
FADD32 Rs, Rd
FSUB32 Rs, Rd
FMUL32 Rs, Rd
FDIV32 Rs, Rd
FSQRT32 Rs, Rd
FCMP32 Rd, Rs
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
FADD32	0x08
FSUB32	0x09
FMUL32	0x0A
FDIV32	0x0B
FSQRT32	0x0C
FCMP32	0x0D

Operation

```
Rd <- Rd op Rs  (FADD32 FSUB32 FMUL32 FDIV32); Rd <- sqrt(Rs) (FSQRT32)
FCMP32: flags as FCMP, comparing Rd with Rs
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps. FCMP32 writes `N Z V C` as FCMP does.

Exceptions None.

A `binary32` lives in the low 32 bits of a register, sign-extended to 64 like every 32-bit result. Results are correctly rounded to nearest even. The old spellings FADDS, FSUBS, ... are aliases.

Why it is here. `binary64` has first-class opcodes because on a 64-bit machine it is the common case; `binary32` is in FESC with the same operations, held in the low 32 bits of a register and sign-extended like every other 32-bit value (ISA section 4.10, and the ABI). The old spellings (FADDS and so on) became aliases when the author fixed the naming rule on 2026-10-21: "putting f in front for floating point ops is ok, `fadd` and I guess `fadd16`" (errata 24).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

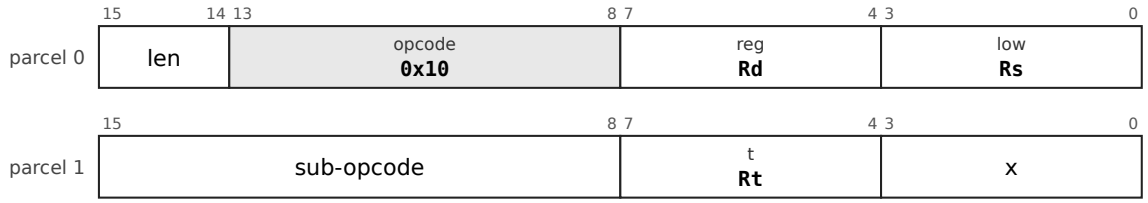
FMADD: Fused multiply-add, binary64

Mnemonics: FMADD, FMSUB, FNMADD, FNMSUB (aliases: FMADDD, FMSUBD, FNMADDD, FNMSUBD)

Syntax

```
FMADD Rs, Rt, Ru, Rd
FMSUB Rs, Rt, Ru, Rd
FNMADD Rs, Rt, Ru, Rd
FNMSUB Rs, Rt, Ru, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
FMADD	0x10
FMSUB	0x11
FNMADD	0x12
FNMSUB	0x13

Operation

```
FMADD : Rd <- Rs*Rt + Ru
FMSUB : Rd <- Rs*Rt - Ru
FNMADD: Rd <- -(Rs*Rt) - Ru
FNMSUB: Rd <- -(Rs*Rt) + Ru          ; each rounded once
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

Four register operands are available at `len = 01`. The old spellings `FMADDD`, ... are aliases.

Why it is here. ISA revision 13 added the whole fused multiply-add family to `FESC` because its absence "was the single worst gap in the floating-point set": one rounding instead of two, and the inner step of dot products, polynomial evaluation and every matrix kernel. `FESC` gives it four register operands (errata 22). It replaces the VAX `POLY`, which section 11 of the ISA calls the best answer before fused multiply-add and now superseded by it.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

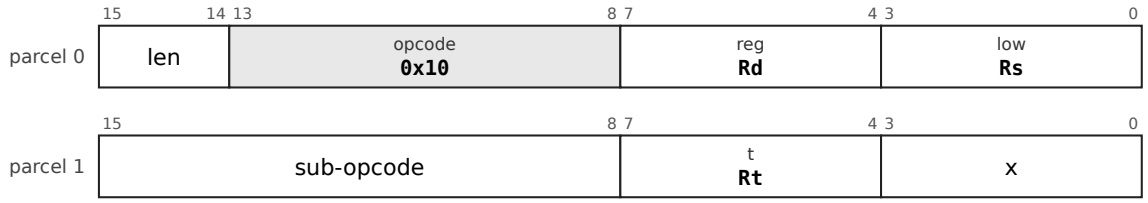
FMADD32: Fused multiply-add, binary32

Mnemonics: `FMADD32`, `FMSUB32`, `FNMADD32`, `FNMSUB32` (aliases: `FMADDS`, `FMSUBS`, `FNMADDS`, `FNMSUBS`)

Syntax

```
FMADD32 Rs, Rt, Ru, Rd
FMSUB32 ...
FNMADD32 ...
FNMSUB32 ...
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
FMADD32	0x18
FMSUB32	0x19
FNMADD32	0x1A

Mnemonic	Sub-opcode
FNMSUB32	0x1B

Operation

```
As FMADD, in binary32, rounded once
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

Exceptions None.

Why it is here. The binary32 forms of the fused multiply-add family, added with it in ISA revision 13.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

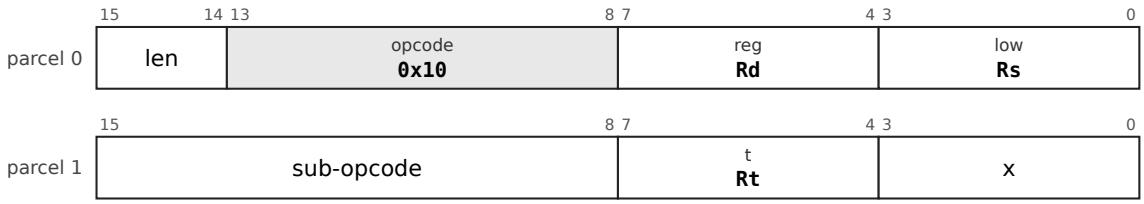
FSIGN: Sign operations on the bits

Mnemonics: FNEG, FABS, FCOPYSIGN

Syntax

```
FNEG Rs, Rd
FABS Rs, Rd
FCOPYSIGN Rs, Rt, Rd
```

Encoding



From len = 01. In this group low is a register, never a mode. Operand roles: reg is the destination Rd, low the first source Rs, t the second Rt and x the third Ru; the assembler writes the destination last.

Mnemonic	Sub-opcode
FNEG	0x20
FABS	0x21
FCOPYSIGN	0x22

Operation

```
FNEG     : Rd <- Rs with the sign bit flipped
FABS     : Rd <- Rs with the sign bit cleared
FCOPYSIGN: Rd <- magnitude of Rs, sign of Rt
```

Flags None. These operate on the bits, do not canonicalise a NaN and raise no exception flag.

Exceptions None.

Why it is here. Part of the floating-point set of the instruction-set text (section 4.10, “sign and selection”): they work on the bits, do not canonicalise a NaN and raise no flag, so they cost almost nothing. When the author asked on 2026-10-21 what the PowerPC 601 had in floating point and “any we may want to include?”, the comparison (made from memory and not checked against a manual) listed negate, absolute value and the like on the PowerPC side as well, and what the machine lacked was recorded in errata 17, 18 and 19. The record gives no separate reason for these three.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

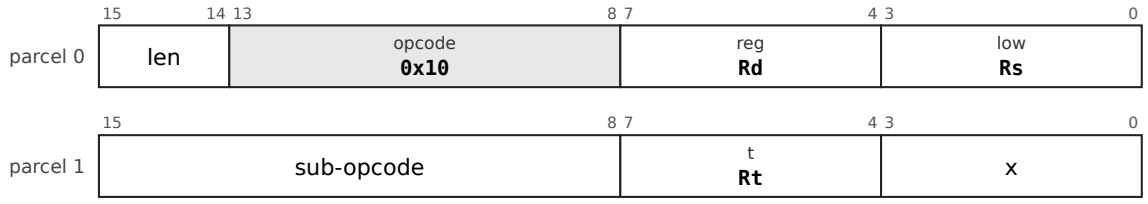
FMIN: Floating minimum and maximum, binary64

Mnemonics: FMIN, FMAX (aliases: FMIND, FMAXD)

Syntax

```
FMIN Rs, Rt, Rd
FMAX Rs, Rt, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
FMIN	0x23
FMAX	0x24

Operation

```
if Rs is a NaN then Rd <- Rt ; else if Rt is a NaN then Rd <- Rs ; else Rd <- smaller (FMIN) or larger
(FMAX)
if both are NaNs the result is the canonical quiet NaN
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

The old spellings `FMIND`, `FMAXD` are aliases.

Why it is here. In the original floating-point set (ISA section 4.10), as the floating counterparts of the integer `MIN` and `MAX` added later on the author's request (errata 23). A NaN operand loses to a number and two NaNs give the canonical quiet NaN (errata 26, decided in `sim64`).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

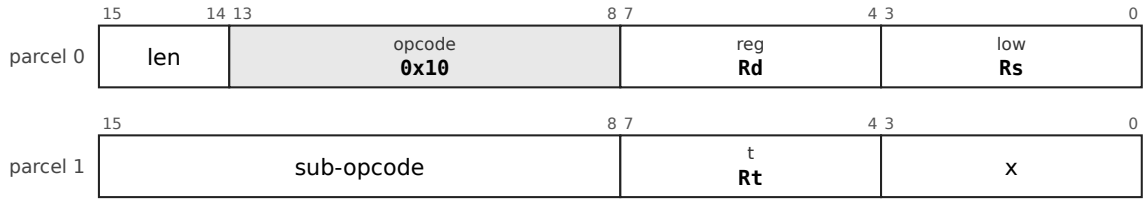
FCLASS: Classify a floating-point value

Mnemonics: `FCLASS`, `FCLASS32`, `FCLASS16` (aliases: `FCLASSD`, `FCLASSS`)

Syntax

```
FCLASS Rs, Rd
FCLASS32 Rs, Rd
FCLASS16 Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
FCLASS	0x28
FCLASS32	0x29
FCLASS16	0x5C

Operation

```
Rd <- a bit vector with exactly one bit set:
bit 0 -infinity      bit 1 -normal      bit 2 -subnormal   bit 3 -zero
bit 4 +zero          bit 5 +subnormal   bit 6 +normal      bit 7 +infinity
bit 8 signalling NaN bit 9 quiet NaN
```

Flags None.

Exceptions None.

Every libm function begins with this dispatch. The old spellings `FCLASSD`, `FCLASSS` are aliases.

Why it is here. "Every libm function begins with this dispatch" (ISA section 4.10): one instruction returns which of ten kinds a value is (signalling or quiet NaN, infinity, normal, subnormal, zero, each signed), where software needs several compares and masks. It is in the original floating-point set; the assistant listed it among the items justified by a workload and not by compiled C, together with the tag branches, and recommended waiting for the frequency table before cutting any (this chat, 2026-10-21). Nothing in the corpus emits it, because the compiler is not taught to.

Status implemented in the reference executor for `FCLASS`, `FCLASS16`; not yet for the others.

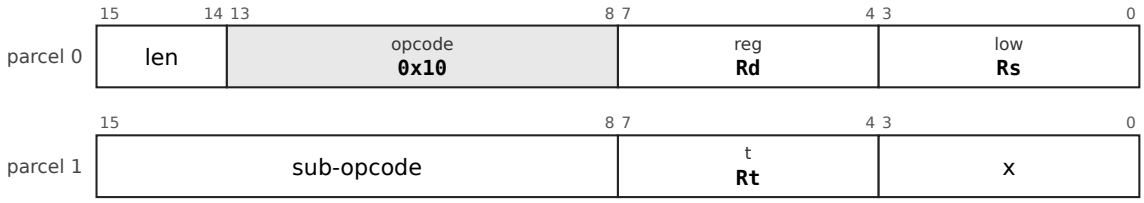
FRINT: Round to an integral value, binary64

Mnemonics: `FRINT`, `FRINTZ`, `FRINTM`, `FRINTP`, `FRINTA` (aliases: `FRINTD`, `FRINTDZ`, `FRINTDM`, `FRINTDP`, `FRINTDA`)

Syntax

```
FRINT Rs, Rd
FRINTZ Rs, Rd
FRINTM Rs, Rd
FRINTP Rs, Rd
FRINTA Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
`FRINT`	`0x30`
`FRINTZ`	`0x31`
`FRINTM`	`0x32`
`FRINTP`	`0x33`
`FRINTA`	`0x34`

Operation

```
FRINT : Rd <- Rs rounded to nearest, ties to even
FRINTZ: toward zero         FRINTM: toward -infinity
FRINTP: toward +infinity    FRINTA: to nearest, ties away from zero (C round())
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

The rounding mode is in the opcode, never in a mode bit. The old spellings `FRINTD`, `FRINTDZ`, ... are aliases.

Why it is here. Rounding to an integral value in each direction (nearest even, toward zero, down, up) is in the original set. **`FRINTA`, ties away from zero, was added on 2026-10-21** after the PowerPC 601 comparison, for C's `round()`, which

is common in libm; it is a qualifier of the round operation and so costs no new operation (errata 19). The author's reaction: "add those 3 things, i like having the rounding variants" (the other two were the exception flags and the unsigned conversions).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

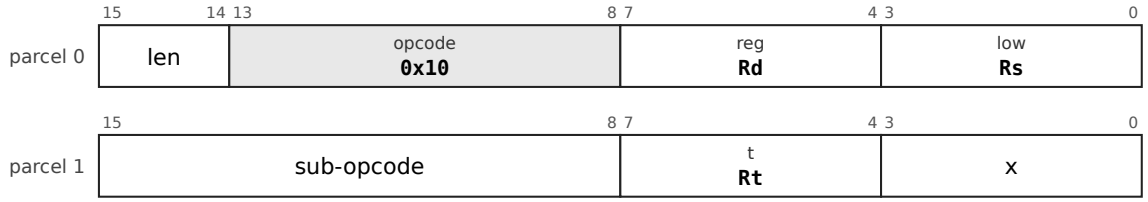
FCVT: Floating conversions

Mnemonics: `FCVTDQ`, `FCVTDQU`, `FCVTQD`, `FCVTQUD`, `FCVTDS`, `FCVTSD`, `FCVTSL`, `FCVTLS`

Syntax

```
FCVTDQ Rs, Rd        ; binary64 to signed quad
FCVTDQU Rs, Rd       ; binary64 to unsigned quad
FCVTQD Rs, Rd        ; signed quad to binary64
FCVTQUD Rs, Rd       ; unsigned quad to binary64
FCVTDS Rs, Rd        ; binary64 to binary32
FCVTSD Rs, Rd        ; binary32 to binary64
FCVTSL Rs, Rd        ; binary32 to signed 32-bit integer
FCVTLS Rs, Rd        ; signed 32-bit integer to binary32
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
FCVTDQ	0x38
FCVTDQU	0x3E
FCVTQD	0x39
FCVTQUD	0x3F
FCVTDS	0x3A
FCVTSD	0x3B
FCVTSL	0x3C
FCVTLS	0x3D

Operation

```
float to integer: truncate toward zero, then saturate; a NaN gives the largest value
integer to float: round to nearest even
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. A NaN or an out-of-range value raises `NV` and saturates.

Exceptions None.

Integer results are 64-bit (`FCVTDQ`, `FCVTDQU`) or 32-bit sign-extended (`FCVTSL`). Unsigned conversion of a negative value is invalid and gives 0.

Why it is here. The conversions were in the original set. **`FCVTDQU` and `FCVTQUD`, unsigned 64-bit to and from `binary64`, were added on 2026-10-21** because C needs both and without them each takes a branch and several instructions; the author approved them with the other two additions ("add those 3 things"; errata 18).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

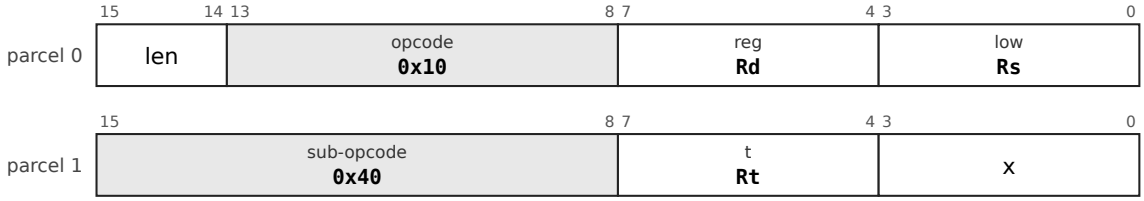
FSFLAGS: Swap the floating-point exception flags

Mnemonics: `FSFLAGS`

Syntax

```
FSFLAGS Rs, Rd
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Operation

```
old <- FPSR
FPSR <- Rs[4:0]
Rd <- old
```

Flags None.

Exceptions None.

`FPSR` is the sticky five-bit register (`NX` bit 0, `UF` 1, `OF` 2, `DZ` 3, `NV` 4). C99 `<fenv.h>` is built on this one instruction.

Why it is here. The one thing missing for IEEE 754 conformance, found on 2026-10-21 while answering the author's question about the PowerPC 601's floating-point operations: a search of the instruction-set text for flags, `fenv`, "inexact" and "sticky" found nothing, yet IEEE requires five status flags and C99 `<fenv.h>` needs software access to them. The fix is a sticky five-bit `FPSR`, laid out as RISC-V's `fflags`, written by every operation that can raise a flag and never trapping, and one swap instruction to read and write it. It is a record of what happened, not a mode, so the rule against mode bits is untouched. The author approved it with "add those 3 things" (; errata 17).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

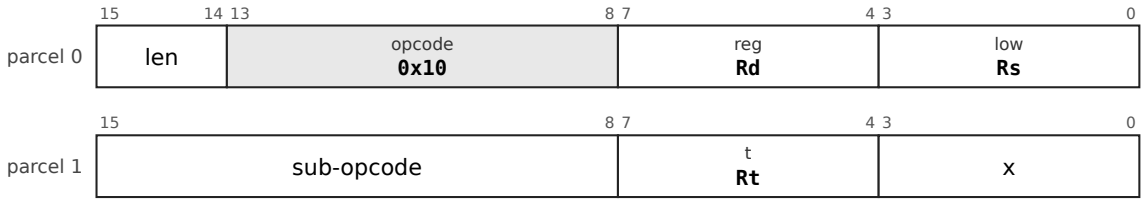
FHALF: Floating arithmetic, binary16

Mnemonics: `FADD16`, `FSUB16`, `FMUL16`, `FDIV16`, `FSQRT16`, `FCMP16`, `FMADD16`, `FMSUB16`, `FNMADD16`, `FNMSUB16`

Syntax

```
FADD16 Rs, Rd
FSUB16 Rs, Rd
FMUL16 Rs, Rd
FDIV16 Rs, Rd
FSQRT16 Rs, Rd
FCMP16 Rd, Rs
FMADD16 Rs, Rt, Ru, Rd ...
```

Encoding



From `len = 01`. In this group `low` is a register, never a mode. Operand roles: `reg` is the destination `Rd`, `low` the first source `Rs`, `t` the second `Rt` and `x` the third `Ru`; the assembler writes the destination last.

Mnemonic	Sub-opcode
`FADD16`	`0x50`

Mnemonic	Sub-opcode
FSUB16	0x51
FMUL16	0x52
FDIV16	0x53
FSQRT16	0x54
FCMP16	0x55
FMADD16	0x58
FMSUB16	0x59
FNMADD16	0x5A
FNMSUB16	0x5B

Operation

```
as the binary32 words, in binary16: Rd <- Rd op Rs ; the fused forms take Rs, Rt, Ru, Rd
a half lives in the low 16 bits of a register, sign-extended to 64
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps. FCMP16 writes N Z V C as FCMP.

Exceptions None.

Results are correctly rounded to nearest even, tininess judged after rounding; a NaN result is the canonical 0x7E00. Half FMIN, FMAX, FRINT*, bfloat16 and conversions to and from 64-bit integers are open items.

Why it is here. The author raised half precision on 2026-10-21 (the assistant's answer opens "Is FP16 a GPU thing?" and goes on "You're right about the gradients"). The answer, from memory and not checked against a manual, was that it began as one (NVIDIA's half, 2002), spread to machine learning and image formats, and is now in CPUs (x86 F16C and AVX-512 FP16, ARMv8.2, RISC-V Zfh and Zfhmin). The assistant laid out two levels, conversions only (about 500 to 1,000 lookup tables, no new operation by the folding rule) and full arithmetic (about 2,000 to 3,500 tables, estimates of its own), and noted that the existing double datapath can compute half correctly rounded. On 2026-10-22 the author decided: "add fp16 conversions and arithmetic". The leftovers (half FMIN, FRINT*, bfloat16, 64-bit integer conversions) are open items. The related question of 16-bit *integer* arithmetic was treated as a design item and decided against (errata 28).

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

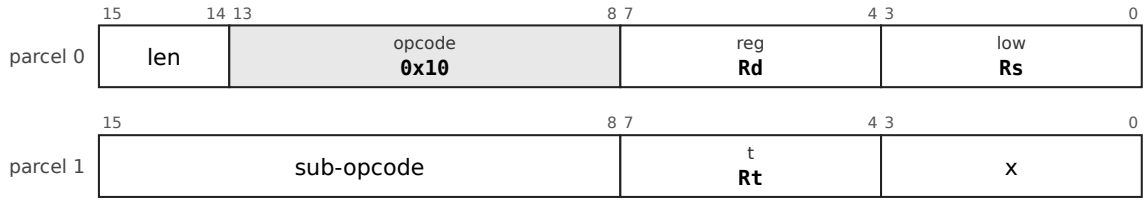
FCVTH: Conversions to and from binary16

Mnemonics: FCVTSH, FCVTHS, FCVTDH, FCVTHD, FCVTHL, FCVTLH

Syntax

```
FCVTSH Rs, Rd        ; binary32 to half
FCVTHS Rs, Rd        ; half to binary32
FCVTDH Rs, Rd        ; binary64 to half
FCVTHD Rs, Rd        ; half to binary64
FCVTHL Rs, Rd        ; half to signed 32-bit integer, toward zero
FCVTLH Rs, Rd        ; signed 32-bit integer to half
```

Encoding



From len = 01. In this group low is a register, never a mode. Operand roles: reg is the destination Rd, low the first source Rs, t the second Rt and x the third Ru; the assembler writes the destination last.

Mnemonic	Sub-opcode
FCVTSH	0x60
FCVTHS	0x61
FCVTDH	0x62
FCVTHD	0x63

Mnemonic	Sub-opcode
FCVTHL	0x64
FCVTLH	0x65

Operation

```
narrowing: round to nearest even ; widening: exact
FCVTHL of a NaN or an infinity is invalid and saturates; truncating a fraction is inexact
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

Exceptions None.

Why it is here. The conversion half of the binary16 decision (see FHALF): on 2026-10-22 the author asked for "fp16 conversions and arithmetic". The conversions are the part machine-learning code needs most, because a half is normally loaded and stored with MOVW, converted, computed in single or double, and converted back.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

9.13 Atomics and ordering

Cf. chapter 7, section 7.12 (the atomic and ordering instructions), section 7.3 (the atomics in detail) and section 7.2 (the memory model).

9.13.1 The atomic escape ATOM (opcode 0x11) holds CAS, SWAP, XADD, XAND, XOR_, XXOR, XMAX, XMIN, XMAXU and XMINU, each with a _DB variant that adds a barrier, and LL and SC; FENCE, WFE, SEV, ISYNC and HINT are in SYS. Their semantics, ordering rules and the reservation monitor are in the chapter named above. *(opcode 0x11)*

Commentary. The instructions are described where their effect is: the multiprocessor chapter. **Impact.** *Kernel:* see 7.12. *Compiler:* a lock acquire is one _DB instruction. *Hardware:* a reservation monitor beside the data cache.

9.14 System instructions, exceptions and interrupts

Cf. chapter 6, section 6.7 (system instructions, exceptions and interrupts), section 6.2 (exceptions and interrupts) and section 6.3 (system registers).

9.14.1 The system escape SYS (opcode 0x14) holds the privileged and system instructions: TRAP, TRAPL, RTI, RTT, RET, HALT, BPT, MTPS, MFPS, SPL, MFSR, MTSR, MTDP, MFDP, MFCPUID, PROBER, PROBEW, PUSHX, POPX, and the translation and cache maintenance instructions (TLBI, TLBIA, TLBSYNC, MTASID, MFASID, ICINV, ICINVA). The user-memory guard instructions STUA and CLUA are described with them. *(opcode 0x14)*

Commentary. The modes, the entry sequence and the system registers are the kernel chapter's; this chapter lists the instructions. **Impact.** *Kernel:* see 6.7. *Compiler:* none, apart from TRAP in system-call stubs. *Hardware:* a class decode and one privilege check.

9.15 Faults, restart and memory management

Cf. chapter 6, section 6.8 (faults, restart and memory management) and section 6.4 (memory management).

9.15.1 A fault is a trap, an abort, a progress stop in a block operation or a machine check, and every instruction either completes or is annulled and restarted with no state to undo (MCPY and its kin keep their progress in registers); the page-table format, the memory types and the translation registers are in the kernel chapter. *(D)*

Commentary. The instruction-set consequence is the restart rule: the register update of an auto-increment mode is held until the instruction retires. **Impact.** *Kernel:* see 6.8. *Compiler:* no scheduling restriction arises from faults. *Hardware:* a single-entry undo buffer and a read-only walker.

9.16 Position-independent code

9.16.1 Position-independent code costs nothing extra on this machine: every access to code or data in the same module is PC-relative by an ordinary addressing mode, so there is no GOT register and no prologue to set one up. *(D, ABI version 1)*

9.16.2 A call to an imported function is CALLI got_slot(PC) and a PLT stub is the single instruction JMPI got_slot(PC); an imported variable's address is one MOVQ got_slot(PC), Rd. *(D)*

9.16.3 A position-independent access to a global is no longer than an absolute one: four bytes within 32 KB and six within 2 GB, against six for a 32-bit absolute address. *(D)*

9.16.4 The thread pointer is `DP`, so the local-exec thread-local model is one four-byte `MOVQ tls_offset(DP), Rd`. *(D)*

9.16.5 A branch at the middle length reaches 4 GB, so no linker veneers are needed for any binary smaller than that; the assembler's and the linker's branch relaxation both reduce to choosing a length. *(D)*

9.16.6 `R12` is the linker's scratch register. Lazy against eager binding, and whether imports are always called without a PLT stub, are decided for version 1 (no stub) and open for later versions. *(D for `R12` and no stub; open for binding)*

Commentary. **Precedent.** On i386 a position-independent executable reserved `%ebx` (one of eight registers) for the GOT base and set it in every prologue, which cost a few percent of size and speed; x86-64 fixed that with RIP-relative addressing, and AArch64 needs linker veneers because its branches reach only 128 MB. **Trade-offs.** The two-byte PC-relative form is the shortest, so position-independent data access is slightly smaller than absolute access; the price is a GOT load per imported call site where a PLT stub would cost a load per function. **Improvements.** Position-independence is a side effect of three decisions made for other reasons: PC-relative as an ordinary mode (adopted because the PC left the register file), variable-length encoding (for density) and `DP` (a 6502 zero-page descendant kept for density), which is exactly the thread pointer. **Benefits.** OpenBSD builds everything position-independent by default, and here that costs nothing. **Modern comparables.** x86-64 RIP-relative addressing; AArch64 `ADRP` with a GOT load and veneers for far calls. **Impact.** *Kernel:* a loader applies only relocations for data pointers, and write-xor-execute and randomisation come with no code penalty. *Compiler:* `-fPIE` is the default and emits the same instructions as `-fno-pic` for local data; no reserved register, no per-function setup. *Hardware:* nothing; the PC-relative mode is in the address adder already.

9.17 What was rejected

9.17.1 The architecture refuses, by design, every feature that adds hidden state, an unbounded instruction, a second operand path or a mode bit that changes what other instructions mean. *(D)*

9.17.2 From the PDP-11 and VAX it refuses general memory-to-memory operands, deferred addressing as a general form (only `JMPI` and `CALLI` are deferred), unbounded instructions with hidden microstate (`MOVC5`, `EDITPC`), packed decimal, hardware call frames (`CALLS`, `CALLG`), the PC as a general register, multiple floating-point formats, `POLY`, the `IOT`, `EMT`, `WAIT` and `RESET` instructions, a distinct low-half unsigned multiply, and the PDP-11's compare operand order. *(D)*

9.17.3 From x86 it refuses partial-register writes that preserve upper bits, operand-size and `REX` prefixes, segmentation, flag writes that vary by instruction, implicit registers for multiply, divide and shift count, the x87 stack, transcendental instructions and byte-granular instruction fetch. *(D)*

9.17.4 From the RISC family it refuses a register hardwired to zero (a sixteenth of a 16-register file), fused compare-and-branch in place of flags, branch delay slots, register windows and software-refilled translation buffers. *(D)*

9.17.5 It also refuses features of its own invention: NUL-terminated string instructions (`MSCN` with an explicit maximum gives a bounded `strlen`), `CRC32` words (carry-less multiply is the primitive under every CRC), reciprocal-estimate instructions, NaN payload propagation, a 32-bit profile, an eight-byte instruction alignment, `MOVT` and `MOVK` partial-immediate inserts, hardware tag checking, store-with-write-barrier, a self-synchronising encoding, and a scalar instruction for sub-1-bit inference (the accelerator of chapter 8 does it). *(D)*

Commentary. **Precedent.** The VAX's `EDITPC` is a bytecode interpreter in microcode, and DEC needed a separate control chip for the PDP-11's commercial instruction set and shipped none; the VAX's hardware call frames ran slower than the equivalent sequence on later models; ARM32's `PC+8` and MIPS's delay slot expose one implementation's pipeline forever. **Trade-offs.** Each refusal gives up a real convenience: the lost `MOV (R1)+,(R2)+` copy loop, the byte-wide `ADDB`, the inline-argument `JSR`. Section 9.18 says what replaces each. **Improvements.** The common pattern is that a refused feature was either subsumed by a more general one (`MCPY` for the copy loop) or paid its cost in every implementation for a few users (`EDITPC`). **Benefits.** An instruction set a small in-order core can implement completely and a compiler can target without exceptions. **Modern comparables.** RISC-V and AArch64 also refuse memory-to-memory operands and delay slots; AArch64 keeps flags where RISC-V refuses them. **Impact.** *Kernel:* no instruction has a restart state except the three that keep their progress in architectural registers, so a fault is always restartable. *Compiler:* no special cases: every operand rule is uniform, one memory operand and the rest registers. *Hardware:* no microcode engine, no second operand path, no hidden-state latch, no alignment rotator in the fetch path.

9.18 Idioms: restored, lost and replaced

9.18.1 Several idioms that the PDP-11 and VAX had and later RISC designs dropped are restored: subtract-one-and-branch (`SOB`, here in both directions), probe an address without faulting (`PROBER`, `PROBEW`), bit-field extract and insert, find-first-set (`CTZ`, `CLZ`), byte swap, block move, fill, compare and scan with their progress held in registers, the register-save mask (`PUSHM`, `POPM`, the useful half of `CALLS`), and carry-less multiply. *(D)*

9.18.2 The VAX's counted-string instructions are largely present: `MOVC3` is `MCPY`, `CMPC3` is `MCMP`, `LOCC` is `MSCN`, `SKPC` is `MSCNN`, and `MOVC5` is `MCPY` plus `MSET`. Table-translated move, table scans and substring search are absent because a per-byte table lookup stops a block operation from streaming. *(D)*

9.18.3 Deliberately lost, with their replacements: memory-to-memory `MOV` (replaced by `MCPY`); autoincrement-deferred and the dispatch-table idiom (replaced by `JTAG` for tag dispatch and a scaled-index `JMP` for the rest); `JSR Rn` and `RTS Rn` with inline arguments and cheap coroutines (replaced by `CALL` and `RET` with the return address on the stack where every unwinder and debugger expects it); the VAX `CASE` and `INDEX` instructions (replaced by a compare, branch and scaled `JMP`); byte and word ALU operations (`MOVZB`, the operation, `MOVB`: three instructions where the PDP-11 had one); and the flag set and clear instructions. *(D)*

9.18.4 Subsumed, so not losses: `SXT` (the conversion moves), `SWAB` (`BSWAP`), `MARK` (the convention it served is gone), `ASH` and `ASHC` (the shifts take a register count), `MFPI` and `MTPI` (`PROBER` with the previous-mode bits and the user-memory guard), `MULU` (identical to `MUL` in the low half), and `ROL` (an assembler alias for `ROR`). *(D)*

9.18.5 Measurements decided what was added: the compiler uses `SET cc, rd` (about 0.4% of code), not integer `ABS`, `MIN` or `MAX` (under 0.1%), not the bit-field instructions (0.03%) and not the auto-increment modes (0.1%); the second opcode page is about 7% of the instructions emitted. *(D, with the figures in the instruction-set text, section 14.4)*

Commentary. **Precedent.** The PDP-11's `JSR Rn` put the return address in a register of the caller's choosing, which made coroutines a pair of `JSR`s and inline arguments free; the VAX replaced it with `CALLS` and a frame, which was slower than the code it replaced. **Trade-offs.** The most frequently felt loss in hand-written code is the byte-wide read-modify-write; the compiler never notices it. A bounds-checked `INDEX` for Pascal and Ada is a plausible later addition. **Improvements.** `SOB` branches either way, with four reaches; the block operations keep their progress in registers, which is what the VAX got wrong, so a fault restarts for free. **Benefits.** The code a compiler emits is the code the table above measures, and the extras are for the people who write assembler. **Modern comparables.** x86 keeps `LOOP` and the string instructions with `REP`; AArch64 has bit-field moves and no memory-to-memory forms; RISC-V has none of them in the base. **Impact.** *Kernel:* `PROBER` and `PROBEW` replace the `nofault` global in the user-copy routines. *Compiler:* idioms the compiler never emits are not built for it, so the compiler's instruction selection stays small. *Hardware:* each restored idiom is a few hundred gates of existing datapath, and each lost one is a decode that never existed.

9.19 Side by side: the PDP-11, the VAX and the pdp64

9.19.1 The word is 64 bits against the PDP-11's 16 and the VAX's 32, the registers are 16 as on the VAX, the virtual address is 39 bits with 40 physical, and the instruction count is 58 assigned opcodes plus 45 on the second page, against the VAX's 304. *(D)*

9.19.2 The program counter is not a register, there is no argument pointer, and the frame pointer and the linker scratch are ordinary software conventions. *(D)*

9.19.3 An instruction has one or two operands (three for the block operations) and at most one memory operand, against the PDP-11's two and the VAX's six. *(D)*

9.19.4 The condition codes are the same four, `N`, `Z`, `V` and `C`; floating point is in the base and in the general registers, with three IEEE formats (half, single, double) and no mode bits that change the meaning of other instructions, where the PDP-11 had `FPS.FD`, `FL` and `FT` and the VAX had rounding bits in the status word. *(D)*

9.19.5 An instruction is two, four or six bytes (one to three parcels of 16 bits) with a one-parcel second page, against the PDP-11's two to six and the VAX's one to more than fifty. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX instruction formats, with their encodings drawn side by side, are in the instruction-set text, section 13. **Trade-offs.** The pdp64 keeps what made the PDP-11 pleasant, orthogonal addressing and a small register set, and what made the VAX powerful, flags, a register-save mask and string operations, and drops what made both expensive: memory-to-memory operands and unbounded instructions. **Improvements.** The compiler's code

averages about 3.3 bytes an instruction, measured on the fair corpus before the second page. **Benefits.** A processor that is smaller than either predecessor while running 64-bit code. **Modern comparables.** The closest family is AArch64 in the number of registers and addressing modes and RISC-V compressed in density; neither has flags and a variable length together. **Impact.** *Kernel:* the exception model is the PDP-11's with the VAX's per-mode stacks. *Compiler:* one memory operand makes instruction selection a register-allocation problem, as on a RISC. *Hardware:* a decoder that reads two bits to find the length, and no microcode.

9.20 What was taken from LoongArch, and what was not

9.20.1 LoongArch64 is the most recent clean-room instruction set designed without an ideological commitment: it borrowed from MIPS, RISC-V and ARM as each had the better answer, which is the posture this book takes. *(the author's design principle)*

9.20.2 Adopted from it: bit manipulation in the base (`CLO`, `CLZ`, `CTZ`, bit-field moves, byte reverse), atomics with the barrier folded in (the `_DB` variants), a richer atomic set (fetch-and-minimum and maximum), and memory type as a page attribute. *(D)*

9.20.3 Declined, with reasons: `R0` hardwired to zero (a sixteenth of a 16-register file, and `CLR` and `NEG` are one parcel already); no condition codes (a third read port in the branch unit against a four-bit latch); `FCSR` rounding-mode bits (hidden state that changes other instructions); four PC-relative address-formation instructions (one `LEA` carries 32 bits); 32 registers (this encoding has no bit to spare for the field); and separate vector register files (chapter 8). *(D)*

9.20.4 Still under consideration: separate floating-point condition flags, software-visible translation-buffer manipulation (to pin an entry for a real-time kernel), and a conditional select for constant-time code. *(P)*

Commentary. **Precedent.** LoongArch has a fixed 32-bit encoding with room for two register fields and a displacement, so it can afford compare-and-branch and needs four instructions to form an address; this machine's variable length reverses both trade-offs. **Trade-offs.** The direct-mapping windows LoongArch uses to avoid translation-buffer misses on kernel text were considered and deferred (errata 44). **Improvements.** The lesson is that the same idea is right or wrong depending on the encoding around it, and the book keeps asking which. **Benefits.** Borrowing the tested parts saves design time and avoids their bugs. **Modern comparables.** LoongArch itself; the RISC-V bit-manipulation extensions (Zbb, Zbs) do much of the same. **Impact.** *Kernel:* atomics with barriers make a lock acquire one instruction. *Compiler:* bit operations in the base mean the compiler can use them without a feature test. *Hardware:* each adopted idea is a few hundred lookup tables and each declined one is a block not built.

9.21 What is still open in the instruction set

9.21.1 Open, with the reasoning in appendix B or in the instruction-set text, section 12: the thin 6-bit displacement of the shortest scaled-index form; the landing-pad decision (optional now, mandatory later); atomics against the bus (the lock against the monitor, settled in favour of the monitor by errata 45); worst-case instruction latency for the block operations (an interrupt-latency claim must state the block granularity); the floating-point condition flags (the signed branches do not treat unordered cleanly); the block operations and the cache (a streaming memory type or a non-temporal variant); directed rounding for arithmetic (a rounding field in the instruction); the leftovers of half precision (minimum, maximum, rounding, bfloat16); conditional select; lazy against eager binding in the ABI; the verification of the memory model against published litmus tests; and the coherence protocol if the core count passes four. *(P and open)*

9.21.2 Resolved since the first listing: the reserved-opcode discipline (a top-level opcode is for an operation whose common case is register to register, written down), the fourth parcel (removed, and the code `11` is a second page of 64 one-parcel opcodes), UTF-8 scope (`MVAL` is committed), and the measured length distribution (2, 4, 6 and 8 bytes at 46.5, 42.8, 10.6 and 0.1 percent, before the second page). *(D)*

Commentary. **Trade-offs.** None of the open items blocks the first implementation; the three that must be settled before silicon, the memory model verification, the coherence choice and the worst-case latency figure, are verification tasks, not design changes. **Impact.** *Kernel:* the open items touching the kernel are the user-memory guard, the floating-point state and the block-operation granularity. *Compiler:* conditional select and directed rounding would change instruction selection. *Hardware:* the coherence protocol must be chosen before the cache is built, not after.

Chapter 10 — Conclusion: from FPGA to silicon

The pdp64 is built to be run first on a modest FPGA, to be small enough that its whole core is about sixteen thousand lookup tables or ninety thousand transistors, and to reach silicon only after an OpenBSD built by the project's own toolchain can be logged into over ssh. This chapter gives the FPGA plan and its numbers, the transistor budget for a chip, why the first implementations are in order and how the architecture is kept ready for a faster one, and the path from today's state to the finish line. Tags: *(D)* decided; *(E)* an estimate, not a measurement.

10.1 The FPGA plan

10.1.1 The first implementation is a soft core on a system-on-chip FPGA whose hard ARM cores serve as the bring-up harness: they load programs, run the tests and provide the console while the pdp64 core is still being debugged. *(D, plan)*

10.1.2 The first board is the Terasic DE10-Nano (Cyclone V SE: about 110,000 logic elements, dual Cortex-A9, 1 GB of memory on the hard processor, HDMI, and the MiSTer ecosystem); the Digilent Zybo Z7-20 or the AMD Kria KV260 are the move when timing becomes the limit; the open-toolchain ULX3S is kept in view but loses the hard-core harness. *(D, plan)*

10.1.3 A complete core is about 16,000 six-input lookup tables, 10 to 16 multiplier blocks and 11 block RAMs, with an error of about ±40%: the register file is 250 lookup tables of distributed RAM, the 64-bit ALU 600, the barrel shifter 450, the caches 300 and 500 plus 8 block RAMs, the bus bridge 600. *(E)*

10.1.4 Block RAM, not logic, is the scarce resource: the caches, the translation buffer, the framebuffer and the blitter scratch compete for about 5.5 megabits. *(E)*

10.1.5 The realistic clock is 50 to 80 MHz on the Cyclone V, 90 to 130 MHz on a Zynq 7020 and 180 to 250 MHz on an UltraScale+; a correct 40 MHz machine is worth more than a fast broken one, and 60 MHz is already some fifty times a PDP-11/70. *(E)*

10.1.6 The work is staged in phases, each ending in a demonstration and a regression suite: specification and tools; an integer core; exceptions; the memory-management unit; the memory system; floating point; a booting kernel; peripherals; and, optionally and last, a second core. *(D, plan)*

10.1.7 Phase 2, the fault model, is not optional and does not move; the block operations are built with it, because their whole specification is about what state is visible when an exception arrives. *(D, plan)*

Commentary. **Precedent.** The PDP-11's own microprocessor, the T-11, was about 12,000 transistors, a complete integer core without a memory-management unit or floating point; the J-11 that implemented the full PDP-11/70 was about 120,000. The MicroVAX 78032 was 125,000 transistors for a subset of the VAX, with 175 of its 304 instructions and no floating point. **Trade-offs.** The 64-bit carry chain is slow on an FPGA and gates the clock more than it would in silicon, the barrel shifter is a large multiplexer tree, and flags serialise branch resolution behind the ALU; none of that argues for changing the instruction set, only for where the pipeline registers go, for example a virtually indexed, physically tagged first-level cache so that the translation buffer is off the critical path. A multiplier built from two to four multiplier blocks over four to eight cycles closes timing where a full combinational one will not. **Improvements.** The register-transfer description is kept vendor-neutral, with every vendor primitive in one thin directory (under 500 lines), so that moving between boards costs that directory and nothing else. **Benefits.** A single engineer working part-time can reach a booting system with floating point in roughly 10 to 19 months, by the plan's wide-error-bar estimate, with phases 2 and 3 the ones that historically double. **Modern comparables.** The RISC-V soft cores Rocket and VexRiscv are the usual FPGA reference points; the plan treats the compiler back end as the schedule risk, as it is for any new architecture.

10.2 The chip: a transistor budget

10.2.1 For a non-pipelined or short-pipeline implementation in a modern process, the whole core excluding caches is estimated at about 92,000 transistors. *(E)*

Block	Transistors (E)
register file, 16 × 64, two read ports and one write	~6,000
ALU, shifter, address adder	~15,000
decode and sequencer	~6,500
bit unit (popcount tree, count zeros, bitfields)	~3,500
block-move sequencer	~1,500
UTF-8 decode and validity	~1,200
translation buffer (32 entries), walker, window compare	~19,000
floating-point unit with fused multiply-add, classify, min and max	~34,000
control, status word, exceptions, undo buffer	~5,500
total, excluding caches	**~92,000**

10.2.2 The bit unit and the UTF-8 decoder together cost less than 5% of the die; the fused multiply-add datapath is the largest single addition and the least negotiable. *(E)*

Commentary. **Precedent.** The figure sits between the T-11 and the MicroVAX: it is a 64-bit machine with floating point and memory management below the 78032's 125,000 transistors, which had neither. **Trade-offs.** The estimate is not a measurement and excludes caches, which dominate the area of a real chip; an implementation of the translation buffer as a content-addressable memory is larger than the figure assumes, which is why the FPGA build uses a four-way set-associative one. **Benefits.** A chip this size fits in the budget of a small process node with room for caches and the bus controller. **Modern comparables.** The ARM1 of 1985 was about 25,000 transistors without a memory-management unit or floating point; the Intel 80386 about 275,000.

10.3 In order now, ready for out of order

10.3.1 The first implementations, on the FPGA and in silicon, are small in-order cores without speculation; the architecture is kept ready for a large out-of-order design. *(D, errata 38)*

10.3.2 The condition flags were the structural obstacle: 68.2% of compiled instructions wrote a flag and only 1.5% read one, so they were reorganised into two groups that nothing writes in part, and moves, loads, stores and the compare-and-branch instructions write none. *(D, errata 39)*

10.3.3 An in-order, non-speculative core has none of the speculative side channels of the last decade today; the rules that keep a future faster core from repeating them are collected in the hardware-vulnerabilities report, and include no speculation across protection domains, no shared predictor state across privilege boundaries without a flush, and operand-independent timing for a declared set of instructions. *(D as a design aim; the rules are proposals)*

10.3.4 Sequential consistency is nearly free on a small in-order core and is the most expensive memory model for a large speculative one; if a large core is built it may be necessary to move to total store order, which is the least painful retreat. *(P)*

Commentary. **Precedent.** One instruction set has often spanned in-order and out-of-order cores: the ARM small cores beside the large ones, the RISC-V Rocket beside the BOOM, x86 from the Pentium to the P6, Alpha from the 21064 to the 21264. The VAX's long, orthogonal instructions with many side effects are generally blamed for making its later fast implementations so hard. **Trade-offs.** Readiness has a cost, visible here as the compiled-code size paid for the flag changes (0.39%) and as design effort spent on a core that may never be built. **Improvements.** The pain-points report finds that the pdp64 avoids the largest classes of kernel pain by design (barriers, floating-point state, a faulting entry, remote translation flushes, the speculative family) and is exposed in a few places: a single page-table base if a core ever speculates, software accessed and dirty bits, uncached DMA buffers, and no specified constant-time or Rowhammer protection. **Benefits.** Any of these can be addressed in the specification while it is still being written, which is cheaper than after silicon. **Modern comparables.** The common vulnerability-driven changes of recent years, page-table isolation and speculation barriers, are what a speculative core of this architecture would have to add.

10.4 The finish line, and the path to it

10.4.1 Before silicon the project delivers five things: an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh`. *(D, errata 52)*

10.4.2 The architecture manual, which includes the instruction set but covers the whole machine, is the primary deliverable; the toolchain is advanced as needed to write and check it. *(D, errata 54)*

10.4.3 Today the assembler and linker exist, with the atomics still to be added to the assembler and shared libraries still to be added to the linker; the pcc compiler back end passes 516 C tests on the simulator and on QEMU; the LLVM back end has its machine-code layer, checked against the project's own assembler on 62 programs and 6,304 instruction forms with no disagreement; the simulator and QEMU run a kernel-mode machine with exceptions, a memory-management unit, a timer and a console; no part of OpenBSD is ported. *(measured)*

10.4.4 The long pole is the LLVM back end and clang. OpenBSD's kernel and base cannot be built by pcc, and the fallback of porting Unix/32V with pcc would not reach the goal, because it has no network stack and no `ssh`. *(D, errata 41 and the roadmap)*

10.4.5 The path, in order, with a test for each step: finish the toolchain gaps (atomics, the user-memory guard, the ABI version); an LLVM back end that passes the C tests; clang, the compiler runtime and `lld`; a machine model for the operating system with an enumerating bus, a disk, a network card and an entropy source (virtio devices first, for speed); the kernel booting to single-user; the userland to a login prompt; the network and `sshd`, and then `ssh user@localhost -p 2222` gives a shell; and finally the same system on the FPGA. *(P, roadmap)*

Commentary. **Precedent.** OpenBSD's riscv64 port is its newest and smallest, and it measures at 35,600 lines of machine-dependent code, of which 14,873 are the core, 11,771 platform drivers, 4,513 the loader and 4,443 headers; arm64 is 61,613 and amd64 79,167. **Trade-offs.** The pdp64 port should be smaller than the riscv64 one in several places: no firmware tables to parse, uniform eight-register devices instead of a platform's driver zoo, a trivial loader, no cache-maintenance code and no floating-point state code; the core machine-dependent code of about 15,000 lines is the part that has to be written by hand *(estimate)*. The LLVM back end is estimated at three to six months for one experienced engineer, a figure that has not been tested. **Benefits.** An OpenBSD that can be logged into is a demonstration that the architecture, the toolchain and the operating-system interface are complete in the way that matters to users, and it forces every piece (the compiler, the linker, the bus, the devices, the boot) to work together. **Modern comparables.** OpenBSD ports to new architectures have followed this shape; the RISC-V port is the nearest in kind.

10.5 What remains open

Everything not yet settled is collected, with its options and its benefits, in appendix B; the ideas that nobody has yet turned into a decision are in appendix A. The principal open choices are the transport and interrupt controller (chapters 2 and 5), the details of the device layout, the interrupt controller and virtualization position, the vector-math position, and the several smaller instruction-set proposals.

Part II — The instruction reference tables

The instruction reference tables

This is the reference part of the manual: **one entry for each instruction**, in the layout of a processor reference manual. Each entry gives its mnemonics, its syntax, its encoding (the parcel diagram and the numbers), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the book explains it. The entries are generated from `spec/pdp64-mnemonics.hbnf`, the one place where the numbers of the architecture are written, so an encoding here cannot disagree with the assembler or the simulators; the words (operation, flags, exceptions) are in `tools/manual/isa_entries.py` and were checked against the reference executor, `common/pdp64exec.c`, which `sim64` and QEMU share. Where the executor does not implement an instruction yet, its entry says so.

Notation. `Rd` is the destination register (the `reg` field of parcel 0), `Rs` the first source (`low`), `Rt` the second (`t` of parcel 1), `Ru` the third (`x`); the assembler writes the destination last. `rm` is a general operand: a register at `len = 00`, or a memory operand, an immediate or an absolute address by one of the eight addressing modes (book section 9.5). `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. A `len = 01` instruction is two parcels (four bytes) and a `len = 10` instruction three (six bytes); `len = 11` is page 2. `N Z V C` are the condition flags. Bit diagrams number the bits of a parcel from 15 at the left.

Cf. lines point to the chapter that explains the matter an instruction touches: the kernel (chapter 6), the multiprocessor model (chapter 7), the bus (chapter 2).

Where the entries are

- Section 6.7, System instructions, exceptions and interrupts: `NOP`, `RTI`, `RET`, `HALT`, `BPT`, `FENCE`, `WFE`, `SEV`, `TLBIA`, `PUSHX`, `ISYNC`, `HINT`, `MFSR`, `MTPS`, `SPL`, `TRAP`, `TLBI`, `MTASID`, `MTDP`, `MTDMW`, `MFCPUID`, `ICINV`, `PROBER`
- Section 7.12, The atomic and ordering instructions: `CAS`, `SWAP`, `XADD`, `XAND`, `XMAX`, `LL`
- Section 9.6, Moves, conversions and address formation: `MOVQ`, `MOVZL`, `LEA`, `MOVUB`
- Section 9.7, Integer arithmetic: `ADD`, `SUB`, `CMP`, `MUL`, `MULH`, `ADDM`, `INC`, `DEC`, `NEG`, `ADC`, `SBC`, `DIV`, `MULHU`, `ABS`, `MIN`, `SET`
- Section 9.8, Logic, shifts and rotates: `BIC`, `BIS`, `BIT`, `XOR`, `AND`, `SHL`, `CLR`, `COM`, `TST`, `ASL`, `ASR`
- Section 9.9, Bits, bytes and text: `CLZ`, `POPCNT`, `BSWAP`, `BITREV`, `ROR`, `RORI`, `RORI32`, `DSHR`, `EXTB`, `INSB`, `XNORPOP`, `CLMULL`, `UTF8LEN`, `UTF8DEC`, `UTF8ENC`
- Section 9.10, Branches and control flow: `BR`, `SOB`, `CBNE`, `BTAG`, `JMP`, `CALL`, `JMPI`, `JTAG`
- Section 9.11, The stack and block operations: `PUSHM`, `POPM`, `PUSH`, `POP`, `MCPY`, `MSET`, `MCMP`, `MSCN`, `MVAL`
- Section 9.12, Floating point: `FADD`, `FCMP`, `FDIV`, `FADD32`, `FMADD`, `FMADD32`, `FNEG`, `FMIN`, `FCLASS`, `FRINT`, `FCVTDQ`, `FSFLAGS`, `FADD16`, `FCVTSH`

The tables

- The second page: one-parcel abbreviations
- Assembler aliases and old spellings
- The architectural constants
- The mnemonics, alphabetically
- The opcodes, in order

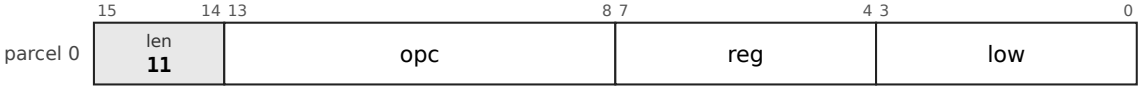
The specification has 270 mnemonic words (234 instructions and 36 aliases) and 45 page-2 opcodes; the reference executor implements 193 of the 234 instructions; the rest are specified and have entries marked as not yet implemented.

Generated by `tools/mkmanual.py`; do not edit.

The second page: one-parcel abbreviations

Page 2 (`len = 11`) is a second opcode map of 64 one-parcel instructions, 45 of them defined. **Every page-2 instruction is exactly the page-1 instruction it abbreviates**: the same result, the same flags and the same faults. The assembler chooses the shortest encoding, so a mnemonic still assembles to one instruction and a program can never change meaning by using a page-2 form. The opcode numbers of page 2 are independent of those of page 1. `reg` is the register (or the high part of an immediate) named first, `low` the field named second.

Generated by `tools/mkmanual.py` from `spec/pdp64-mnemonics.hbnf` and `tools/manual/isa_entries.py`; do not edit.



Opcode	Name	Operation	reg	low	Long form it abbreviates
0x00	ld.q	MOVQ load through (rn)	rd	rn	MOVQ (rn), rd
0x01	st.q	MOVQ store through (rn)	rs	rn	MOVQ rs, (rn)
0x02	ld.l	MOVL load through (rn)	rd	rn	MOVL (rn), rd
0x03	st.l	MOVL store through (rn)	rs	rn	MOVL rs, (rn)
0x04	ld.w	MOVW load through (rn)	rd	rn	MOVW (rn), rd
0x05	st.w	MOVW store through (rn)	rs	rn	MOVW rs, (rn)
0x06	ld.b	MOVB load through (rn)	rd	rn	MOVB (rn), rd
0x07	st.b	MOVB store through (rn)	rs	rn	MOVB rs, (rn)
0x08	ldz.l	MOVZL load through (rn)	rd	rn	MOVZL (rn), rd
0x09	ldz.w	MOVZW load through (rn)	rd	rn	MOVZW (rn), rd
0x0A	ldz.b	MOVZB load through (rn)	rd	rn	MOVZB (rn), rd
0x0B	lea.sp	LEA 8*d(SP), rd	rd	d (0 to 15)	LEA 8*d(SP), rd
0x0C	ldq.sp0	MOVQ load from SP, offset 8*low, 0 to 120	rd	offset / 8	MOVQ d(SP), rd
0x0D	ldq.sp1	MOVQ load from SP, offset 8*(16+low), 128 to 248	rd	offset / 8 - 16	MOVQ d(SP), rd
0x0E	stq.sp	MOVQ store to SP, offset 8*low, 0 to 120	rs	offset / 8	MOVQ rs, d(SP)
0x10	ldl.sp0	MOVL load from SP, offset 4*low, 0 to 60	rd	offset / 4	MOVL d(SP), rd
0x11	ldl.sp1	MOVL load from SP, offset 4*(16+low), 64 to 124	rd	offset / 4 - 16	MOVL d(SP), rd
0x12	stl.sp0	MOVL store to SP, offset 4*low, 0 to 60	rs	offset / 4	MOVL rs, d(SP)
0x13	stl.sp1	MOVL store to SP, offset 4*(16+low), 64 to 124	rs	offset / 4 - 16	MOVL rs, d(SP)
0x14	ldq.d1	MOVQ load from rn at offset 8	rd	rn	MOVQ 8(rn), rd
0x15	ldq.d2	MOVQ load from rn at offset 16	rd	rn	MOVQ 16(rn), rd
0x16	ldq.d3	MOVQ load from rn at offset 24	rd	rn	MOVQ 24(rn), rd
0x17	ldq.d4	MOVQ load from rn at offset 32	rd	rn	MOVQ 32(rn), rd
0x18	stq.d1	MOVQ store to rn at offset 8	rs	rn	MOVQ rs, 8(rn)
0x19	stq.d2	MOVQ store to rn at offset 16	rs	rn	MOVQ rs, 16(rn)
0x1A	ldl.d1	MOVL load from rn at offset 4	rd	rn	MOVL 4(rn), rd
0x1B	ldl.d2	MOVL load from rn at offset 8	rd	rn	MOVL 8(rn), rd
0x1C	ldl.d3	MOVL load from rn at offset 12	rd	rn	MOVL 12(rn), rd
0x1D	ldl.d4	MOVL load from rn at offset 16	rd	rn	MOVL 16(rn), rd
0x1E	pushm8	push the registers R8 to R15 named by the 8-bit mask reg:low (bit i is R(8+i))	mask high	mask low	PUSHM #(mask << 8)
0x1F	popm8	pop the registers named by the mask, as pushm8	mask high	mask low	POPM #(mask << 8)
0x20	sub.sp	SUB #(8*n), SP, n = reg:low (0 to 255)	n high	n low	SUB #(8*n), SP
0x21	add.sp	ADD #(8*n), SP, n = reg:low	n high	n low	ADD #(8*n), SP
0x22	movq.i	MOVQ #imm, rd, imm sign-extended 4 bits (-8 to 7)	rd	imm	MOVQ #imm, rd

Opcode	Name	Operation	reg	low	Long form it abbreviates
0x23	shl.i	SHL by low (0 to 15)	rd	count	SHL #n, rd
0x24	shr.i	SHR by low (0 to 15)	rd	count	SHR #n, rd
0x25	sar.i	SAR by low (0 to 15)	rd	count	SAR #n, rd
0x26	and.i	AND with a 4-bit sign-extended immediate	rd	imm	AND #imm, rd
0x27	bis.i	BIS with a 4-bit sign-extended immediate	rd	imm	BIS #imm, rd
0x28	xor.i	XOR with a 4-bit sign-extended immediate	rd	imm	XOR #imm, rd
0x29	add.i	ADD with a 4-bit sign-extended immediate	rd	imm	ADD #imm, rd
0x2A	pushx	PUSHX (reg and low zero)	0	0	SYSX sub-opcode 0
0x2B	popx	POPX (reg and low zero)	0	0	SYSX sub-opcode 1
0x2C	isync	ISYNC (reg and low zero)	0	0	SYSX sub-opcode 2
0x2D	hint	HINT #n, n = reg:low (0 to 255)	n high	n low	SYSX sub-opcode 3

Every page-2 opcode not listed (0x0F and 0x2E to 0x3F, 19 of 64) is reserved; a reserved page-2 opcode is an illegal-instruction abort.

Why there is a second page

On 2026-10-22 the author asked: “so... why do we need 8 byte instructions? only amd64 has more than 6 bytes. can we squeeze into 3 parcels?” The fourth parcel had been there for 48-bit immediates and 44-bit displacements, and only 49 of 373,635 compiled instructions (0.013%) used it. Removing it freed the length code 11, and the author’s next question was “ok so... you ready to update the isa with all those new 1 parcel forms?”: the code became a second page of one-parcel instructions, each an exact abbreviation of a long form, chosen from the forms the compiler emits most. The estimate from static counts on the 827-file corpus was about 118 KB, 7% of the 1.67 MB image (errata 31; CHANGES section 12). The rule that keeps the page honest: a page-2 instruction has the same result, flags and faults as the long form it abbreviates, so the assembler choosing the shortest encoding can never change what a program does.

Assembler aliases and old spellings

A mnemonic that assembles to another instruction. One mnemonic is one instruction (book section 9.18); the disassembler never prints an alias.

Generated by `tools/mkmanual.py` *from* `spec/pdp64-mnemonics.hbnf`*; do not edit.*

Written	Assembles to	Group
BHIS	BCC	branch
BLO	BCS	branch
CBHIS	CBCC	cbranch
CBLO	CBCS	cbranch
FADDD	FADD	alu
FADDS	FADD32	fesc
FCLASSD	FCLASS	fesc
FCLASSS	FCLASS32	fesc
FCMPD	FCMP	alu
FCMPS	FCMP32	fesc
FDIVD	FDIV	fesc
FDIVS	FDIV32	fesc
FMADDD	FMADD	fesc
FMADDS	FMADD32	fesc
FMAXD	FMAX	fesc
FMIND	FMIN	fesc
FMSUBD	FMSUB	fesc
FMSUBS	FMSUB32	fesc
FMULD	FMUL	alu
FMULS	FMUL32	fesc
FNMADDD	FNMADD	fesc
FNMADDS	FNMADD32	fesc
FNMSUBD	FNMSUB	fesc
FNMSUBS	FNMSUB32	fesc
FRINTD	FRINT	fesc
FRINTDA	FRINTA	fesc
FRINTDM	FRINTM	fesc
FRINTDP	FRINTP	fesc
FRINTDZ	FRINTZ	fesc
FSQRTD	FSQRT	fesc
FSQRTS	FSQRT32	fesc
FSUBD	FSUB	alu
FSUBS	FSUB32	fesc
MULU	MUL	alu
ROL	RORI	besc
WAIT	WFE	sys-none
ROL rs, #n, rd	RORI rs, #(64-n), rd	assembler, immediate count only

The architectural constants

The numbers of the processor status word, the modes, the exception vectors, the page-table entry, the memory types, the translation geometry, the system registers, the addressing modes and the condition codes, each read from the machine-readable specification (`spec/pdp64-system.hbnf` and `spec/pdp64-mnemonics.hbnf`).
Generated by `tools/mkmanual.py`; do not edit.

The processor status word (16 bits)

Field	Bit	Width	Meaning
C	0	1	carry (borrow after a subtract)
V	1	1	signed overflow
Z	2	1	zero
N	3	1	negative
T	4	1	trace: take a trace trap after the next instruction
PRI	5	3	interrupt priority level, 0 to 7
MODE	8	2	current mode
PREV	10	2	previous mode (the mode before the last exception)

Bits 12 to 15 are reserved and read as zero. Bit 12 is the user-memory guard UA of errata 43 (not yet in the specification file).

The modes

Mode	Value
kernel	0
supervisor	1
user	3

Value 2 is reserved: loading it into either mode field of the status word is an illegal instruction.

The exception vectors

The table has 512 entries of eight bytes at the physical address `VBR + 8 * vector`; ECAUSE equals the vector number.

Vector	Number	Cause
RESET	0	processor reset: the first instruction fetch
ILLEGAL	2	a reserved or undefined opcode, a reserved encoding or an unassigned system register
PRIVILEGED	3	a system instruction executed outside kernel mode
MISALIGNED	4	a data access whose address is not a multiple of its size (the MOVU forms excepted)
NONCANONICAL	5	a virtual address whose bits 63 to 39 differ from bit 38
IFAULT	6	instruction fetch from a page that is not mapped
LFAULT	7	load from a page that is not mapped
SFAULT	8	store to a page that is not mapped
IPROT	9	instruction fetch denied by the page permissions
LPROT	10	load denied by the page permissions
SPROT	11	store denied by the page permissions
DIVZERO	12	division by zero (DIV, DIVU, MOD, MODU)
BUSFAULT	13	a read of non-existent memory (a precise abort)
LPAD	14	an indirect transfer that did not land on LPAD (enforcement off in ABI version 1)
BPT	15	the BPT instruction
TIMER	32	the timer interrupt (level 6)
COUNT	512	the number of vectors; the table has this many eight-byte entries

The page-table entry (64 bits)

Field	Bit	Width	Meaning
`V`	0	1	valid
`R`	1	1	readable
`W`	2	1	writable
`X`	3	1	executable
`U`	4	1	accessible from user mode
`G`	5	1	global: ignores the ASID
`A`	6	1	accessed (managed by software)
`D`	7	1	dirty (managed by software)
`MT`	8	2	memory type
`PPN`	12	28	physical page number: physical address bits 12 to 39, in place

A non-leaf entry has `V` = 1 and `R` = `W` = `X` = 0 and points to the next table. Every bit not named is zero.

The memory types (`MT`)

Type	Value
cached	0
uncached	1
device	2

The translation geometry

Quantity	Value
levels	3

The system registers (`MFSR`, `MTSR`)

Name	Number	Access
`PTBR`	0	read and write
`EPC`	1	read and write
`EVA`	2	read only
`ECAUSE`	3	read only
`DMW0`	4	read and write
`DMW1`	5	read and write
`USP`	6	read and write
`SSP`	7	read and write
`EPSW`	8	read and write
`VBR`	9	read and write
`MMUCTL`	10	read and write
`ASID`	11	read and write
`CPUID`	12	read only

The addressing modes

Mode	Value
base	`0000`
index	`0001`
pcrel	`0010`
dp	`0011`
imm	`0100`
abs	`0101`
postinc	`0110`
predec	`0111`

Modes `1000` to `1111` are reserved. The syntax and the parcel layout of each mode are in the book, section 9.5.

The condition codes

Name	Number	Aliases
al	0	
ne	1	
eq	2	
pl	3	
mi	4	
vc	5	
vs	6	
cc	7	hs
cs	8	lo
ge	9	
lt	10	
gt	11	
le	12	
hi	13	
los	14	

The mnemonics, alphabetically

Generated by `tools/mkmanual.py`; do not edit.

Mnemonic	Entry	Opcode	Sub	Group
ABS	ABS (section 9.7)	0x0E	0x8	Arithmetic (AESC)
ADC	ADC (section 9.7)	0x13	0x6	Single operand (SOP)
ADD	ADD (section 9.7)	0x16		Moves, address formation and the ALU
ADDM	ADDM (section 9.7)	0x0F	0x8	Memory escape (MESC)
AND	AND (section 9.8)	0x1D		Moves, address formation and the ALU
ASL	ASL (section 9.8)	0x13	0x8	Single operand (SOP)
ASR	ASR (section 9.8)	0x13	0x9	Single operand (SOP)
BCC	BCC (section 9.10)	0x37		Branches
BCS	BCC (section 9.10)	0x38		Branches
BEQ	BCC (section 9.10)	0x32		Branches
BGE	BCC (section 9.10)	0x39		Branches
BGT	BCC (section 9.10)	0x3B		Branches
BHI	BCC (section 9.10)	0x3D		Branches
BHIS (alias of BCC)	BCC (section 9.10)	0x37		Branches
BIC	BIC (section 9.8)	0x18		Moves, address formation and the ALU
BICM	ADDM (section 9.7)	0x0F	0xA	Memory escape (MESC)
BIS	BIS (section 9.8)	0x19		Moves, address formation and the ALU
BISM	ADDM (section 9.7)	0x0F	0xB	Memory escape (MESC)
BIT	BIT (section 9.8)	0x1B		Moves, address formation and the ALU
BITREV	BITREV (section 9.9)	0x0C	0x8	Bits and fields (BESC)
BLE	BCC (section 9.10)	0x3C		Branches
BLO (alias of BCS)	BCC (section 9.10)	0x38		Branches
BLOS	BCC (section 9.10)	0x3E		Branches
BLT	BCC (section 9.10)	0x3A		Branches
BMI	BCC (section 9.10)	0x34		Branches
BNE	BCC (section 9.10)	0x31		Branches
BNTAG	BTAG (section 9.10)	0x15	0x1	Branches
BPL	BCC (section 9.10)	0x33		Branches
BPT	BPT (section 6.7)	0x14	0x5	System (SYS)
BR	BCC (section 9.10)	0x30		Branches
BSWAP	BSWAP (section 9.9)	0x0C	0x5	Bits and fields (BESC)
BSWAPL	BSWAP (section 9.9)	0x0C	0x7	Bits and fields (BESC)
BSWAPW	BSWAP (section 9.9)	0x0C	0x6	Bits and fields (BESC)
BTAG	BTAG (section 9.10)	0x15	0x0	Branches
BVC	BCC (section 9.10)	0x35		Branches
BVS	BCC (section 9.10)	0x36		Branches
CALL	CALL (section 9.10)	0x13	0xD	Single operand (SOP)
CALLI	JMPI (section 9.10)	0x12	0x1	Indirect transfers (XFER)
CAS	CAS (section 7.12)	0x11	0x0	Atomics (ATOM)
CASDB	CAS (section 7.12)	0x11	0x1	Atomics (ATOM)
CBCC	CBCC (section 9.10)	0x28	0x7	Branches
CBCS	CBCC (section 9.10)	0x28	0x8	Branches
CBEQ	CBCC (section 9.10)	0x28	0x2	Branches
CBGE	CBCC (section 9.10)	0x28	0x9	Branches
CBGT	CBCC (section 9.10)	0x28	0xB	Branches
CBHI	CBCC (section 9.10)	0x28	0xD	Branches

Mnemonic	Entry	Opcode	Sub	Group
CBHIS (alias of CBCC)	CBCC (section 9.10)	0x28	0x7	Branches
CBLE	CBCC (section 9.10)	0x28	0xC	Branches
CBLO (alias of CBCS)	CBCC (section 9.10)	0x28	0x8	Branches
CBLOS	CBCC (section 9.10)	0x28	0xE	Branches
CBLT	CBCC (section 9.10)	0x28	0xA	Branches
CBMI	CBCC (section 9.10)	0x28	0x4	Branches
CBNE	CBCC (section 9.10)	0x28	0x1	Branches
CBPL	CBCC (section 9.10)	0x28	0x3	Branches
CBVC	CBCC (section 9.10)	0x28	0x5	Branches
CBVS	CBCC (section 9.10)	0x28	0x6	Branches
CLMULH	CLMUL (section 9.9)	0x0C	0x32	Bits and fields (BESC)
CLMULL	CLMUL (section 9.9)	0x0C	0x31	Bits and fields (BESC)
CLO	COUNT (section 9.9)	0x0C	0x2	Bits and fields (BESC)
CLR	CLR (section 9.8)	0x13	0x0	Single operand (SOP)
CLZ	COUNT (section 9.9)	0x0C	0x0	Bits and fields (BESC)
CMP	CMP (section 9.7)	0x1A		Moves, address formation and the ALU
COM	COM (section 9.8)	0x13	0x1	Single operand (SOP)
CTO	COUNT (section 9.9)	0x0C	0x3	Bits and fields (BESC)
CTZ	COUNT (section 9.9)	0x0C	0x1	Bits and fields (BESC)
DEC	DEC (section 9.7)	0x13	0x3	Single operand (SOP)
DIV	DIV (section 9.7)	0x0E	0x0	Arithmetic (AESC)
DIVU	DIV (section 9.7)	0x0E	0x1	Arithmetic (AESC)
DSHR	DSHR (section 9.9)	0x0C	0x20	Bits and fields (BESC)
EXTB	EXTB (section 9.9)	0x0C	0x21	Bits and fields (BESC)
EXTZB	EXTB (section 9.9)	0x0C	0x22	Bits and fields (BESC)
FABS	FSIGN (section 9.12)	0x10	0x21	Floating point (FESC)
FADD	FADD (section 9.12)	0x24		Moves, address formation and the ALU
FADD16	FHALF (section 9.12)	0x10	0x50	Floating point (FESC)
FADD32	FSINGLE (section 9.12)	0x10	0x8	Floating point (FESC)
FADDD (alias of FADD)	FADD (section 9.12)	0x24		Moves, address formation and the ALU
FADDS (alias of FADD32)	FSINGLE (section 9.12)	0x10	0x8	Floating point (FESC)
FCLASS	FCLASS (section 9.12)	0x10	0x28	Floating point (FESC)
FCLASS16	FCLASS (section 9.12)	0x10	0x5C	Floating point (FESC)
FCLASS32	FCLASS (section 9.12)	0x10	0x29	Floating point (FESC)
FCLASSD (alias of FCLASS)	FCLASS (section 9.12)	0x10	0x28	Floating point (FESC)
FCLASSS (alias of FCLASS32)	FCLASS (section 9.12)	0x10	0x29	Floating point (FESC)
FCMP	FCMP (section 9.12)	0x27		Moves, address formation and the ALU
FCMP16	FHALF (section 9.12)	0x10	0x55	Floating point (FESC)
FCMP32	FSINGLE (section 9.12)	0x10	0xD	Floating point (FESC)
FCMPD (alias of FCMP)	FCMP (section 9.12)	0x27		Moves, address formation and the ALU
FCMPS (alias of FCMP32)	FSINGLE (section 9.12)	0x10	0xD	Floating point (FESC)
FCOPYSIGN	FSIGN (section 9.12)	0x10	0x22	Floating point (FESC)
FCVTDH	FCVTH (section 9.12)	0x10	0x62	Floating point (FESC)
FCVTDQ	FCVT (section 9.12)	0x10	0x38	Floating point (FESC)
FCVTDQU	FCVT (section 9.12)	0x10	0x3E	Floating point (FESC)
FCVTDS	FCVT (section 9.12)	0x10	0x3A	Floating point (FESC)
FCVTHD	FCVTH (section 9.12)	0x10	0x63	Floating point (FESC)
FCVTHL	FCVTH (section 9.12)	0x10	0x64	Floating point (FESC)
FCVTHS	FCVTH (section 9.12)	0x10	0x61	Floating point (FESC)
FCVTLH	FCVTH (section 9.12)	0x10	0x65	Floating point (FESC)
FCVTLS	FCVT (section 9.12)	0x10	0x3D	Floating point (FESC)
FCVTQD	FCVT (section 9.12)	0x10	0x39	Floating point (FESC)
FCVTQUD	FCVT (section 9.12)	0x10	0x3F	Floating point (FESC)
FCVTSD	FCVT (section 9.12)	0x10	0x3B	Floating point (FESC)
FCVTSH	FCVTH (section 9.12)	0x10	0x60	Floating point (FESC)
FCVTSL	FCVT (section 9.12)	0x10	0x3C	Floating point (FESC)

Mnemonic	Entry	Opcode	Sub	Group
FDIV	FDIV (section 9.12)	0x10	0x0	Floating point (FESC)
FDIV16	FHALF (section 9.12)	0x10	0x53	Floating point (FESC)
FDIV32	FSINGLE (section 9.12)	0x10	0xB	Floating point (FESC)
FDIVD (alias of FDIV)	FDIV (section 9.12)	0x10	0x0	Floating point (FESC)
FDIVS (alias of FDIV32)	FSINGLE (section 9.12)	0x10	0xB	Floating point (FESC)
FENCE	FENCE (section 6.7)	0x14	0x6	System (SYS)
FMADD	FMADD (section 9.12)	0x10	0x10	Floating point (FESC)
FMADD16	FHALF (section 9.12)	0x10	0x58	Floating point (FESC)
FMADD32	FMADD32 (section 9.12)	0x10	0x18	Floating point (FESC)
FMADDD (alias of FMADD)	FMADD (section 9.12)	0x10	0x10	Floating point (FESC)
FMADDS (alias of FMADD32)	FMADD32 (section 9.12)	0x10	0x18	Floating point (FESC)
FMAX	FMIN (section 9.12)	0x10	0x24	Floating point (FESC)
FMAXD (alias of FMAX)	FMIN (section 9.12)	0x10	0x24	Floating point (FESC)
FMIN	FMIN (section 9.12)	0x10	0x23	Floating point (FESC)
FMIND (alias of FMIN)	FMIN (section 9.12)	0x10	0x23	Floating point (FESC)
FMSUB	FMADD (section 9.12)	0x10	0x11	Floating point (FESC)
FMSUB16	FHALF (section 9.12)	0x10	0x59	Floating point (FESC)
FMSUB32	FMADD32 (section 9.12)	0x10	0x19	Floating point (FESC)
FMSUBD (alias of FMSUB)	FMADD (section 9.12)	0x10	0x11	Floating point (FESC)
FMSUBS (alias of FMSUB32)	FMADD32 (section 9.12)	0x10	0x19	Floating point (FESC)
FMUL	FADD (section 9.12)	0x26		Moves, address formation and the ALU
FMUL16	FHALF (section 9.12)	0x10	0x52	Floating point (FESC)
FMUL32	FSINGLE (section 9.12)	0x10	0xA	Floating point (FESC)
FMULD (alias of FMUL)	FADD (section 9.12)	0x26		Moves, address formation and the ALU
FMULS (alias of FMUL32)	FSINGLE (section 9.12)	0x10	0xA	Floating point (FESC)
FNEG	FSIGN (section 9.12)	0x10	0x20	Floating point (FESC)
FNMADD	FMADD (section 9.12)	0x10	0x12	Floating point (FESC)
FNMADD16	FHALF (section 9.12)	0x10	0x5A	Floating point (FESC)
FNMADD32	FMADD32 (section 9.12)	0x10	0x1A	Floating point (FESC)
FNMADDD (alias of FNMADD)	FMADD (section 9.12)	0x10	0x12	Floating point (FESC)
FNMADDS (alias of FNMADD32)	FMADD32 (section 9.12)	0x10	0x1A	Floating point (FESC)
FNMSUB	FMADD (section 9.12)	0x10	0x13	Floating point (FESC)
FNMSUB16	FHALF (section 9.12)	0x10	0x5B	Floating point (FESC)
FNMSUB32	FMADD32 (section 9.12)	0x10	0x1B	Floating point (FESC)
FNMSUBD (alias of FNMSUB)	FMADD (section 9.12)	0x10	0x13	Floating point (FESC)
FNMSUBS (alias of FNMSUB32)	FMADD32 (section 9.12)	0x10	0x1B	Floating point (FESC)
FRINT	FRINT (section 9.12)	0x10	0x30	Floating point (FESC)
FRINTA	FRINT (section 9.12)	0x10	0x34	Floating point (FESC)
FRINTD (alias of FRINT)	FRINT (section 9.12)	0x10	0x30	Floating point (FESC)
FRINTDA (alias of FRINTA)	FRINT (section 9.12)	0x10	0x34	Floating point (FESC)
FRINTDM (alias of FRINTM)	FRINT (section 9.12)	0x10	0x32	Floating point (FESC)
FRINTDP (alias of FRINTP)	FRINT (section 9.12)	0x10	0x33	Floating point (FESC)
FRINTDZ (alias of FRINTZ)	FRINT (section 9.12)	0x10	0x31	Floating point (FESC)
FRINTM	FRINT (section 9.12)	0x10	0x32	Floating point (FESC)
FRINTP	FRINT (section 9.12)	0x10	0x33	Floating point (FESC)
FRINTZ	FRINT (section 9.12)	0x10	0x31	Floating point (FESC)
FSFLAGS	FSFLAGS (section 9.12)	0x10	0x40	Floating point (FESC)
FSQRT	FDIV (section 9.12)	0x10	0x1	Floating point (FESC)
FSQRT16	FHALF (section 9.12)	0x10	0x54	Floating point (FESC)
FSQRT32	FSINGLE (section 9.12)	0x10	0xC	Floating point (FESC)
FSQRTD (alias of FSQRT)	FDIV (section 9.12)	0x10	0x1	Floating point (FESC)
FSQRTS (alias of FSQRT32)	FSINGLE (section 9.12)	0x10	0xC	Floating point (FESC)
FSUB	FADD (section 9.12)	0x25		Moves, address formation and the ALU
FSUB16	FHALF (section 9.12)	0x10	0x51	Floating point (FESC)
FSUB32	FSINGLE (section 9.12)	0x10	0x9	Floating point (FESC)
FSUBD (alias of FSUB)	FADD (section 9.12)	0x25		Moves, address formation and the ALU

Mnemonic	Entry	Opcode	Sub	Group
FSUBS (alias of FSUB32)	FSINGLE (section 9.12)	0x10	0x9	Floating point (FESC)
HALT	HALT (section 6.7)	0x14	0x4	System (SYS)
HINT	HINT (section 6.7)	0x14	0x3	System (SYS)
ICINV	ICINV (section 6.7)	0x14	0xC	System (SYS)
ICINVA	TLBA (section 6.7)	0x14	0xC	System (SYS)
INC	INC (section 9.7)	0x13	0x2	Single operand (SOP)
INSB	INSB (section 9.9)	0x0C	0x23	Bits and fields (BESC)
ISYNC	ISYNC (section 6.7)	0x14	0x2	System (SYS)
JMP	JMP (section 9.10)	0x13	0xC	Single operand (SOP)
JMPI	JMPI (section 9.10)	0x12	0x0	Indirect transfers (XFER)
JTAG	JTAG (section 9.10)	0x12	0x2	Indirect transfers (XFER)
LEA	LEA (section 9.6)	0x0B		Moves, address formation and the ALU
LL	LLSC (section 7.12)	0x11	0x20	Atomics (ATOM)
LPAD	NOP (section 6.7)	0x14	0x9	System (SYS)
MAX	MINMAX (section 9.7)	0x0E	0x12	Arithmetic (AESC)
MAXU	MINMAX (section 9.7)	0x0E	0x13	Arithmetic (AESC)
MCMP	MCMP (section 9.11)	0x0D	0x3	Block memory (MBLK)
MCPY	MCPY (section 9.11)	0x0D	0x0	Block memory (MBLK)
MFASID	MTASID (section 6.7)	0x14	0x7	System (SYS)
MFCPUID	MFCPUID (section 6.7)	0x14	0xB	System (SYS)
MFDP	MTDP (section 6.7)	0x14	0x9	System (SYS)
MFPS	MTPS (section 6.7)	0x14	0x2	System (SYS)
MFSR	MFSR (section 6.7)	0x14	0xD	System (SYS)
MIN	MINMAX (section 9.7)	0x0E	0x10	Arithmetic (AESC)
MINU	MINMAX (section 9.7)	0x0E	0x11	Arithmetic (AESC)
MMOV	MCPY (section 9.11)	0x0D	0x1	Block memory (MBLK)
MOD	DIV (section 9.7)	0x0E	0x2	Arithmetic (AESC)
MODU	DIV (section 9.7)	0x0E	0x3	Arithmetic (AESC)
MOVB	MOV (section 9.6)	0x06		Moves, address formation and the ALU
MOVL	MOV (section 9.6)	0x02		Moves, address formation and the ALU
MOVQ	MOV (section 9.6)	0x00		Moves, address formation and the ALU
MOVUB	MOVU (section 9.6)	0x0F	0x0	Memory escape (MESC)
MOVUL	MOVU (section 9.6)	0x0F	0x4	Memory escape (MESC)
MOVUQ	MOVU (section 9.6)	0x0F	0x6	Memory escape (MESC)
MOVUW	MOVU (section 9.6)	0x0F	0x2	Memory escape (MESC)
MOVW	MOV (section 9.6)	0x04		Moves, address formation and the ALU
MOVZB	MOVZ (section 9.6)	0x0A		Moves, address formation and the ALU
MOVZL	MOVZ (section 9.6)	0x08		Moves, address formation and the ALU
MOVZW	MOVZ (section 9.6)	0x09		Moves, address formation and the ALU
MSCN	MSCN (section 9.11)	0x0D	0x4	Block memory (MBLK)
MSCNN	MSCN (section 9.11)	0x0D	0x5	Block memory (MBLK)
MSET	MSET (section 9.11)	0x0D	0x2	Block memory (MBLK)
MTASID	MTASID (section 6.7)	0x14	0x6	System (SYS)
MTDMW	MTDMW (section 6.7)	0x14	0xA	System (SYS)
MTDP	MTDP (section 6.7)	0x14	0x8	System (SYS)
MTPS	MTPS (section 6.7)	0x14	0x1	System (SYS)
MTSR	MFSR (section 6.7)	0x14	0xE	System (SYS)
MUL	MUL (section 9.7)	0x1E		Moves, address formation and the ALU
MULH	MULH (section 9.7)	0x1F		Moves, address formation and the ALU
MULHU	MULHU (section 9.7)	0x0E	0x4	Arithmetic (AESC)
MULU (alias of MUL)	MUL (section 9.7)	0x1E		Moves, address formation and the ALU
MVAL	MVAL (section 9.11)	0x0D	0x6	Block memory (MBLK)
NEG	NEG (section 9.7)	0x13	0x4	Single operand (SOP)
NOP	NOP (section 6.7)	0x14	0x0	System (SYS)
POP	POP (section 9.11)	0x13	0xF	Single operand (SOP)
POPCNT	POPCNT (section 9.9)	0x0C	0x4	Bits and fields (BESC)

Mnemonic	Entry	Opcode	Sub	Group
POPM	POPM (section 9.11)	0x13	0xB	Single operand (SOP)
POPX	PUSHX (section 6.7)	0x14	0x1	System (SYS)
PROBER	PROBE (section 6.7)	0x14	0xD	System (SYS)
PROBEW	PROBE (section 6.7)	0x14	0xE	System (SYS)
PUSH	PUSH (section 9.11)	0x13	0xE	Single operand (SOP)
PUSHM	PUSHM (section 9.11)	0x13	0xA	Single operand (SOP)
PUSHX	PUSHX (section 6.7)	0x14	0x0	System (SYS)
RET	RET (section 6.7)	0x14	0x3	System (SYS)
ROL (alias of RORI)	RORI (section 9.9)	0x0C	0x11	Bits and fields (BESC)
ROR	ROR (section 9.9)	0x0C	0x10	Bits and fields (BESC)
RORI	RORI (section 9.9)	0x0C	0x11	Bits and fields (BESC)
RORI32	RORI32 (section 9.9)	0x0C	0x12	Bits and fields (BESC)
RTI	RTI (section 6.7)	0x14	0x1	System (SYS)
RTT	RTI (section 6.7)	0x14	0x2	System (SYS)
SAR	SHIFT (section 9.8)	0x22		Moves, address formation and the ALU
SBC	SBC (section 9.7)	0x13	0x7	Single operand (SOP)
SC	LLSC (section 7.12)	0x11	0x21	Atomics (ATOM)
SET	SET (section 9.7)	0x0E	0x20	Arithmetic (AESC)
SEV	SEV (section 6.7)	0x14	0x8	System (SYS)
SHL	SHIFT (section 9.8)	0x20		Moves, address formation and the ALU
SHR	SHIFT (section 9.8)	0x21		Moves, address formation and the ALU
SOB	SOB (section 9.10)	0x3F	0x3F	Branches
SPL	SPL (section 6.7)	0x14	0x3	System (SYS)
SUB	SUB (section 9.7)	0x17		Moves, address formation and the ALU
SUBM	ADDM (section 9.7)	0x0F	0x9	Memory escape (MESC)
SWAP	SWAP (section 7.12)	0x11	0x2	Atomics (ATOM)
SWAPDB	SWAP (section 7.12)	0x11	0x3	Atomics (ATOM)
TLBI	TLBI (section 6.7)	0x14	0x5	System (SYS)
TLBIA	TLBA (section 6.7)	0x14	0xA	System (SYS)
TLBSYNC	TLBA (section 6.7)	0x14	0xB	System (SYS)
TRAP	TRAP (section 6.7)	0x14	0x4	System (SYS)
TRAPL	TRAP (section 6.7)	0x14	0xF	System (SYS)
TST	TST (section 9.8)	0x13	0x5	Single operand (SOP)
UTF8DEC	UTF8DEC (section 9.9)	0x0C	0x41	Bits and fields (BESC)
UTF8ENC	UTF8ENC (section 9.9)	0x0C	0x42	Bits and fields (BESC)
UTF8LEN	UTF8LEN (section 9.9)	0x0C	0x40	Bits and fields (BESC)
WAIT (alias of WFE)	WFE (section 6.7)	0x14	0x7	System (SYS)
WFE	WFE (section 6.7)	0x14	0x7	System (SYS)
XADD	XADD (section 7.12)	0x11	0x4	Atomics (ATOM)
XADDDB	XADD (section 7.12)	0x11	0x5	Atomics (ATOM)
XAND	XAND (section 7.12)	0x11	0x6	Atomics (ATOM)
XANDDB	XAND (section 7.12)	0x11	0x7	Atomics (ATOM)
XMAX	XMAX (section 7.12)	0x11	0xC	Atomics (ATOM)
XMAXDB	XMAX (section 7.12)	0x11	0xD	Atomics (ATOM)
XMAXU	XMAX (section 7.12)	0x11	0x10	Atomics (ATOM)
XMAXUDB	XMAX (section 7.12)	0x11	0x11	Atomics (ATOM)
XMIN	XMAX (section 7.12)	0x11	0xE	Atomics (ATOM)
XMINDB	XMAX (section 7.12)	0x11	0xF	Atomics (ATOM)
XMINU	XMAX (section 7.12)	0x11	0x12	Atomics (ATOM)
XMINUDB	XMAX (section 7.12)	0x11	0x13	Atomics (ATOM)
XNORPOP	XNORPOP (section 9.9)	0x0C	0x30	Bits and fields (BESC)
XOR	XOR (section 9.8)	0x1C		Moves, address formation and the ALU
XOR_	XAND (section 7.12)	0x11	0x8	Atomics (ATOM)
XOR_DB	XAND (section 7.12)	0x11	0x9	Atomics (ATOM)
XXOR	XAND (section 7.12)	0x11	0xA	Atomics (ATOM)
XXORDB	XAND (section 7.12)	0x11	0xB	Atomics (ATOM)

The opcodes, in order

Generated by `tools/mkmanual.py`; do not edit.

Page 1

Opcode	Use
0x00	MOVQ
0x01	reserved
0x02	MOVL
0x03	reserved
0x04	MOVW
0x05	reserved
0x06	MOVB
0x07	reserved
0x08	MOVZL
0x09	MOVZW
0x0A	MOVZB
0x0B	LEA
0x0C	CLZ, CTZ, CLO, CTO, POPCNT and 17 more (sub-opcodes in the entries)
0x0D	MCPY, MMOV, MSET, MCMP, MSCN and 2 more (sub-opcodes in the entries)
0x0E	DIV, DIVU, MOD, MODU, MULHU and 6 more (sub-opcodes in the entries)
0x0F	MOVUB, MOVUW, MOVUL, MOVUQ, ADDM and 3 more (sub-opcodes in the entries)
0x10	FDIV, FSQRT, FADD32, FSUB32, FMUL32 and 49 more (sub-opcodes in the entries)
0x11	CAS, CASDB, SWAP, SWAPDB, XADD and 17 more (sub-opcodes in the entries)
0x12	JMPI, CALLI, JTAG
0x13	CLR, COM, INC, DEC, NEG and 11 more (sub-opcodes in the entries)
0x14	NOP, RTI, RTT, RET, HALT and 29 more (sub-opcodes in the entries)
0x15	BTAG, BNTAG
0x16	ADD
0x17	SUB
0x18	BIC
0x19	BIS
0x1A	CMP
0x1B	BIT
0x1C	XOR
0x1D	AND
0x1E	MUL
0x1F	MULH
0x20	SHL
0x21	SHR
0x22	SAR
0x23	spare ALU slot (decodes, traps)
0x24	FADD
0x25	FSUB
0x26	FMUL
0x27	FCMP
0x28	CBNE, CBEQ, CBPL, CBMI, CBVC and 9 more (sub-opcodes in the entries)
0x29	reserved
0x2A	reserved
0x2B	reserved
0x2C	reserved
0x2D	reserved
0x2E	reserved
0x2F	reserved

Opcode	Use
0x30	BR
0x31	BNE
0x32	BEQ
0x33	BPL
0x34	BMI
0x35	BVC
0x36	BVS
0x37	BCC
0x38	BCS
0x39	BGE
0x3A	BLT
0x3B	BGT
0x3C	BLE
0x3D	BHI
0x3E	BLOS
0x3F	SOB

Page 2

Opcode	Name
0x00	ld.q
0x01	st.q
0x02	ld.l
0x03	st.l
0x04	ld.w
0x05	st.w
0x06	ld.b
0x07	st.b
0x08	ldz.l
0x09	ldz.w
0x0A	ldz.b
0x0B	lea.sp
0x0C	ldq.sp0
0x0D	ldq.sp1
0x0E	stq.sp
0x10	ldl.sp0
0x11	ldl.sp1
0x12	stl.sp0
0x13	stl.sp1
0x14	ldq.d1
0x15	ldq.d2
0x16	ldq.d3
0x17	ldq.d4
0x18	stq.d1
0x19	stq.d2
0x1A	ldl.d1
0x1B	ldl.d2
0x1C	ldl.d3
0x1D	ldl.d4
0x1E	pushm8
0x1F	popm8
0x20	sub.sp
0x21	add.sp
0x22	movq.i
0x23	shl.i
0x24	shr.i
0x25	sar.i
0x26	and.i
0x27	bis.i
0x28	xor.i
0x29	add.i
0x2A	pushx
0x2B	popx
0x2C	isync
0x2D	hint

Part III — Appendices

Appendix A — Possible improvements not decided on yet

These are ideas that would make the machine better in one respect or another and that nobody has yet turned into a decision. They come from the reports on kernel pain points, hardware vulnerabilities and OpenBSD's mitigations, and from the open items of the instruction-set text. Each is stated as a possibility, with its benefit, its cost and what other machines do, so that it can be taken up, rejected or promoted to a decision in appendix B. Nothing here is built or specified unless it says so. The same ordering principle as the rest of the book applies: code size, then easy recoverability, then speed and efficiency through fewer gates and tighter hot loops.

A.1 Reserve a page bit that enforces landing pads

A.1.1 The `LPAD` instruction marks a legal target of an indirect jump or call; a page bit could make the hardware fault when an indirect transfer lands anywhere else, with enforcement off in ABI version 1 and on from version 2.

Commentary. **Precedent.** `LPAD` already exists, and the ABI leaves landing pads off in version 1, with no switch to enforce them. **Trade-offs.** The bit costs one comparator and two bytes per address-taken function, and deciding later gets more expensive because every binary built in the meantime lacks the pads. **Benefits.** It blocks jump-oriented attacks at the transfer, with no recovery code beyond the fault handler that already exists. **Modern comparables.** Intel's indirect-branch tracking and ARM's branch target identification, both of which OpenBSD enforces by default where the hardware has them. **Impact.** *Kernel:* A landing-pad violation is a fault the handler already handles; the kernel sets the page bit at exec time. *Compiler:* The compiler emits `LPAD` at every address-taken function, two bytes each. *Hardware:* One page bit and one comparator on an indirect-transfer target.

A.2 A page bit that says a system call may be made from here

A.2.1 A page-table bit could let the hardware refuse a system call from any page not so marked, so that the kernel's check of where a call came from moves into the instruction.

Commentary. **Precedent.** OpenBSD's `msyscall` and `pinsyscalls` restrict where system calls may be issued from. **Trade-offs.** One bit and one gate; the kernel would no longer need to check the origin on every call. **Benefits.** About four instructions removed from every system call, and region pinning with no hot-path check. **Modern comparables.** No architecture surveyed has this bit; it is done by the kernel in software. **Impact.** *Kernel:* The kernel no longer checks where a system call came from on every call, which removes about four instructions from each. *Compiler:* None. *Hardware:* One page bit and one gate at the `TRAP` decode.

A.3 A check of the stack pointer at system-call entry

A.3.1 The hardware could check that the stack pointer lies in a region registered as a stack when a system call or a fault is taken.

Commentary. **Precedent.** OpenBSD checks the stack pointer against a `MAP_STACK` region on system calls and faults. **Trade-offs.** It adds a comparison to entry, which is the path the design most wants short, and gives no benefit over the kernel making the same check. **Verdict so far.** Not recommended; recorded because the question is natural. **Impact.** *Kernel:* None over the check the kernel already makes. *Compiler:* None. *Hardware:* A comparator and a region register on the entry path, which is the path the design most wants short.

A.4 Protecting the return address

A.4.1 The return address lives on the data stack, because `CALL` pushes it and there is no link register; there are four ways to protect it: software canaries and OpenBSD's return-address protection in the compiler, a hardware return stack, a shadow stack, and nothing.

Commentary. **Precedent.** x86 has the same exposure and uses stack canaries, `retguard` in OpenBSD's clang, and the shadow stacks of Intel's control-flow enforcement; ARM has pointer authentication. **Trade-offs.** A shadow stack is the strongest and brings the most recovery code (a second stack to overflow, switch and unwind), so the analysis does not recommend it by default; software detection costs a few instructions per function and no hardware. **Benefits.** Detection with little or no recovery code. **Verdict so far.** Start with the compiler; consider a hardware mechanism only if code size allows. **Impact.** *Kernel:* A shadow stack is a second stack for the kernel to manage and unwind; software detection needs only a fault handler. *Compiler:* The compiler adds a canary or a return-address check per function. *Hardware:* Software costs nothing; a hardware return stack or a shadow stack costs a second stack structure and its control.

A.5 Tag or key bits in pointers

A.5.1 The 25 spare bits of a 64-bit pointer (a 39-bit virtual address uses the rest) could carry a tag or a key, as a memory-tagging or pointer-authentication scheme does.

Commentary. **Precedent.** ARM's top-byte-ignore, pointer authentication and memory tagging. **Trade-offs.** The cost is canonical-address rules: pointers must be masked before use, and the hardware must define what a tagged pointer does. **Verdict so far.** No threat that the operating system asked for requires it; it costs gates and state, so it is not recommended now. **Impact.** *Kernel:* The kernel would have to define what a tagged pointer means across the user and kernel boundary. *Compiler:* Every pointer use needs masking, and pointer arithmetic must preserve tags. *Hardware:* Gates and state in the address path.

A.6 An optional 48-bit address mode

A.6.1 A fourth page-table level, switched on by a feature bit, could raise the virtual address from 39 to 48 bits.

Commentary. **Trade-offs.** The user half of the 39-bit space is 38 bits, which gives address-space randomisation far less entropy than the 47 bits of amd64; the price of fixing it is a longer walk on every miss. **Benefits.** Entropy and a bigger address space for a database or a memory-mapped file. **Modern comparables.** x86-64 and AArch64 have four levels (and x86-64 five); RISC-V offers three, four and five by feature. **Impact.** *Kernel:* A mode for the kernel to switch on, with a fourth-level table to allocate and walk. *Compiler:* None beyond pointer size, which does not change. *Hardware:* A fourth walker level and a longer walk on every miss.

A.7 A second page-table base register

A.7.1 A second page-table base for the kernel half of the address space would let a kernel and a user program run with different tables without flushing, if a speculative core ever needs the isolation.

Commentary. **Precedent.** The cost of separating kernel and user page tables on x86 after Meltdown was about 5% typical and about 30% in the worst case in the developers' own estimates, and the tagged translation buffers (PCID) made it tolerable. **Trade-offs.** The current single base is correct and free on a core that does not speculate; the second base register costs a register and a rule, and the question is whether to pay it before the need exists. **Benefits.** The isolation is in the hardware from the start. **Modern comparables.** AArch64 has two translation-table base registers, one for each half of the address space. **Impact.** *Kernel:* The kernel switches one base register at the privilege change instead of managing a shared table. *Compiler:* None. *Hardware:* A second base register and a rule choosing which half uses which.

A.8 Two-byte forms of the user-memory guard instructions

A.8.1 `STUA` and `CLUA` could have one-parcel page-2 forms, saving four bytes per user-copy routine.

Commentary. **Trade-offs.** It uses two of the 19 reserved page-2 slots, which is not worth it until a kernel exists to count the copy routines. **Benefits.** Smaller and faster user copies. **Verdict so far.** Hold the slots and decide when the kernel exists. **Impact.** *Kernel:* Each user copy routine is four bytes smaller. *Compiler:* None. *Hardware:* Two reserved page-2 slots used.

A.9 Thirty-two-bit forms of add, subtract and shift

A.9.1 If the extension instructions the compiler emits after narrow arithmetic are more than a few percent of the code, 32-bit forms of the common operations on page 2 would remove them.

Commentary. **Precedent.** AArch64's `w` forms and RISC-V's `ADDW` family. **Trade-offs.** A second set of opcodes in the page-2 map against extension instructions in the compiled code; the overhead has never been measured over the whole corpus. **Verdict so far.** Measure first; it is one script over the compiled corpus. **Impact.** *Kernel:* None. *Compiler:* The compiler

stops emitting an extension after common 32-bit operations. *Hardware:* A second set of opcodes in the page-2 map and a little ALU decode.

A.10 Packed-lane instructions in the integer registers

A.10.1 A small set of lane operations (add, subtract, saturating add, compare, minimum, maximum, average, sum of absolute differences, shuffle on 8, 16 and 32-bit lanes) in the existing registers could speed up string, codec and image loops.

Commentary. **Precedent.** Alpha's motion-video instructions and PA-RISC's multimedia instructions did exactly this, in integer registers. **Trade-offs.** It adds no state and little hardware, but it is 64 bits wide so the gain per instruction is modest, and compilers use it poorly. **Verdict so far.** Collect the evidence first: the compiler corpus was built at a size setting that hides how much vectorisable code there is (appendix B, S1). **Impact.** *Kernel:* String and checksum routines get faster. *Compiler:* Vectorisation into integer lanes is weak, so the library routines would be hand-written. *Hardware:* A lane-aware ALU: little extra hardware and no new state.

A.11 Streaming and non-temporal block operations

A.11.1 A large `MCPY` evicts the whole data cache for data never read again; a non-temporal variant or a streaming memory type would avoid it.

Commentary. **Precedent.** x86 has non-temporal stores; ARM has non-temporal load and store pairs. **Trade-offs.** It shows immediately on an 8 KB cache; the fix is an opcode or a memory type, the non-fix is to leave the cache alone. **Verdict so far.** Open since the instruction-set text's open items; decide when the memory system is built. **Impact.** *Kernel:* Large copies would not evict the kernel's working set. *Compiler:* `memcpy` would choose the streaming form above a size. *Hardware:* A non-temporal write path in the cache.

A.12 Smaller possibilities

- `MULHSU` (the high half of a signed-by-unsigned product, for a signed top limb meeting an unsigned one) is reserved in the arithmetic escape; it is one correction step in software and has one caller, a bignum library's innermost loop.
- **Hardware page zeroing**, and **guard-page states in the page-table entry that do not consume translation-buffer entries**, would make the allocator hardening of OpenBSD's `malloc` cheaper; nobody has asked.
- **A mixed leaf size for the kernel's text** is needed if the kernel is to be write-xor-execute with one-gigabyte leaf pages.
- **Hardware breakpoints and debug registers** are not designed; a debugger today has only the breakpoint instruction and the trace bit.
- **A sealed (immutable) page bit and a hardware mask for `pledge`** have no benefit over the kernel doing the checks, and are recorded to be rejected.

Commentary. **Trade-offs.** None of these is ranked above the decisions in appendix B; the first four sections of this appendix (landing pads, the system-call page bit, the stack check and return-address protection) are the ones OpenBSD's developers have shown the most interest in.

Appendix B — Decisions pending

These are the decisions the author has not yet made, each with its question, the proposed answer, and the precedent, trade-offs and benefits that surround it. Each has five options with their costs in the working document `../decisions-pending.md` (the identifiers P, R, S, N and B below are that document's), and the boot choices are worked through in `../boot.md`. Decisions already made (errata 42 to 57) are not repeated here. Each entry is judged by the three principles in order: code size, easy recoverability (less recovery code), and speed and efficiency through fewer gates and tighter hot loops. Every entry is a proposal *(P)* until the author decides it.

B.1 The instruction set and the system

B.1.1 The wake rule of `WFE` (P1)

B.1.1.1 The question is whether `WFE` ends its sleep when an interrupt is pending even if the priority masks it; the proposal is yes. *(P)*

Commentary. **Precedent.** An idle loop must check for work and sleep without a gap, or an interrupt that arrives between the two is lost. V7's idle routine lowered the priority and executed `wait` inside a loop that rechecked the run queue after every interrupt, so a lost wake-up cost at most one clock tick. **Trade-offs.** With priority 7 on entry, "lower the priority, then sleep" misses an interrupt that lands between the two, and "sleep at priority 7" never wakes. **Benefits.** The idle loop becomes `spl #7`, check for work, `wfe`, `spl #0`, with no gap and no timer tick to bound it; no opcode, a few gates. **Modern comparables.** AArch64 and RISC-V `wfi` resume on a pending enabled interrupt (RISC-V's manual says the global enable bit is ignored and that an implementation may wake for any reason, so "the same as RISC-V" holds only if the pdp64's priority mask is treated like that bit); x86 closes the gap with the one-instruction interrupt shadow of `STI`, which is hidden state. **Impact.** *Kernel:* The idle loop needs no tick to bound a lost wake-up. *Compiler:* Spin-wait and idle code use `WFE` and `HINT #1`. *Hardware:* A pending-interrupt line into the wait logic, a few gates.

B.1.2 An unprivileged time counter (P3)

B.1.2.1 The proposal is `RDTIME rd`, an unprivileged read of a free-running, constant-rate, monotonic counter. *(P)*

Commentary. **Precedent.** The PDP-11 and VAX kept the time in memory updated by a line clock and read it with a system call. **Trade-offs.** A user-readable counter is a timing side channel, so the kernel needs an enable bit; the counter's width and rate become architecture. **Benefits.** A clock read without a trap, which is what a fast `clock_gettime` in a shared page needs. **Modern comparables.** x86 `RDTSC`, RISC-V `rdtime`, PowerPC `mftb`, s390 `STCKF`, MIPS `RDHWR`, LoongArch `rdtime`; the x86 counter's history of unreliability is the argument for fixing monotonicity and constant rate at the start. **Use.** Timestamps, timeouts, and the fast path of every library's clock call. **Impact.** *Kernel:* A fast `clock_gettime` with no trap. *Compiler:* An intrinsic for the counter read. *Hardware:* One counter, a user-read enable bit and a `SYSX` decode.

B.1.3 Feature and identification registers (P4)

B.1.3.1 The proposal is a block of read-only system registers (numbers 13 and up) giving the version, the optional features, the cache geometry, the page sizes and the address width. *(P)*

Commentary. **Precedent.** The VAX had a system-identification register the kernel switched on; x86 has `CPUID`, which has grown hundreds of leaves and traps under virtualisation. **Trade-offs.** Each register's layout must be written once in a grammar so it cannot drift. **Benefits.** One binary adapts to the machine, and a kernel need not patch itself per model (OpenBSD carries 75 references to its boot-time patching). **Modern comparables.** AArch64's identification registers, RISC-V's `misa`. **Impact.** *Kernel:* The kernel need not patch itself per model. *Compiler:* Runtime selection of routines by feature bits. *Hardware:* A small block of read-only registers.

B.1.4 TLB invalidation by address-space tag (P5)

B.1.4.1 The proposal is one new instruction, `TLBIASID`, that invalidates every translation of one address space, broadcast like `TLBI`. *(P)*

Commentary. **Precedent.** The VAX flushed process translations on every context switch and kept system ones; x86 added tags (PCID) and `INVPCID` much later. **Trade-offs.** A flush-all works until the first rollover of the tag numbers, after which every process loses its translations. **Benefits.** Process exit and tag reuse become cheap. **Modern comparables.** AArch64 `TLBI ASIDE1IS`, RISC-V `sfence.vma` with an address-space operand, x86 `INVPCID`. **Impact.** *Kernel:* Process exit and tag reuse become one broadcast. *Compiler:* None. *Hardware:* A tag compare in the translation buffer's invalidate path.

B.1.5 What a multi-register instruction leaves behind when it faults (P12)

B.1.5.1 The proposal is a rule and a test: every multi-access instruction (`PUSHM`, `POPM`, `PUSHX`, `POPX`, the block operations) is either all-or-nothing or safe to repeat from the start, and an emulator test interrupts each at every access and checks that a restart reaches the same state. *(P)*

Commentary. **Precedent.** The PDP-11 needed a status register (`MMR1`) and, on the 11/40, a kernel routine that decoded the faulting instruction to undo it; the VAX needed a first-part-done bit; ARM's load-multiple restarts with the base register restored. **Trade-offs.** The area below the stack pointer is dead memory, so a partial push is invisible; the risk is another observer (a device or core) seeing a half-written area. **Benefits.** The fault handler restarts blindly, with no instruction-specific cases, which is what recoverability means. **Modern comparables.** RISC-V has no multi-register instruction; ARM and the VAX restart as above. **Impact.** *Kernel:* The fault handler restarts every multi-access instruction blindly. *Compiler:* None. *Hardware:* Each multi-access instruction checks every address before acting, or is repeatable.

B.1.6 Priority 7 on interrupt entry as well as exception entry (P14)

B.1.6.1 Exceptions enter at priority 7, which is decided; the open question is whether interrupts do too, which the implementation does. *(exceptions D, errata 40; interrupts P)*

Commentary. **Precedent.** The PDP-11 loaded each vector's own priority and pushed the old state on a stack, so an interrupt at any time could not lose it; trap vectors conventionally held priority 7. **Trade-offs.** Entering in registers needs the priority raised, or a higher-level interrupt could overwrite the saved state before the handler stores it; the cost is one `SPL` in a handler that wants to be interruptible. The alternatives are per-vector priority (the PDP-11's choice), a hardware push, and an interrupt shadow. **Modern comparables.** x86, AArch64 and RISC-V all mask interrupts on entry. **Impact.** *Kernel:* One `SPL` in a handler that wants to be interruptible. *Compiler:* None. *Hardware:* Priority 7 loaded into the new status word at entry.

B.1.7 A prefetch instruction (P15)

B.1.7.1 The proposal is none for now, with the shape of a non-faulting `PREFETCH` with a memory operand reserved on a free sub-opcode. *(P)*

Commentary. **Precedent.** Neither the PDP-11 nor the VAX had one; caches there were transparent. **Trade-offs.** The kernel survey counts over a hundred prefetch instructions across the four kernels, mostly in tuned list and copy code; an in-order core with no data cache gains nothing. **Modern comparables.** AArch64 `PRFM`, ARM `PLD`, PowerPC `DCBT`, x86 `PREFETCHT0`. **Impact.** *Kernel:* List walks and copy loops can warm the cache. *Compiler:* `PREFETCH` would need compiler heuristics. *Hardware:* A non-faulting load path with no destination; nothing on a core with no data cache.

B.1.8 Performance counters (P16)

B.1.8.1 The proposal is two free-running counters, cycles and instructions retired, readable in user mode, added after the clock (P3) with the same enable bit. *(P)*

Commentary. **Precedent.** Unix profiled by sampling the program counter at clock ticks. **Trade-offs.** A user-readable cycle counter is a timing channel; a full counter block with event selection costs a large block and interrupt machinery. **Modern comparables.** x86 `RDPMC`, the AArch64 PMU, RISC-V's `hpm` counters. **Impact.** *Kernel:* A profiler reads counters instead of sampling at ticks. *Compiler:* Intrinsics. *Hardware:* Two counters and an enable bit.

B.1.9 Constant-time execution (P17)

B.1.9.1 The question is how to guarantee that instruction time does not depend on operand values; the proposal is a core set of instructions that is always constant-time plus a mode bit that extends the promise to divide, `MSCN`, `MCMP` and floating point. *(P, the author's request, option 3)*

Commentary. **Trade-offs.** Making all integer instructions constant-time would cost about 1% on average by a rough estimate, because divide is only 0.04% of compiled instructions (149 of 370,115; multiply costs nothing, since a fixed-latency multiplier needs less logic than one with an early-out); the mode bit keeps fast paths for ordinary code, which the author prefers. A constant-time instruction does not make code constant-time: a secret-dependent branch or table index still leaks through the predictor and the cache. **Precedent.** Neither ancestor addressed it; the PDP-11's multiply and divide were microcoded with operand-dependent times. **Modern comparables.** Arm's `DIT` bit, Intel's data-operand-independent timing mode, RISC-V's scalar-crypto timing subset. **Use.** Cryptography on secret keys. The published performance cost of those modes is being researched. **Impact.** *Kernel:* Crypto code sets the mode around secret-dependent work. *Compiler:* The back end must not turn a constant-time sequence into a variable-time one. *Hardware:* The core set costs nothing; the mode bit needs the fast paths of divide and the scanners to be gateable.

B.1.10 The literal pool and execute-only code (P18)

B.1.10.1 The proposal is an ABI rule: literal pools move out of `.text` into a read-only section beside it, so code pages hold only instructions and can be execute-only. *(P)*

Commentary. **Precedent.** The PDP-11's immediates sit in the instruction stream, and with separate instruction and data spaces a program could not read its own instructions as data. **Trade-offs.** The PC-relative reach limits how far the pool may be, and the linker must keep the two sections near each other. **Benefits.** Zero hardware; a memory-disclosure bug no longer reveals code. **Modern comparables.** OpenBSD makes code execute-only on amd64 and arm64; ARM compilers offer a flag to keep pools out of code. **Impact.** *Kernel:* None. *Compiler:* Pools are emitted into a read-only section beside the text. *Hardware:* None; the linker keeps the sections within PC-relative reach.

B.1.11 Vector and SIMD math (S1 to S6)

B.1.11.1 The device route is decided: the processor has no vector register file and no SIMD instruction set, and wide data-parallel work (graphics, AI and large-language-model inference, the array languages APL and K) is done by memory-mapped devices (chapter 8). *(D, errata 59)*

B.1.11.2 What remains open from the six questions is the number formats and operations of the devices and the reconciliation of the device descriptors with the standard descriptor; the six questions were: where data-parallel work lives (coprocessor, packed lanes in the integer registers, a fixed-width vector file, a scalable one, a matrix unit); which registers hold vectors; fixed or scalable length; how the machine state is saved on a context switch and whether the kernel may use it; which number formats (half precision, bfloat16, 8-bit integer dot products, 8-bit floating point); and the ABI and compiler support. *(P)*

Commentary. **Precedent.** The PDP-11 sent heavy numerical work to attached array processors; the VAX had a vector extension on its 6000 series, with 16 vector registers of 64 elements, supported by VMS; neither Unix used it much. **Trade-offs.** A fixed-width vector file brings a second register file and its context-switch cost, opcodes, and a new width for every generation (the history of SSE, AVX and AVX-512); a scalable design runs one binary on a small and a large implementation at the price of the most complex specification. Lazy saving of vector state was a security hole on x86, and Linux went back to eager restore. **Benefits.** The coprocessor needs no opcodes and no processor state. **Modern comparables.** ARM SVE and RISC-V V (scalable), ARM NEON, x86 AVX, PowerPC VSX, Alpha MVI. **Use.** Neural-network inference (the LB64 accelerator), graphics, signal processing and the library's string and memory routines. The evidence needed first: compile the corpus at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and decide the target workloads. **Impact.** *Kernel:* No vector state to save. *Compiler:* No vectoriser for the processor; the runtime lowers array primitives to descriptors. *Hardware:* No register file; the devices carry the width.

B.2 The bus, boot and devices

B.2.1 Data-cache maintenance by address (P6)

B.2.1.1 The proposal is to reserve nothing: DMA buffers are uncached, so no cache instruction is needed until a non-coherent device appears. *(P)*

Commentary. **Precedent.** The 11/70's cache watched DMA writes; the VAX's adapter had data paths to purge. **Trade-offs.** Uncached buffers are slow to read; cache maintenance would speed them and needs three reserved sub-opcodes. **Modern comparables.** ARM, MIPS and embedded PowerPC with non-coherent DMA wrap cache maintenance in a DMA-mapping layer; x86 DMA is coherent. **Impact.** *Kernel:* Drivers need no cache code. *Compiler:* None. *Hardware:* No cache-maintenance operations.

B.2.2 Firmware calls (P7)

B.2.2.1 The proposal is none: everything goes through devices, with a required reset register and a watchdog in the bus controller. The kernel survey disagrees and would specify a firmware-call convention on reserved trap vectors. *(P)*

Commentary. **Precedent.** The PDP-11 kernel never called firmware; the VAX had console registers the kernel read and wrote. **Trade-offs.** No firmware means no firmware to trust and no extra privilege level, and every platform must say where its console and reset registers are, which the others get from firmware. **Modern comparables.** RISC-V's supervisor binary interface, ARM's PSCI and secure monitor calls, Alpha's PALcode, SPARC's Open Firmware. **Impact.** *Kernel:* Reboot and power-off through a device register and a watchdog. *Compiler:* None. *Hardware:* A reset register and a watchdog timer in the controller.

B.2.3 Double-width compare-and-swap and a random-number instruction (P8, P9)

B.2.3.1 The proposals are to add neither: the evidence for a 128-bit compare-and-swap is one Linux use (software can tag a pointer with the 25 spare bits instead), and random numbers come from an entropy device on the bus rather than an instruction. *(P)*

Commentary. **Precedent.** The VAX solved the lock-free queue with whole-queue interlocked instructions; no machine of the period had a random-number instruction. **Modern comparables.** x86 `CMPXCHG16B`, AArch64 `CASP`, RISC-V `AMOCAS.Q`; x86 `RDRAND` and `RDSEED`, PowerPC `DARN`, s390 `PRNO`, AArch64 `RNDR`. **Impact.** *Kernel:* Entropy from a device at boot. *Compiler:* Tagged pointers in software if a double-width compare-and-swap is not wanted. *Hardware:* An entropy source device; no new opcode.

B.2.4 The interrupt controller and routing to cores (R1)

B.2.4.1 The proposal is wired interrupt lines to core 0 with a per-line target register, keeping the eight priority levels, with message-signalled interrupts added later. *(P)*

Commentary. **Precedent.** The PDP-11 had no controller; a device asserted one of four request lines and supplied its vector. **Trade-offs.** The smallest design adds no new failure cases and cannot give a fast multi-queue device its own interrupt. **Modern comparables.** x86's APIC, ARM's GIC, RISC-V's PLIC and AIA. **Impact.** *Kernel:* A handler sees one interrupt model. *Compiler:* None. *Hardware:* Wired lines to one core with a target register.

B.2.5 Inter-processor interrupts and starting other cores (R2, B2)

B.2.5.1 The proposal is a per-core doorbell register and a per-core start register in the bus controller (chapter 4), with `SEV` and `WFE` for waiting. *(P)*

Commentary. **Precedent.** The VAX's console processor started the other processors. **Modern comparables.** x86's INIT and START-UP sequence, ARM's PSCI and spin table, RISC-V's hart-start call. **Impact.** *Kernel:* One register write per core and one per doorbell. *Compiler:* None. *Hardware:* A start register and a doorbell per core.

B.2.6 How coherence scales (R3)

B.2.6.1 The proposal is to keep snooping for two to four cores and to record the ceiling. *(P)*

Commentary. **Trade-offs.** Directory protocols scale and are large; software coherence is cheap and fragile. **Modern comparables.** x86's snoop filters and directories across sockets, ARM's coherent interconnects. **Impact.** *Kernel:* Cache coherence is invisible to the kernel. *Compiler:* None. *Hardware:* Snooping on two to four cores.

B.2.7 The bus transport (R4)

B.2.7.1 The proposal is AXI as the transport, with Q64 as the model of ordering, memory types and the hand-off on top. *(P)*

Commentary. **Trade-offs.** It saves a bridge and makes the atomics a monitor naturally, and the Unibus's lock is no longer a native transaction. **Modern comparables.** ARM's AMBA, RISC-V's TileLink or AXI, FPGA systems on AXI throughout. **Impact.** *Kernel:* None. *Compiler:* None. *Hardware:* The bridge to AXI is not needed if AXI is the transport.

B.2.8 The device layout and the descriptor (R5)

B.2.8.1 The proposal is the uniform layout and the 32-byte descriptor of chapter 5, with an identification register first, and virtio-compatible devices only if reusing OpenBSD's drivers saves more than it costs. *(P)*

Commentary. **Precedent.** The RK11 and the DEUNA, as in chapter 5. **Trade-offs.** The single ring per device makes duplex devices two slots. **Modern comparables.** `virtio`, NVMe, SATA's command lists, Intel Ethernet's done bit. **Impact.** *Kernel:* One driver skeleton. *Compiler:* The descriptor is a generated C structure. *Hardware:* One ring engine behind all devices.

B.2.9 DMA addressing and protection (R6)

B.2.9.1 The proposal is no translation unit for now: every device must be able to address all 40 bits of physical memory. *(P)*

Commentary. **Precedent.** The Unibus carried only 18 address bits, so the VAX and the larger PDP-11s had map registers that translated device addresses, in effect a simple translation unit for necessity. **Trade-offs.** No protection from a faulty or malicious device; a translation unit matters when virtualisation or untrusted devices do. **Modern comparables.** x86 VT-d, ARM SMMU, the RISC-V IOMMU; Linux's bounce buffers for limited devices. **Impact.** *Kernel:* A driver passes physical addresses. *Compiler:* None. *Hardware:* No translation unit for devices.

B.2.10 Errors raised by devices and the bus (R7)

B.2.10.1 The proposal is the precise and imprecise paths the ISA already has plus a small register block in the bus controller for the cause. *(P)*

Commentary. **Precedent.** A PDP-11 bus timeout trapped through vector 4 and usually ended in a panic; the VAX had a machine-check frame. **Modern comparables.** x86's machine-check banks and PCI Express error reporting, ARM's RAS extension. **Impact.** *Kernel:* One recovery path for device errors. *Compiler:* None. *Hardware:* A small error register block in the controller.

B.2.11 Virtualisation (R8)

B.2.11.1 The proposal is to declare it out of scope for System v1 and fix five constraints now: privileged state stays privileged or controllable, the page-table walker could be wrapped by a second level, interrupts are injectable, system-register access can trap, and the translation-enable bit and software accessed and dirty bits are reviewed for what a hypervisor would need. *(P)*

Commentary. **Precedent.** The PDP-11 was not designed for virtualisation and has sensitive instructions that do not trap in user mode; the VAX was not cleanly virtualisable either, and a 1991 paper described the changes needed. **Trade-offs.** Adding virtualisation late cost the VAX instruction changes; settling the constraints early costs a paragraph. **Modern comparables.** x86 VMX with nested paging, ARM exception level 2, the RISC-V hypervisor extension, IBM's trap-and-emulate machines. **Impact.** *Kernel:* A hypervisor could be added without changing a guest. *Compiler:* None. *Hardware:* Constraints on privileged state, the walker and interrupt injection.

B.2.12 The bus controller's mechanisms (N1 to N3)

B.2.12.1 The proposals are a fixed 4 KB page per slot with the mapping register for exceptions (N1), an interrupt with a small event queue and a rescan on overflow (N2), and an absent mark with a precise fault and the queued work left in memory (N3). *(P)*

Commentary. See section 2.6 and its commentary: the VAX's nexus scan, PCI's configuration space and all-ones reads, USB's status-change endpoint. **Impact.** *Kernel:* Enumeration and hot-plug are one loop and one queue. *Compiler:* None. *Hardware:* A fixed page per slot, an event queue and a presence bit.

B.2.13 The boot decisions (B1, B3, B4, B5, B8)

B.2.13.1 The proposals are a boot ROM that reads block 0 (B1), a boot-information block of memory, cores, the bus controller's address and two slots (B3), an `INIT` bit with an optional firmware descriptor (B4), no UEFI, with a simple hand-off a shim could use (B5), and a block list in the protective MBR's boot code (B8). *(P)*

Commentary. See chapter 4 and `../boot.md`: the PDP-11's block 0 and `ODT`, the VAX's boot block and restart parameter block, the BIOS and UEFI. **Impact.** *Kernel:* A boot-information struct and a core start register. *Compiler:* The loader and first block are built by the project's compiler. *Hardware:* A boot ROM, a block-0 reader and per-core start registers.

B.2.14 How USB attaches, and virtio first

B.2.14.1 The question is whether USB is a controller that shows each attached device as a bus slot, or a conventional ring-based host controller driven by an existing operating-system USB stack; and whether the emulator uses `virtio` devices first to reach an OpenBSD login sooner. *(P, for the author)*

Commentary. **Trade-offs.** Per-slot USB moves the protocol into the controller chip and takes the stack out of the operating system; the conventional controller uses code that already exists. `virtio` reuses OpenBSD's existing drivers in the emulator and defers the project's own eight-register devices to silicon; it gets to `ssh` faster and puts native devices later. **Modern comparables.** xHCI for the conventional controller; `virtio-mmio` on every emulated RISC-V and ARM board. **Impact.** *Kernel:* A disk driver serves a USB disk; virtio reuses the existing drivers. *Compiler:* None. *Hardware:* USB protocol in the controller costs gates; virtio costs nothing for the emulator.

Appendix C — Glossary

Terms the book uses, collected in one place. Each entry gives the plain meaning and, in brackets, the chapter or section where the book deals with it. Terms for the instruction groups (the "escape" opcodes) are listed by name and opcode; the full encodings are in the instruction-set text `../pdp64-isa.md`.

C.1 Architecture and encoding

Term	Meaning
pdp64	the 64-bit extension of the PDP-11 this book describes, as the PDP-11 extended the PDP-10's ideas down and the VAX extended the PDP-11 up (chapter 1)
parcel	a 16-bit unit of instruction encoding; an instruction is one to three parcels, two to six bytes (9.4)
`len`	bits 15 and 14 of the first parcel: the number of extension parcels for `00`, `01` and `10`; the value `11` selects the second opcode page (9.4)
page 1, page 2	the two opcode maps. Page 1 has 64 top-level opcodes, 58 assigned; page 2 (`len = 11`) has 64 one-parcel entries, 45 defined, each an exact abbreviation of a page-1 instruction (9.1)
top-level opcode	the six bits that select an instruction group; reserved for an operation whose common case is register-to-register (9.1)
escape opcode	a top-level opcode that selects a group by a sub-opcode: `BESC` (bits, `0x0C`), `MBLK` (block memory, `0x0D`), `AESC` (arithmetic, `0x0E`), `MESC` (unaligned and memory ALU, `0x0F`), `FESC` (floating point, `0x10`), `ATOM` (atomics, `0x11`), `XFER` (indirect transfers, `0x12`), `SOP` (single operand, `0x13`), `SYS` (system, `0x14`), `TAGOP` (tag branch, `0x15`)
`CBCC`	compare-immediate-and-branch on condition, opcode `0x28`, the one fused compare and branch (9.10)
`RORI32`	rotate right by an immediate, 32-bit, sign-extended result, a `BESC` sub-opcode
`SOB`	subtract one and branch, either direction (9.10)
`MCPY`, `MMOV`, `MSET`, `MCMP`, `MSCN`, `MSCNN`	the block operations: copy, move (overlap-safe), fill, compare, scan for a byte, scan for the first byte not equal; progress is held in registers so a fault restarts for free (9.11)
`MVAL`	block scan of valid UTF-8, one of the string instructions of the same family (9.9)
`PUSHM`, `POPM`	push and pop a register set named by a mask; the useful half of the VAX's `CALLS` without the frame (9.11)
`JMPI`, `CALLI`	jump and call through a memory word; the only deferred forms, used for PLT stubs and imports (9.10, 9.16)
`JTAG`, `TAGOP`	multiway dispatch on a pointer's low tag bits, and the tag-branch group (9.10)
`LPAD`	marks a legal target of an indirect jump or call; optional in ABI version 1 (appendix A.1)
`CLMUL`	carry-less multiply, the primitive under every CRC and Galois-field operation (9.9)
`XNORPOP`, `POPCNT`	bit-population instructions used by binary operations and bitsets (8.3)
`SET cc, rd`	set a register to 0 or 1 from a condition (9.7)
`DP`	the direct pointer: in user mode it is the thread pointer, in kernel mode the per-CPU pointer; a descendant of the 6502's zero page (9.3, 9.16)
`PSW`	the processor status word: flags, mode, priority and the guard bit (6.1)
`N Z V C`	the four condition flags: negative, zero, overflow, carry (9.3)
canonical address	a 64-bit address whose bits above 38 equal bit 38; any other value faults, so software cannot hide tags in the high bits (6.4)
`lowtag`	a type tag stored in the low bits of a pointer, free because objects are quad-aligned (instruction-set text, section 6.2; not otherwise used in this book)
SWAR	SIMD within a register: using one wide scalar register as several narrow lanes through masking and carry-aware arithmetic (instruction-set text, glossary; the book's bit and byte instructions of 9.9 make it effective)
`FPSR`	the sticky floating-point exception flag register, read and written by `FSFLAGS` (9.12)
ulp	unit in the last place: the spacing between adjacent floating-point numbers, the unit of rounding error
Payne–Hanek	the argument-reduction algorithm that computes x mod $\pi/2$ to full precision for large x; what hardware transcendentals cannot do (instruction-set text, section 9.7.1; the book's account of floating point is 9.12)
Table Maker's Dilemma	that correct rounding of a transcendental may need unboundedly many bits to resolve a near-tie, so no fixed-time circuit can guarantee it (instruction-set text, section 9.7.1)
block floating point	integer mantissas sharing one exponent, used by the LB64 accelerator so fixed-point accumulation is exact (8.3)

C.2 Memory, protection and the kernel

Term	Meaning
`MT`	memory type, a two-bit page attribute: normal cached, normal uncached, device strongly ordered (2.1, 6.4)
`ASID`	address space identifier: tags translation-buffer entries so a context switch flushes nothing (6.4)
`G` **bit**	global: a translation-buffer entry that ignores the `ASID` (6.4)
TLB	translation lookaside buffer: the cache of page-table entries; `TLBI` invalidates one, `TLBIA` all, `TLBSYNC` waits for other cores (7.5)
`DMW`	direct mapping window: a region translated by a fixed offset with no page table; deferred (errata 44)
`MMUCTL`	the register that enables translation and sets the table base (6.3, 6.4)
walker	the hardware that reads page-table entries on a translation-buffer miss (6.8)
`STUA` / `CLUA`	set and clear the user-access guard bit, which lets kernel code touch user memory only inside the copy routines (errata 43, 6.5)
`PROBER` / `PROBEW`	test whether an address is readable or writable in a given mode without faulting (6.7)
W^X	write xor execute: no page is both writable and executable; OpenBSD enforces it (appendix A)
PIE	position-independent executable: loadable at any address; costs nothing here (9.16)
GOT, PLT	global offset table and procedure linkage table: the indirection that lets position-independent code reach imported symbols (9.16)
ABI	application binary interface: calling convention, register use, data layout; version 1 is accepted (errata 46)
trap, exception, interrupt	a deliberate entry (`TRAP #n`), a fault raised by an instruction, and a signal from a device; all enter through the vector table (6.2)
precise fault	a fault after which every earlier instruction has completed and no later one has changed state (6.8)
`SPL`	set the interrupt priority level (6.2)
`WFE`, `SEV`	wait for an event and send an event: the idle and wake-up pair (7.6)
`FENCE`	orders memory accesses; owed to devices and to other cores (2.2, 7.2)
`ATOM`, **monitor**	the atomic operations, built on a reservation monitor as in AXI and ARM, not a locked bus cycle (errata 45, 7.3)
SMP	symmetric multiprocessing: several identical cores sharing memory (chapter 7)
coherence, snooping MSI	the protocol that keeps each core's cache consistent with the others; snooping means each cache watches the bus; MSI is its three states (modified, shared, invalid) (7.7)
sequential consistency	the memory model in which all cores see all accesses in one order; the pdp64 declares it (7.2)

C.3 The bus, devices and boot

Term	Meaning
Q64	the system bus: the PDP-11 Q-bus lineage widened to 64-bit data and 40-bit addresses (2.1)
Unibus, Q-bus	the PDP-11's buses; Q64 keeps their programming model, with devices in the address space (2.1)
`DATIP` / `DATO`	Unibus and Q-bus transaction types forming a locked read-modify-write (2.5)
space bit	bit 39 of the physical address: set selects device space, clear selects memory (2.1.6, errata 60)
slot	a device's own 4 KB (or larger) window of device space, found by enumeration (2.6, 5.1)
bus controller	the device that is the bus: it enumerates slots, maps them, and reports arrivals and departures (2.6)
MMIO	memory-mapped input/output: device registers reached by ordinary loads and stores (5.1)
DMA	direct memory access: a device moving data to or from memory without the processor (5.4)
ring, descriptor, `OWN`**, doorbell**	the common device protocol: a ring of 32-byte descriptors in memory, each with an ownership bit the host sets and the device clears, and a register written to tell the device to look (5.2)
`ID` / `CONTROL` / `STATUS` / `RING_BASE` / `RING_SIZE` / `DOORBELL` / `IRQ` / `DEVICE`	the eight common registers every device starts with (5.2)
`INIT`, `READY`	the control bit that makes a device initialise itself and the status bit that says it has (5.2, 5.3)
virtio	a standard family of virtual devices of the same ring-and-descriptor shape; the emulator's first drivers (5.5, appendix B)
NVMe, SCSI, ATA, SATA	storage interfaces, each a command queue in memory and a doorbell, which is the same device pattern (errata 61, 5.5)

Term	Meaning
HDMI transmitter	the chip that takes a pixel stream and does the link protocol; the pdp64 side is a pixel generator and registers (3.3)
`VIC-64X`, `SID-64X`, `BLT64`	the video, sound and blitter devices of the graphics terminal (chapter 3)
blitter, raster operation	a device that copies and combines rectangles of pixels; the raster operation says how source and destination combine (3.1)
BIOS, boot ROM	the first software that runs: it brings the hardware up, finds the disk label and loads the next stage; open source (4.1, errata 47)
GPT, disk label	the GUID partition table wrapped around the disk's own label so other systems see the disk is in use (4.3, errata 50)
LB64, MX64, VEC64	the inference accelerator and its companions: sign-flip-accumulate layers, floating-point multiply-accumulate for attention, and element-wise vector work (8.3)
APL, K	array languages whose primitives work on whole arrays; one primitive maps to one descriptor (8.4)

C.4 Process and sources

Term	Meaning
errata	the numbered decision log of the project; every decision in the book cites its item (`../errata.md`)
revision 20	the last full revision of the instruction-set text before the streamlined set; changes since are in `../CHANGES-SINCE-R20.md`
fair corpus	the 827 or 841 C files that every back end compiles, used for all size comparisons (1.5)
`pcc`	the portable C compiler the project extends to emit pdp64 code (1.5)
`sim64`, QEMU	the instruction-set simulator and the full-system emulator the tests run on (1.5, 10.4; not otherwise described in this book)
(D), (P), (S)	the tags on a numbered statement: decided, proposed, stub (index)
Impact	the line ending each decision's commentary: effect on the kernel, on the compiler and on the hardware layout (index)

Appendix D — The ABI, version 1

The application binary interface fixes how compiled code, the linker and the operating system agree: the sizes of types, which registers carry arguments and results, who saves what, how the stack looks, and how position-independent code reaches its data. These are accepted choices; the instruction-set text leaves the ABI open (book section 9.16 states what the instruction set makes cheap). The source is `../abi.md`; the compiler, the assembler and the linker follow it, and the version number is how a later change is made without breaking old binaries (errata 46: `long double` is `double` in version 1).

D.1 The conventions

Data model	LP64: `char` 8, `short` 16, `int` 32, `long` 64, `long long` 64, pointers 64. Little-endian. `char` is signed. `long double` is `double`.
Alignment	natural; the stack pointer is a multiple of **8** at all times (data alignment is architectural, ISA 7.4: a quad access to an address not a multiple of 8 aborts; the 2-byte parcel alignment is for instructions only). No 16-byte alignment: no type or register is wider than a quad.
Registers	`R0`-`R5` arguments, `R0` (and `R1`) results, `R0`-`R7` and `R12` caller-saved; `R8`-`R11`, `R15` callee-saved; `R12` linker scratch; `R13` frame pointer (callee-saved, optional); `R14` stack pointer.
Floating point	`binary64` in a general register, `binary32` in the low 32 bits sign-extended (ISA 2.3); passed and returned exactly like integers.
Arguments	in order in `R0`-`R5` (a struct of up to 16 bytes takes one or two consecutive, as many as needed); the rest pushed right to left, each in an 8-byte slot (a larger struct in as many slots as it needs, by value).
Results	`R0`; `R0:R1` for a 128-bit pair or a struct up to 16 bytes; a larger struct through a hidden pointer passed in `R0` (the arguments then start at `R1`), and returned in `R0`.
Calls	`CALL`/`CALLI` push the return address; `RET` pops it. The caller removes stack arguments.
Varargs	the callee stores `R0`-`R5` in a save area directly below the stack arguments; `va_list` is a pointer into the area, then on into the stack arguments. One area suffices because floats use the same registers.
Red zone	none (version 1).
Symbols	no leading underscore.
Landing pads	off in version 1; the assembler provides `.lpad`.
PIC	globals through `d(PC)`; imported symbols through `MOVQ got(PC),Rd`; calls to imports through `CALLI got(PC)`; TLS local-exec `MOVQ off(DP),Rd`.

D.2 Variadic calls

In a call to a function declared with …, every argument goes in an 8-byte slot, in order: slots 0-5 are `r0`-`r5`, later slots are on the stack. A structure of any size is passed by value in consecutive slots and may split between `r5` and the stack. A variadic callee leaves its named parameters in their slots (`fp+16+8k`) and `va_start` is the address after the last named slot; `va_list` is a `char *` and `va_copy` is assignment. The prologue pops the return address, pushes `r5`..`r0` and pushes the return address again, so the register save area is contiguous with the stack arguments.

D.3 What the ABI leaves open

Lazy against eager binding, mandatory landing pads and whether imports are always called without a PLT stub beyond version 1 are open (book section 9.16, decisions in appendix B).

Appendix E — The assembly language and the object format

The assembler's source layout is the V7 assembler's, `as`; its lexical rules are C's. This appendix is the syntax of a source file: lines, labels, numbers, strings, operators, directives, registers, the operand modes, the mnemonic conventions and the object format. The grammar itself is `spec/pdp64-asm.hbnf` (the machine-readable specification, whose text rules are checked by `tools/check-asmsyntax.sh`); the source of this appendix is `../asm-syntax.md`. The mnemonics are in the instruction dictionary.

The grammar is `spec/pdp64-asm.hbnf` (hbnf text rules); `tools/check-asmsyntax.sh` generates the parser and runs `tests/asmsyntax.c` over it, and `tools/check-as64.sh` assembles `tests/asm/*.s` and compares the bytes with hand-worked expectations.

The layout of a source file is the V7 assembler's, `as`; the lexical rules are C's.

- **Lines.** One statement to a line. `;` starts a comment that runs to the end of the line.
- **Labels.** `name:` defines a label; `1:` defines a numeric label, referenced as `1b` (the nearest one before) or `1f` (the nearest after). `sym = expr` assigns. `.` is the location counter. Names are letters, digits, `_`, `.` and `$` and any non-ASCII code point (`café`, `Ж–ラ`); a name may start with a digit as long as the whole bareword is not a number (`3dfx:` is a label; `3`, `0x3d`, `0b101`, `0o17` are numbers; `1f` and `1b` are local references), and a bareword of digits only is a numeric local label. Names are case-sensitive and never truncated. ELF symbol names are any bytes, so a name with other characters is written in double quotes (`"odd name!"`). A mnemonic or register name cannot be a label, and neither can `r` followed only by digits: `r16` is an error, not a name (`r2d2` is a name).
- **UTF-8.** A source file is UTF-8, and the parser refuses anything that is not (a stray byte, an overlong form, a surrogate, a truncated sequence), in names, strings, characters and comments alike; a NUL byte is refused too. In a string, UTF-8 text is copied as bytes and `\u00e9`, `\U0001F600` are written as UTF-8; `'é'` is the code point 0xE9, as is `'\u00e9'`.
- **Numbers and characters** are C's: decimal by default (`010` is ten), `0x`, `0b` or `0o` when asked for; `'a'`, `'\n'`, `'\x41'`, `'\101'`.
- **Strings** are `"text\n"` with C's escapes (`\n \t \r \a \b \f \v \0 \\ \' \" \xHH \ooo`), in `.byte` and `.ascii`.
- **Operators** are C's: `* / % & << >>` and `+ - | ^`, unary `- ~ !`, and parentheses. `/` is division. `*`, `/`, `%`, `&` and the shifts need constants; `+` and `-` accept one relocatable symbol, and the difference of two symbols of one section is a constant.
- **Directives.** `.text .data .bss .globl .comm .byte .word .long .quad .ascii .even .align .space .lpad .pool`. `.word` is 16 bits, `.long` 32, `.quad` 64 (the ISA's PDP-11 terminology). `.align n` aligns to n bytes (a power of two).
- **Registers.** `r0`..`r15`; `sp` is `r14`, `fp` is `r13`. `pc` and `dp` appear only as a base: `d(pc)`, `d(dp)`.
- **Operand modes** (ISA section 5): `rN`; `#expr`; `@expr`; `expr(rN)` and `(rN)`; `expr(rN)(rX*s)` with `s` 1, 2, 4 or 8; `expr(pc)`; `expr(dp)`; `(rN)+` and `(rN)+d`; `-(rN)` and `(rN)-d` (errata 15); a bare expression is a PC-relative reference.
- **Mnemonics** are lowercase (`movq`, `add`, `bne`). A multi-register instruction writes its destination last: `add src, rd`, `xnorpop ra, rb, rd`. `cmp` is the exception, `cmp rd, src`, because the encoding computes `rd - src` (errata 13).
- **Compare and branch.** `cbCC rd, #imm, label` compares `rd` with a 5-bit signed immediate (-16 to 15) and branches on the condition: `cbeq r3, #5, label`. The mnemonics are `cbne cbeq cbpl cbmi cbvc cbvs cbcc cbcs cbge cblt cbgt cble cbhi cblos` and the aliases of the branch group (ISA §4.4.1). With a register as the second operand, `cbCC ra, rb, label` compares two registers (`cbeq r1, r2, label`, ISA §4.4.1), in four bytes or, beyond 4 KB, six. `rori32 rs, #n, rd` rotates the low 32 bits right and sign-extends the result (ISA §4.6). The immediate of `cbCC` may be up to 16 bits (`cbeq r3, #255, label`); the assembler chooses the four-byte, the wide or the long encoding, and refuses a wide immediate whose target is more than 64 KB away (ISA §4.4.1).
- **Literal pool: `=expr`.** An operand written with a leading `=` is **syntactic sugar for a pooled constant, and nothing else**: `movq =0x123456789ABC, r1` puts the 64-bit value in the literal pool and loads it with one PC-relative `movq`. It is accepted where a quad memory source is (`movq`, `push`, the ALU group with the register first for `cmp`: `cmp rd, =K`, and the floating-point ALU ops), and refused for sized moves, shifts, stores, `lea` and a relocatable symbol. `=` always pools, even a small value; use `#imm` for a constant that fits the signed 32-bit field. A bare `#K` that does not fit that field is an **error** ("use =K to load it from the literal pool"): the assembler never pools silently. Entries are 8 bytes, 8-byte aligned, one per distinct value per pool,

in order of first use, emitted into `.text` at the end of the section or at the directive **`.pool`** (flush now; code must jump over a mid-section pool). The `=` marks the sugar on the page, which is why it is allowed (errata 33).

- **Length.** The assembler chooses the shortest encoding, including the one-parcel forms of page 2 (ISA §4.18); those forms have no mnemonics of their own, so `movq (r3), r1` assembles to the one-parcel form when it fits and to the long form when it does not.
- **`llvm-mc`.** The LLVM back end's assembler (`llvm/`, ISA-checked against `as64` by `llvm/check.sh`) accepts this syntax with these exceptions: no literal pool (`=K` and `.pool`), no names that start with a digit, no non-ASCII names, and `.comm` needs its alignment written (`as64` implies 8). `(pc)` and `(dp)` without a displacement are accepted by `as64` (displacement 0) and by `llvm-mc`.
- **Object format.** ELF64, little-endian, `e_machine` 0xF064 (`EM_PDP64`); relocations `R_PDP64_D64`, `R_PDP64_D32`, `R_PDP64_INSN_ABS`, `R_PDP64_INSN_PC`, `R_PDP64_INSN_BR` (`as64/relocs.h`; LLVM: `ELFRelocs/PDP64.def`).

What it leaves out of V7 `as`: octal numbers by default, `<string>` and the `'\e` escape, `\/ \< \>` and `[ ]` in expressions, `.if`, `.stabs`, symbols cut to eight characters.